

Dokumentacja Techniczna Projektu

PLIK: `celery_app.py`

```
"""celery_app.py
```

Konfiguracja Celery i taski generowania PDF.

WZORZEC: minimalna aplikacja Flask dla workera z QueuePool + dispose po fork().

Zamiast NullPool (które otwiera nowe TCP przy każdym zapytaniu) stosujemy:

1. Standardowy QueuePool w procesie macierzystym.
2. Sygnał `worker_process_init`: `db.engine.dispose(close=False)` – porzuca odziedziczone deskryptory bez wysyłania QUIT do bazy, więc każdy podproces workera buduje własną, czystą pulę od pierwszego zapytania.
3. Sygnał `worker_process_shutdown`: `db.session.remove()` + `db.engine.dispose()` – czyste zamknięcie po zatrzymaniu procesu.

Parametry puli (`pool_size=2`, `max_overflow=3`, `pool_timeout=10`) dobrane pod typowe obciążenie Celery: 2 równoległe zapytania na worker + krótki overflow dla spiętrzonych tasków.

```
"""
```

```
import os
```

```
import logging
```

```
from celery import Celery, signals
```

```
from flask import Flask
```

```
logger = logging.getLogger(__name__)
```

```
BROKER_URL = os.environ['CELERY_BROKER_URL']
```

```
RESULT_URL = os.environ['CELERY_RESULT_BACKEND']
```

```
PDF_OUTPUT_DIR = os.environ.get('PDF_OUTPUT_DIR', '/app/pdf_output')
```

```
TEX_SERVICE_URL = os.environ.get('TEX_SERVICE_URL', 'http://tex-service:5002')
```

— 1. Celery – konfiguracja

```
celery = Celery(

    'ans_praktyki',

    broker=BROKER_URL,

    backend=RESULT_URL,

    include=['celery_app'],

)

celery.conf.update(

    task_serializer='json',

    result_serializer='json',

    accept_content=['json'],

    timezone='Europe/Warsaw',

    enable_utc=True,

    task_track_started=True,

    broker_connection_retry_on_startup=True,

    result_expires=3600,

    beat_schedule={

        'cleanup-old-pdfs': {

            'task': 'cleanup_old_pdfs',

            'schedule': 3600.0,    # co godzinę

        },

    },

)
```

— 2. Minimalna aplikacja Flask dla workera

```
def _create_worker_app() -> Flask:

    """

    Tworzy odchudzoną aplikację Flask bez blueprintów ani rozszerzeń webowych.

    Używa QueuePool (domyślna pula SQLAlchemy) z parametrami dopasowanymi do

    środowiska Celery. Właściwa izolacja połączeń po fork() realizowana jest

    przez sygnał worker_process_init.

    """
```

```

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] = os.environ['DATABASE_URL']

app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

# QueuePool: pula utrzymywana per-proces, odbudowywana po fork() via dispose.

app.config['SQLALCHEMY_ENGINE_OPTIONS'] = {

    'pool_size':      2,      # stałe połączenia na worker

    'max_overflow':   3,      # dodatkowe przy spiętrzeniu tasków

    'pool_timeout':   10,     # sekund oczekiwania na wolne połączenie

    'pool_pre_ping':  True,   # wykrywa martwe połączenia przed użyciem

}

app.config['SECRET_KEY'] = os.environ['SECRET_KEY']

app.config['TEX_SERVICE_URL'] = TEX_SERVICE_URL


from core.extensions import db

db.init_app(app)


return app

```

```

_worker_app: Flask | None = None

```

```

@signals.worker_process_init.connect

```

```

def _init_worker(**kwargs):

```

```

    """

```

```

    Wywoływany w każdym nowo sforkowanym podprocesie workera.

```

```

    db.engine.dispose(close=False) porzuca odziedziczone po procesie
    macierzystym deskryptory socketów TCP bez wysyłania komendy zamykającej
    do PostgreSQL. Pierwsze zapytanie w tym podprocesie otworzy świeże,
    wyłącznie mu przynależące połączenia – bez ryzyka kolizji protokołu.

```

```

    """

```

```

    global _worker_app

```

```

    _worker_app = _create_worker_app()

```

```
ctx = _worker_app.app_context()
```

```
ctx.push()
```

```
from core.extensions import db
```

```
db.engine.dispose(close=False)
```

```
logger.info("Worker process initialized (QueuePool, dispose after fork)")
```

```
@signals.worker_process_shutdown.connect
```

```
def _shutdown_worker(**kwargs):
```

```
    """Sprząta połączenia przy zamykaniu procesu workera."""
```

```
    from core.extensions import db
```

```
    try:
```

```
        db.session.remove()
```

```
        db.engine.dispose()
```

```
    except Exception:
```

```
        pass
```

```
    logger.info("Worker process shut down cleanly")
```

```
def _get_app() -> Flask:
```

```
    """Zwraca aplikację workera; fallback dla wywołań spoza kontekstu Celery (np. testy)."""
```

```
    global _worker_app
```

```
    if _worker_app is None:
```

```
        _worker_app = _create_worker_app()
```

```
        _worker_app.app_context().push()
```

```
        from core.extensions import db
```

```
        db.engine.dispose(close=False)
```

```
    return _worker_app
```

```
# — Wyjątki domenowe (nie ponawiamy) —————
```

```
class ZapisNieIstnieje(Exception):
```

```
"""Brak rekordu w bazie – błąd stały, retry bez sensu."""
```

```
# — 3. Taski
```

```
@celery.task(bind=True, name='generate_pdf_dziennik',
              max_retries=3, default_retry_delay=10)

def generate_pdf_dziennik(self, enrollment_id: str) -> dict:

    import uuid

    from pathlib import Path

    import httpx

    from core.extensions import db

    from core.modele import InternshipEnrollment

    from core.uslugi.dokumenty import buduj_kontekst

    zapis = db.session.get(InternshipEnrollment, uuid.UUID(enrollment_id))

    if not zapis:

        raise ZapisNieIstnieje(f'Zapis {enrollment_id} nie istnieje')

    context = buduj_kontekst(zapis, 'ZAL_6')

    tex_url = _get_app().config.get('TEX_SERVICE_URL', TEX_SERVICE_URL)

    try:

        resp = httpx.post(

            f"{tex_url}/generuj",

            json={'template': 'zal6_dziennik.tex.j2', 'context': context},

            timeout=60.0,

        )

        resp.raise_for_status()

    except httpx.HTTPError as exc:

        logger.warning("Tex-service error (retry %d/%d): %s",

                      self.request.retries, self.max_retries, exc)

        raise self.retry(exc=exc, countdown=10 * (self.request.retries + 1))
```

```

output_dir = Path(PDF_OUTPUT_DIR)

output_dir.mkdir(parents=True, exist_ok=True)

filename = f"dziennik_{enrollment_id}.pdf"

output_path = output_dir / filename

output_path.write_bytes(resp.content)

return {

    'status': 'success',

    'path': str(output_path),

    'filename': f"dziennik_{zapis.student.last_name}_{zapis.student.first_name}.pdf",

}

```

```

@celery.task(name='cleanup_old_pdfs')

```

```

def cleanup_old_pdfs(max_age_hours: int = 24) -> dict:

    """Usuwa pliki PDF starsze niż max_age_hours z katalogu PDF_OUTPUT_DIR."""

    import time

    from pathlib import Path

    output_dir = Path(PDF_OUTPUT_DIR)

    if not output_dir.exists():

        return {'deleted': 0, 'errors': 0}

    cutoff = time.time() - max_age_hours * 3600

    deleted = errors = 0

    for f in output_dir.glob('*.pdf'):

        try:

            if f.stat().st_mtime < cutoff:

                f.unlink()

                deleted += 1

        except Exception as exc:

            logger.warning("Nie udało się usunąć %s: %s", f, exc)

            errors += 1

```

```
logger.info("Czyszczenie PDF: usunięto %d, błędy %d (próg: %dh)", deleted, errors, max_age_hours)

return {'deleted': deleted, 'errors': errors}
```

```
@celery.task(bind=True, name='generuj_pdf_z_szablonu',

              max_retries=3, default_retry_delay=10)

def generuj_pdf_z_szablonu(self, template_name: str, context: dict,

                           filename_prefix: str) -> dict:
```

```
    """Kompiluje PDF z nazwanego szablonu Jinja2 i słownika danych.
```

```
    Jedyna dozwolona ścieżka asynchronicznej kompilacji – nie przyjmuje
    surowego kodu TeX, eliminując wektor Command Injection.
```

Args:

```
    template_name: nazwa pliku szablonu z katalogu templates/
                   (np. 'zal6_dziennik.tex.j2')
    context:       słownik danych domenowych (JSON-serializowalny)
    filename_prefix: prefiks nazwy pliku wynikowego
```

```
    """
```

```
import uuid
```

```
from pathlib import Path
```

```
try:
```

```
    from tex_service.compiler import compile_pdf

    pdf_bytes = compile_pdf(template_name, context)
```

```
except Exception as exc:
```

```
    logger.warning("LaTeX compile error (retry %d/%d): %s",

                  self.request.retries, self.max_retries, exc)
```

```
    raise self.retry(exc=exc, countdown=10 * (self.request.retries + 1))
```

```
output_dir = Path(PDF_OUTPUT_DIR)
```

```
output_dir.mkdir(parents=True, exist_ok=True)
```

```
filename = f"{filename_prefix}_{uuid.uuid4().hex[:8]}.pdf"
```

```
output_path = output_dir / filename
```

```
output_path.write_bytes(pdf_bytes)
```

```
return {
```

```
    'status': 'success',
```

```
    'path': str(output_path),
```

```
    'filename': filename,
```

```
}
```


PLIK: docker-compose.patch.yml

```
version: '3.8'
```

```
services:
```

```
  tex-service:
```

```
    build:
```

```
      context: ./tex_service
```

```
      dockerfile: Dockerfile
```

```
    container_name: sop_tex_service
```

```
    restart: unless-stopped
```

```
    environment:
```

```
      - TZ=Europe/Warsaw
```

```
    volumes:
```

```
      - ./tex_engine/templates:/app/templates:ro
```

```
    healthcheck:
```

```
      test: ["CMD", "curl", "-f", "http://localhost:5002/health"]
```

```
      interval: 10s
```

```
      timeout: 5s
```

```
      retries: 3
```

```
    expose:
```

```
      - "5002"
```

PLIK: docker-compose.yml

services:

db:

image: postgres:16-alpine

container_name: sop_db

restart: unless-stopped

environment:

POSTGRES_USER: \${POSTGRES_USER:-ans_admin}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}

POSTGRES_DB: \${POSTGRES_DB:-ans_praktyki}

networks:

- internal

volumes:

- ./database/init.sql:/docker-entrypoint-initdb.d/init.sql

- pgdata:/var/lib/postgresql/data

deploy:

resources:

limits:

cpus: '1.0'

memory: 512M

redis:

image: redis:7-alpine

container_name: sop_redis

restart: unless-stopped

networks:

- internal

volumes:

- redis_data:/data

deploy:

resources:

limits:

cpus: '0.25'

memory: 128M

admin:

build:

context: .

dockerfile: app_admin/Dockerfile

container_name: sop_admin

restart: unless-stopped

environment:

- FLASK_ENV=\${FLASK_ENV:-production}
- DATABASE_URL=postgresql+psycopg2://\${POSTGRES_USER:-ans_admin}:\${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}@db:5432/\${POSTGRES_DB:-ans_praktyki}
- SECRET_KEY=\${SECRET_KEY:?Set SECRET_KEY in .env}
- FILE_ENCRYPTION_KEY=\${FILE_ENCRYPTION_KEY:?Set FILE_ENCRYPTION_KEY in .env}
- FILESERVER_URL=\${FILESERVER_URL:-http://fileserver:5003}
- FILESERVER_API_KEY=\${FILESERVER_API_KEY:?Set FILESERVER_API_KEY in .env}
- TEX_SERVICE_URL=http://tex-service:5002
- CELERY_BROKER_URL=redis://redis:6379/0
- CELERY_RESULT_BACKEND=redis://redis:6379/0
- AZURE_CLIENT_ID=\${AZURE_CLIENT_ID:?Set AZURE_CLIENT_ID in .env}
- AZURE_CLIENT_SECRET=\${AZURE_CLIENT_SECRET:?Set AZURE_CLIENT_SECRET in .env}
- AZURE_TENANT_ID=\${AZURE_TENANT_ID:?Set AZURE_TENANT_ID in .env}

ports:

- "5000:5000"

networks:

- internal
- public

depends_on:

tex-service:

condition: service_healthy

db:

condition: service_started

redis:

condition: service_started

fileserver:

condition: service_healthy

```
volumes:

  - pdf_data:/app/pdf_output

  - ./app_admin:/app/app_admin:ro

  - ./core:/app/core:ro

read_only: true

tmpfs:

  - /tmp

  - /app/app_admin/logs

security_opt:

  - no-new-privileges:true

cap_drop:

  - ALL

deploy:

  resources:

    limits:

      cpus: '0.5'

      memory: 256M

student:

  build:

    context: .

    dockerfile: app_student/Dockerfile

  container_name: sop_student

  restart: unless-stopped

  environment:

    - FLASK_ENV=${FLASK_ENV:-production}

    - DATABASE_URL=postgresql+psycpg2://${POSTGRES_USER:-ans_admin}:${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}@db:5432/${POSTGRES_DB:-ans_praktyki}

    - SECRET_KEY=${SECRET_KEY:?Set SECRET_KEY in .env}

    - FILE_ENCRYPTION_KEY=${FILE_ENCRYPTION_KEY:?Set FILE_ENCRYPTION_KEY in .env}

    - FILESERVER_URL=${FILESERVER_URL:-http://fileserver:5003}

    - FILESERVER_API_KEY=${FILESERVER_API_KEY:?Set FILESERVER_API_KEY in .env}

    - TEX_SERVICE_URL=http://tex-service:5002

    - CELERY_BROKER_URL=redis://redis:6379/0

    - CELERY_RESULT_BACKEND=redis://redis:6379/0
```

- AZURE_CLIENT_ID=\${AZURE_CLIENT_ID:?Set AZURE_CLIENT_ID in .env}
- AZURE_CLIENT_SECRET=\${AZURE_CLIENT_SECRET:?Set AZURE_CLIENT_SECRET in .env}
- AZURE_TENANT_ID=\${AZURE_TENANT_ID:?Set AZURE_TENANT_ID in .env}

ports:

- "5001:5001"

networks:

- internal
- public

depends_on:

tex-service:

condition: service_healthy

db:

condition: service_started

redis:

condition: service_started

fileserver:

condition: service_healthy

volumes:

- pdf_data:/app/pdf_output
- ./app_student:/app/app_student:ro
- ./core:/app/core:ro

read_only: true

tmpfs:

- /tmp

security_opt:

- no-new-privileges:true

cap_drop:

- ALL

deploy:

resources:

limits:

cpus: '0.5'

memory: 256M

celery-worker:

build:

context: .

dockerfile: app_admin/Dockerfile

container_name: sop_celery_worker

restart: unless-stopped

environment:

- FLASK_ENV=\${FLASK_ENV:-production}

- DATABASE_URL=postgresql+psycopg2://\${POSTGRES_USER:-ans_admin}:\${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}@db:5432/\${POSTGRES_DB:-ans_praktyki}

- SECRET_KEY=\${SECRET_KEY:?Set SECRET_KEY in .env}

- FILE_ENCRYPTION_KEY=\${FILE_ENCRYPTION_KEY:?Set FILE_ENCRYPTION_KEY in .env}

- FILESERVER_URL=\${FILESERVER_URL:-http://fileserver:5003}

- FILESERVER_API_KEY=\${FILESERVER_API_KEY:?Set FILESERVER_API_KEY in .env}

- TEX_SERVICE_URL=http://tex-service:5002

- CELERY_BROKER_URL=redis://redis:6379/0

- CELERY_RESULT_BACKEND=redis://redis:6379/0

command: celery -A celery_app worker --loglevel=info --concurrency=2

networks:

- internal

depends_on:

redis:

condition: service_started

db:

condition: service_started

tex-service:

condition: service_healthy

volumes:

- pdf_data:/app/pdf_output

- ./app_admin:/app/app_admin:ro

- ./core:/app/core:ro

read_only: true

tmpfs:

- /tmp

- /app/app_admin/logs

```
security_opt:

  - no-new-privileges:true

cap_drop:

  - ALL

deploy:

  resources:

    limits:

      cpus: '1.0'

      memory: 512M

celery-beat:

  build:

    context: .

    dockerfile: app_admin/Dockerfile

  container_name: sop_celery_beat

  restart: unless-stopped

  environment:

    - FLASK_ENV=${FLASK_ENV:-production}

    - DATABASE_URL=postgresql+psycopg2://${POSTGRES_USER:-ans_admin}:${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}@db:5432/${POSTGRES_DB:-ans_praktyki}

    - SECRET_KEY=${SECRET_KEY:?Set SECRET_KEY in .env}

    - FILE_ENCRYPTION_KEY=${FILE_ENCRYPTION_KEY:?Set FILE_ENCRYPTION_KEY in .env}

    - FILESERVER_URL=${FILESERVER_URL:-http://fileserver:5003}

    - FILESERVER_API_KEY=${FILESERVER_API_KEY:?Set FILESERVER_API_KEY in .env}

    - TEX_SERVICE_URL=http://tex-service:5002

    - CELERY_BROKER_URL=redis://redis:6379/0

    - CELERY_RESULT_BACKEND=redis://redis:6379/0

    - PDF_OUTPUT_DIR=/app/pdf_output

  command: celery -A celery_app beat --loglevel=info --schedule=/tmp/celerybeat-schedule

  networks:

    - internal

  depends_on:

    redis:

      condition: service_started

    db:
```

```
    condition: service_started

volumes:

  - pdf_data:/app/pdf_output

  - ./core:/app/core:ro

read_only: true

tmpfs:

  - /tmp

security_opt:

  - no-new-privileges:true

cap_drop:

  - ALL

deploy:

  resources:

    limits:

      cpus: '0.25'

      memory: 128M

fileserver:

  build:

    context: ./fileserver

    dockerfile: Dockerfile

  container_name: sop_fileserver

  restart: unless-stopped

  environment:

    - FILESERVER_API_KEY=${FILESERVER_API_KEY:?Set FILESERVER_API_KEY in .env}

  networks:

    - internal

  volumes:

    - fileserver_data:/data/files

  healthcheck:

    test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:5003/health')"]

    interval: 10s

    timeout: 5s

    retries: 5
```


start_period: 10s

tex-service:

build:

context: ./tex_service

dockerfile: Dockerfile

container_name: sop_tex_service

restart: unless-stopped

environment:

- TZ=Europe/Warsaw
- TEXMFVAR=/tmp/texmf-var
- TEXMFCACHE=/tmp/texmf-var
- HOME=/tmp

volumes:

- ./tex_service/templates:/app/templates:ro

expose:

- "5002"

networks:

- internal

healthcheck:

test: ["CMD", "curl", "-f", "http://localhost:5002/health"]

interval: 10s

timeout: 5s

retries: 5

start_period: 30s

read_only: true

tmpfs:

- /tmp:size=256m,mode=1777
- /root:size=128m,mode=0700

security_opt:

- no-new-privileges:true

cap_drop:

- ALL

deploy:

resources:

limits:

cpus: '1.0'

memory: 512M

reservations:

memory: 128M

networks:

Sieć wewnętrzna – bez dostępu do internetu.

Wszystkie serwisy backendowe komunikują się wyłącznie przez tę sieć.

internal:

internal: true

Sieć publiczna – wyłącznie dla admin i student (porty 5000/5001).

public:

volumes:

pgdata:

pdf_data:

fileserver_data:

redis_data:

PLIK: x.py

```
import os

import unicodedata

from fpdf import FPDF, XPos, YPos


# Funkcja ratunkowa: usunie polskie znaki, jeśli czcionki systemowe zawiodą,
# żeby skrypt się nie wywalił na literce "ż" lub "ł".

def usun_polskie_znaki(tekst):

    return unicodedata.normalize('NFKD', tekst).encode('ASCII', 'ignore').decode('ASCII')


class CodeToPDF(FPDF):

    def header(self):

        if self.page_no() == 1:

            self.set_font(self.font_family_name, 'B', 16)

            # Zamiast ln=True używamy nowych standardów fpdf2

            self.cell(0, 10, 'Dokumentacja Techniczna Projektu', new_x=XPos.LMARGIN, new_y=YPos.NEXT, align='C')

            self.ln(10)


def create_pdf_from_project(source_dir, output_filename):

    if not os.path.exists(source_dir):

        print(f" BŁĄD: Folder '{source_dir}' nie istnieje!")

        return

    print(f" Rozpoczynam skanowanie folderu: {os.path.abspath(source_dir)}")

    pdf = CodeToPDF()

    pdf.set_auto_page_break(auto=True, margin=15)

    # --- SZUKANIE CZCIONKI ---

    font_name = 'Courier'

    ma_polskie_znaki = False

    # Lista popularnych czcionek (Consolas jest najlepsza do kodu)

    czcionki_do_sprawdzenia = [
```

```

('Consolas', r'C:\Windows\Fonts\consola.ttf', r'C:\Windows\Fonts\consolab.ttf'),

('Arial', r'C:\Windows\Fonts\arial.ttf', r'C:\Windows\Fonts\arialbd.ttf'),

('CourierNew', r'C:\Windows\Fonts\cour.ttf', r'C:\Windows\Fonts\courbd.ttf')

]

for nazwa, plik_zwykly, plik_pogrubiony in czcionki_do_sprawdzenia:

    if os.path.exists(plik_zwykly) and os.path.exists(plik_pogrubiony):

        try:

            pdf.add_font(nazwa, '', plik_zwykly)

            pdf.add_font(nazwa, 'B', plik_pogrubiony)

            font_name = nazwa

            ma_polskie_znaki = True

            print(f" Znaleziono czcionkę obsługującą polskie znaki: {nazwa}")

            break

        except Exception:

            continue

if not ma_polskie_znaki:

    print(" UWAGA: Nie mogę załadować systemowej czcionki TTF.")

    print(" Uruchamiam protokół ratunkowy: konwersja na ASCII (ż -> z, ł -> l).")

pdf.font_family_name = font_name

ext_list = ('.py', '.html', '.css', '.yaml', '.yml', 'Dockerfile')

znalezione_pliki = 0

for root, dirs, files in os.walk(source_dir):

    # Ignorujemy niepotrzebne foldery środowiskowe

    dirs[:] = [d for d in dirs if d not in ['.git', 'venv', '.venv', '__pycache__', 'node_modules']]

    for file in files:

        if file.endswith(ext_list) or file == 'Dockerfile':

            znalezione_pliki += 1

            file_path = os.path.join(root, file)

```

```

rel_path = os.path.relpath(file_path, source_dir)

print(f" Przetwarzam: {rel_path}")

pdf.add_page()

# --- NAGŁÓWEK PLIKU ---

pdf.set_fill_color(240, 240, 240)

pdf.set_text_color(0, 51, 102)

pdf.set_font(font_name, 'B', 11)

tytul_pliku = f" PLIK: {rel_path}"

if not ma_polskie_znaki: tytul_pliku = usun_polskie_znaki(tytul_pliku)

# Nowe wywołania metody cell

pdf.cell(0, 8, tytul_pliku, new_x=XPos.LMARGIN, new_y=YPos.NEXT, fill=True)

pdf.ln(4)

# --- TREŚĆ KODU ---

pdf.set_text_color(30, 30, 30)

pdf.set_font(font_name, '', 8)

try:

    with open(file_path, 'r', encoding='utf-8', errors='replace') as f:

        for linia in f:

            format_linii = linia.replace('\t', '    ')

            if not ma_polskie_znaki:

                format_linii = usun_polskie_znaki(format_linii)

            pdf.multi_cell(0, 4, format_linii)

except Exception as e:

    pdf.set_text_color(200, 0, 0)

    pdf.set_font(font_name, 'B', 10)

    blad = f"Bład odczytu: {e}"

    if not ma_polskie_znaki: blad = usun_polskie_znaki(blad)

```

```
pdf.cell(0, 10, blad, new_x=XPos.LMARGIN, new_y=YPos.NEXT)
```

```
if znalezione_pliki == 0:
```

```
    print(f" Nie znaleziono plików.")
```

```
    return
```

```
pdf.output(output_filename)
```

```
print(f" SUKCES! Gotowe. Złożono {znalezione_pliki} plików w dokumencie: '{output_filename}'.")
```

```
# --- ODPALAMY ---
```

```
folder_zrodlowy = '.'
```

```
nazwa_pdf = 'dokumentacja_kodu.pdf'
```

```
create_pdf_from_project(folder_zrodlowy, nazwa_pdf)
```

PLIK: y.py

```
import os

import unicodedata

from fpdf import FPDF, XPos, YPos


def usun_polskie_znaki(tekst):

    """Awaryjne usuwanie ogonków, gdyby system zablokował czcionki."""

    return unicodedata.normalize('NFKD', tekst).encode('ASCII', 'ignore').decode('ASCII')


class TreePDF(FPDF):

    def header(self):

        if self.page_no() == 1:

            self.set_font(self.font_family_name, 'B', 16)

            self.cell(0, 10, 'Struktura Drzewa Projektu', new_x=XPos.LMARGIN, new_y=YPos.NEXT, align='C')

            self.ln(5)


def generate_tree(dir_path, prefix="", ignore_dirs=None):

    """Rekurencyjnie generuje linie tekstu reprezentujące drzewo folderów."""

    if ignore_dirs is None:

        # Lista folderów, które nie będą zaśmiecać drzewa

        ignore_dirs = {'.git', 'venv', '.venv', '__pycache__', 'node_modules', '.idea', '.vscode'}

    try:

        items = sorted(os.listdir(dir_path))

    except PermissionError:

        return [f"{prefix}[Brak dostępu]"]

    # Filtrujemy ignorowane foldery

    filtered_items = []

    for item in items:

        full_path = os.path.join(dir_path, item)

        if os.path.isdir(full_path) and item in ignore_dirs:

            continue

        filtered_items.append(item)
```

```

lines = []

for i, item in enumerate(filtered_items):

    is_last = (i == len(filtered_items) - 1)

    # Używamy bezpiecznych znaków ASCII do rysowania gałęzi

    connector = "\-- " if is_last else "+-- "

    lines.append(f"{prefix}{connector}{item}")

    full_path = os.path.join(dir_path, item)

    if os.path.isdir(full_path):

        extension = "    " if is_last else "|    "

        lines.extend(generate_tree(full_path, prefix=prefix + extension, ignore_dirs=ignore_dirs))

return lines

def create_tree_pdf(source_dir, output_filename):

    if not os.path.exists(source_dir):

        print(f" BŁĄD: Folder '{source_dir}' nie istnieje!")

        return

    print(f" Skanuję strukturę folderu: {os.path.abspath(source_dir)}")

    pdf = TreePDF()

    pdf.set_auto_page_break(auto=True, margin=15)

    # --- SZUKANIE BEZPIECZNEJ CZCIONKI MONOSPACE ---

    font_name = 'Courier'

    ma_polskie_znaki = False

    czcionki_do_sprawdzenia = [

        ('Consolas', r'C:\Windows\Fonts\consola.ttf', r'C:\Windows\Fonts\consolab.ttf'),

        ('CourierNew', r'C:\Windows\Fonts\cour.ttf', r'C:\Windows\Fonts\courbd.ttf')

    ]

```



```

for nazwa, plik_zwykly, plik_pogrubiony in czcionki_do_sprawdzenia:

    if os.path.exists(plik_zwykly) and os.path.exists(plik_pogrubiony):

        try:

            pdf.add_font(nazwa, '', plik_zwykly)

            pdf.add_font(nazwa, 'B', plik_pogrubiony)

            font_name = nazwa

            ma_polskie_znaki = True

            break

        except Exception:

            continue

pdf.font_family_name = font_name

pdf.add_page()

# --- RYSOWANIE DRZEWA ---

pdf.set_text_color(30, 30, 30)

pdf.set_font(font_name, 'B', 10)

# Nazwa głównego folderu

root_name = os.path.basename(os.path.abspath(source_dir))

if not root_name: root_name = "Projekt"

if not ma_polskie_znaki: root_name = usun_polskie_znaki(root_name)

pdf.cell(0, 6, f"Katalog: {root_name}", new_x=XPos.LMARGIN, new_y=YPos.NEXT)

pdf.set_font(font_name, '', 10)

# Pobieranie linii drzewa z funkcji

drzewo_linie = generate_tree(source_dir)

# Zapisywanie każdej linii do PDF

for linia in drzewo_linie:

    tekst_linii = linia

    if not ma_polskie_znaki:

```

```
tekst_linii = usun_polskie_znaki(tekst_linii)

pdf.cell(0, 5, tekst_linii, new_x=XPos.LMARGIN, new_y=YPos.NEXT)

pdf.output(output_filename)

print(f" SUKCES! Struktura projektu zapisana jako: '{output_filename}'.")

# --- KONFIGURACJA ---

folder_zrodlowy = '.' # '.' oznacza obecny folder

nazwa_pdf = 'struktura_projektu.pdf'

create_tree_pdf(folder_zrodlowy, nazwa_pdf)
```

PLIK: app_admin\config.py

```
import os


def _require(name: str) -> str:

    v = os.environ.get(name)

    if not v:

        raise RuntimeError(

            f"Zmienna środowiskowa {name!r} nie jest ustawiona. "

            "Uzupełnij plik .env lub zmienne środowiskowe kontenera."

        )

    return v


class Config:

    # Osobne nazwy ciasteczek dla admina i studenta

    SESSION_COOKIE_NAME = 'admin_session'

    SECRET_KEY = _require('SECRET_KEY')

    SQLALCHEMY_DATABASE_URI = _require('DATABASE_URL')

    SQLALCHEMY_TRACK_MODIFICATIONS = False

    # Microsoft Entra ID (Azure AD) – OAuth 2.0

    AZURE_CLIENT_ID = _require('AZURE_CLIENT_ID')

    AZURE_CLIENT_SECRET = _require('AZURE_CLIENT_SECRET')

    AZURE_TENANT_ID = _require('AZURE_TENANT_ID')

    # Rate limiting (flask-limiter + Redis)

    RATELIMIT_STORAGE_URI = os.environ.get('CELERY_BROKER_URL', 'redis://localhost:6379/0')


class DevelopmentConfig(Config):

    DEBUG = True

    ENV = 'development'
```

```
class ProductionConfig(Config):

    DEBUG = False

    ENV = 'production'

    SESSION_COOKIE_SECURE = True

    SESSION_COOKIE_HTTPONLY = True

    SESSION_COOKIE_SAMESITE = 'Lax'


# Słownik ułatwiający Fabryce Aplikacji wybór odpowiedniej klasy
config_dict = {

    'development': DevelopmentConfig,

    'production': ProductionConfig,

    'default': DevelopmentConfig

}
```

PLIK: app_admin\Dockerfile

```
FROM python:3.12-slim
```

```
ENV PYTHONDONTWRITEBYTECODE=1 \
```

```
    PYTHONUNBUFFERED=1 \
```

```
    PYTHONPATH=/app
```

```
WORKDIR /app
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
```

```
    curl \
```

```
    libmagic1 \
```

```
    && rm -rf /var/lib/apt/lists/*
```

```
COPY app_admin/requirements.txt requirements.txt
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
# Bezpieczeństwo: dedykowany nieuprzywilejowany użytkownik (nie root)
```

```
RUN addgroup --system appgroup \
```

```
    && adduser --system --ingroup appgroup --no-create-home appuser \
```

```
    && chown -R appuser:appgroup /app
```

```
USER appuser
```

```
EXPOSE 5000
```

```
# gevent worker: współdzieli greenlets w pętli zdarzeń I/O – eliminuje blokadę
```

```
# wątku przez time.sleep w endpointach SSE przy zachowaniu --workers 2.
```

```
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--worker-class", "gevent", "--worker-connections", "100",  
"app_admin:create_app()"]
```

PLIK: app_admin__init__.py

```
import os

from flask import Flask

from jinja2 import select_autoescape

from core.extensions import db, login_manager, csrf, limiter

from core.error_handlers import register_error_handlers

import logging

from logging.handlers import RotatingFileHandler

from pathlib import Path


def create_app():

    from pathlib import Path as _Path

    from flask import Blueprint as _Blueprint

    app = Flask(__name__)

    # — Core static assets (CSS design system + templates) —————

    core_bp = _Blueprint(

        'core_static', __name__,

        static_folder=str(_Path(__file__).parent.parent / 'core' / 'static'),

        static_url_path='/core/static',

        template_folder=str(_Path(__file__).parent.parent / 'core' / 'templates'),

    )

    app.register_blueprint(core_bp)

    app.jinja_options = app.jinja_options.copy()

    app.jinja_options.update(dict(

        autoescape=select_autoescape(['html', 'xml'])

    ))

    # Configure file-based logging so exceptions are captured when console is silent

    logs_dir = Path(app.root_path) / 'logs'

    try:

        logs_dir.mkdir(exist_ok=True)
```

```

file_handler = RotatingFileHandler(logs_dir / 'app.log', maxBytes=1000000, backupCount=5)

file_handler.setLevel(logging.INFO)

formatter = logging.Formatter('%(asctime)s %(levelname)s: %(message)s [in %(pathname)s:%(lineno)d]')

file_handler.setFormatter(formatter)

app.logger.addHandler(file_handler)

app.logger.setLevel(logging.INFO)

except Exception:

    pass


from app_admin.config import config_dict

env = os.environ.get('FLASK_ENV', 'development')

app.config.from_object(config_dict.get(env, config_dict['default']))


db.init_app(app)

login_manager.init_app(app)

csrf.init_app(app)

limiter.init_app(app)

register_error_handlers(app)


login_manager.login_view = 'auth.logowanie'

# Polish translation for Flask-Login unauthorized message

login_manager.login_message = 'Zaloguj się, aby uzyskać dostęp do tej strony.'

login_manager.login_message_category = 'warning'

with app.app_context():

    from core.autoryzacja import stworz_blueprint_auth, wymaga_rol

    from core.pliki import stworz_blueprint_pliki

    from core.modele import UserRole

    from app_admin.routes.pulpit import dashboard_bp

    from app_admin.routes.zarzadzanie import zarzadzanie_bp

    from app_admin.routes.ocenianie import evaluation_bp

    from app_admin.routes.dziennik import journal_bp

    from app_admin.routes.dokumenty import documents_bp


auth_bp = stworz_blueprint_auth(

```

```

        dozwolone_role=[UserRole.ADMIN, UserRole.UOPZ],

        template_logowania='auth/logowanie.html',

        template_zmiany_hasla='auth/zmien_haslo.html',

    )

uploads_bp = stworz_blueprint_pliki()


app.register_blueprint(auth_bp)

app.register_blueprint(dashboard_bp, url_prefix='/panel')

app.register_blueprint(zarzadzanie_bp, url_prefix='/zarzadzanie')

app.register_blueprint(evaluation_bp, url_prefix='/oceny')

app.register_blueprint(journal_bp, url_prefix='/dzienniki')

app.register_blueprint(documents_bp, url_prefix='/dokumenty')

app.register_blueprint(uploads_bp, url_prefix='/uploads')

```

```
@app.context_processor
```

```
def inject_nav_counts():
```

```
    counts = {}
```

```
    try:
```

```
        from flask_login import current_user
```

```
        if current_user.is_authenticated:
```

```
            from core.repozytoria import RepozytoriumZapisow
```

```
            row = RepozytoriumZapisow().liczniki_nav()
```

```
            counts['nav_oczekujace'] = row['oczekujace']
```

```
            counts['nav_komisja'] = row['komisja']
```

```
            counts['nav_dziekan'] = row['dziekan']
```

```
    except Exception:
```

```
        pass
```

```
    return counts
```

```
@app.context_processor
```

```
def inject_tlumacz():
```

```
    return dict(tlumacz_status=lambda val: {
```

```
        'PENDING': 'Oczekuje',
```

```
        'AWAITING_APPROVAL': 'Oczekuje na akceptację',
```



```
        'IN_PROGRESS':      'W trakcie',
        'COMPLETED':       'Zakończona',
        'REJECTED':         'Odrzucony',
        'ACTIVE':           'Aktywna',
        'INACTIVE':         'Nieaktywna',
    }.get(val, val))
```

```
    return app
```

```
@login_manager.user_loader
```

```
def wczytaj_uzytkownika(user_id):
```

```
    from core.modele import User
```

```
    return db.session.get(User, user_id)
```

PLIK: app_admin\routes\dokumenty.py

```
import uuid

import json

import time


try:

    # Przy gevent workerze monkeypatching jest aktywny – używamy gevent.sleep,

    # które oddaje procesor innym greenletom zamiast blokować wątek OS.

    # Przy sync workerze (testy, dev) fallback na time.sleep.

    from gevent import sleep as _sleep

except ImportError:

    from time import sleep as _sleep


from flask import Blueprint, render_template, redirect, url_for, flash, request, jsonify, abort, send_file, current_app, Response

from flask_login import login_required, current_user

from pathlib import Path


from core.modele import InternshipEnrollment, DocumentAuditLog, UserRole

from core.extensions import db, limiter

from core.autoryzacja import wymaga roli

from core.repozytoria import RepozytoriumZapisow, RepozytoriumLogow


_repo_zapisow = RepozytoriumZapisow()

_repo_logow = RepozytoriumLogow()


import logging

logger = logging.getLogger(__name__)


documents_bp = Blueprint('documents', __name__)


DOCUMENTS = {

    'ZAL_1': ('zal1_porozumienie.tex.j2', 'Załącznik 1 (Porozumienie)'),

    'ZAL_2a': ('zal2a_harmonogram.tex.j2', 'Załącznik 2a (Harmonogram Uczelniiany)'),

    'ZAL_3': ('zal3_skierowanie.tex.j2', 'Załącznik 3 (Skierowanie i ocena ZOPZ)'),
```

```

'ZAL_4': ('zal4_efekty.tex.j2', 'Załącznik 4 (Efekty Uczenia Się)'),
'ZAL_4a': ('zal4a_komisja.tex.j2', 'Załącznik 4a (Opinia Komisji do weryfikacji)'),
'ZAL_4b': ('zal4b_wniosek.tex.j2', 'Załącznik 4b (Wniosek o działalność gospodarczą)'),
'ZAL_6': ('zal6_dziennik.tex.j2', 'Załącznik 6 (Dziennik Praktyki)'),
'ZAL_7': ('zal7_sprawozdanie.tex.j2', 'Załącznik 7 (Sprawozdanie Instytucji)'),
'ZAL_7a': ('zal7a_wniosek.tex.j2', 'Załącznik 7a (Wniosek o pracę zawodową)'),
'ZAL_8': ('zal8_protokol.tex.j2', 'Załącznik 8 (Protokół Egzaminacyjny)'),
'ZAL_9': ('zal9_zobowiazanie.tex.j2', 'Załącznik 9 (Zobowiązanie poufności)'),
}

```

```

def log_audit(zapis_id, doc_type, action, details=""):

```

```

    db.session.add(DocumentAuditLog(

        id=uuid.uuid4(),

        user_id=current_user.id,

        enrollment_id=zapis_id,

        document_type=doc_type,

        action=action,

        details=details

    ))

    db.session.commit()

```

```

@documents_bp.route('/zapis/<uuid:id>')

```

```

@wymaga_roli(UserRole.ADMIN, UserRole.UOPZ)

```

```

def panel(id):

```

```

    zapis = _repo_zapisow.znajdz_po_id(id) or abort(404)

    logs = _repo_logow.ostatnie_dla_zapisu(id)

    return render_template('documents/panel.html', zapis=zapis, docs=DOCUMENTS, logs=logs)

```

```

@documents_bp.route('/zapis/<uuid:id>/generuj_auto/<doc_type>', methods=['POST'])

```

```

@wymaga_roli(UserRole.ADMIN, UserRole.UOPZ)

```

```

@limiter.limit("60 per hour")

```

```

def generuj_auto(id, doc_type):

```

```

    zapis = _repo_zapisow.znajdz_po_id(id) or abort(404)

    if doc_type not in DOCUMENTS:

```

```
abort(404)
```

```
template_name, _ = DOCUMENTS[doc_type]
```

```
from core.uslugi.dokumenty import buduj_kontekst
```

```
from celery_app import generuj_pdf_z_szablonu
```

```
context = buduj_kontekst(zapis, doc_type)
```

```
try:
```

```
    task = generuj_pdf_z_szablonu.delay(template_name, context, doc_type.lower())
```

```
    log_audit(zapis.id, doc_type, 'GENERATED_AUTO', f'Wygenerowano {doc_type}')
```

```
    return jsonify({'task_id': task.id, 'status': 'PENDING'})
```

```
except Exception as e:
```

```
    return jsonify({'error': str(e)}), 500
```

```
@documents_bp.route('/status/<task_id>')
```

```
@wymaga_rolu(UserRole.ADMIN, UserRole.UOPZ)
```

```
def status_pdf(task_id):
```

```
    """Legacy polling endpoint – zachowany dla kompatybilności wstecznej."""
```

```
    try:
```

```
        from celery_app import celery
```

```
        task = celery.AsyncResult(task_id)
```

```
        if task.state == 'SUCCESS':
```

```
            res = task.result
```

```
            if isinstance(res, dict) and res.get('status') == 'error':
```

```
                return jsonify({'status': 'FAILURE', 'error': res.get('message')})
```

```
            return jsonify({'status': 'SUCCESS', 'download_url': url_for('documents.pobierz_pdf', task_id=task_id)})
```

```
        elif task.state == 'FAILURE':
```

```
            return jsonify({'status': 'FAILURE', 'error': str(task.info)})
```

```
        else:
```

```
            return jsonify({'status': task.state})
```

```
except Exception as e:
```

```
    return jsonify({'error': str(e)}), 500
```

```
_SSE_MAX_SECONDS = 120 # po tym czasie strumień jest zamykany niezależnie od stanu
```

```
@documents_bp.route('/stream/<task_id>')
```

```
@wymaga_rolu(UserRole.ADMIN, UserRole.UOPZ)
```

```
def stream_status(task_id):
```

```
    """Server-Sent Events – status generowania PDF.
```

```
    Strumień jest automatycznie zamykany po _SSE_MAX_SECONDS sekund,  
    co chroni wątki serwera przed blokadą przez osierocone połączenia  
    (np. po zamknięciu karty przez użytkownika).
```

```
    """
```

```
    from celery_app import celery
```

```
def generate():
```

```
    deadline = time.monotonic() + _SSE_MAX_SECONDS
```

```
    try:
```

```
        while time.monotonic() < deadline:
```

```
            try:
```

```
                task = celery.AsyncResult(task_id)
```

```
                if task.state == 'SUCCESS':
```

```
                    res = task.result
```

```
                    if isinstance(res, dict) and res.get('status') == 'error':
```

```
                        data = {'status': 'FAILURE', 'error': res.get('message', 'Nieznany błąd')}
```

```
                    else:
```

```
                        data = {'status': 'SUCCESS',
```

```
                                'download_url': url_for('documents.pobierz_pdf', task_id=task_id)}
```

```
                        yield f"data: {json.dumps(data)}\n\n"
```

```
                    return
```

```
                elif task.state == 'FAILURE':
```

```
                    yield f"data: {json.dumps({'status': 'FAILURE', 'error': str(task.info)})}\n\n"
```

```
                return
```

```
            else:
```

```
                yield f"data: {json.dumps({'status': task.state})}\n\n"
```

```

        except GeneratorExit:

            return

        except Exception as exc:

            logger.error("SSE stream error for task %s: %s", task_id, exc)

            yield f"data: {json.dumps({'status': 'FAILURE', 'error': str(exc)})}\n\n"

            return

        _sleep(1)

    finally:

        # Wysyłamy zamknięcie – klient może zareagować komunikatem timeout

        yield f"data: {json.dumps({'status': 'TIMEOUT'})}\n\n"

return Response(generate(), mimetype='text/event-stream',

                headers={'Cache-Control': 'no-cache', 'X-Accel-Buffering': 'no'})

@documents_bp.route('/pobierz/<task_id>')

@wymaga_rolu(UserRole.ADMIN, UserRole.UOPZ)

def pobierz_pdf(task_id):

    try:

        from celery_app import celery

        task = celery.AsyncResult(task_id)

        if task.state != 'SUCCESS':

            abort(404)

        result = task.result

        if isinstance(result, dict) and result.get('status') == 'error':

            abort(500)

        pdf_path = Path(result['path'])

        if not pdf_path.exists():

            abort(404)

        return send_file(pdf_path, mimetype='application/pdf', as_attachment=True, download_name=result['filename'])

    except Exception:

        abort(500)

```

PLIK: app_admin\routes\dziennik.py

```
"""

app_admin/routes/dziennik.py

Monitor dzienników praktyk – podgląd wpisów, wykrywanie przerw, generowanie PDF.

Przemianowano z journal.py.

"""

import io

from datetime import date, timedelta

from flask import Blueprint, render_template, request, abort, send_file

from flask_login import login_required, current_user

from core.modele import UserRole

from core.repozytoria import RepozytoriumZapisow, RepozytoriumWpisow


journal_bp = Blueprint('journal', __name__)


_repo_zapisow = RepozytoriumZapisow()

_repo_wpisow = RepozytoriumWpisow()


def _wykryj_przerwy(wpisy, prog_dni=3):

    """Wykrywa przerwy dłuższe niż prog_dni roboczych między wpisami."""

    if not wpisy:

        return []

    daty = sorted(w.entry_date for w in wpisy)

    przerwy = []

    for i in range(1, len(daty)):

        delta = 0

        d = daty[i - 1]

        while d < daty[i]:

            d += timedelta(days=1)

            if d.weekday() < 5:

                delta += 1

        if delta > prog_dni:
```

```

        przerwy.append({'od': daty[i - 1], 'do': daty[i], 'dni_roboczych': delta})

    return przerwy


@journal_bp.route('/')

@login_required

def lista_dziennikow():

    szukaj      = request.args.get('szukaj', '').strip()

    strona      = request.args.get('strona', 1, type=int)

    uopz_id     = current_user.id if current_user.role == UserRole.UOPZ else None

    zapisy = _repo_zapisow.w_trakcie_strona(

        szukaj=szukaj, supervisor_id=uopz_id, strona=strona

    )

    stats = _repo_wpisow.statystyki_dla_zapisow([z.id for z in zapisy.items])

    dane      = []

    dzisiaj   = date.today()

    for z in zapisy.items:

        ostatni, liczba_wpisow = stats.get(z.id, (None, 0))

        alert = False

        if ostatni:

            delta = 0

            d = ostatni

            while d < dzisiaj:

                d += timedelta(days=1)

                if d.weekday() < 5:

                    delta += 1

            alert = delta > 3

        dane.append({'zapis': z, 'liczba_wpisow': liczba_wpisow, 'ostatni_wpis': ostatni, 'alert': alert})

    return render_template('journal/lista.html', dane=dane, zapisy=zapisy)

```



```

@journal_bp.route('/zapis/<uuid:id>')

@login_required

def dziennik_zapisu(id):

    zapis      = _repo_zapisow.znajdz_po_id(id) or abort(404)

    wpisy      = _repo_wpisow.dla_zapisu(id, malejaco=True)

    przerwy    = _wykryj_przerwy(wpisy)

    suma_godzin = sum(w.duration_hours for w in wpisy)

    return render_template('journal/dziennik.html', zapis=zapis, wpisy=wpisy,

                           przerwy=przerwy, suma_godzin=suma_godzin)

```

```

@journal_bp.route('/zapis/<uuid:id>/pdf')

@login_required

def pdf_dziennik(id):

    zapis = _repo_zapisow.znajdz_po_id(id) or abort(404)

    try:

        from tex_service.compiler import compile_pdf

        from core.uslugi.dokumenty import buduj_kontekst

        ctx      = buduj_kontekst(zapis, 'ZAL_6')

        pdf_bytes = compile_pdf('zal6_dziennik.tex.j2', ctx)

        nazwa     = f"dziennik_{zapis.student.last_name}_{zapis.student.first_name}.pdf"

        return send_file(io.BytesIO(pdf_bytes), mimetype='application/pdf',

                        as_attachment=True, download_name=nazwa)

    except EnvironmentError:

        abort(503)

    except Exception:

        abort(500)

```

PLIK: app_admin\routes\ocenie.py

```
"""

app_admin/routes/ocenie.py

Oceny efektów uczenia się – operuje na InternshipEnrollment (enrollment).

Przemianowano z evaluation.py.

"""

import uuid

from flask import Blueprint, render_template, redirect, url_for, flash, request, abort

from flask_login import login_required, current_user

from core.modele import (InternshipEnrollment, OutcomeAssessment, LearningOutcome,
                          UserRole, EnrollmentStatus, AssessmentResult)

from core.extensions import db

from core.autoryzacja import wymaga_rol

from core.repozytoria import RepozytoriumZapisow, RepozytoriumEfektow, RepozytoriumOcen

_repo_zapisow = RepozytoriumZapisow()

_repo_efektow = RepozytoriumEfektow()

_repo_ocen = RepozytoriumOcen()

evaluation_bp = Blueprint('evaluation', __name__)

def _parse_grade(val: str):

    """Konwertuje ocenę z formularza na float, obsługując przecinki i kropki."""

    if not val or not val.strip():

        return None

    try:

        return float(val.strip().replace(',', '.'))

    except (ValueError, AttributeError):

        return None

from core.uslugi import SerwisOceniania
```

```

def get_pilne_oceny(uopz_id=None):

    return SerwisOceniania.get_pilne_oceny(uopz_id)


@evaluation_bp.route('/')

@login_required

def lista_ocen():

    SerwisOceniania.auto_complete_internships()


    uopz_id = current_user.id if current_user.role == UserRole.UOPZ else None

    zapisy = _repo_zapisow.aktywne_i_zakonczzone(supervisor_id=uopz_id)


    from datetime import date, timedelta


    zapisy_z_deadlinami = []

    for zapis in zapisy:

        deadline = None

        dni_do_deadline = None

        przekroczony = False


        if zapis.end_date and zapis.status == EnrollmentStatus.COMPLETED:

            deadline = zapis.end_date + timedelta(days=7)

            dni_do_deadline = (deadline - date.today()).days

            przekroczony = dni_do_deadline < 0


    zapisy_z_deadlinami.append({

        'zapis': zapis,

        'deadline': deadline,

        'dni_do_deadline': dni_do_deadline,

        'przekroczony': przekroczony,

        'w_trakcie': zapis.status == EnrollmentStatus.IN_PROGRESS,

        'zakonczone': zapis.status == EnrollmentStatus.COMPLETED,

```

```
})
```

```
w_trakcie = [z for z in zapisy_z_deadlinami if z['w_trakcie']]  
zakonczone = [z for z in zapisy_z_deadlinami if z['zakonczone']]
```

```
return render_template('evaluation/lista_ocen.html',  
                        zapisy_z_deadlinami=zapisy_z_deadlinami,  
                        w_trakcie=w_trakcie,  
                        zakonczone=zakonczone)
```

```
@evaluation_bp.route('/zapis/<uuid:id>/karta_ocen', methods=['GET', 'POST'])
```

```
@wymaga_rola(UserRole.ADMIN, UserRole.UOPZ)
```

```
def ocen_praktyke(id):
```

```
    zapis = db.session.get(InternshipEnrollment, id) or abort(404)
```

```
    if request.method == 'POST':
```

```
        from core.uslugi.ocenianie import GradeFormData
```

```
    dane = GradeFormData(  
        report_grade
```

```
            = _parse_grade(request.form.get('ocena_sprawozdania', '')),
```

```
        supervisor_grade
```

```
            = _parse_grade(request.form.get('ocena_uopz', '')),
```

```
        workplace_grade
```

```
            = _parse_grade(request.form.get('ocena_zopz', '')),
```

```
        supervisor_grade_description = request.form.get('ocena_opisowa_uopz'),
```

```
        workplace_grade_description = request.form.get('ocena_opisowa_zopz'),
```

```
        exam_question_1
```

```
            = request.form.get('sprawdzian_pytanie_1'),
```

```
        exam_grade_1
```

```
            = _parse_grade(request.form.get('sprawdzian_ocena_1', '')),
```

```
        exam_question_2
```

```
            = request.form.get('sprawdzian_pytanie_2'),
```

```
        exam_grade_2
```

```
            = _parse_grade(request.form.get('sprawdzian_ocena_2', '')),
```

```
        exam_question_3
```

```
            = request.form.get('sprawdzian_pytanie_3'),
```

```
        exam_grade_3
```

```
            = _parse_grade(request.form.get('sprawdzian_ocena_3', '')),
```

```
        finalize
```

```
            = bool(request.form.get('zakonczone')),
```

```
)
```

```
wynik = SerwisOceniania.zapisz_oceny(zapis, dane)
```

```

    if not wynik.success:

        flash(wynik.error_message, 'danger')

        return redirect(url_for('evaluation.ocen_praktyke', id=zapis.id))

    db.session.commit()

    flash('Oceny zostały zapisane.', 'success')

    return redirect(url_for('evaluation.ocen_praktyke', id=zapis.id))

from flask_wtf import FlaskForm

csrf_form = FlaskForm()

return render_template('evaluation/karta_ocen.html', practically=zapis, zapis=zapis, csrf_form=csrf_form)

@evaluation_bp.route('/zapis/<uuid:id>/sprawozdanie')

@wymaga_rolu(UserRole.ADMIN, UserRole.UOPZ)

def podglad_sprawozdania(id):

    zapis = db.session.get(InternshipEnrollment, id) or abort(404)

    return render_template('evaluation/podglad_sprawozdania.html', zapis=zapis)

@evaluation_bp.route('/zapis/<uuid:id>', methods=['GET', 'POST'])

@wymaga_rolu(UserRole.ADMIN, UserRole.UOPZ)

def ocen_zapis(id):

    zapis = db.session.get(InternshipEnrollment, id) or abort(404)

    efekty = _repo_efektow.wszystkie()

    istniejace = {

        str(o.learning_outcome_id): o

        for o in _repo_ocen.dla_zapisu(id)

    }

    if request.method == 'POST':

        for efekt in efekty:

            wynik_str = request.form.get(f'wynik_{efekt.id}')

            uwagi = request.form.get(f'uwagi_{efekt.id}', '').strip()

```

```

        if not wynik_str:

            continue

        try:

            wynik = AssessmentResult[wynik_str]

        except KeyError:

            continue

        ocena = istniejace.get(str(efekt.id))

        if ocena:

            ocena.result = wynik

            ocena.notes = uwagi or None

        else:

            db.session.add(OutcomeAssessment(

                id = uuid.uuid4(),

                enrollment_id = zapis.id,

                learning_outcome_id = efekt.id,

                result = wynik,

                notes = uwagi or None,

            ))

        db.session.commit()

        flash('Oceny zostały zapisane.', 'success')

        return redirect(url_for('evaluation.ocen_zapis', id=id))

    return render_template('evaluation/formularz_ocen.html',

                           zapis=zapis, efekty=efekty, istniejace=istniejace)

@evaluation_bp.route('/zapis/<uuid:id>/zakonczone', methods=['POST'])

@wymaga_rol(UserRole.ADMIN, UserRole.UOPZ)

def zakoncz_zapis(id):

    zapis = db.session.get(InternshipEnrollment, id) or abort(404)

    from core.uslugi.workflow import ZapisFSM

    ZapisFSM(zapis).zakonczone()

    db.session.commit()

    flash(f'Praktyka studenta {zapis.student.first_name} {zapis.student.last_name} została zakończona.', 'success')

    return redirect(url_for('evaluation.lista_ocen'))

```

```

@evaluation_bp.route('/zapis/<uuid:id>/protokol')

@wymaga_rola(UserRole.ADMIN, UserRole.UOPZ)

def generuj_protokol(id):

    """Generuje Protokół egzaminu (zał.8) przez tex-service."""

    import httpx, unicodedata

    from flask import make_response, current_app

    from datetime import date

    zapis = db.session.get(InternshipEnrollment, id) or abort(404)

    ok = zapis.final_grades

    sp = zapis.examination

    if not (ok and ok.supervisor_grade):

        flash('Protokół dostępny dopiero po wystawieniu oceny UOPZ.', 'warning')

        return redirect(url_for('evaluation.ocen_praktyke', id=id))

    s = zapis.student

    tex_url = current_app.config.get('TEX_SERVICE_URL', 'http://tex-service:5002')

    dm = zapis.workplace_details

    firma_nazwa = (zapis.firma.name if zapis.firma else (dm.company_name if dm else None)) or ''

    def _f(v):

        return float(v) if v is not None else None

    ctx = {

        'zapis': {

            'firma_nazwa':        firma_nazwa,

            'termin_od':          zapis.start_date.strftime('%d.%m.%Y') if zapis.start_date else '',

            'termin_do':          zapis.end_date.strftime('%d.%m.%Y') if zapis.end_date else '',

            'ocena_sprawozdania':  _f(ok.report_grade if ok else None),

            'ocena_uopz':          _f(ok.supervisor_grade if ok else None),

            'ocena_zopz':          _f(ok.workplace_grade if ok else None),

            'sprawdzian_pytanie_1': sp.question_1 if sp else None,

            'sprawdzian_ocena_1':  _f(sp.grade_1 if sp else None),

```

```

'sprawdzian_pytanie_2': sp.question_2 if sp else None,

'sprawdzian_ocena_2': _f(sp.grade_2 if sp else None),

'sprawdzian_pytanie_3': sp.question_3 if sp else None,

'sprawdzian_ocena_3': _f(sp.grade_3 if sp else None),

'uopz': {'first_name': zapis.uopz.first_name, 'last_name': zapis.uopz.last_name} if zapis.uopz else None,
},

'student': {

    'imie': s.first_name, 'nazwisko': s.last_name,

    'first_name': s.first_name, 'last_name': s.last_name,

    'nr_albumu': s.album_number or '', 'album_number': s.album_number or '',

    'kierunek': getattr(s, 'field_of_study', 'Informatyka') or 'Informatyka',
},

'specjalnosc': getattr(s, 'specialization', '') or getattr(zapis, 'specialization', '') or '',

'praktyka': {

    'rok_uczelnianny': zapis.internship.academic_year if zapis.internship else '',

    'semestr': zapis.internship.semester if zapis.internship else '',

    'wymiar_godzin': zapis.internship.required_hours if zapis.internship else 960,
},

'data_egzaminu': date.today().strftime('%d.%m.%Y'),
}

try:

    r = httpx.post(f'{tex_url}/generuj',

        json={'template': 'zal8_protokol.tex.j2', 'context': ctx, 'filename': 'zal8_protokol.pdf'},

        timeout=60)

    if r.status_code == 200:

        safe = unicodedata.normalize('NFKD', s.last_name).encode('ascii', 'ignore').decode('ascii') or 'student'

        resp = make_response(r.content)

        resp.headers['Content-Type'] = 'application/pdf'

        resp.headers['Content-Disposition'] = f'attachment; filename="zal8_protokol_{safe}.pdf"'

        return resp

    flash(f'Błąd generowania protokołu: {r.text[:200]}', 'danger')

except Exception as e:

    flash(f'Błąd połączenia z tex-service: {str(e)}', 'danger')

return redirect(url_for('evaluation.ocen_praktyke', id=id))

```


PLIK: app_admin\routes\pulpit.py

```
from flask import Blueprint, render_template

from flask_login import login_required, current_user

from core.modele import UserRole, EnrollmentStatus

from core.repozytoria import RepozytoriumZapisow, RepozytoriumPraktyk, RepozytoriumUzytkownikow

from flask_wtf import FlaskForm


dashboard_bp = Blueprint('dashboard', __name__)


_repo_zapisow      = RepozytoriumZapisow()
_repo_praktyk      = RepozytoriumPraktyk()
_repo_uzytk        = RepozytoriumUzytkownikow()


@dashboard_bp.route('/')

@login_required

def index():

    uopz_id = current_user.id if current_user.role == UserRole.UOPZ else None

    stats_zapisow = _repo_zapisow.statystyki_pulpit(supervisor_id=uopz_id)

    statystyki = {

        **stats_zapisow,

        'zakonczone':      _repo_praktyk.liczba_nieaktywnych(),

        'liczba_studentow': _repo_uzytk.liczba_aktywnych_studentow(),

    }

    ostatnie_zapisy = _repo_zapisow.ostatnie(supervisor_id=uopz_id, limit=8)

    pilne_oceny = []

    if current_user.role == UserRole.UOPZ:

        from datetime import date, timedelta

        for zapis in _repo_zapisow.zakonczone_dla_uopz(current_user.id):

            if zapis.termin_do:

                deadline      = zapis.termin_do + timedelta(days=7)
```

```
dni_do_deadline = (deadline - date.today()).days

if dni_do_deadline <= 2:

    pilne_oceny.append({

        'zapis':        zapis,

        'deadline':     deadline,

        'dni_do_deadline': dni_do_deadline,

        'przekroczony':  dni_do_deadline < 0,

    })
```

```
csrf_form = FlaskForm()
```

```
return render_template('dashboard/index.html',

    statystyki=statystyki,

    ostatnie_zapisy=ostatnie_zapisy,

    pilne_oceny=pilne_oceny,

    csrf_form=csrf_form)
```

PLIK: app_admin\routes\zarzadzanie\dokumenty_studentow.py

```
"""

app_admin/routes/zarzadzanie/dokumenty_studentow.py

Lista dokumentów studentów - widok dla ADMIN i UOPZ.

"""


import logging

import unicodedata


import httpx

from flask import render_template, request, abort, make_response, current_app

from flask_login import login_required, current_user


from core.modele import EnrollmentStatus, UserRole

from core.uslugi.dokumenty import DOC_CONFIG, STATIC_TEMPLATES, rozwiaz_dokumenty, buduj_kontekst

from core.repozytoria import RepozytoriumUzytkownikow, RepozytoriumZapisow


from . import zarzadzanie_bp


logger = logging.getLogger(__name__)


_repo_uzytok = RepozytoriumUzytkownikow()

_repo_zapisow = RepozytoriumZapisow()


# — Helpers —————

def _get_tex_url() -> str:

    return current_app.config.get('TEX_SERVICE_URL', 'http://tex-service:5002')


def _sciezka(zapis) -> str:

    return zapis.track_type.value if zapis.track_type else 'STANDARD'
```

— Lista studentów

```
@zarzadzanie_bp.route('/dokumenty')
```

```
@login_required
```

```
def dokumenty_studentow():
```

```
    strona      = request.args.get('strona', 1, type=int)
```

```
    szukaj      = request.args.get('szukaj', '').strip()
```

```
    uopz_id     = current_user.id if current_user.role == UserRole.UOPZ else None
```

```
    studenci_page = _repo_uzytk.studenci_strona(
```

```
        szukaj=szukaj, supervisor_id=uopz_id, strona=strona
```

```
    )
```

```
    aktywne_statusy = [
```

```
        EnrollmentStatus.AWAITING_APPROVAL, EnrollmentStatus.COMMISSION_REVIEW,
```

```
        EnrollmentStatus.DEAN_APPROVAL, EnrollmentStatus.IN_PROGRESS,
```

```
    ]
```

```
    student_ids = [s.id for s in studenci_page.items]
```

```
    aktywne_counts = _repo_zapisow.liczba_aktywnych_dla_studentow(
```

```
        student_ids, aktywne_statusy
```

```
    )
```

```
    uopz_cache: dict = {}
```

```
    studenci_info = []
```

```
    for s in studenci_page.items:
```

```
        uopz = None
```

```
        if s.uopz_id:
```

```
            if s.uopz_id not in uopz_cache:
```

```
                uopz_cache[s.uopz_id] = _repo_uzytk.znajdz_po_id(s.uopz_id)
```

```
            uopz = uopz_cache[s.uopz_id]
```

```
    studenci_info.append({
```

```
        'student': s,
```

```
        'uopz':     uopz,
```

```
        'aktywne': aktywne_counts.get(s.id, 0),
    })
```

```
return render_template(
    'zarzadzanie/dokumenty_studentow.html',
    studenci=studenci_page,
    studenci_info=studenci_info,
    szukaj=szukaj,
)
```

— Dokumenty jednego studenta —————

```
@zarzadzanie_bp.route('/dokumenty/student/<uuid:student_id>')
```

```
@login_required
```

```
def dokumenty_studenta(student_id):
```

```
    from core.modele import Student
```

```
    from core.extensions import db
```

```
    student = db.session.get(Student, student_id) or abort(404)
```

```
    if current_user.role == UserRole.UOPZ and student.supervisor_id != current_user.id:
```

```
        abort(403)
```

```
    aktywne_statusy = [
```

```
        EnrollmentStatus.AWAITING_APPROVAL,
```

```
        EnrollmentStatus.COMMISSION_REVIEW,
```

```
        EnrollmentStatus.DEAN_APPROVAL,
```

```
        EnrollmentStatus.IN_PROGRESS,
```

```
        EnrollmentStatus.COMPLETED,
```

```
    ]
```

```
    zapisy = _repo_zapisow.aktywne_dla_studenta(student_id, aktywne_statusy)
```

```
    uopz = _repo_uzytz.znajdz_po_id(student.uopz_id) if student.uopz_id else None
```

```
    dokumenty_list = [
```

```

        {'zapis': z, 'docs': rozwiadz_dokumenty(z), 'sciezka': _sciezka(z)}

    for z in zapisy

]

```

```

return render_template(

    'zarzadzanie/dokumenty_studenta.html',

    student=student,

    uopz=uopz,

    dokumenty_list=dokumenty_list,

)

```

— Pobieranie dokumentów —————

```

@zarzadzanie_bp.route('/dokumenty/pobierz/<uuid:zapis_id>/<typ>')

@login_required

def dokumenty_pobierz(zapis_id, typ):

    if typ not in DOC_CONFIG:

        abort(404)

    zapis = _repo_zapisow.znajdz_po_id(zapis_id) or abort(404)

    if current_user.role == UserRole.UOPZ and zapis.supervisor_id != current_user.id:

        abort(403)

    template_name, filename = DOC_CONFIG[typ]

    ctx = buduj_kontekst(zapis, typ)

    try:

        response = httpx.post(

            f"{{_get_tex_url()}}/generuj",

            json={'template': template_name, 'context': ctx, 'filename': filename},

            timeout=60,

        )

        if response.status_code != 200:

            abort(500)

        safe_name = (

```

```

        unicodedata.normalize('NFKD', zapis.student.last_name or '')

        .encode('ascii', 'ignore').decode('ascii') or 'student'

    )

    pdf_name = template_name.replace('.tex.j2', '')

    resp = make_response(response.content)

    resp.headers['Content-Type'] = 'application/pdf'

    resp.headers['Content-Disposition'] = f'attachment; filename="{pdf_name}_{safe_name}.pdf"'

    return resp

except Exception:

    logger.exception("Błąd generowania %s dla %s", typ, zapis_id)

    abort(500)


@zarzadzanie_bp.route('/dokumenty/staly/<klucz>')

@login_required

def dokumenty_pobierz_staly(klucz):

    if klucz not in STATIC_TEMPLATES:

        abort(404)

    template_name, filename = STATIC_TEMPLATES[klucz]

    try:

        response = httpx.post(

            f"{_get_tex_url()}/generuj",

            json={'template': template_name, 'context': {}, 'filename': filename},

            timeout=30,

        )

        if response.status_code != 200:

            abort(500)

        resp = make_response(response.content)

        resp.headers['Content-Type'] = 'application/pdf'

        resp.headers['Content-Disposition'] = f'attachment; filename="{filename}"'

        return resp

    except Exception:

        logger.exception("Błąd pobierania stałego dokumentu %s", klucz)

        abort(500)

```

PLIK: app_admin\routes\zarzadzanie\dziekan.py

```
import uuid

import csv

import io

import datetime

from datetime import timezone as _tz

from flask import (Blueprint, render_template, redirect, url_for,

                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError

from werkzeug.security import generate_password_hash


from core.modele import (User, Student, Internship, InternshipEnrollment, InternshipSchedule, LearningOutcome,

                          UserRole, InternshipStatus, EnrollmentStatus, UploadedDocument, Company)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytkownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from core.repozytoria import RepozytoriumZapisow


_repo_zapisow = RepozytoriumZapisow()


from . import zarzadzanie_bp

from .formularze import *


# — Dziekan —————

@zarzadzanie_bp.route('/dziekan')

@wymaga_rol(UserRole.ADMIN)

def dziekan_lista():

    strona    = request.args.get('page', 1, type=int)
```



```
wnioski    = _repo_zapisow.wnioski_dziekana_strona(strona=strona)

csrf_form = FlaskForm()

return render_template('zarzadzanie/dziedkan/lista.html', wnioski=wnioski, csrf_form=csrf_form)
```

```
@zarzadzanie_bp.route('/dziedkan/<uuid:id>/decyzja', methods=['GET', 'POST'])
```

```
@wymaga_rola(UserRole.ADMIN)
```

```
def dziedkan_decyzja(id):
```

```
    from flask_wtf import FlaskForm
```

```
    from wtforms import TextAreaField, SelectField, SubmitField
```

```
    from wtforms.validators import DataRequired, Optional
```

```
    zapis = db.session.get(InternshipEnrollment, id) or abort(404)
```

```
    if zapis.status != EnrollmentStatus.DEAN_APPROVAL:
```

```
        flash('Wniosek nie wymaga decyzji dziekana.', 'warning')
```

```
        return redirect(url_for('zarzadzanie.dziedkan_lista'))
```

```
class FormularzDziekana(FlaskForm):
```

```
    decyzja    = SelectField('Decyzja dziekana', choices=[
```

```
        ('APPROVED', 'Wyrażam zgodę na zaliczenie praktyki'),
```

```
        ('REJECTED', 'Nie wyrażam zgody na zaliczenie'),
```

```
    ], validators=[DataRequired()])
```

```
    komentarz = TextAreaField('Komentarz dziekana', validators=[Optional()])
```

```
    submit     = SubmitField('Zapisz decyzję')
```

```
    form = FormularzDziekana()
```

```
    if form.validate_on_submit():
```

```
        from core.modele import ProcessEvent, EventType
```

```
        from datetime import datetime, timezone as _tz
```

```
        from core.uslugi.workflow import ZapisFSM, IllegalTransitionError
```

```
        try:
```

```
with ZapisFSM.lock(id) as fsm:

    if fsm.zapis.status != EnrollmentStatus.DEAN_APPROVAL:

        flash('Wniosek zmienił status podczas przetwarzania – spróbuj ponownie.', 'warning')

        return redirect(url_for('zarzadzanie.dziekana_lista'))

    if form.decyzja.data == 'APPROVED':

        fsm.zatwierdz_przez_dziekana()

        flash('Wniosek zatwierdzony przez dziekana. Student może kontynuować praktykę.', 'success')

    else:

        fsm.odrzuc()

        db.session.add(ProcessEvent(

            enrollment_id=fsm.zapis.id, event_type=EventType.SUPERVISOR_COMMENT,

            comment=f"Dziekan nie wyraził zgody: {form.komentarz.data}",

            executed_by_id=current_user.id, executed_at=datetime.now(_tz.utc),

        ))

        flash('Wniosek odrzucony przez dziekana.', 'warning')

        db.session.add(ProcessEvent(

            enrollment_id=fsm.zapis.id, event_type=EventType.DEAN_DECISION,

            decision=form.decyzja.data, comment=form.komentarz.data,

            executed_by_id=current_user.id, executed_at=datetime.now(_tz.utc),

        ))

        db.session.commit()

except IllegalTransitionError as e:

    flash(str(e), 'danger')

    return redirect(url_for('zarzadzanie.dziekana_lista'))

return render_template('zarzadzanie/dziekana/decyzja.html', form=form, zapis=zapis)
```

PLIK: app_admin\routes\zarzadzanie\firmy.py

```
import uuid

from flask import (Blueprint, render_template, redirect, url_for, flash, request, abort)

from flask_login import login_required

from flask_wtf import FlaskForm


from core.modele import Company, EnrollmentStatus, UserRole

from core.extensions import db

from core.autoryzacja import wymaga_rol

from core.repozytoria import RepozytoriumFirm


from . import zarzadzanie_bp

from .formularze import FormularzFirmy


_repo_firm = RepozytoriumFirm()


_AKTYWNE_STATUSY = [

    EnrollmentStatus.AWAITING_APPROVAL, EnrollmentStatus.IN_PROGRESS,

    EnrollmentStatus.COMMISSION_REVIEW, EnrollmentStatus.DEAN_APPROVAL,

]


@zarzadzanie_bp.route('/firmy')

@wymaga_rol(UserRole.ADMIN)

def lista_firm():

    strona = request.args.get('page', 1, type=int)

    szukaj = request.args.get('szukaj', '').strip()

    status = request.args.get('status', 'wszystkie')


    firmy = _repo_firm.lista_strona(szukaj=szukaj, status=status, strona=strona)

    csrf_form = FlaskForm()

    return render_template('zarzadzanie/firmy/lista.html', firmy=firmy, csrf_form=csrf_form)
```

```

@zarzadzanie_bp.route('/firmy/dodaj', methods=['GET', 'POST'])

@wymaga_rola(UserRole.ADMIN)

def dodaj_firme():

    form = FormularzFirmy()

    if form.validate_on_submit():

        if _repo_firm.znajdz_po_nazwie_aktywna(form.nazwa.data.strip()):

            flash('Firma o tej nazwie już istnieje w systemie.', 'error')

            return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='dodaj')

        if form.nip_krs.data and form.nip_krs.data.strip():

            istniejaca_nip = _repo_firm.znajdz_po_nip_aktywna(form.nip_krs.data.strip())

            if istniejaca_nip:

                flash(f'Firma z numerem NIP/KRS "{form.nip_krs.data.strip()}" już istnieje ({istniejaca_nip.name}).',
                    'error')

                return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='dodaj')

        firma = Company(

            id      = uuid.uuid4(),

            name    = form.nazwa.data.strip(),

            address = form.adres.data.strip() if form.adres.data else None,

            city    = form.miasto.data.strip() if form.miasto.data else None,

            tax_id  = form.nip_krs.data.strip() if form.nip_krs.data else None,

        )

        db.session.add(firma)

        db.session.commit()

        flash('Firma została dodana do systemu.', 'success')

        return redirect(url_for('zarzadzanie.lista_firm'))

    return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='dodaj')


@zarzadzanie_bp.route('/firmy/<uuid:id>/edytuj', methods=['GET', 'POST'])

@wymaga_rola(UserRole.ADMIN)

def edytuj_firme(id):

    firma = _repo_firm.znajdz_po_id(id) or abort(404)

```

```
form = FormularzFirmy(obj=firma)
```

```
if form.validate_on_submit():
```

```
    if _repo_firm.znajdz_po_nazwie_aktywna(form.nazwa.data.strip(), pominij_id=firma.id):
```

```
        flash('Firma o tej nazwie już istnieje w systemie.', 'error')
```

```
        return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='edytuj', firma=firma)
```

```
if form.nip_krs.data and form.nip_krs.data.strip():
```

```
    istniejaca_nip = _repo_firm.znajdz_po_nip_aktywna(form.nip_krs.data.strip(), pominij_id=firma.id)
```

```
    if istniejaca_nip:
```

```
        flash(f'Firma z NIP/KRS "{form.nip_krs.data.strip()}" już istnieje ({istniejaca_nip.name}).', 'error')
```

```
        return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='edytuj', firma=firma)
```

```
firma.name = form.nazwa.data.strip()
```

```
firma.address = form.adres.data.strip() if form.adres.data else None
```

```
firma.city = form.miasto.data.strip() if form.miasto.data else None
```

```
firma.tax_id = form.nip_krs.data.strip() if form.nip_krs.data else None
```

```
db.session.commit()
```

```
flash('Dane firmy zostały zaktualizowane.', 'success')
```

```
return redirect(url_for('zarzadzanie.lista_firm'))
```

```
return render_template('zarzadzanie/firmy/formularz.html', form=form, tryb='edytuj', firma=firma)
```

```
@zarzadzanie_bp.route('/firmy/<uuid:id>/usun', methods=['POST'])
```

```
@wymaga_roli(UserRole.ADMIN)
```

```
def usun_firme(id):
```

```
    firma = _repo_firm.znajdz_po_id(id) or abort(404)
```

```
    wszystkie_praktyki = _repo_firm.liczba_praktyk(firma.id)
```

```
    if wszystkie_praktyki > 0:
```

```
        flash(f'Nie można usunąć firmy - ma {wszystkie_praktyki} praktyk w historii.', 'error')
```

```
        return redirect(url_for('zarzadzanie.lista_firm'))
```

```
    nazwa_firmy = firma.name
```

```
    _repo_firm.usun(firma)
```

```
db.session.commit()

flash(f'Firma "{nazwa_firmy}" została trwale usunięta z systemu.', 'success')

return redirect(url_for('zarzadzanie.lista_firm'))
```

```
@zarzadzanie_bp.route('/firma/<uuid:id>/przelacz-aktywnosc', methods=['POST'])
```

```
@wymaga_rola(UserRole.ADMIN)
```

```
def przelacz_aktywnosc_firmy(id):
```

```
    firma = _repo_firm.znajdz_po_id(id) or abort(404)
```

```
    if firma.is_active:
```

```
        aktywne_praktyki = _repo_firm.liczba_aktywnych_praktyk(firma.id, _AKTYWNE_STATUSY)
```

```
        if aktywne_praktyki > 0:
```

```
            flash(f'Nie można dezaktywować firmy - ma {aktywne_praktyki} aktywnych praktyk.', 'error')
```

```
            return redirect(url_for('zarzadzanie.lista_firm'))
```

```
        firma.is_active = False
```

```
        flash('Firma została dezaktywowana.', 'success')
```

```
    else:
```

```
        firma.is_active = True
```

```
        flash('Firma została aktywowana.', 'success')
```

```
db.session.commit()
```

```
return redirect(url_for('zarzadzanie.lista_firm'))
```

PLIK: app_admin\routes\zarzadzanie\formularze.py

```
import uuid

import csv

import io

import datetime

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError


from core.modele import User, Student

from core.repozytoria import RepozytoriumUzytkownikow


_repo_uzyt = RepozytoriumUzytkownikow()


# --- Formularze -----


class FormularzStudenta(FlaskForm):

    imie = StringField('Imię', validators=[DataRequired(), Length(max=100)])

    nazwisko = StringField('Nazwisko', validators=[DataRequired(), Length(max=100)])

    email = StringField('E-mail', validators=[DataRequired(), Email(), Length(max=255)])

    numer_albumu = StringField('Nr albumu', validators=[DataRequired(), Length(max=20)])

    plec = SelectField('Płeć', choices=[('', '--- Wybierz ---'), ('M', 'Mężczyzna'), ('K', 'Kobieta')],
validators=[Optional()])

    kierunek = StringField('Kierunek studiów', validators=[Optional(), Length(max=100)])

    specjalnosc = StringField('Specjalność', validators=[Optional(), Length(max=100)])

    tryb_studiow = SelectField('Tryb studiów', choices=[('', '--- Wybierz ---'), ('stacjonarne', 'Stacjonarne'),
('niestacjonarne', 'Niestacjonarne')], validators=[Optional()])

    uopz_id = SelectField('Opiekun uczelniany (UOPZ)', choices=[], validators=[DataRequired(message='Wybierz
opiekuna UOPZ.')])

    def validate_email(self, pole):

        u = _repo_uzyt.znajdz_po_emailu(pole.data.lower().strip())

        if u:
```

```

        raise ValidationError('Konto z tym e-mailem już istnieje.')

def validate_numer_albumu(self, pole):

    s = _repo_uzytz.znajdz_studenta_po_albumie(pole.data.strip())

    if s:

        raise ValidationError('Student z tym nr albumu już istnieje.')


class FormularzEdycjiStudenta(FormularzStudenta):

    def __init__(self, uzytkownik_id=None, *args, **kwargs):

        super().__init__(*args, **kwargs)

        self._uid = uzytkownik_id


    def validate_email(self, pole):

        u = _repo_uzytz.znajdz_po_emailu(pole.data.lower().strip())

        if u and str(u.id) != str(self._uid):

            raise ValidationError('Konto z tym e-mailem już istnieje.')


    def validate_numer_albumu(self, pole):

        s = _repo_uzytz.znajdz_studenta_po_albumie(pole.data.strip())

        if s and str(s.id) != str(self._uid):

            raise ValidationError('Student z tym nr albumu już istnieje.')


class FormularzPracownika(FlaskForm):

    imie      = StringField('Imię',      validators=[DataRequired(), Length(max=100)])

    nazwisko  = StringField('Nazwisko',  validators=[DataRequired(), Length(max=100)])

    email     = StringField('E-mail',    validators=[DataRequired(), Email(), Length(max=255)])

    rola      = SelectField('Rola', choices=[

        ('UOPZ', 'Opiekun uczelni (UOPZ)'),

        ('ADMIN', 'Administrator'),

    ], validators=[DataRequired()])


    def validate_email(self, pole):

```



```
u = _repo_uzytik.znajdz_po_emailu(pole.data.lower().strip())

if u:

    raise ValidationError('Konto z tym e-mailem już istnieje.')
```

```
class FormularzImportuCSV(FlaskForm):
```

```
    plik = FileField('Plik CSV', validators=[

        DataRequired(),

        FileAllowed(['csv'], 'Tylko pliki CSV.')

    ])
```

```
    uopz_id = SelectField('Opiekun uczelniany (UOPZ)', choices=[], validators=[DataRequired(message='Wybierz opiekuna UOPZ.')])
```

```
class FormularzFirmy(FlaskForm):
```

```
    nazwa = StringField('Nazwa firmy', validators=[DataRequired(), Length(max=255)])

    adres = StringField('Adres', validators=[Optional(), Length(max=255)])

    miasto = StringField('Miasto', validators=[Optional(), Length(max=100)])

    nip_krs = StringField('NIP/KRS', validators=[Optional(), Length(max=50)])
```

```
class FormularzPraktyki(FlaskForm):
```

```
    rok_uczelniany = StringField('Rok uczelniany', validators=[DataRequired(), Length(max=9)])

    semestr = SelectField('Semestr', choices=[

        ('zimowy', 'Zimowy'),

        ('letni', 'Letni'),

    ], validators=[DataRequired()])

    wymiar_godzin = StringField('Wymiar godzin (h)', validators=[DataRequired()])
```

PLIK: app_admin\routes\zarzadzanie\komisja.py

```
import uuid

import csv

import io

import datetime

from datetime import timezone as _tz

from flask import (Blueprint, render_template, redirect, url_for,

                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError

from werkzeug.security import generate_password_hash


from core.modele import (User, Student, Internship, InternshipEnrollment, InternshipSchedule, LearningOutcome,

                          UserRole, InternshipStatus, EnrollmentStatus, UploadedDocument, Company)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytkownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from core.repozytoria import RepozytoriumZapisow, RepozytoriumDokumentowStudenta


_repo_zapisow = RepozytoriumZapisow()

_repo_docs    = RepozytoriumDokumentowStudenta()


from . import zarzadzanie_bp

from .formularze import *


# — Komisja weryfikująca —————

@zarzadzanie_bp.route('/komisja')

@wymaga_rol(UserRole.ADMIN, UserRole.UOPZ)

def komisja_lista():
```

```

strona    = request.args.get('page', 1, type=int)

wnioski   = _repo_zapisow.wnioski_komisja_strona(strona=strona)

csrf_form = FlaskForm()

return render_template('zarzadzanie/komisja/lista.html', wnioski=wnioski, csrf_form=csrf_form)


@zarzadzanie_bp.route('/komisja/<uuid:id>/weryfikuj', methods=['GET', 'POST'])
@wymaga_rola(UserRole.ADMIN, UserRole.UOPZ)
def komisja_weryfikuj(id):

    from flask_wtf import FlaskForm

    from wtforms import TextAreaField, SelectField, SubmitField

    from wtforms.validators import DataRequired, Optional

    zapis = db.session.get(InternshipEnrollment, id) or abort(404)

    if zapis.status not in (EnrollmentStatus.COMMISSION_REVIEW, EnrollmentStatus.AWAITING_APPROVAL,
EnrollmentStatus.REVISION_REQUIRED):

        flash('Wniosek nie wymaga weryfikacji komisji.', 'warning')

        return redirect(url_for('zarzadzanie.komisja_lista'))


class FormularzKomisji(FlaskForm):

    decyzja = SelectField('Decyzja komisji', choices=[

        ('APPROVED',          'Zatwierdzam - kieruję do dziekana'),

        ('PARTIALLY_APPROVED', 'Zatwierdzam częściowo - wymaga uzupełnień'),

        ('REJECTED',          'Odrzucam wniosek'),

    ], validators=[DataRequired()])

    komentarz = TextAreaField('Komentarz komisji', validators=[Optional()])

    submit    = SubmitField('Zapisz decyzję')


form = FormularzKomisji()


if form.validate_on_submit():

    from core.modele import ProcessEvent, EventType

    from datetime import datetime, timezone as _tz

    from core.uslugi.workflow import ZapisFSM, IllegalTransitionError

```

```

try:

    with ZapisFSM.lock(id) as fsm:

        if fsm.zapis.status not in (EnrollmentStatus.COMMISSION_REVIEW, EnrollmentStatus.AWAITING_APPROVAL,
EnrollmentStatus.REVISION_REQUIRED):

            flash('Wniosek zmienił status podczas przetwarzania – spróbuj ponownie.', 'warning')

            return redirect(url_for('zarzadzanie.komisja_lista'))

if form.decyzja.data == 'APPROVED':

    decyzja_db = 'APPROVED'

    fsm.zatwierdz_przez_komisje()

    flash('Wniosek zatwierdzony i przekazany do dziekana.', 'success')

elif form.decyzja.data == 'PARTIALLY_APPROVED':

    decyzja_db = 'PARTIALLY_APPROVED'

    fsm.zadaj_poprawki()

    db.session.add(ProcessEvent(

        enrollment_id=fsm.zapis.id, event_type=EventType.SUPERVISOR_COMMENT,

        comment=f"Komisja: {form.komentarz.data}",

        executed_by_id=current_user.id, executed_at=datetime.now(_tz.utc),

    ))

    flash('Wniosek wymaga uzupełnień - student zostanie powiadomiony.', 'info')

else:

    decyzja_db = 'REJECTED'

    fsm.odrzuc()

    db.session.add(ProcessEvent(

        enrollment_id=fsm.zapis.id, event_type=EventType.SUPERVISOR_COMMENT,

        comment=f"Wniosek odrzucony przez komisję: {form.komentarz.data}",

        executed_by_id=current_user.id, executed_at=datetime.now(_tz.utc),

    ))

    flash('Wniosek został odrzucony.', 'warning')

db.session.add(ProcessEvent(

    enrollment_id=fsm.zapis.id, event_type=EventType.COMMITTEE_DECISION,

    decision=decyzja_db, comment=form.komentarz.data,

    executed_by_id=current_user.id, executed_at=datetime.now(_tz.utc),

```

```
    ))

    db.session.commit()

except IllegalTransitionError as e:

    flash(str(e), 'danger')

return redirect(url_for('zarzadzanie.komisja_lista'))


dokumenty = _repo_docs.dla_zapisu_studenta(id, zapis.student_id)

return render_template('zarzadzanie/komisja/weryfikuj.html',

                       form=form, zapis=zapis, dokumenty=dokumenty)
```

PLIK: app_admin\routes\zarzadzanie\praktyki.py

```
import uuid

import csv

import io

import datetime

from datetime import timezone as _tz

from flask import (Blueprint, render_template, redirect, url_for,

                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError

from werkzeug.security import generate_password_hash


from core.modele import (User, Student, Internship, InternshipEnrollment, InternshipSchedule, LearningOutcome,

                          UserRole, InternshipStatus, EnrollmentStatus, InternshipPath, UploadedDocument, Company)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytkownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from core.repozytoria import (RepozytoriumPraktyk, RepozytoriumZapisow,

                              RepozytoriumUzytkownikow, RepozytoriumEfektow,

                              RepozytoriumDokumentowStudenta)


_repo_praktyk = RepozytoriumPraktyk()

_repo_zapisow = RepozytoriumZapisow()

_repo_uzytk = RepozytoriumUzytkownikow()

_repo_efektow = RepozytoriumEfektow()

_repo_docs = RepozytoriumDokumentowStudenta()


from . import zarzadzanie_bp

from .formularze import *
```

```
@zarzadzanie_bp.route('/praktyki')

@login_required

def lista_praktyk():

    strona = request.args.get('strona', 1, type=int)

    praktyki = _repo_praktyk.lista_strona(strona=strona)

    csrf_form = FlaskForm()

    return render_template('zarzadzanie/praktyki.html', praktyki=praktyki, csrf_form=csrf_form)


@zarzadzanie_bp.route('/praktyki/nowa', methods=['GET', 'POST'])

@wymaga_rol(UserRole.ADMIN)

def nowa_praktyka():

    form = FormularzPraktyki()

    if form.validate_on_submit():

        rok = (form.rok_uczelnianny.data or '').strip()

        try:

            wymiar = int(form.wymiar_godzin.data)

        except Exception:

            flash('Wymiar godzin musi być liczbą całkowitą.', 'danger')

            return render_template('zarzadzanie/formularz_praktyki.html', form=form)

        p = Internship(

            id = uuid.uuid4(),

            academic_year = rok,

            semester = form.semestr.data,

            required_hours = wymiar,

            status = InternshipStatus.INACTIVE, # noqa: forms use Polish field names deliberately

        )

        db.session.add(p)

        db.session.commit()

        flash('Internship została utworzona.', 'success')

        return redirect(url_for('zarzadzanie.lista_praktyk'))

    return render_template('zarzadzanie/formularz_praktyki.html', form=form)
```

```

@zarzadzanie_bp.route('/praktyki/<uuid:id>/aktywnosc', methods=['POST'])

@wymaga_rola(UserRole.ADMIN)

def przelacz_aktywnosc_praktyki(id):

    p = db.session.get(Internship, id) or abort(404)

    p.status = InternshipStatus.INACTIVE if p.status == InternshipStatus.ACTIVE else InternshipStatus.ACTIVE

    db.session.commit()

    stan = 'aktywowana' if p.status == InternshipStatus.ACTIVE else 'dezaktywowana'

    flash(f'Praktyka {p.academic_year} ({p.semester}) została {stan}.', 'success')

    return redirect(url_for('zarzadzanie.lista_praktyk'))

```

— Zgłoszenia studentów —————

```

class FormularzPrzypiszUOPZ(FlaskForm):

    uopz_id = SelectField('Opiekun uczelniany (UOPZ)', choices=[], validators=[Optional()])

@zarzadzanie_bp.route('/zgloszenia')

@wymaga_rola(UserRole.ADMIN)

def lista_zgloszen():

    strona          = request.args.get('page', 1, type=int)

    status_filter   = request.args.get('status', '').strip()

    try:

        zgloszenia = _repo_zapisow.lista_zgloszen_strona(status_filter=status_filter, strona=strona)

    except ValueError:

        flash(f'Nieznany status: {status_filter}', 'warning')

        zgloszenia = _repo_zapisow.lista_zgloszen_strona(strona=strona)

    csrf_form = FlaskForm()

    return render_template('zarzadzanie/enrollments/list.html', zgloszenia=zgloszenia, csrf_form=csrf_form)

```



```

@zarzadzanie_bp.route('/zgloszenia/<uuid:id>/przypisz-uopz', methods=['GET', 'POST'])

@wymaga_rola(UserRole.ADMIN)

def przypisz_uopz(id):

    zapis = db.session.get(InternshipEnrollment, id) or abort(404)

    form = FormularzPrzypiszUOPZ()

    uopz_list = _repo_uzytk.aktywni_uopz()

    form.uopz_id.choices = [('', '--- brak ---')] + [(str(u.id), f"{u.first_name} {u.last_name}") for u in uopz_list]

    if form.validate_on_submit():

        if form.uopz_id.data:

            from core.uslugi.workflow import ZapisFSM, IllegalTransitionError

            try:

                with ZapisFSM.lock(id) as fsm:

                    fsm.zapis.supervisor_id = form.uopz_id.data

                    fsm.wyslij_do_akceptacji()

                    db.session.commit()

                    flash('Opiekun UOPZ przypisany, zgłoszenie przekazane do zatwierdzenia.', 'success')

            except IllegalTransitionError as e:

                flash(str(e), 'danger')

        else:

            flash('Nie wybrano opiekuna.', 'warning')

    return redirect(url_for('zarzadzanie.lista_zgloszen'))

if request.method == 'GET':

    form.uopz_id.data = str(zapis.supervisor_id) if zapis.supervisor_id else ''

    return render_template('zarzadzanie/enrollments/przypisz_uopz.html', form=form, zapis=zapis)

@zarzadzanie_bp.route('/zgloszenia/<uuid:id>/szczegoly', methods=['GET', 'POST'])

@wymaga_rola(UserRole.ADMIN, UserRole.UOPZ)

def szczegoly_zgloszenia(id):

    zapis = db.session.get(InternshipEnrollment, id) or abort(404)

```

```

if current_user.role == UserRole.UOPZ and zapis.supervisor_id != current_user.id:

    abort(403)

harmonogram      = _repo_zapisow.harmonogram_dla_zapisu(id)

harmonogram_dict = {h.learning_outcome_id: h for h in harmonogram}

efekty           = _repo_efektow.wszystkie()

from flask_wtf import FlaskForm

from wtforms import TextAreaField, SubmitField

class FormularzKomentarza(FlaskForm):

    komentarz = TextAreaField('Komentarz do studenta')

    zatwierdz = SubmitField('Zatwierdź zgłoszenie')

    odrzuc     = SubmitField('Wymagane poprawki')

form = FormularzKomentarza()

if form.validate_on_submit():

    from core.modele import ProcessEvent, EventType

    from datetime import datetime, timezone as _tz

    from core.uslugi.workflow import ZapisFSM, IllegalTransitionError

    try:

        with ZapisFSM.lock(id) as fsm:

            if form.zatwierdz.data:

                if form.komentarz.data:

                    db.session.add(ProcessEvent(

                        enrollment_id=fsm.zapis.id,

                        event_type=EventType.ADMIN_COMMENT if current_user.role == UserRole.ADMIN else
EventType.SUPERVISOR_COMMENT,

                        comment=form.komentarz.data,

                        executed_by_id=current_user.id, executed_at=datetime.now(_tz.utc),

                    ))

                fsm.zatwierdz_przez_uopz()

            flash('Zgłoszenie zostało zatwierdzone!', 'success')

```

```

elif form.odrzuc.data:

    db.session.add(ProcessEvent(

        enrollment_id=fsm.zapis.id,

                                event_type=EventType.ADMIN_COMMENT if current_user.role == UserRole.ADMIN else
EventType.SUPERVISOR_COMMENT,

        comment=form.komentarz.data,

        executed_by_id=current_user.id, executed_at=datetime.now(_tz.utc),

    ))

    db.session.add(ProcessEvent(

        enrollment_id=fsm.zapis.id, event_type=EventType.STUDENT_NOTIFICATION,

        executed_by_id=current_user.id, executed_at=datetime.now(_tz.utc),

    ))

    fsm.odrzuc()

    flash('Wysłano prośbę o poprawki do studenta.', 'info')

    db.session.commit()

except IllegalTransitionError as e:

    flash(str(e), 'danger')

return redirect(url_for('zarzadzanie.lista_zgloszen') if current_user.role == UserRole.ADMIN

                    else url_for('zarzadzanie.moje_zgloszenia'))

uploaded_docs = _repo_docs.dla_zapisu_studenta(id, zapis.student_id)

return render_template('zarzadzanie/enrollments/szczegoly.html',

                        zapis=zapis, harmonogram_dict=harmonogram_dict,

                        efekty=efekty, form=form, uploaded_docs=uploaded_docs)

@zarzadzanie_bp.route('/zgloszenia/<uuid:id>/zatwierdz-zaklad', methods=['POST'])

@wymaga_rol(UserRole.UOPZ, UserRole.ADMIN)

def zatwierdz_zaklad(id):

    from core.uslugi.workflow import ZapisFSM, IllegalTransitionError

    try:

        with ZapisFSM.lock(id) as fsm:

            fsm.zatwierdz_przez_uopz()

            db.session.commit()

```

```

        flash('Zakład zatwierdzony. Internship rozpoczęła się.', 'success')

except IllegalTransitionError as e:

    flash(str(e), 'danger')

return redirect(url_for('zarzadzanie.lista_zgloszen'))


@zarzadzanie_bp.route('/zgloszenia/<uuid:id>/potwierdz', methods=['POST'])

@wymaga_rol(UserRole.ADMIN, UserRole.UOPZ)

def potwierdz_zapis(id):

    from core.uslugi.workflow import ZapisFSM, IllegalTransitionError

    try:

        with ZapisFSM.lock(id) as fsm:

            fsm.zatwierdz_przez_uopz()

            if current_user.role == UserRole.UOPZ:

                fsm.zapis.supervisor_id = current_user.id

            db.session.commit()

            flash('Zapis studenta na praktykę został potwierdzony. Zostałeś/aś przypisany/a jako opiekun.', 'success')

except IllegalTransitionError as e:

    flash(str(e), 'danger')

return redirect(request.referrer or url_for('dashboard.index'))


@zarzadzanie_bp.route('/moje-zgloszenia')

@wymaga_rol(UserRole.UOPZ)

def moje_zgloszenia():

    """Lista zgłoszeń przypisanych do aktualnego UOPZ"""

    from app_admin.routes.ocenianie import get_pilne_oceny

    strona          = request.args.get('page', 1, type=int)

    status_filter   = request.args.get('status', '').strip()

    try:

        zgloszenia = _repo_zapisow.dla_uopz_strona(

            current_user.id, status_filter=status_filter, strona=strona

```

```

    )

except ValueError:

    flash(f'Nieznany status: {status_filter}', 'warning')

    zgloszenia = _repo_zapisow.dla_uopz_strona(current_user.id, strona=strona)

    liczniki = _repo_zapisow.liczniki_dla_uopz(current_user.id)

    pilne_oceny = get_pilne_oceny(current_user.id)

    csrf_form = FlaskForm()

    return render_template('zarzadzanie/enrollments/moje_lista.html',

                           zgloszenia=zgloszenia, liczniki=liczniki,

                           pilne_oceny=pilne_oceny, csrf_form=csrf_form)


@zarzadzanie_bp.route('/praktyki/<uuid:id>/usun', methods=['POST'])
@wymaga_rola(UserRole.ADMIN)
def usun_praktyke(id):

    p = db.session.get(Internship, id) or abort(404)

    opis = f'{p.academic_year} ({p.semester})'

    from sqlalchemy import text as _text

    db.session.execute(_text("""

        DELETE FROM uploaded_documents

        WHERE enrollment_id IN (

            SELECT id FROM internship_enrollments WHERE internship_id = :pid

        )

    """), {'pid': str(id)})

    db.session.delete(p)

    db.session.commit()

    flash(f'Internship {opis} została usunięta.', 'success')

    return redirect(url_for('zarzadzanie.lista_praktyk'))

```

PLIK: app_admin\routes\zarzadzanie\uzytownicy.py

```
import uuid

import csv

import io

import datetime

from flask import (Blueprint, render_template, redirect, url_for,
                   flash, request, abort)

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from flask_wtf.file import FileField, FileAllowed

from wtforms import StringField, SelectField

from wtforms.validators import DataRequired, Email, Length, Optional, ValidationError


from core.modele import (User, Student, Internship, InternshipEnrollment, InternshipSchedule, LearningOutcome,
                          UserRole, InternshipStatus, EnrollmentStatus, UploadedDocument, Company)

from core.extensions import db

from core.uslugi import UsługaUzytkownikow as _UsługaUzytkownikow

_serwis_uzytownikow = _UsługaUzytkownikow()

from core.autoryzacja import wymaga_rol

from core.repozytoria import RepozytoriumUzytkownikow

_repo_uzyt = RepozytoriumUzytkownikow()


from . import zarzadzanie_bp

from .formularze import *


# — Użytkownicy —————

@zarzadzanie_bp.route('/uzytownicy')

@login_required

def lista_uzytownikow():

    strona      = request.args.get('strona', 1, type=int)

    szukaj      = request.args.get('szukaj', '').strip()

    filtr_rola  = request.args.get('rola', '').strip()
```

```

uzytkownicy = _repo_uzytk.szukaj_strona(szukaj=szukaj, filtr_rola=filtr_rola, strona=strona)

csrf_form = FlaskForm()

return render_template('zarzadzanie/uzytkownicy.html',

                        uzytkownicy=uzytkownicy,

                        csrf_form=csrf_form)


@zarzadzanie_bp.route('/uzytkownicy/nowy-student', methods=['GET', 'POST'])
@wymaga_rola(UserRole.ADMIN)
def nowy_student():

    form = FormularzStudenta()

    uopz_list = _repo_uzytk.aktywni_uopz()

    form.uopz_id.choices = [(str(u.id), f"{u.first_name} {u.last_name}") for u in uopz_list]

    if form.validate_on_submit():

        u = _serwis_uzytkownikow.utworz_studenta(

            email          = form.email.data.lower().strip(),

            haslo          = '',

            imie           = form.imie.data,

            nazwisko       = form.nazwisko.data,

            numer_albumu   = form.numer_albumu.data,

            gender         = form.plec.data          or None,

            field_of_study = form.kierunek.data      or None,

            specialization  = form.specjalnosc.data  or None,

            study_mode      = form.tryb_studiow.data or None,

            supervisor_id  = form.uopz_id.data       or None,

            require_password_change=False,

        )

        flash(

            f'Konto studenta {u.imie} {u.nazwisko} (nr alb. {u.numer_albumu}) '

            f'zostało utworzone. Student może się teraz zalogować przez Microsoft ({u.email}).',

            'success'

        )

    return redirect(url_for('zarzadzanie.lista_uzytkownikow'))

```

```
return render_template('zarzadzanie/formularz_studenta.html', form=form, uzytkownik=None)
```

```
@zarzadzanie_bp.route('/uzytkownicy/<uuid:id>/edytuj-student', methods=['GET', 'POST'])
```

```
@wymaga_rola(UserRole.ADMIN)
```

```
def edytuj_studenta(id):
```

```
    u = db.session.get(User, id) or abort(404)
```

```
    form = FormularzEdycjiStudenta(uzytkownik_id=id, obj=u)
```

```
    uopz_list = _repo_uzytk.aktywni_uopz()
```

```
    form.uopz_id.choices = [(str(x.id), f"{x.first_name} {x.last_name}") for x in uopz_list]
```

```
    if request.method == 'GET':
```

```
        form.imie.data = u.first_name
```

```
        form.nazwisko.data = u.last_name
```

```
        form.email.data = u.email
```

```
        form.numer_albumu.data = u.album_number
```

```
    if form.validate_on_submit():
```

```
        u.first_name = form.imie.data.strip()
```

```
        u.last_name = form.nazwisko.data.strip()
```

```
        u.email = form.email.data.lower().strip()
```

```
        u.album_number = form.numer_albumu.data.strip()
```

```
        u.gender = form.plec.data or None
```

```
        u.field_of_study = form.kierunek.data or None
```

```
        u.specialization = form.specjalnosc.data or None
```

```
        u.study_mode = form.tryb_studiow.data or None
```

```
        db.session.commit()
```

```
        flash('Dane studenta zostały zaktualizowane.', 'success')
```

```
        return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

```
    return render_template('zarzadzanie/formularz_studenta.html', form=form, uzytkownik=u)
```



```

@zarzadzanie_bp.route('/uzytkownicy/nowy-pracownik', methods=['GET', 'POST'])

@wymaga_rol(UserRole.ADMIN)

def nowy_pracownik():

    form = FormularzPracownika()

    if form.validate_on_submit():

        u = User(

            id = uuid.uuid4(),

            first_name = form.imie.data.strip(),

            last_name = form.nazwisko.data.strip(),

            email = form.email.data.lower().strip(),

            role = UserRole[form.rola.data],

            password_hash = '',

            require_password_change = False,

            is_active = True,

        )

        db.session.add(u)

        db.session.commit()

        flash(

            f'Konto {u.first_name} {u.last_name} [{u.role.value}] utworzone. '

            f'Użytkownik może się zalogować przez Microsoft ({u.email}).',

            'success'

        )

        return redirect(url_for('zarzadzanie.lista_uzytkownikow'))

    return render_template('zarzadzanie/formularz_pracownika.html', form=form, uzytkownik=None)

```

```

@zarzadzanie_bp.route('/uzytkownicy/import-csv', methods=['GET', 'POST'])

@wymaga_rol(UserRole.ADMIN)

def import_csv():

    form = FormularzImportuCSV()

    uopz_list = _repo_uzytk.aktywni_uopz()

    form.uopz_id.choices = [('', '- wybierz -')] + [(str(u.id), f"{u.first_name} {u.last_name}") for u in uopz_list]

    wyniki = None

```

```

if form.validate_on_submit():

    uopz_id    = form.uopz_id.data or None

    plik       = form.plik.data

    zawartosc  = plik.read().decode('utf-8-sig')

    czytnik    = csv.DictReader(io.StringIO(zawartosc))

    utworzono, pominieto, bledy = 0, 0, []

    wiersze_csv = []

    for nr_wiersza, wiersz in enumerate(czytnik, start=2):

        imie      = (wiersz.get('imie')          or wiersz.get('Imię')          or '').strip()

        nazwisko   = (wiersz.get('nazwisko')       or wiersz.get('Nazwisko')    or '').strip()

        email      = (wiersz.get('email')          or wiersz.get('Email')        or '').strip().lower()

        nr_albumu  = (wiersz.get('numer_albumu')    or wiersz.get('Nr albumu')    or '').strip()

        plec       = (wiersz.get('plec')            or wiersz.get('Płeć')          or '').strip().upper() or None

        kierunek   = (wiersz.get('kierunek')        or wiersz.get('Kierunek')      or '').strip() or None

        specjalnosc = (wiersz.get('specjalnosc')     or wiersz.get('Specjalność') or '').strip() or None

        tryb_studiow = (wiersz.get('tryb_studiow')   or wiersz.get('Tryb')          or '').strip().lower() or None

        if not all([imie, nazwisko, email, nr_albumu, plec, kierunek, tryb_studiow]):

            bledy.append(f'Wiersz {nr_wiersza}: brakujące dane (wymagane: imie, nazwisko, email, numer_albumu, plec, kierunek, tryb_studiow)')

            pominieto += 1

            continue

        wiersze_csv.append({

            'nr': nr_wiersza, 'imie': imie, 'nazwisko': nazwisko,

            'email': email, 'nr_albumu': nr_albumu, 'plec': plec,

            'kierunek': kierunek, 'specjalnosc': specjalnosc,

            'tryb_studiow': tryb_studiow,

        })

    if wiersze_csv:

        all_emails = [w['email'] for w in wiersze_csv]

        all_albums = [w['nr_albumu'] for w in wiersze_csv]

```

```

existing = _repo_uzytk.znajdz_istniejace_po_email_lub_albumie(
    all_emails, all_albums
)

existing_emails = {row.email for row in existing}
existing_albums = {row.album_number for row in existing if row.album_number}

for w in wiersze_csv:
    if w['email'] in existing_emails or w['nr_albumu'] in existing_albums:
        bledy.append(f"Wiersz {w['nr']}: {w['email']} lub nr {w['nr_albumu']} już istnieje")
        pominieto += 1
        continue
    try:
        _serwis_uzytkownikow.utworz_studenta(
            email = w['email'].lower().strip(),
            haslo = '',
            imie = w['imie'],
            nazwisko = w['nazwisko'],
            numer_albumu = w['nr_albumu'],
            gender = w['plec'],
            field_of_study = w['kierunek'],
            specialization = w['specjalnosc'],
            study_mode = w['tryb_studiow'],
            supervisor_id = uuid.UUID(uopz_id) if uopz_id else None,
            require_password_change = False,
        )
        existing_emails.add(w['email'])
        existing_albums.add(w['nr_albumu'])
        utworzono += 1
    except Exception as e:
        bledy.append(f"Wiersz {w['nr']}: {str(e)}")
        pominieto += 1

```

```
db.session.commit()
```

```
wyniki = {'utworzono': utworzono, 'pominieto': pominieto, 'bledy': bledy}
```

```
if utworzono:
```

```
    flash(f'Import zakończony: {utworzono} kont utworzonych.', 'success')
```

```
return render_template('zarzadzanie/import_csv.html', form=form, wyniki=wyniki)
```

```
@zarzadzanie_bp.route('/uzytkownicy/<uuid:id>/aktywnosc', methods=['POST'])
```

```
@wymaga_rola(UserRole.ADMIN)
```

```
def przelacz_aktywnosc(id):
```

```
    u = db.session.get(User, id) or abort(404)
```

```
    if str(u.id) == str(current_user.id):
```

```
        flash('Nie możesz dezaktywować własnego konta.', 'danger')
```

```
        return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

```
    u.is_active = not u.is_active
```

```
    db.session.commit()
```

```
    stan = 'aktywowane' if u.is_active else 'dezaktywowane'
```

```
    flash(f'Konto {u.first_name} {u.last_name} zostało {stan}.', 'success')
```

```
    return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

```
@zarzadzanie_bp.route('/uzytkownicy/<uuid:id>/usun', methods=['POST'])
```

```
@wymaga_rola(UserRole.ADMIN)
```

```
def usun_uzytkownika(id):
```

```
    u = db.session.get(User, id) or abort(404)
```

```
    if str(u.id) == str(current_user.id):
```

```
        flash('Nie możesz usunąć własnego konta.', 'danger')
```

```
        return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

```
    imie_nazwisko = f'{u.first_name} {u.last_name}'
```

```
    db.session.delete(u)
```

```
    db.session.commit()
```

```
    flash(f'Konto {imie_nazwisko} zostało trwale usunięte.', 'success')
```

```
return redirect(url_for('zarzadzanie.lista_uzytkownikow'))
```

PLIK: app_admin\routes\zarzadzanie__init__.py

```
from flask import Blueprint
```

```
zarzadzanie_bp = Blueprint('zarzadzanie', __name__)
```

```
from . import uzytkownicy, praktyki, firmy, komisja, dziekan, dokumenty_studentow
```

```
/* =====  
  
admin.css – Punkt wejścia arkusza stylów panelu  
  
System Praktyk Zawodowych – ANS Elbląg  
  
Moduły ładowane w kolejności zależności:  
  
1. zmienne – tokeny CSS, reset, dostępność (WCAG 2.1 AA)  
2. komunikaty – flash messages i powiadomienia  
3. formularze – pola input, checkboxy  
4. logowanie – strona logowania i zmiana hasła  
5. układ – sidebar, panel-main, nagłówek  
6. karty – komponent .karta  
7. tabele – tabele danych  
8. przyciski – przyciski i grupy akcji  
9. statusy – odznaki statusów i ról  
10. komponenty – pasek postępu, paginacja, filtry  
11. dashboard – statystyki, strona błędu  
  
===== */  
  
@import url('/core/static/css/modul/zmienne.css');  
  
@import url('/core/static/css/modul/komunikaty.css');  
  
@import url('/core/static/css/modul/formularze.css');  
  
@import url('/core/static/css/modul/logowanie.css');  
  
@import url('/core/static/css/modul/uklad.css');  
  
@import url('/core/static/css/modul/karty.css');  
  
@import url('/core/static/css/modul/tabele.css');  
  
@import url('/core/static/css/modul/przyciski.css');  
  
@import url('/core/static/css/modul/statusy.css');  
  
@import url('/core/static/css/modul/komponenty.css');  
  
@import url('/core/static/css/modul/dashboard.css');
```

PLIK: app_admin\static\css\student.css

```
/* =====  
  
student.css – punkt wejścia dla widoków studenta renderowanych  
  
wewnątrz app_admin (podgląd dokumentów, panel komisji itp.)  
  
Nie zawiera własnych definicji – importuje kanoniczne moduły  
z core/static, identycznie jak app_student/static/css/student.css.  
Zmiana stylu w jednym miejscu (core/static/css/modul/) automatycznie  
obowiązuje w obu aplikacjach.  
  
===== */  
  
@import url('/core/static/css/modul/zmienne.css');  
  
@import url('/core/static/css/modul/komunikaty.css');  
  
@import url('/core/static/css/modul/formularze.css');  
  
@import url('/core/static/css/modul/logowanie.css');  
  
@import url('/core/static/css/modul/uklad.css');  
  
@import url('/core/static/css/modul/karty.css');  
  
@import url('/core/static/css/modul/tabele.css');  
  
@import url('/core/static/css/modul/przyciski.css');  
  
@import url('/core/static/css/modul/statusy.css');  
  
@import url('/core/static/css/modul/komponenty.css');  
  
@import url('/core/static/css/modul/dashboard.css');
```


PLIK: app_admin\templates\auth\logowanie.html

```
{% extends "layouts/base_auth.html" %}

{% block tytul %}Logowanie{% endblock %}

{% block body_class %}strona-logowania{% endblock %}

{% block body %}

<div class="panel-logowania" role="main">

    <aside class="panel-logowania__branding" aria-label="Informacje o instytucji">

        <div class="branding__tresc">

            <div class="branding__logo-kontener">

                

            </div>

            <div class="branding__separator" aria-hidden="true"></div>

            <p class="branding__nazwa-systemu">System zarządzania<br>praktykami zawodowymi</p>

            <p class="branding__podtytul">Kierunek Informatyka</p>

            <div class="branding__dekoracja" aria-hidden="true">

                <span></span><span></span><span></span><span></span>

            </div>

        </div>

    </aside>

    <main class="panel-logowania__formularz" id="main-content">

        <div class="formularz__kontener">

            <div class="formularz__naglowek">

                <h1 class="formularz__tytul">Panel administracyjny</h1>

                <p class="formularz__opis">

                    Zaloguj się kontem Microsoft przypisanym do uczelni.

                </p>

            </div>

        </div>

    </main>

</div>

{% endblock %}
```

</p>

</div>

```
{% with wiadomosci = get_flashed_messages(with_categories=True) %}
```

```
{% if wiadomosci %}
```

```
<div role="alert" aria-live="assertive" class="komunikaty">
```

```
{% for kategoria, wiadomosc in wiadomosci %}
```

```
<div class="komunikat komunikat--{{ kategoria }}">
```

```
<span class="komunikat__ikona" aria-hidden="true">
```

```
{% if kategoria == 'danger' %}{% elif kategoria == 'success' %}{% else %}{% endif %}
```

```
</span>
```

```
<span>{{ wiadomosc }}</span>
```

```
</div>
```

```
{% endfor %}
```

```
</div>
```

```
{% endif %}
```

```
{% endwith %}
```

```
<div class="logowanie-microsoft">
```

```
<a
```

```
href="{{ url_for('auth.ms_login', next=request.args.get('next', '')) }}"
```

```
class="przycisk-microsoft"
```

```
aria-label="Zaloguj się przez konto Microsoft"
```

```
>
```

```
<svg class="przycisk-microsoft__logo" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 21 21"
aria-hidden="true">
```

```
<rect x="1" y="1" width="9" height="9" fill="#f25022"/>
```

```
<rect x="11" y="1" width="9" height="9" fill="#7fba00"/>
```

```
<rect x="1" y="11" width="9" height="9" fill="#00a4ef"/>
```

```
<rect x="11" y="11" width="9" height="9" fill="#ffb900"/>
```

```
</svg>
```

```
<span>Zaloguj się przez Microsoft</span>
```

```
</a>
```

```
</div>
```

```
<footer class="formularz__stopka">
```

```
<p class="formularz__stopka-tekst">
```

```
    Dostęp wyłącznie dla upoważnionych pracowników Instytutu.
```

```
</p>
```

```
<p class="formularz__stopka-tekst">
```

```
    Problemy z logowaniem?
```

```
<a href="mailto:admin@ans.elblag.edu.pl" class="formularz__link">
```

```
    Skontaktuj się z administratorem
```

```
</a>
```

```
</p>
```

```
</footer>
```

```
</div>
```

```
</main>
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_admin\templates\dashboard\index.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dashboard{% endblock %}

{% block naglowek %}Przegląd systemu{% endblock %}


{% block naglowek_akcje %}

    {% if current_user.role.value == 'ADMIN' %}

        <div class="akcje-panelu" style="display: flex; gap: 0.5rem; align-items: center;">

            <a href="{{ url_for('zarzadzanie.nowy_student') }}" class="przycisk przycisk--glowny">

                <span aria-hidden="true">+</span> Dodaj studenta

            </a>

            <a href="{{ url_for('zarzadzanie.nowy_pracownik') }}" class="przycisk przycisk--glowny">

                <span aria-hidden="true">+</span> Dodaj pracownika

            </a>

        </div>

    {% endif %}

{% endblock %}


{% block tresc %}


{% if current_user.role.value == 'UOPZ' and pilne_oceny %}

<div style="margin-bottom: 2rem;">

    {% for item in pilne_oceny %}

        <div class="komunikat komunikat--{% if item.przekroczony %}danger{% else %}warning{% endif %}" style="margin-bottom: 1rem;">

            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" style="flex-shrink: 0; margin-top: 2px;">

                <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/>

                <line x1="12" y1="9" x2="12" y2="13"/>

                <line x1="12" y1="17" x2="12.01" y2="17"/>

            </svg>

            <div>

                <div style="font-weight: 600;">

                    {% if item.przekroczony %}
```

```

        Przekroczony deadline oceny: {{ item.zapis.student.last_name }} {{ item.zapis.student.first_name }}

    {% elif item.dni_do_deadline == 0 %}

        Dziś ostatni dzień oceny: {{ item.zapis.student.last_name }} {{ item.zapis.student.first_name }}

    {% else %}

        Pilna ocena za {{ item.dni_do_deadline }} dni: {{ item.zapis.student.last_name }} {{
item.zapis.student.first_name }}

    {% endif %}

</div>

<div style="font-size: 0.9rem; margin-top: 0.5rem;">

    Deadline: {{ item.deadline.strftime('%d.%m.%Y') }}

    <a href="{{ url_for('evaluation.ocen_praktyke', id=item.zapis.id) }}"

        style="margin-left: 1rem; color: inherit; text-decoration: underline;">

        Wystaw ocenę →

    </a>

</div>

</div>

</div>

{% endfor %}

</div>

{% endif %}

```

```

<section class="siatka-statystyk" aria-label="Kluczowe wskaźniki">

```

```

<div class="karta-stat">

    <span class="karta-stat__ikona" aria-hidden="true"><svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="10"/><polyline points="12 6 12 12 16 14"/></svg></span>

    <span class="karta-stat__liczba">{{ statystyki.praktyki_aktywne }}</span>

    <span class="karta-stat__etykieta">W trakcie realizacji</span>

</div>

```

```

<div class="karta-stat">

    <span class="karta-stat__ikona" aria-hidden="true"><svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M9 11l3 3L22 4"/><path
d="M21 12v7a2 2 0 0 1-2 2H5a2 2 0 0 1-2 2V5a2 2 0 0 1 2-2h11"/></svg></span>

    <span class="karta-stat__liczba">{{ statystyki.oczekujace_oceny }}</span>

    <span class="karta-stat__etykieta">Czeka na ocenę</span>

```

```
</div>
```

```
<div class="karta-stat">
```

```
    <span class="karta-stat__ikona" aria-hidden="true"><svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M22 10v6M2 10 5-10
5z"/><path d="M6 12v5c0 2 4 4 6 4s6-2 6-4v-5"/></svg></span>
```

```
    <span class="karta-stat__liczba">{{ statystyki.zakonczone }}</span>
```

```
    <span class="karta-stat__etykieta">Zakończzone</span>
```

```
</div>
```

```
<div class="karta-stat">
```

```
    <span class="karta-stat__ikona" aria-hidden="true"><svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M17 21v-2a4 4 0 0
0-4-4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1
0 7.75"/></svg></span>
```

```
    <span class="karta-stat__liczba">{{ statystyki.liczba_studentow }}</span>
```

```
    <span class="karta-stat__etykieta">Aktywnych studentów</span>
```

```
</div>
```

```
</section>
```

```
<div class="uklad-dashboardu" style="display: grid; grid-template-columns: 2fr 1fr; gap: 2rem;">
```

```
    <section class="karta" aria-labelledby="tytul-ostatnie">
```

```
        <header class="karta__naglowek">
```

```
            <h2 id="tytul-ostatnie" class="karta__tytul">Ostatnie aktywności</h2>
```

```
                <a href="{{ url_for('zarzadzanie.lista_praktyk') }}" class="przycisk przycisk--drugorzedny
przycisk--maly">Wszystkie</a>
```

```
        </header>
```

```
    <div class="karta__tresc">
```

```
        <div class="tabela-wrapper">
```

```
            <table class="tabela">
```

```
                <thead>
```

```
                    <tr>
```

```
                        <th scope="col">Student</th>
```

```
                        <th scope="col">Praktyka</th>
```

```
                        <th scope="col">Status</th>
```

```

        <th scope="col" style="text-align:right;">Akcje</th>

    </tr>

</thead>

<tbody>

    {% for z in ostatnie_zapisy %}

    <tr>

        <td>{{ z.student.first_name }} {{ z.student.last_name }}</td>

        <td>{{ z.praktyka.rok_uczelnianny }} ({{ z.praktyka.semestr }})</td>

        <td>

            <span class="status status--{{ z.status.value|lower|replace('_', '-') }}">

                {{ tlumacz_status(z.status.value) }}

            </span>

        </td>

        <td style="text-align:right;">

            {% if z.status.value == 'PENDING' %}

                <form method="POST" action="{{ url_for('zarzadzanie.potwierdz_zapis', id=z.id) }}"
style="display:inline;">

                    {{ csrf_form.hidden_tag() }}

                    <button type="submit" class="przycisk przycisk--drugorzecny przycisk--maly" onclick="return
confirm('Potwierdzić ten zapis?')">Zatwierdź</button>

                </form>

            {% endif %}

        </td>

    </tr>

    {% else %}

    <tr>

        <td colspan="3" class="tabela--pusta">Brak aktywnych zapisów.</td>

    </tr>

    {% endfor %}

</tbody>

</table>

</div>

</div>

</section>

```

```
<section class="karta" aria-labelledby="tytul-oczekujace">

  <header class="karta__naglowek">

    <h2 id="tytul-oczekujace" class="karta__tytul">Aktywne praktyki</h2>

  </header>

  <div class="karta__tresc">

    <ul class="lista-prosta" role="list" style="list-style: none; padding: 0;">

      {% for z in ostatnie_zapisy %}

        <li style="padding: 1rem 0; border-bottom: 1px solid var(--kolor-ramka); display: flex; justify-content: space-between; align-items: center;">

          <div>

            <span style="display: block; font-weight: 600;">{{ z.student.first_name }} {{ z.student.last_name }}</span>

            <span style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">{{ z.praktyka.rok_uczelnianny }}</span>

          </div>

        </li>

        {% else %}

        <li style="padding: 2rem; text-align: center; color: var(--kolor-tekst-drugor);">Brak zapisów.</li>

        {% endfor %}

      </ul>

    </div>

  </section>

</div>

{% endblock %}
```


PLIK: app_admin\templates\documents\edytor.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Edytor Dokumentów (LaTeX){% endblock %}

{% block naglowek %}Tryb Ręczny: {{ doc_title }}{% endblock %}


{% block naglowek_akcje %}

<a href="{{ url_for('documents.panel', id=zapis.id) }}" class="przycisk przycisk--drugorzeczny">Powrót do panelu</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto; padding-bottom: 4rem;">

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__naglowek" style="background: var(--kolor-tlo); display: flex; justify-content: space-between; align-items: center;">

            <h2 class="karta__tytul">Surowy kod LaTeX</h2>

            <span style="font-size: 0.85rem; color: var(--kolor-blad); font-weight: 600;">Uwaga: Jesteś w trybie "Manual Override"</span>

        </div>

        <div class="karta__tresc">

            <p style="font-size: 0.9rem; color: var(--kolor-tekst-drugor); margin-bottom: 1.5rem;">

                Edytujesz właśnie wyrenderowany szablon. Możesz wprowadzać dowolne zmiany zgodnie z syntaktyką LaTeX (LuaLaTeX). Zmieniony dokument nie zostanie nadpisany w systemie, lecz od razu skompilowany do PDF. Jeśli system automatyczny generuje błędne dane, możesz je tutaj poprawić i pobrać PDF ręcznie.

            </p>

            <form id="tex-form">

                <input type="hidden" name="csrf_token" value="{{ csrf_token() }}" />

                <div style="border: 1px solid var(--kolor-ramka); border-radius: 4px; overflow: hidden; margin-bottom: 1.5rem;">

                    <textarea name="tex_source" id="tex_source" style="width: 100%; height: 60vh; max-height: 800px; font-family: 'IBM Plex Mono', monospace; font-size: 0.85rem; padding: 1rem; border: none; background: #fbfbfb; resize: vertical; spellcheck="false">{{ raw_tex }}</textarea>

                </div>

            </div>

        </div>

    </div>

</div>
```

```

<div style="display: flex; justify-content: space-between; align-items: center;">

  <div id="status-box" style="font-size: 0.9rem; font-weight: 500; color: var(--kolor-tekst-drugor);">

    Gotowy do kompilacji

  </div>

  <button type="submit" id="btn-kompiluj" class="przycisk przycisk--glowny" style="padding: 0.75rem 2.5rem; font-size: 1.05rem;">

    Kompiluj do PDF

  </button>

</div>

</form>

</div>

</div>

</div>

```

```
{% block scripts_extra %}
```

```
<script>
```

```

document.addEventListener('DOMContentLoaded', () => {

  const form = document.getElementById('tex-form');

  const btn = document.getElementById('btn-kompiluj');

  const statusBox = document.getElementById('status-box');

  form.addEventListener('submit', async (e) => {

    e.preventDefault();

    btn.disabled = true;

    btn.innerHTML = 'Kompilowanie w tle...';

    statusBox.innerHTML = '<span style="color: var(--kolor-glowny);">Czekam na uruchomienie zadania...</span>';

    try {

      const formData = new FormData(form);

      const res = await fetch("{ url_for('documents.kompiluj_recznie', id=zapis.id, doc_type=doc_type) }", {

        method: 'POST',

        body: formData,

```

```

        headers: {

            'X-Requested-With': 'XMLHttpRequest'

        }
    });

    const data = await res.json();

    if(!res.ok) {

        throw new Error(data.error || 'Błąd serwera');

    }

    const taskId = data.task_id;

    watchTaskSSE(taskId);

} catch(err) {

    statusBox.innerHTML = `

```

```

    } else if (data.status === 'FAILURE') {

        es.close();

        statusBox.innerHTML = `

```

PLIK: app_admin\templates\documents\panel.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dokumenty i Załączniki{% endblock %}

{% block naglowek %}Generowanie dokumentów: {{ zapis.student.imie }} {{ zapis.student.nazwisko }}{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.szczegoly_praktyki', id=zapis.internship_id) %}" class="przycisk
przycisk--drugorzeczny">Powrót do szczegółów</a>

{% endblock %}

{% block tresc %}

<div class="uklad-szczegolow" style="display: grid; grid-template-columns: 2fr 1fr; gap: 2rem;">

    <div class="lewa-kolumna" style="display: flex; flex-direction: column; gap: 2rem;">

        <!-- Lista dokumentów -->

        <div class="karta">

            <div class="karta__naglowek" style="background: var(--kolor-tlo); display: flex; justify-content: space-between;
            align-items: center;">

                <h2 class="karta__tytul">Dostępne szablony (LaTeX)</h2>

            </div>

            <div class="karta__tresc">

                <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">

                    Wybierz dokument, który chcesz wygenerować ze stałych szablonów. Kliknij "Tryb ręczny" aby edytować surowy
                    szablon TeX przed kompilacją.

                </p>

                <div style="display: flex; flex-direction: column; gap: 1rem;">

                    {% for doc_key, doc_info in docs.items() %}

                        <div style="display: flex; justify-content: space-between; align-items: center; padding: 1rem; border: 1px
                        solid var(--kolor-ramka); border-radius: 8px;">

                            <div style="font-weight: 500;">

                                {{ doc_info[1] }}

                            </div>

                            <div style="display: flex; gap: 0.5rem; align-items: center;">

                                <span id="status-{{ doc_key }}" style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); font-weight:
```

```
600; margin-right: 0.5rem;"></span>

        <button onclick="triggerAuto('{{ doc_key }}')" id="btn-auto-{{ doc_key }}" class="przycisk
przycisk--drugorzedny przycisk--maly">Wyg. Automatycznie</button>

        <a href="{{ url_for('documents.edytuj', id=zapis.id, doc_type=doc_key) }}" class="przycisk przycisk--glowny
przycisk--maly" style="background: #e67e22; border-color: #e67e22;">Tryb ręczny (TeX)</a>

    </div>

</div>

{% endfor %}

</div>

</div>

</div>

</div>

</div>

<div class="prawa-kolumna" style="display: flex; flex-direction: column; gap: 2rem;">

    <!-- Audit log -->

    <div class="karta">

        <div class="karta__naglowek" style="background: var(--kolor-tlo);">

            <h2 class="karta__tytul">Ostatnie modyfikacje (Log)</h2>

        </div>

        <div class="karta__tresc py-3">

            {% if logs %}

            <ul style="list-style: none; padding: 0; margin: 0; font-size: 0.85rem;">

                {% for log in logs %}

                <li style="margin-bottom: 1rem; border-bottom: 1px dashed var(--kolor-ramka); padding-bottom: 0.5rem;">

                    <strong style="color: var(--kolor-tekst);">{{ log.action }}</strong> – <span style="font-family:
monospace;">{{ log.document_type }}</span>

                    <br>

                    <span style="color: var(--kolor-tekst-drugor);">

                        {{ log.created_at.strftime('%Y-%m-%d %H:%M') }} | Kto: {{ log.user.imie if log.user else 'System' }}

                    </span>

                    {% if log.details %}

                    <div style="font-style: italic; margin-top: 0.25rem;">{{ log.details }}</div>

                    {% endif %}

                </li>

                {% endfor %}

            </ul>

        </div>

    </div>

</div>
```

```

    {% else %}

    <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Brak historii dokumentów.</p>

    {% endif %}

</div>

</div>

</div>

</div>

{% block scripts_extra %}

<script>

async function triggerAuto(docType) {

    const btn = document.getElementById('btn-auto-' + docType);

    const statusBox = document.getElementById('status-' + docType);

    btn.disabled = true;

    statusBox.innerHTML = '<span style="color: var(--kolor-glowny);">Uruchamianie...</span>';

    try {

        const url = `/dokumenty/zapis/{{ zapis.id }}/generuj_auto/${docType}`;

        const csrfToken = document.querySelector('meta[name="csrf-token"]').content;

        const res = await fetch(url, {

            method: 'POST',

            headers: {'X-Requested-With': 'XMLHttpRequest', 'X-CSRFToken': csrfToken}

        });

        const data = await res.json();

        if (!res.ok) throw new Error(data.error || 'Błąd serwera');

        // SSE – jedno trwałe połączenie zamiast pollingu co 2 sekundy

        watchTaskSSE(data.task_id, btn, statusBox);

    } catch (err) {

        statusBox.innerHTML = '<span style="color: var(--kolor-blad);">Błąd: ${err.message}</span>`;

        btn.disabled = false;

    }

```

```
}
```

```
function watchTaskSSE(taskId, btn, statusBox) {

    const es = new EventSource(`/dokumenty/stream/${taskId}`);

    statusBox.innerHTML = '<span style="color: var(--kolor-glowny);"> Trwa kompilacja...</span>';

    es.onmessage = (event) => {

        const data = JSON.parse(event.data);

        if (data.status === 'SUCCESS') {

            es.close();

            statusBox.innerHTML = `<span style="color: var(--kolor-sukces);"> Gotowe! Pobieranie...</span>`;

            btn.disabled = false;

            window.location.href = data.download_url;

            setTimeout(() => { statusBox.innerHTML = ''; window.location.reload(); }, 3000);

        } else if (data.status === 'FAILURE') {

            es.close();

            statusBox.innerHTML = `<span style="color: var(--kolor-blad);"> Błąd generacji.</span>`;

            btn.disabled = false;

            alert('Błąd kompilacji: ' + data.error);

        } else {

            const prog = data.progress || 0;

            statusBox.innerHTML = `<span style="color: var(--kolor-glowny);"> Kompilacja... ${prog}%</span>`;

        }

    };

    es.onerror = () => {

        es.close();

        statusBox.innerHTML = `<span style="color: var(--kolor-blad);">Błąd połączenia SSE.</span>`;

        btn.disabled = false;

    };

}
```

</script>

{% endblock %}

{% endblock %}

PLIK: app_admin\templates\evaluation\egzamin.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Egzamin praktyki{% endblock %}

{% block naglowek %}Protokół egzaminacyjny{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.szczegoly_praktyki', id=praktyka.id) %}" class="przycisk
przycisk--drugorzeczny">Powrót</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">

    <!-- Info o studencie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-akcent);">

        <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <h2 style="font-size: 1.1rem; margin-bottom: 0.25rem;">{{ praktyka.student.imie }} {{ praktyka.student.nazwisko
                }}</h2>

                <p style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

                    Zakład: {{ praktyka.zaklad.nazwa }} | Godziny: {{ praktyka.lacznie_godzin }} / 160 h

                </p>

            </div>

            <div style="text-align: right;">

                <span class="status status--{{ praktyka.status.value|lower|replace('_', '-') }}">{{
                tlumacz_status(praktyka.status.value) }}</span>

            </div>

        </div>

    </div>

</div>

<!-- Wzór -->

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Wzór obliczania oceny końcowej (Zał. 8)</h3>
```

[illegible]

```

        {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

        <option value="{{ stopien }}" {% if praktyka.ocena_sprawozdania == stopien %}selected{% endif %}>{{ stopien
    }}</option>

        {% endfor %}

    </select>

</div>


<div class="pole-formularza">

    <label class="pole-formularza__etykieta">Ocena Opiekuna Uczelnianego (U)</label>

    <select name="ocena_uopz" class="filtr-select" style="width: 100%;" required>

        <option value="">Wybierz...</option>

        {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

            <option value="{{ stopien }}" {% if praktyka.ocena_uopz == stopien %}selected{% endif %}>{{ stopien
    }}</option>

            {% endfor %}

        </select>

    </div>


<div class="pole-formularza">

    <label class="pole-formularza__etykieta">Ocena Opiekuna Zakładowego (Z)</label>

    <select name="ocena_zopz" class="filtr-select" style="width: 100%;" required>

        <option value="">Wybierz...</option>

        {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

            <option value="{{ stopien }}" {% if praktyka.ocena_zopz == stopien %}selected{% endif %}>{{ stopien
    }}</option>

            {% endfor %}

        </select>

    </div>


</div>

</div>

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__tresc" style="text-align: center; padding: 1.5rem;">

```

```

<!-- Wyliczenie K (JS) -->

```

```

        <div style="font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.08em; color:
var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">Ocena końcowa K</div>

        <div id="wynik-k" style="font-size: 3rem; font-weight: 800; color: var(--kolor-glowny);">--</div>

    </div>

</div>

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__tresc">

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Członkowie komisji egzaminacyjnej (oddzieleni przecinkami)</label>

            <input type="text" name="czlonkowie_komisji" class="pole-formularza__input"

                placeholder="np. dr Jan Kowalski, mgr Anna Nowak"

                {% if praktyka.committee_members %}value="{{ praktyka.committee_members | join(', ') }}" {% endif %}>

        </div>

    </div>

</div>

<div style="display: flex; justify-content: center; padding-bottom: 4rem;">

    <button type="submit" class="przycisk przycisk--glowny" style="padding: 1rem 3rem; font-size: 1.1rem; box-shadow: 0
4px 14px rgba(26, 58, 92, 0.3);">

        Zatwierdź egzamin i zakończ praktykę

    </button>

</div>

</form>

</div>

{% endblock %}

{% block scripts_extra %}

<script>

function obliczK() {

    const e = parseFloat(document.querySelector('[name="ocena_egzamin"]').value) || 0;

    const s = parseFloat(document.querySelector('[name="ocena_sprawozdanie"]').value) || 0;

    const u = parseFloat(document.querySelector('[name="ocena_uopz"]').value) || 0;

    const z = parseFloat(document.querySelector('[name="ocena_zopz"]').value) || 0;

    const wynik = document.getElementById('wynik-k');

```

```
if (e && s && u && z) {

    const k = (0.4 * e + 0.1 * s + 0.2 * u + 0.3 * z).toFixed(1);

    wynik.textContent = k;

    wynik.style.color = k >= 3.0 ? 'var(--kolor-sukces)' : 'var(--kolor-blad)';

} else {

    wynik.textContent = '-';

    wynik.style.color = 'var(--kolor-glowny)';

}

}

document.querySelectorAll('select').forEach(s => s.addEventListener('change', obliczK));

</script>

{% endblock %}
```

PLIK: app_admin\templates\evaluation\formularz_ocen.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Ocena praktyki{% endblock %}

{% block naglowek %}Karta oceny efektów uczenia się{% endblock %}

{% block naglowek_akcje %}

<a href="{{ url_for('evaluation.lista_ocen') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}

{% block tresc %}

<div class="formularz-ocen-kontener" style="max-width: 1000px; margin: 0 auto;">

    <!-- Info o studentie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-glowny);">

        <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <h2 style="font-size: 1.1rem; margin-bottom: 0.25rem;">{{ praktyka.student.imie }} {{ praktyka.student.nazwisko
                }}</h2>

                <p style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">Zakład: {{ praktyka.zaklad.nazwa }} | Ścieżka: {{
                praktyka.typ_sciezki.value }}</p>

            </div>

            <div style="text-align: right;">

                <span class="status status--oczekuje">Oczekiwanie na ocenę</span>

            </div>

        </div>

    </div>

    <form method="POST">

        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

        <!-- 13 Efektów uczenia się -->

        <section class="karta" style="margin-bottom: 2rem;">

            <header class="karta__naglowek">

                <h3 class="karta__tytul">Ocena 13 efektów uczenia się (Załącznik 4/4a)</h3>

            </header>

        </section>

    </form>

</div>
```

```

<div class="karta__tresc" style="padding: 0;">

  <div class="tabela-wrapper">

    <table class="tabela">

      <thead>

        <tr>

          <th scope="col" style="width: 60px;">Kod</th>

          <th scope="col">Opis efektu</th>

          <th scope="col" style="width: 280px;">Wynik</th>

        </tr>

      </thead>

      <tbody>

        {% for efekt in efekty %}

        {% set ocena = istniejace.get(efekt.id|string) %}

        <tr>

          <td><strong>{{ efekt.kod }}</strong></td>

          <td style="font-size: 0.85rem; line-height: 1.4;">{{ efekt.opis }}</td>

          <td>

            <div style="display: flex; gap: 0.75rem; flex-wrap: wrap;">

              <label class="radio-ocena">

                <input type="radio" name="wynik_{{ efekt.id }}" value="OSIAGNIETO" {% if ocena and ocena.wynik.name
== 'OSIAGNIETO' %}>checked{% endif %} required>

                <span>Osiągnięto</span>

              </label>

              {% if praktyka.typ_sciezki.value in ['ZATRUDNIENIE', 'DZIALALNOSC'] %}

              <label class="radio-ocena">

                <input type="radio" name="wynik_{{ efekt.id }}" value="CZESCIOWO" {% if ocena and ocena.wynik.name
== 'CZESCIOWO' %}>checked{% endif %}>

                <span>Częściowo</span>

              </label>

              {% endif %}

              <label class="radio-ocena">

                <input type="radio" name="wynik_{{ efekt.id }}" value="NIE_OSIAGNIETO" {% if ocena and
ocena.wynik.name == 'NIE_OSIAGNIETO' %}>checked{% endif %}>

                <span>Nie osiągnięto</span>

              </label>

            </div>

          </td>

        </tr>

        {% endfor %}

      </tbody>

    </table>

  </div>

</div>

```



```

        </label>

    </div>

    {% if praktyka.typ_sciezki.value in ['ZATRUDNIENIE', 'DZIALALNOSC'] %}

    <input type="text" name="uwagi_{{ efekt.id }}" value="{{ ocena.uwagi_oceniaczajacego if ocena else '' }}"

        placeholder="Uwagi (wymagane przy częściowym)"

        style="width: 100%; margin-top: 0.5rem; padding: 0.4rem; font-size: 0.8rem; border: 1px solid
var(--kolor-ramka); border-radius: 4px;">

        {% endif %}

    </td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

</section>

<!-- Oceny ogólne i opisowa -->

<div class="uklad-dashboardu" style="display: grid; grid-template-columns: 1fr 1fr; gap: 2rem;">

    <section class="karta">

        <header class="karta__naglowek">

            <h3 class="karta__tytul">Oceny parametryczne</h3>

        </header>

        <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 1.5rem;">

            <div class="pole-formularza">

                <label class="pole-formularza__etykieta">Ocena sprawozdania (S)</label>

                <select name="ocena_sprawozdania" class="filtr-select" style="width: 100%;">

                    <option value="">Wybierz...</option>

                    {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

                        <option value="{{ stopien }}" {% if praktyka.ocena_sprawozdania == stopien %>selected{% endif %}>{{ stopien
}}</option>

                    {% endfor %}

                </select>

            </div>

```

```

<div class="pole-formularza">

    <label class="pole-formularza__etykieta">Ocena Opiekuna Uczelnianego (U)</label>

    <select name="ocena_uopz" class="filtr-select" style="width: 100%;">

        <option value="">Wybierz...</option>

        {% for stopien in [2.0, 3.0, 3.5, 4.0, 4.5, 5.0] %}

            <option value="{{ stopien }}" {% if praktyka.ocena_uopz == stopien %}selected{% endif %}>{{ stopien
    }}</option>

        {% endfor %}

    </select>

</div>

</div>

</section>

<section class="karta">

    <header class="karta__naglowek">

        <h3 class="karta__tytul">Opinia opisowa UOPZ</h3>

    </header>

    <div class="karta__tresc">

        <div class="pole-formularza">

            <label for="ocena_opisowa_uopz" class="pole-formularza__etykieta">Uzasadnienie oceny i uwagi</label>

            <textarea id="ocena_opisowa_uopz" name="ocena_opisowa_uopz" class="pole-formularza__input" style="min-height:
120px; resize: vertical;">{{ praktyka.ocena_opisowa_uopz or '' }}</textarea>

        </div>

    </div>

</section>

</div>

<div style="margin-top: 2rem; display: flex; justify-content: center; padding-bottom: 4rem;">

    <button type="submit" class="przycisk przycisk--glowny" style="padding: 1rem 3rem; font-size: 1.1rem; box-shadow: 0
4px 14px rgba(26, 58, 92, 0.3);">

        Zapisz arkusz oceny

    </button>

</div>

</form>

```

```
</div>
```

```
<style>
```

```
.radio-ocena {  
  
  display: flex;  
  
  align-items: center;  
  
  gap: 0.4rem;  
  
  font-size: 0.85rem;  
  
  cursor: pointer;  
  
  background: var(--kolor-tlo);  
  
  padding: 0.3rem 0.6rem;  
  
  border-radius: 4px;  
  
  border: 1px solid var(--kolor-ramka);  
  
  transition: all 0.2s;  
  
}  
  
.radio-ocena:hover {  
  
  border-color: var(--kolor-glowny-jasny);  
  
}  
  
.radio-ocena input:checked + span {  
  
  font-weight: 600;  
  
  color: var(--kolor-glowny);  
  
}  
  
.radio-ocena input {  
  
  accent-color: var(--kolor-glowny);  
  
}
```

```
</style>
```

```
{% endblock %}
```

PLIK: app_admin\templates\evaluation\index.html

<h1>Evaluations</h1>

PLIK: app_admin\templates\evaluation\karta_ocen.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Karta Ocen Praktyki{% endblock %}

{% block naglowek %}Karta Ocen (Zał. 3 / Zał. 8){% endblock %}

{% block naglowek_akcje %}

<a href="{{ url_for('evaluation.lista_ocen') }}" class="przycisk przycisk--drugorzeczny">Powrót do listy</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 900px; margin: 0 auto;">

    <!-- Info o studencie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-akcent);">

        <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <h2 style="font-size: 1.1rem; margin-bottom: 0.25rem;">{{ zapis.student.imie }} {{ zapis.student.nazwisko }}</h2>

                <p style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

                    Firma: {{ zapis.firma_nazwa or 'Brak nazwy firmy' }} | ZOPZ: {{ zapis.zopz_imie_nazwisko or 'Brak ZOPZ' }} |

                    Godziny: {{ zapis.lacznie_godzin }} h

                </p>

            </div>

            <div style="text-align: right;">

                <span class="status status--{{ zapis.status.value|lower|replace('_', '-') }}">{{

                    tlumacz_status(zapis.status.value) }}</span>

            </div>

        </div>

    </div>

</div>

<form method="POST">

    {{ csrf_form.hidden_tag() }}

    <!-- Karta Ocen (Zał. 3) -->

    <div class="karta" style="margin-bottom: 2rem;">
```

```

<div class="karta__naglowek" style="background: var(--kolor-tlo);">

    <h3 class="karta__tytul">Karta oceny praktyki (Zař. 3)</h3>

</div>

<div class="karta__tresc">

    <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1rem;">

        Wprowadź oceny parametryczne. Ocenę ZOPZ przepis� z dostarczonego formularza papierowego podpisanej Karty nr 3.

    </p>

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; margin-bottom: 1.5rem;">

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Ocena Sprawozdania (S)</label>

            <input type="number" step="0.5" min="2" max="5" name="ocena_sprawozdania" class="pole-formularza__input ocen_field" value="{{ zapis.ocena_sprawozdania or '' }}" placeholder="2.0 - 5.0">

        </div>

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Ocena UOPZ (U)</label>

            <input type="number" step="0.5" min="2" max="5" name="ocena_uopz" class="pole-formularza__input ocen_field" value="{{ zapis.ocena_uopz or '' }}" placeholder="2.0 - 5.0">

        </div>

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Ocena z papierowej Karty ZOPZ (Z)</label>

            <input type="number" step="0.5" min="2" max="5" name="ocena_zopz" class="pole-formularza__input ocen_field" value="{{ zapis.ocena_zopz or '' }}" placeholder="2.0 - 5.0">

        </div>

    </div>

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem;">

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Opinia opisowa Uczelnianego Opiekuna Praktyk Zawodowych (UOPZ)</label>

            <textarea name="ocena_opisowa_uopz" class="pole-formularza__input" rows="4" placeholder="Wpisz treść opinii...">{{ zapis.ocena_opisowa_uopz or '' }}</textarea>

        </div>

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Opinia opisowa z dostarczonej karty ZOPZ (opcjonalnie)</label>

            <textarea name="ocena_opisowa_zopz" class="pole-formularza__input" rows="4" placeholder="Przepisz jeśli obecna...">{{ zapis.ocena_opisowa_zopz or '' }}</textarea>

        </div>

    </div>

```

</div>

</div>

</div>

</div>

<!-- Komisja Egzaminacyjna (Załącznik 8) -->

<div class="karta" style="margin-bottom: 2rem;">

<div class="karta__naglowek" style="background: var(--kolor-tlo);">

<h3 class="karta__tytul">Protokół Egzaminacyjny (Załącznik 8)</h3>

</div>

<div class="karta__tresc">

<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1.5rem;">

Wprowadź 3 pytania zadane studentowi oraz ocenę (od 2.0 do 5.0) za każdą odpowiedź. Ocena E jest automatyczną średnią z poniższych.

</p>

<div style="display: grid; grid-template-columns: 3fr 1fr; gap: 1rem; margin-bottom: 1rem; align-items: end;">

<div class="pole-formularza">

<label class="pole-formularza__etykieta">Pytanie 1</label>

<input type="text" name="sprawdzian_pytanie_1" class="pole-formularza__input" value="{ { zapis.sprawdzian_pytanie_1 or '' } }" placeholder="Treść pytania...">

</div>

<div class="pole-formularza">

<label class="pole-formularza__etykieta">Ocena z odp. 1</label>

<input type="number" step="0.5" min="2" max="5" name="sprawdzian_ocena_1" class="pole-formularza__input egzamin_odp" value="{ { zapis.sprawdzian_ocena_1 or '' } }" placeholder="2.0 - 5.0">

</div>

</div>

<div style="display: grid; grid-template-columns: 3fr 1fr; gap: 1rem; margin-bottom: 1rem; align-items: end;">

<div class="pole-formularza">

<label class="pole-formularza__etykieta">Pytanie 2</label>

<input type="text" name="sprawdzian_pytanie_2" class="pole-formularza__input" value="{ { zapis.sprawdzian_pytanie_2 or '' } }" placeholder="Treść pytania...">

</div>

<div class="pole-formularza">

```

<label class="pole-formularza__etykieta">Ocena z odp. 2</label>

    <input type="number" step="0.5" min="2" max="5" name="sprawdzian_ocena_2" class="pole-formularza__input
egzamin_odp" value="{{ zapis.sprawdzian_ocena_2 or '' }}" placeholder="2.0 - 5.0">

</div>

</div>

<div style="display: grid; grid-template-columns: 3fr 1fr; gap: 1rem; align-items: end;">

    <div class="pole-formularza">

        <label class="pole-formularza__etykieta">Pytanie 3</label>

            <input type="text" name="sprawdzian_pytanie_3" class="pole-formularza__input" value="{{
zapis.sprawdzian_pytanie_3 or '' }}" placeholder="Treść pytania...">

        </div>

        <div class="pole-formularza">

            <label class="pole-formularza__etykieta">Ocena z odp. 3</label>

            <input type="number" step="0.5" min="2" max="5" name="sprawdzian_ocena_3" class="pole-formularza__input
egzamin_odp" value="{{ zapis.sprawdzian_ocena_3 or '' }}" placeholder="2.0 - 5.0">

        </div>

    </div>

</div>

</div>

</div>

<!-- Podsumowanie Wyników -->

<div class="karta" style="margin-bottom: 2rem; border-top: 4px solid var(--kolor-akcent);">

    <div class="karta__tresc" style="display: flex; justify-content: space-around; align-items: center; padding:
2rem;">

        <div style="text-align: center;">

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); text-transform: uppercase;">Średnia Egzaminu
(E)</div>

            <div id="wynik-e" style="font-size: 2.5rem; font-weight: 700; color: var(--kolor-glowny);">--</div>

        </div>

        <div style="font-size: 1.5rem; color: var(--kolor-ramka);">+</div>

        <div style="text-align: center;">

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); text-transform: uppercase;">Wzór
Obliczeniowy</div>

```



```

<div style="font-family: monospace; font-size: 1.1rem; font-weight: 500; margin-top: 0.5rem; color: #444;">K =
0.4*E + 0.1*S + 0.2*U + 0.3*Z</div>

</div>

<div style="font-size: 1.5rem; color: var(--kolor-ramka);">=</div>

<div style="text-align: center;">

    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); text-transform: uppercase;">Ocena Końcowa
(K)</div>

    <div id="wynik-k" style="font-size: 2.5rem; font-weight: 700; color: var(--kolor-glowny);">—</div>

</div>

</div>

<!-- Akcje -->

<div style="display: flex; justify-content: space-between; align-items: center; padding-bottom: 3rem;">

    <span style="font-size: 0.85rem; color: var(--kolor-blad);" id="ostrzezenie-z">Obliczenie K i zatwierdzenie jest
zablokowane do momentu wpisania oceny "Z".</span>

    <div style="display: flex; gap: 1rem; flex-wrap: wrap;">

        <button type="submit" name="zapisz" value="1" class="przycisk przycisk--drugorzecny" style="box-shadow: 0 2px 8px
rgba(0,0,0,0.1);">Zapisz Wersję Roboczą</button>

        <button type="submit" name="zakoncz" value="1" id="btn-zakoncz" class="przycisk przycisk--glowny"
style="box-shadow: 0 4px 14px rgba(26, 58, 92, 0.3);" disabled>Zakończ i Zatwierdź Ocenę</button>

        {% if zapis.ocena_uopz is not none %}

        <a href="{% url_for('evaluation.generuj_protokol', id=zapis.id) %}"

            class="przycisk przycisk--drugorzecny" style="box-shadow: 0 2px 8px rgba(0,0,0,0.1);">

                Pobierz Protokół (Zał. 8)

        </a>

        {% endif %}

    </div>

</div>

</div>

</form>

</div>

```

```
{% block scripts_extra %}  
  
<script>  
  
document.addEventListener('DOMContentLoaded', () => {  
  
    const p1 = document.querySelector('[name="sprawdzian_ocena_1"]');  
  
    const p2 = document.querySelector('[name="sprawdzian_ocena_2"]');  
  
    const p3 = document.querySelector('[name="sprawdzian_ocena_3"]');  
  
  
    const os = document.querySelector('[name="ocena_sprawozdania"]');  
  
    const ou = document.querySelector('[name="ocena_uopz"]');  
  
    const oz = document.querySelector('[name="ocena_zopz"]');  
  
  
  
    const we = document.getElementById('wynik-e');  
  
    const wk = document.getElementById('wynik-k');  
  
    const btnZakoncz = document.getElementById('btn-zakoncz');  
  
    const ostrzezenie = document.getElementById('ostrzezenie-z');  
  
  
  
    function obl() {  
  
        let e1 = parseFloat(p1.value) || 0;  
  
        let e2 = parseFloat(p2.value) || 0;  
  
        let e3 = parseFloat(p3.value) || 0;  
  
  
  
        let s = parseFloat(os.value) || 0;  
  
        let u = parseFloat(ou.value) || 0;  
  
        let z = parseFloat(oz.value) || 0;  
  
  
  
  
        let exam_avg = 0;  
  
        let numPyt = 0;  
  
        if(e1>0) numPyt++;  
  
        if(e2>0) numPyt++;  
  
        if(e3>0) numPyt++;  
  
        if(numPyt > 0) exam_avg = (e1+e2+e3)/numPyt;  
  
  
  
        we.textContent = exam_avg > 0 ? exam_avg.toFixed(2) : '-';  
  
    }  
  
    btnZakoncz.addEventListener('click', () => {  
        ostrzezenie.textContent = 'Zakończ test';  
        obl();  
    });  
  
});
```

```

if(exam_avg > 0 && s > 0 && u > 0 && z > 0) {

    let k = (0.4 * exam_avg) + (0.1 * s) + (0.2 * u) + (0.3 * z);

    wk.textContent = k.toFixed(2);

    wk.style.color = k >= 3.0 ? 'var(--kolor-sukces)' : 'var(--kolor-blad)';

    btnZakoncz.disabled = false;

    ostrzezenie.style.display = 'none';

} else {

    wk.textContent = '-';

    wk.style.color = 'var(--kolor-glowny)';

    btnZakoncz.disabled = true;

    ostrzezenie.style.display = 'inline-block';

}

}

[p1, p2, p3, os, ou, oz].forEach(el => el.addEventListener('input', obl));

obl();

});

</script>

{% endblock %}

{% endblock %}

```

PLIK: app_admin\templates\evaluation\lista_ocen.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Oceny i efekty{% endblock %}

{% block naglowek %}Studenci – oceny i efekty{% endblock %}


{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto;">


{% macro tabela_studentow(items, tytul, kolor) %}

<div class="karta" style="margin-bottom: 1.5rem;">

    <div class="karta__naglowek" style="border-left: 4px solid {{ kolor }};">

        <h2 class="karta__tytul">{{ tytul }} <span style="font-size:0.85rem; font-weight:400;
color:var(--kolor-tekst-drugor);">({{ items|length }})</span></h2>

    </div>

    <div class="karta__tresc">

        {% if items %}

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>

                    <tr>

                        <th>Student</th>

                        <th>Kierunek / Specjalność</th>

                        <th>Zakład pracy</th>

                        <th>Termin</th>

                        <th>Godziny</th>

                        <th>Akcje</th>

                    </tr>

                </thead>

                <tbody>

                    {% for item in items %}

                    {% set p = item.zapis %}

                    <tr {% if item.przekroczony %}style="background: rgba(220,38,38,0.04);" {% endif %}>

                        <td>
```

```

<div style="font-weight:600;">{{ p.student.last_name }} {{ p.student.first_name }}</div>

<div style="font-size:0.8rem; color:var(--kolor-tekst-drugor);">

    Nr alb.: {{ p.student.album_number or '-' }}

    {% if p.student.plec %} · {{ 'K' if p.student.plec == 'K' else 'M' }}{% endif %}

</div>

</td>

<td style="font-size:0.85rem;">

    {{ p.student.kierunek or 'Informatyka' }}<br>

    <span style="color:var(--kolor-tekst-drugor);">{{ p.student.specjalnosc or p.specjalnosc or '-' }}</span>

</td>

<td style="font-size:0.85rem;">

    {{ (p.firma.nazwa if p.firma else p.firma_nazwa) or '-' }}

</td>

<td style="font-size:0.85rem;">

    {{ p.termin_od.strftime('%d.%m.%Y') if p.termin_od else '-' }} -

    {{ p.termin_do.strftime('%d.%m.%Y') if p.termin_do else '-' }}

    {% if item.deadline %}

        <div style="font-size:0.75rem; {% if item.przekroczony %}color:var(--kolor-blad); font-weight:600;{% elif
item.dni_do_deadline <= 2 %}color:#d97706;{% else %}color:var(--kolor-tekst-drugor);{% endif %}">

            {% if item.przekroczony %}Deadline przekroczony o {{ -item.dni_do_deadline }} dni

            {% elif item.dni_do_deadline == 0 %}Dziś ostatni dzień!

            {% else %}Deadline za {{ item.dni_do_deadline }} dni{% endif %}

        </div>

    {% endif %}

</td>

<td><strong>{{ p.total_hours_logged or 0 }} h</strong></td>

<td>

    <a href="{{ url_for('evaluation.ocen_praktyke', id=p.id) }}"

        class="przycisk przycisk--maly {% if item.przekroczony %}przycisk--niebezpieczny{% elif item.zakonczone
%}przycisk--glowny{% else %}przycisk--drugorzedy{% endif %}">

        {% if item.przekroczony %}PILNE: Ocen{% elif item.zakonczone %}Wystaw ocenę{% else %}Podgląd{% endif %}

    </a>

</td>

</tr>

{% endfor %}

```

```
</tbody>

</table>

</div>

{% else %}

<div style="text-align:center; padding:2rem; color:var(--kolor-tekst-drugor);">Brak studentów.</div>

{% endif %}

</div>

</div>

{% endmacro %}


{{ tabela_studentow(zakonczone, 'Gotowi do oceny (zakończzone praktyki)', '#dc2626') }}

{{ tabela_studentow(w_trakcie, 'W trakcie praktyki', '#3b82f6') }}


</div>

{% endblock %}
```

PLIK: app_admin\templates\evaluation\podglad_sprawozdania.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Podgląd Sprawozdania{% endblock %}

{% block naglowek %}Sprawozdanie: {{ zapis.student.imie }} {{ zapis.student.nazwisko }}{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.szczegoly_praktyki', id=zapis.internship_id) %}" class="przycisk przycisk--drugorzeczny">Powrót do szczegółów</a>

<a href="{% url_for('evaluation.ocen_praktyke', id=zapis.id) %}" class="przycisk przycisk--glowny">Przejdź do oceniania</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 900px; margin: 0 auto; padding-bottom: 3rem;">

    {% if not zapis.sprawozdanie %}

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

        <p style="color: var(--kolor-tekst-drugor);">Student jeszcze nie rozpoczął generowania / pisania sprawozdania.</p>

    </div>

    {% else %}

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__naglowek" style="background: var(--kolor-tlo);">

            <h2 class="karta__tytul">Sekcja I: Charakterystyka miejsca roboczego</h2>

        </div>

        <div class="karta__tresc">

            <div style="white-space: pre-wrap; font-size: 0.95rem; line-height: 1.6;">{{ zapis.sprawozdanie.charakterystyka_miejsca or 'Brak' }}</div>

        </div>

    </div>

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__naglowek" style="background: var(--kolor-tlo);">

            <h2 class="karta__tytul">Sekcja II: Opis i analiza prac</h2>
```

```
</div>

<div class="karta__tresc">

    <div style="white-space: pre-wrap; font-size: 0.95rem; line-height: 1.6;">{{ zapis.sprawozdanie.opis_i_analiza or
'Brak' }}</div>

</div>

</div>


<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek" style="background: var(--kolor-tlo);">

        <h2 class="karta__tytul">Seksja III (Z Dziennika)</h2>

    </div>

    <div class="karta__tresc">

        <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Tabela zliczająca godziny dla każdego efektu z
dziennika (w PDF).</p>

        <a href="{{ url_for('journal.podglad', id=zapis.id) }}" class="przycisk przycisk--drugorzecny przycisk--maly"
style="margin-top: 1rem;">Zobacz pełny dziennik wpisów</a>

    </div>

</div>


{% endif %}


</div>

{% endblock %}
```


PLIK: app_admin\templates\journal\dziennik.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dziennik praktyki{% endblock %}

{% block naglowek %}Dziennik praktyki{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    <a href="{{ url_for('journal.lista_dziennikow') }}" class="przycisk przycisk--drugorzeczny">Powrót do listy</a>

    <button id="btn-pdf" onclick="zlecPDF()" class="przycisk przycisk--glowny">

        <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"
stroke-linecap="round" stroke-linejoin="round" style="vertical-align: -1px;"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0
1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>

        <span id="btn-pdf-label">Pobierz PDF</span>

    </button>

    <span id="pdf-status" style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); display: none;"></span>

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 1000px; margin: 0 auto;">

    <!-- Info o zapisie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-glowny);">

        <div class="karta__tresc">

            <div style="display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 1.5rem;">

                <div>

                    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom:
0.25rem;">Student</div>

                    <div style="font-weight: 600;">{{ zapis.student.first_name }} {{ zapis.student.last_name }}</div>

                    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ zapis.student.album_number or '-'
}}</div>

                </div>

                <div>


```

```

        <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Praktyka</div>

        <div style="font-weight: 600;">{{ zapis.praktyka.rok_uczelnianny }}</div>

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ zapis.praktyka.semestr }}</div>

    </div>

    <div>

        <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Postęp godzin</div>

        <div style="font-weight: 700; font-size: 1.2rem; color: var(--kolor-glowny);">{{ suma_godzin }} / {{ zapis.praktyka.wymiar_godzin }} h</div>

        <div class="pasek-postep" style="margin-top: 0.5rem; height: 8px;">

            <div class="pasek-postep__wypelnienie {{ 'pasek-postep__wypelnienie--zakonczone' if suma_godzin >= zapis.praktyka.wymiar_godzin else '' }}"

                style="width: {{ [(suma_godzin / zapis.praktyka.wymiar_godzin) * 100]|round|int, 100]|min }}%;"></div>

            </div>

        </div>

    </div>

</div>

</div>

</div>

<!-- Alerty przerw -->

{% if przerwy %}

<div style="margin-bottom: 2rem;">

    {% for p in przerwy %}

        <div class="komunikat komunikat--danger" style="margin-bottom: 0.5rem;">

            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" style="flex-shrink: 0; margin-top: 2px;">

                <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/>

                <line x1="12" y1="9" x2="12" y2="13"/>

                <line x1="12" y1="17" x2="12.01" y2="17"/>

            </svg>

            <span>Przerwa <strong>{{ p.dni_roboczych }} dni roboczych</strong> między {{ p.od.strftime('%d.%m')} } a {{ p.do.strftime('%d.%m.%Y')} }</span>

        </div>

    {% endfor %}

</div>

{% endif %}

```

```

<!-- Wpisy -->

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Wpisy dziennika ({{ wpisy|length }})</h3>

    </div>

    <div class="karta__tresc" style="padding: 0;">

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>

                    <tr>

                        <th scope="col" style="width: 110px;">Data</th>

                        <th scope="col" style="width: 80px;">Godziny</th>

                        <th scope="col">Opis czynności</th>

                        <th scope="col" style="width: 70px;">Efekt</th>

                    </tr>

                </thead>

                <tbody>

                    {% for w in wpisy %}

                    <tr>

                        <td><strong>{{ w.entry_date.strftime('%d.%m.%Y') }}</strong></td>

                        <td style="text-align: center;">{{ w.duration_hours }} h</td>

                        <td style="font-size: 0.85rem; line-height: 1.5;">{{ w.description }}</td>

                        <td style="white-space: nowrap; text-align: center;">

                            {% if w.efekty_uczenia %}

                                {% for e in w.efekty_uczenia %}

                                    <span style="font-size: 0.75rem; background: var(--kolor-tlo); padding: 0.2rem 0.4rem; border-radius: 4px; font-weight: 600;">{{ e.kod }}</span>

                                {% endfor %}

                            {% else %}—{% endif %}

                        </td>

                    </tr>

                    {% else %}

                    <tr>

                        <td colspan="4" class="tabela--pusta">Student nie dodał jeszcze żadnych wpisów.</td>

```

```

        </tr>

        {% endfor %}

    </tbody>

</table>

</div>

</div>

</div>

</div>

<script>

const ZAPIS_ID = "{{ zapis.id }}";

const URL_ZLEC = `/dzienniki/zapis/${ZAPIS_ID}/pdf`;

const URL_STATUS = (taskId) => `/dzienniki/zapis/${ZAPIS_ID}/pdf/status/${taskId}`;

const URL_POBIERZ = (taskId) => `/dzienniki/zapis/${ZAPIS_ID}/pdf/pobierz/${taskId}`;


const btn = document.getElementById('btn-pdf');

const btnLabel = document.getElementById('btn-pdf-label');

const statusEl = document.getElementById('pdf-status');


let pollInterval = null;


function zlecPDF() {

    btn.disabled = true;

    btnLabel.textContent = 'Generowanie...';

    statusEl.style.display = 'inline';

    statusEl.textContent = 'Zlecam generowanie PDF...';


    fetch(URL_ZLEC, {

        method: 'POST',

        headers: { 'Content-Type': 'application/json', 'X-CSRFToken': getCsrf() },

    })

    .then(r => r.json())

    .then(data => {

```

```

    if (data.task_id) {

        statusEl.textContent = 'Kompilacja w toku...';

        pollInterval = setInterval(() => sprawdzStatus(data.task_id), 2000);

    } else {

        blad('Błąd zlecenia: ' + (data.error || 'nieznany'));

    }

})

.catch(() => blad('Błąd połączenia z serwerem.'));

}

```

```

function sprawdzStatus(taskId) {

    fetch(URL_STATUS(taskId))

    .then(r => r.json())

    .then(data => {

        if (data.status === 'SUCCESS') {

            clearInterval(pollInterval);

            statusEl.textContent = 'Gotowe!';

            btnLabel.textContent = 'Pobierz PDF';

            btn.disabled = false;

            // Automatyczne pobranie

            window.location.href = URL_POBIERZ(taskId);

        } else if (data.status === 'FAILURE') {

            clearInterval(pollInterval);

            blad('Błąd kompilacji: ' + (data.error || 'sprawdź logi workera'));

        } else {

            const progress = data.progress || 0;

            statusEl.textContent = `Kompilacja... ${progress}%`;

        }

    })

    .catch(() => {});

}

```

```

function blad(msg) {

    clearInterval(pollInterval);

}

```

```
btn.disabled = false;

btnLabel.textContent = 'Pobierz PDF';

statusEl.textContent = msg;

statusEl.style.color = 'var(--kolor-blad)';

}


function getCsrftoken() {

    const meta = document.querySelector('meta[name="csrf-token"]');

    return meta ? meta.getAttribute('content') : '';

}

</script>

{% endblock %}
```

PLIK: app_admin\templates\journal\lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dzienniki praktyk{% endblock %}

{% block naglowek %}Monitor dzienników{% endblock %}

{% block tresc %}

<div class="karta">

    <div class="karta__naglowek">

        <form class="pasek-filtrow" method="GET" action="{{ url_for('journal.lista_dziennikow') }}">

            <div class="filtr-szukaj">

                <span class="filtr-szukaj__ikona" aria-hidden="true"><svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="11" cy="11"
r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg></span>

                <input type="text" name="szukaj" value="{{ request.args.get('szukaj', '') }}" class="filtr-szukaj__input"
placeholder="Szukaj po nazwisku studenta..." aria-label="Wyszukaj dziennik">

            </div>

            <button type="submit" class="przycisk przycisk--drugorzedny">Szukaj</button>

        </form>

    </div>

    <div class="karta__tresc">

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>

                    <tr>

                        <th scope="col">Student</th>

                        <th scope="col">Praktyka</th>

                        <th scope="col">Godziny</th>

                        <th scope="col">Wpisy</th>

                        <th scope="col">Ostatni wpis</th>

                        <th scope="col">Status</th>

                        <th scope="col">Akcje</th>

                    </tr>

                </thead>

                <tbody>
```

```

{% for d in dane %}

<tr {% if d.alert %}style="background: rgba(192, 57, 43, 0.05);"{% endif %}>

    <td>

        <div style="font-weight: 600;">{{ d.zapis.student.last_name }} {{ d.zapis.student.first_name }}</div>

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ d.zapis.student.album_number or
'-' }}</div>

    </td>

    <td>

        <div>{{ d.zapis.praktyka.rok_uczelnianny }}</div>

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ d.zapis.praktyka.semestr | capitalize
}}</div>

    </td>

    <td>

        <div style="font-weight: 600;">{{ d.zapis.total_hours_logged }} / {{ d.zapis.praktyka.wymiar_godzin }}

h</div>

        <div class="pasek-postep" style="margin-top: 4px;">

            <div class="pasek-postep__wypelnienie {% if 'pasek-postep__wypelnienie--zakonczone' if
d.zapis.total_hours_logged >= d.zapis.praktyka.wymiar_godzin else '' %}"

                style="width: {{ [(d.zapis.total_hours_logged / d.zapis.praktyka.wymiar_godzin) * 100]|round|int,
100]|min }}%;"></div>

            </div>

        </td>

    <td><strong>{{ d.liczba_wpisow }}</strong></td>

    <td>

        {% if d.ostatni_wpis %}

            {{ d.ostatni_wpis.strftime('%d.%m.%Y') }}

        {% else %}

            <span style="color: var(--kolor-tekst-trzeciorzed);">Brak wpisów</span>

        {% endif %}

    </td>

    <td>

        {% if d.alert %}

            <span class="status" style="background: #fef2f2; color: var(--kolor-blad);">

                Przerwa &gt;3 dni

            </span>

        {% else %}

            <span class="status status--in-progress">OK</span>


```



```

        {% endif %}

    </td>

    <td>

        <a href="{{ url_for('journal.dziennik_zapisu', id=d.zapis.id) }}" class="przycisk przycisk--drugorzedny przycisk--maly">Podgląd</a>

    </td>

</tr>

{% else %}

<tr>

    <td colspan="7" class="tabela--pusta">Brak aktywnych dzienników.</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

```

```

{% if zapisy.pages > 1 %}

<nav class="paginacja" aria-label="Nawigacja po stronach">

    <span class="paginacja__info">Strona {{ zapisy.page }} z {{ zapisy.pages }}</span>

    <div class="paginacja__przyciski">

        {% for strona in zapisy.iter_pages() %}

            {% if strona %}

                <a href="{{ url_for('journal.lista_dziennikow', strona=strona, szukaj=request.args.get('szukaj', '')) }}"

                    class="paginacja__przycisk {{ 'paginacja__przycisk--aktywny' if strona == zapisy.page else '' }}"

                    {{ strona }}

                </a>

            {% else %}

                <span class="paginacja__separator">...</span>

            {% endif %}

        {% endfor %}

    </div>

</nav>

{% endif %}

</div>

</div>

```

{% endblock %}

PLIK: app_admin\templates\layouts\base.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <meta name="csrf-token" content="{ { csrf_token() } }">

    <title>{% block tytuł %}{% endblock %} – System Praktyk ANS Elbląg</title>

    <link rel="stylesheet" href="{ { url_for('static', filename='css/admin.css') } }">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono&display=swap"
rel="stylesheet">

</head>

<body class="{% block body_class %}{% endblock %}">

    <a href="#main-content" class="skip-link">Przejdź do treści głównej</a>


    {% block body %}{% endblock %}


    {% block scripts_extra %}{% endblock %}

</body>

</html>
```

PLIK: app_admin\templates\layouts\base_panel.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-scale=1.0">

    <meta name="description" content="Panel administracyjny – System Praktyk Zawodowych IIS ANS Elbląg">

    <title>{% block tytuł %}Panel{% endblock %} – IIS ANS Elbląg</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/admin.css') }}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono&display=swap"
rel="stylesheet">

    {% block head_extra %}{% endblock %}

</head>

<body class="panel-body">


    <a href="#main-content" class="skip-link">Przejdź do głównej treści</a>


    <nav class="sidebar" aria-label="Nawigacja główna panelu">

        <div class="sidebar__naglowek">

            <div class="branding__logo-kontener" style="max-width: 140px; margin-bottom: 10px;">

                

            </div>

            <span class="sidebar__system-label">Panel administracyjny</span>

        </div>


        <div class="sidebar__uzytkownik" aria-label="Zalogowany użytkownik">

            <div class="sidebar__uzytkownik-avatar" aria-hidden="true">

                {% if current_user.is_authenticated %}

                    {{ current_user.imie[0] }}{{ current_user.nazwisko[0] }}

                {% else %}

                    ??

                {% endif %}

            </div>

            <div class="sidebar__uzytkownik-info">
```

```

{% if current_user.is_authenticated %}

<span class="sidebar__uzytkownik-imie">{{ current_user.imie }} {{ current_user.nazwisko }}</span>

<span class="sidebar__uzytkownik-rola">

    {% if current_user.is_authenticated and current_user.rola and current_user.rola.value == 'ADMIN'
}%Administrator

    {% elif current_user.is_authenticated and current_user.rola and current_user.rola.value == 'UOPZ' %}Opiekun
uczelnianny

    {% endif %}

</span>

{% else %}

<span class="sidebar__uzytkownik-imie">Niezalogowany</span>

{% endif %}

</div>

</div>

<ul class="sidebar__menu" role="list">

    <li class="sidebar__menu-pozycja">

        <a href="{{ url_for('dashboard.index') }}" class="sidebar__link {% if request.endpoint == 'dashboard.index'
}%sidebar__link--aktywny{% endif %}" {% if request.endpoint == 'dashboard.index' %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><rect x="3" y="3" width="7"
height="7"/><rect x="14" y="3" width="7" height="7"/><rect x="3" y="14" width="7" height="7"/><rect x="14" y="14"
width="7" height="7"/></svg></span> Przegląd

        </a>

    </li>

    <li class="sidebar__menu-sekcja" aria-hidden="true">Praktyki</li>

    <li class="sidebar__menu-pozycja">

        <a href="{{ url_for('zarzadzanie.lista_praktyk') }}" class="sidebar__link {% if 'lista_praktyk' in
(request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'lista_praktyk' in (request.endpoint or '')
}%aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M16 4h2a2 2 0 0 1 2 2v14a2 2
0 0 1-2 2H6a2 2 0 0 1-2-2V6a2 2 0 0 1 2-2h2"/><rect x="8" y="2" width="8" height="4" rx="1" ry="1"/><line x1="9" y1="12"
x2="15" y2="12"/><line x1="9" y1="16" x2="15" y2="16"/></svg></span> Wszystkie praktyki

        </a>

    </li>

    {% if current_user.is_authenticated and current_user.rola and current_user.rola.value == 'ADMIN' %}

    <li class="sidebar__menu-pozycja">

        <a href="{{ url_for('zarzadzanie.lista_zgloszen') }}" class="sidebar__link {% if 'zgloszenia' in
(request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'zgloszenia' in (request.endpoint or '')
}%aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2

```

```

0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14,2 14,8 20,8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16"
y1="17" x2="8" y2="17"/><polyline points="10,9 9,9 8,9"/></svg></span> Zgłoszenia studentów{% if nav_oczekujace %} <span
style="display:inline-flex;align-items:center;justify-content:center;background:#ef4444;color:white;border-radius:9999px;
min-width:1.25rem;height:1.25rem;font-size:0.65rem;font-weight:700;padding:0
0.25rem;margin-left:0.35rem;">{{
nav_oczekujace }}</span>{% endif %}

</a>

</li>

{% elif current_user.is_authenticated and current_user.rola and current_user.rola.value == 'UOPZ' %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.moje_zgloszenia') }}" class="sidebar__link {% if 'moje_zgloszenia' in
(request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'moje_zgloszenia' in (request.endpoint or '')
%}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2
0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14,2 14,8 20,8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16"
y1="17" x2="8" y2="17"/><polyline points="10,9 9,9 8,9"/></svg></span> Moje zgłoszenia

    </a>

</li>

{% endif %}

{% if current_user.is_authenticated and current_user.rola and current_user.rola.value in ['ADMIN', 'UOPZ'] %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('evaluation.lista_ocen') }}" class="sidebar__link {% if 'evaluation' in (request.endpoint or
'') %}sidebar__link--aktywny{% endif %}" {% if 'evaluation' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M9 11l3 3L22 4"/><path d="M21
12v7a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11"/></svg></span> Oceny i efekty

    </a>

</li>

{% endif %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('journal.lista_dziennikow') }}" class="sidebar__link {% if 'journal' in (request.endpoint or
'') %}sidebar__link--aktywny{% endif %}" {% if 'journal' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5
17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/><line x1="9" y1="7" x2="16"
y2="7"/><line x1="9" y1="11" x2="16" y2="11"/><line x1="9" y1="15" x2="13" y2="15"/></svg></span> Dzienniki

    </a>

</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.dokumenty_studentow') }}" class="sidebar__link {% if 'dokumenty_studentow' in
(request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'dokumenty_studentow' in (request.endpoint or '')
%}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><rect x="9" y="9" width="13" height="13" rx="2"/><path
d="M5 15H4a2 2 0 0 1-2-2V4a2 2 0 0 1 2-2h9a2 2 0 0 1 2 2v1"/></svg></span> Dokumenty studentów

    </a>

```

```

</li>

{% if current_user.is_authenticated and current_user.rola and current_user.rola.value in ['ADMIN', 'UOPZ'] %}

<li class="sidebar__menu-sekcja" aria-hidden="true">Weryfikacja</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.komisja_lista') }}" class="sidebar__link {% if 'komisja' in (request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'komisja' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M9 12l2 4-4"/><path d="M21 12c0 4.97-4.03 9-9 9s-9-4.03-9-9 4.03-9 9-9c.83 0 1.63.11 2.38.32"/></svg></span> Komisja weryfikująca{% if nav_komisja %}
        <span style="display:inline-flex;align-items:center;justify-content:center;background:#ef4444;color:white;border-radius:9999px;min-width:1.25rem;height:1.25rem;font-size:0.65rem;font-weight:700;padding:0 0.25rem;margin-left:0.35rem;">{% nav_komisja %}</span>{% endif %}

    </a>

</li>

{% endif %}

{% if current_user.is_authenticated and current_user.rola and current_user.rola.value == 'ADMIN' %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.dziekan_lista') }}" class="sidebar__link {% if 'dziekan' in (request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'dziekan' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M12 21l3.09 6.26L22 9.27l-5 4.87 1.18 6.88L12 17.77l-6.18 3.25L7 14.14 2 9.27l6.91-1.01L12 2z"/></svg></span> Decyzje dziekana{% if nav_dziekan %}
        <span style="display:inline-flex;align-items:center;justify-content:center;background:#ef4444;color:white;border-radius:9999px;min-width:1.25rem;height:1.25rem;font-size:0.65rem;font-weight:700;padding:0 0.25rem;margin-left:0.35rem;">{% nav_dziekan %}</span>{% endif %}

    </a>

</li>

<li class="sidebar__menu-sekcja" aria-hidden="true">Zarządzanie</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="sidebar__link {% if 'uzytkownik' in (request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'uzytkownik' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1 0 7.75"/></svg></span> Użytkownicy

    </a>

</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.lista_firm') }}" class="sidebar__link {% if 'firma' in (request.endpoint or '') %}sidebar__link--aktywny{% endif %}" {% if 'firma' in (request.endpoint or '') %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24"><rect x="4" y="2" width="16" height="20" rx="2" ry="2"/><line x1="9" y1="6" x2="9" y2="6.01"/><line x1="15" y1="6" x2="15" y2="6.01"/><line x1="9" y1="10" x2="9" y2="10.01"/><line x1="15" y1="10" x2="15" y2="10.01"/><line x1="9" y1="14" x2="9" y2="14.01"/><line x1="15" y1="14" x2="15" y2="14.01"/><path d="M9 18h6"/></svg></span> Firmy


```

```

        </a>

    </li>

{% endif %}

</ul>

<div class="sidebar__stopka">

    <form method="POST" action="{% url_for('auth.wylogowanie') %}">

        <input type="hidden" name="csrf_token" value="{% csrf_token() %}">

        <button type="submit" class="sidebar__wylogowanie">

            <svg viewBox="0 0 24 24"><path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4"/><polyline points="16 17 21 12 16 7"/><line x1="21" y1="12" x2="9" y2="12"/></svg> Wyloguj się

        </button>

    </form>

</div>

</nav>

<div class="panel-main">

    <header class="panel-main__naglowek">

        <div class="panel-main__naglowek-tresc">

            <h1 class="panel-main__tytul">{% block naglowek %}{% endblock %}</h1>

            {% block naglowek_akcje %}{% endblock %}

        </div>

        {# flash messages moved to bottom of the page for minimal display #}

    </header>

    <main id="main-content" class="panel-main__tresc" tabindex="-1">

        {% block tresc %}{% endblock %}

    </main>

</div>

{% with wiadomosci = get_flashed_messages(with_categories=True) %}

{% if wiadomosci %}

    <div class="komunikaty komunikaty--fixed" role="status" aria-live="polite">

        {% for kategoria, wiadomosc in wiadomosci %}

            <div class="komunikat komunikat--{% kategoria %} komunikat--minimal" role="alert">

                <span class="komunikat__ikona" aria-hidden="true">

```



```
        {% if kategoria == 'danger' %}{% elif kategoria == 'success' %}{% else %}{% endif %}

    </span>

    <span class="komunikat__tekst">{{ wiadomosc }}</span>

        <button type="button" class="komunikat__zamknij" aria-label="Zamknij powiadomienie"
onclick="this.parentElement.remove()">x</button>

    </div>

    {% endfor %}

</div>

{% endif %}

{% endwith %}

</body>

</html>
```

PLIK: app_admin\templates\layouts\panel.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-scale=1.0">

    <meta name="description" content="Panel administracyjny – System Praktyk Zawodowych IIS ANS Elbląg">

    <title>{% block tytuł %}Panel{% endblock %} – IIS ANS Elbląg</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/admin.css') }}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono&display=swap"
rel="stylesheet">

    {% block head_extra %}{% endblock %}

</head>

<body class="panel-body">

    <a href="#main-content" class="skip-link">Przejdź do głównej treści</a>

    <nav class="sidebar" aria-label="Nawigacja główna panelu">

        <div class="sidebar__naglowek">

            <div class="branding__logo-kontener" style="max-width: 140px; margin-bottom: 10px;">

                

            </div>

            <span class="sidebar__system-label">Panel administracyjny</span>

        </div>

        <div class="sidebar__uzytkownik" aria-label="Zalogowany użytkownik">

            <div class="sidebar__uzytkownik-avatar" aria-hidden="true">

                {{ current_user.imie[0] }}{{ current_user.nazwisko[0] }}

            </div>

            <div class="sidebar__uzytkownik-info">

                <span class="sidebar__uzytkownik-imie">{{ current_user.imie }} {{ current_user.nazwisko }}</span>

                <span class="sidebar__uzytkownik-rola">

                    {% if current_user.rola.value == 'ADMIN' %}Administrator

                    {% elif current_user.rola.value == 'UOPZ' %}Opiekun uczelniany
```

```

        {% elif current_user.rola.value == 'ZOPZ' %}Opiekun zakładowy

        {% endif %}

    </span>

</div>

</div>

<ul class="sidebar__menu" role="list">

    <li class="sidebar__menu-pozycja">

        <a href="{% url_for('dashboard.index') %}" class="sidebar__link {% if request.endpoint == 'dashboard.index' %}sidebar__link--aktywny{% endif %}" {% if request.endpoint == 'dashboard.index' %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"></span> Przegląd

        </a>

    </li>

    <li class="sidebar__menu-sekcja" aria-hidden="true">Praktyki</li>

    <li class="sidebar__menu-pozycja">

        <a href="{% url_for('zarzadzanie.lista_praktyk') %}" class="sidebar__link {% if 'lista_praktyk' in request.endpoint %}sidebar__link--aktywny{% endif %}" {% if 'lista_praktyk' in request.endpoint %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"></span> Wszystkie praktyki

        </a>

    </li>

    {% if current_user.rola.value in ['ADMIN', 'UOPZ'] %}

    <li class="sidebar__menu-pozycja">

        <a href="{% url_for('evaluation.lista_ocen') %}" class="sidebar__link {% if 'evaluation' in request.endpoint %}sidebar__link--aktywny{% endif %}" {% if 'evaluation' in request.endpoint %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"></span> Oceny i efekty

        </a>

    </li>

    {% endif %}

    {% if current_user.rola.value == 'ADMIN' %}

    <li class="sidebar__menu-sekcja" aria-hidden="true">Zarządzanie</li>

    <li class="sidebar__menu-pozycja">

        <a href="{% url_for('zarzadzanie.lista_uzytkownikow') %}" class="sidebar__link {% if 'uzytkownik' in request.endpoint %}sidebar__link--aktywny{% endif %}" {% if 'uzytkownik' in request.endpoint %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"></span> Użytkownicy

        </a>

```

```
</li>

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('zarzadzanie.lista_zakladow') }}" class="sidebar__link {% if 'zaklad' in request.endpoint %}sidebar__link--aktywny{% endif %}" {% if 'zaklad' in request.endpoint %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"></span> Zakłady pracy

    </a>

</li>

{% endif %}

</ul>


<div class="sidebar__stopka">

    <form method="POST" action="{{ url_for('auth.wylogowanie') }}">

        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

        <button type="submit" class="sidebar__wylogowanie">

            <span aria-hidden="true"></span> Wyloguj się

        </button>

    </form>

</div>

</nav>


<div class="panel-main">

    <header class="panel-main__naglowek">

        <div class="panel-main__naglowek-tresc">

            <h1 class="panel-main__tytul">{% block naglowek %}{% endblock %}</h1>

            {% block naglowek_akcje %}{% endblock %}

        </div>

        {% with wiadomosci = get_flashed_messages(with_categories=True) %}

            {% if wiadomosci %}

                <div class="komunikaty" role="status" aria-live="polite">

                    {% for kategoria, wiadomosc in wiadomosci %}

                        <div class="komunikat komunikat--{{ kategoria }}" role="alert">

                            <span class="komunikat__ikona" aria-hidden="true">

                                {% if kategoria == 'danger' %}{% elif kategoria == 'success' %}{% else %}{% endif %}

                            </span>

                            <span>{{ wiadomosc }}</span>

                        </div>

                    {% endfor %}

                </div>

            {% endif %}

        {% endwith %}

    </div>

</div>
```

```
                                <button type="button" class="komunikat__zamknij" aria-label="Zamknij powiadomienie"
onclick="this.parentElement.remove()">x</button>
```

```
        </div>
```

```
    {% endfor %}
```

```
</div>
```

```
{% endif %}
```

```
{% endwith %}
```

```
</header>
```

```
<main id="main-content" class="panel-main__tresc" tabindex="-1">
```

```
    {% block tresc %}{% endblock %}
```

```
</main>
```

```
</div>
```

```
</body>
```

```
</html>
```

PLIK: app_admin\templates\zarzadzanie\dokumenty_studenta.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dokumenty - {{ student.first_name }} {{ student.last_name }}{% endblock %}

{% block naglowek %}Dokumenty studenta{% endblock %}


{% block naglowek_akcje %}

<a href="{{ url_for('zarzadzanie.dokumenty_studentow') }}" class="przycisk przycisk--drugorzeczny">

    ← Powrót do listy

</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 900px; margin: 0 auto;">


    {# — Info o studencie ————— #}

    <div class="karta" style="margin-bottom: 1.5rem;">

        <div class="karta__tresc">

            <div style="display: flex; align-items: center; gap: 1rem;">

                <div style="width: 3rem; height: 3rem; border-radius: 50%; background: var(--kolor-glowny);

                    display: flex; align-items: center; justify-content: center;

                    font-weight: 700; font-size: 1rem; color: #fff; flex-shrink: 0;">

                    {{ (student.first_name or '?')[0] }}{{ (student.last_name or '?')[0] }}

                </div>

                <div>

                    <div style="font-size: 1.1rem; font-weight: 600;">{{ student.first_name }} {{ student.last_name }}</div>

                    <div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-top: 0.2rem;">

                        Nr albumu: <strong>{{ student.album_number or '-' }}</strong>

                        {% if student.kierunek %} · {{ student.kierunek }}{% endif %}

                        {% if student.tryb_studiow %} ({{ student.tryb_studiow }}){% endif %}

                        {% if uopz %} · Opiekun: {{ uopz.first_name }} {{ uopz.last_name }}{% endif %}

                    </div>

                </div>

            </div>

        </div>

    </div>

</div>
```

```
</div>
```

```
{# — Brak praktyk ————— #}
```

```
{% if not dokumenty_list %}
```

```
<div class="karta">
```

```
    <div class="karta__tresc" style="text-align: center; padding: 3rem 2rem;">
```

```
        <svg viewBox="0 0 24 24" style="width: 3rem; height: 3rem; fill: none; stroke: currentColor; stroke-width: 2;
opacity: 0.3; margin: 0 auto 1rem;">
```

```
            <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>
```

```
            <polyline points="14 2 14 8 20 8"/>
```

```
</svg>
```

```
<div style="font-size: 1.1rem; font-weight: 600; margin-bottom: 0.5rem;">Brak dostępnych dokumentów</div>
```

```
<div style="color: var(--kolor-tekst-drugor);">Student nie ma aktywnych ani zakończonych praktyk.</div>
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
{# — Dokumenty per zapis ————— #}
```

```
{% for wpis in dokumenty_list %}
```

```
{% set zapis = wpis.zapis %}
```

```
<div class="karta" style="margin-bottom: 1.5rem;">
```

```
    <div class="karta__naglowek">
```

```
        <div>
```

```
            <div style="font-weight: 600;">
```

```
                {{ zapis.praktyka.rok_uczelniany if zapis.praktyka else '-' }}
```

```
                {% if zapis.praktyka %} · Semestr {{ zapis.praktyka.semestr }}{% endif %}
```

```
            </div>
```

```
            <div style="font-size: 0.82rem; color: var(--kolor-tekst-drugor); margin-top: 0.2rem;">
```

```
                {% if zapis.firma %}{{ zapis.firma.nazwa }}
```

```
                {% elif zapis.firma_nazwa %}{{ zapis.firma_nazwa }}{% endif %}
```

```
                · Ścieżka:
```

```
                {% if wpis.sciezka == 'STANDARD' %}Standardowa
```

```
                {% elif wpis.sciezka == 'EMPLOYMENT' %}Praca etatowa
```

```
                {% elif wpis.sciezka == 'OWN_BUSINESS' %}Działalność gospodarcza
```

```
                {% else %}{{ wpis.sciezka }}{% endif %}
```

```

    </div>

</div>

<span class="status

    {% if zapis.status.value == 'COMPLETED' %}status--zakonczone

    {% elif zapis.status.value == 'IN_PROGRESS' %}status--aktywna

    {% else %}status--oczekuje{% endif %}">

    {% if zapis.status.value == 'COMPLETED' %}Zakończona

    {% elif zapis.status.value == 'IN_PROGRESS' %}W trakcie

    {% elif zapis.status.value == 'AWAITING_APPROVAL' %}Oczekuje na akceptację

    {% elif zapis.status.value == 'COMMISSION_REVIEW' %}Komisja

    {% elif zapis.status.value == 'DEAN_APPROVAL' %}Dziekan

    {% else %}{{ zapis.status.value }}{% endif %}

</span>

</div>


<div class="karta__tresc">

    <div style="display: grid; gap: 0.5rem;">

        {% for doc in wpis.docs %}

            {% if doc.separator is defined and doc.separator %}

                <div style="display: flex; align-items: center; gap: 0.75rem; margin: 0.4rem 0;">

                    <hr style="flex: 1; border: none; border-top: 1px dashed #d1d5db;">

                    <span style="font-size: 0.78rem; color: var(--kolor-tekst-drugor); white-space: nowrap; font-weight: 500;">{{
doc.nazwa }}</span>

                    <hr style="flex: 1; border: none; border-top: 1px dashed #d1d5db;">

                </div>

            {% else %}

                <div style="display: flex; justify-content: space-between; align-items: center;

                    padding: 0.7rem 1rem;

                    background: {% if doc.dostepny %}#f8fafc{% else %}#f3f4f6{% endif %};

                    border: 1px solid {% if doc.dostepny %}#e2e8f0{% else %}#d1d5db{% endif %};

                    border-radius: 0.5rem;">

                    <div>

                        <div style="font-weight: 500; font-size: 0.9rem; {% if not doc.dostepny %}color: var(--kolor-tekst-drugor);{%
endif %}">

                            {{ doc.nazwa }}

```



```
</div>

{% if doc.opis %}

<div style="font-size: 0.78rem; color: var(--kolor-tekst-drugor);">{{ doc.opis }}</div>

{% endif %}

{% if not doc.dostepny and doc.powod %}

<div style="font-size: 0.78rem; color: #d97706; margin-top: 0.15rem;">{{ doc.powod }}</div>

{% endif %}

</div>
```

```
{% if doc.dostepny %}

{% if doc.staly is defined and doc.staly %}

<a href="{{ url_for('zarzadzanie.dokumenty_pobierz_staly', klucz=doc.klucz_staly) }}"

    class="przycisk przycisk--drugorzeczny przycisk--maly">Pobierz PDF</a>

{% elif doc.dynamiczny is defined and doc.dynamiczny %}

<a href="{{ url_for('zarzadzanie.dokumenty_pobierz', zapis_id=doc.zapis_id, typ=doc.typ) }}"

    class="przycisk przycisk--drugorzeczny przycisk--maly">Pobierz PDF</a>

{% endif %}

{% else %}
```

```
    <span style="font-size: 0.78rem; color: var(--kolor-tekst-drugor); padding: 0.3rem 0.625rem; white-space:
nowrap;">Niedostępny</span>
```

```
{% endif %}
```

```
</div>
```

```
{% endif %}
```

```
{% endfor %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endfor %}
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\dokumenty_studentow.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Dokumenty studentów{% endblock %}

{% block naglowek %}Dokumenty studentów{% endblock %}


{% block tresc %}

<div class="karta">

    <div class="karta__naglowek">

        <form method="GET" class="pasek-filtrow" role="search" aria-label="Filtruj studentów">

            <div class="filtr-szukaj">

                <span class="filtr-szukaj__ikona" aria-hidden="true">

                    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none"
                        stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round">

                            <circle cx="11" cy="11" r="8"></circle>

                            <path d="m21 21-4.3-4.3"></path>

                        </svg>

                    </span>

                    <input type="search" name="szukaj" id="szukaj" class="filtr-szukaj__input"
                        value="{{ szukaj }}"
                        placeholder="Szukaj po imieniu, nazwisku lub nr albumu..."
                        aria-label="Szukaj studenta">

                </div>

                <button type="submit" class="przycisk przycisk--drugorzedny">Szukaj</button>

            {% if szukaj %}

                <a href="{{ url_for('zarzadzanie.dokumenty_studentow') }}" class="przycisk przycisk--drugorzedny">

                    Wyczyść filtry

                </a>

            {% endif %}

        </form>


        <span style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

            {{ studenci.total }} studentów

        </span>

    </div>

</div>
```

```

<div class="tabela-wrapper">

  <table class="tabela" aria-label="Lista studentów - dokumenty">

    <thead>

      <tr>

        <th scope="col">Student</th>

        <th scope="col">Nr albumu</th>

        <th scope="col">Kierunek</th>

        <th scope="col">Opiekun (UOPZ)</th>

        <th scope="col">Aktywne praktyki</th>

        <th scope="col"><span class="sr-only">Akcje</span></th>

      </tr>

    </thead>

    <tbody>

      {% for wpis in studenci_info %}

      {% set s = wpis.student %}

      <tr>

        <td>

          <div style="display: flex; flex-direction: column; gap: 0.15rem;">

            <span style="font-weight: 500;">{{ s.first_name }} {{ s.last_name }}</span>

            <span style="font-size: 0.8rem; color: var(--kolor-tekst-trzeciorzed);">{{ s.email }}</span>

          </div>

        </td>

        <td>{{ s.album_number or '-' }}</td>

        <td>

          <div style="font-size: 0.875rem;">{{ s.kierunek or '-' }}</div>

          {% if s.tryb_studiow %}<div style="font-size: 0.78rem; color: var(--kolor-tekst-drugor);">{{ s.tryb_studiow }}</div>{% endif %}

        </td>

        <td style="font-size: 0.875rem;">

          {% if wpis.uopz %}{{ wpis.uopz.first_name }} {{ wpis.uopz.last_name }}{% else %}<span style="color: var(--kolor-tekst-trzeciorzed);">-</span>{% endif %}

        </td>

        <td>

          {% if wpis.aktywne %}

```

```

        <span class="status status--aktywna">{{ wpis.aktwyne }}</span>

        {% else %}

        <span style="color: var(--kolor-tekst-trzeciorzed); font-size: 0.85rem;">—</span>

        {% endif %}

    </td>

    <td style="white-space: nowrap;">

        <a href="{{ url_for('zarzadzanie.dokumenty_studenta', student_id=s.id) }}"

            class="przycisk przycisk--drugorzeczny przycisk--maly">

            Dokumenty

        </a>

    </td>

</tr>

{% else %}

<tr>

    <td colspan="6" class="tabela--pusta">

        {% if szukaj %}

            Nie znaleziono studentów pasujących do zapytania „{{ szukaj }}”.

        {% else %}

            Brak studentów w systemie.

        {% endif %}

    </td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

{% if studenci.pages > 1 %}

<nav class="paginacja" aria-label="Paginacja listy studentów">

    <span class="paginacja__info">

        Wyświetlanie {{ studenci.per_page * (studenci.page - 1) + 1 }}-{{ [studenci.per_page * studenci.page,
studenci.total] | min }}

        z {{ studenci.total }}

    </span>

    <div class="paginacja__przyciski" role="list">

```

```

{% if studenci.has_prev %}

    <a href="{{ url_for('zarzadzanie.dokumenty_studentow', strona=studenci.prev_num, szukaj=szukaj) }}"

        class="paginacja__przycisk" aria-label="Poprzednia strona"></a>

{% endif %}

{% for nr in studenci.iter_pages(left_edge=1, right_edge=1, left_current=2, right_current=2) %}

    {% if nr %}

        <a href="{{ url_for('zarzadzanie.dokumenty_studentow', strona=nr, szukaj=szukaj) }}"

            class="paginacja__przycisk {% if nr == studenci.page %}paginacja__przycisk--aktywny{% endif %}"

            {% if nr == studenci.page %}aria-current="page"{% endif %}>{{ nr }}</a>

    {% else %}

        <span class="paginacja__przycisk" aria-hidden="true">...</span>

    {% endif %}

{% endfor %}

{% if studenci.has_next %}

    <a href="{{ url_for('zarzadzanie.dokumenty_studentow', strona=studenci.next_num, szukaj=szukaj) }}"

        class="paginacja__przycisk" aria-label="Następna strona"></a>

    {% endif %}

</div>

</nav>

{% endif %}

</div>

<style>

.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip: rect(0,0,0,0);
white-space: nowrap; border: 0; }

</style>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\formularz_pracownika.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Nowy pracownik{% endblock %}

{% block naglowek %}Dodaj pracownika{% endblock %}

{% block naglowek_akcje %}

    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzeczny">← Powrót</a>

{% endblock %}

{% block tresc %}

<div class="formularz-kontener">

    <div class="karta">

        <div class="karta__naglowek">

            <h2 class="karta__tytul">Dane konta pracownika</h2>

            <span style="font-size: 0.825rem; color: var(--kolor-tekst-drugor); background: #faeeda; padding: 0.25rem 0.75rem;
border-radius: 999px; font-weight: 500;">

                Logowanie przez Microsoft – e-mail musi być kontem w domenie uczelni

            </span>

        </div>

        <div class="karta__tresc">

            <form method="POST" novalidate class="formularz-panel" aria-label="Formularz pracownika">

                {{ form.hidden_tag() }}

                <div class="formularz-panel__siatka">

                    <div class="pole-formularza {% if form.imie.errors %}pole-formularza--blad{% endif %}">

                        <label for="imie" class="pole-formularza__etykieta">Imię <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

                        {{ form.imie(class="pole-formularza__input") }}

                        {% if form.imie.errors %}<span class="pole-formularza__blad" role="alert">{{ form.imie.errors[0] }}</span>{%
endif %}

                    </div>

                    <div class="pole-formularza {% if form.nazwisko.errors %}pole-formularza--blad{% endif %}">

                        <label for="nazwisko" class="pole-formularza__etykieta">Nazwisko <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

                        {{ form.nazwisko(class="pole-formularza__input") }}

                        {% if form.nazwisko.errors %}<span class="pole-formularza__blad" role="alert">{{ form.nazwisko.errors[0]
}}</span>{% endif %}

                    </div>

                </div>

            </form>

        </div>

    </div>

</div>

</div>
```

```
</div>
```

```
<div class="pole-formularza {% if form.email.errors %}pole-formularza--blad{% endif %}">
```

```
    <label for="email" class="pole-formularza__etykieta">E-mail <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>
```

```
    {{ form.email(class="pole-formularza__input", autocomplete="off") }}
```

```
    {% if form.email.errors %}<span class="pole-formularza__blad" role="alert">{{ form.email.errors[0] }}</span>{%
endif %}
```

```
</div>
```

```
<div class="pole-formularza {% if form.rola.errors %}pole-formularza--blad{% endif %}">
```

```
    <label for="rola" class="pole-formularza__etykieta">Rola w systemie <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>
```

```
    {{ form.rola(class="filtr-select", style="width:100%") }}
```

```
    {% if form.rola.errors %}<span class="pole-formularza__blad" role="alert">{{ form.rola.errors[0] }}</span>{%
endif %}
```

```
</div>
```

```
<div style="background: #f4f6f9; border-radius: var(--promien); padding: 0.875rem 1rem; font-size: 0.875rem; color:
var(--kolor-tekst-drugor);">
```

Pracownik zaloguje się e-mailem i hasłem tymczasowym (adres z e-mail, wszystko przed znakiem @), a następnie zostanie poproszony o ustawienie własnego hasła.

```
</div>
```

```
<div style="display: flex; gap: 0.75rem; flex-wrap: wrap; margin-top: 0.5rem; justify-content: flex-end;">
```

```
    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>
```

```
    <button type="submit" class="przycisk przycisk--glowny">Utwórz konto pracownika</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\formularz_praktyki.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Nowa praktyka{% endblock %}

{% block naglowek %}Nowa praktyka{% endblock %}


{% block naglowek_akcje %}

    <a href="{{ url_for('zarzadzanie.lista_praktyk') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 480px; margin: 0 auto;">

    <form method="POST" action="{{ url_for('zarzadzanie.nowa_praktyka') }}" novalidate>

        {{ form.hidden_tag() }}


        <div class="karta">

            <div class="karta__naglowek">

                <h2 class="karta__tytul">Dane praktyki</h2>

            </div>

            <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 1.25rem;">


                <div class="pole-formularza">

                    {{ form.rok_uczelniany.label(class="pole-formularza__etykieta") }}

                    <input type="text" name="rok_uczelniany" class="pole-formularza__input" style="width:100%;" placeholder="np.
2024/2025" value="{{ form.rok_uczelniany.data or ' ' }}">

                    {% for e in form.rok_uczelniany.errors %}

                        <span class="pole-formularza__blad">{{ e }}</span>

                    {% endfor %}

                </div>


                <div class="pole-formularza">

                    {{ form.semestr.label(class="pole-formularza__etykieta") }}

                    {{ form.semestr(class="filtr-select", style="width:100%") }}

                    {% for e in form.semestr.errors %}

                        <span class="pole-formularza__blad">{{ e }}</span>

                    </div>

            </div>

        </div>

    </form>

</div>

</div>
```



```
{% endfor %}
```

```
</div>
```

```
<div class="pole-formularza">
```

```
  {{ form.wymiar_godzin.label(class="pole-formularza__etykieta") }}
```

```
  {{ form.wymiar_godzin(class="pole-formularza__input", type="number", min="1", placeholder="np. 160") }}
```

```
  {% for e in form.wymiar_godzin.errors %}
```

```
    <span class="pole-formularza__blad">{{ e }}</span>
```

```
  {% endfor %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div style="display: flex; gap: 0.5rem; margin-top: 1.25rem; justify-content: flex-end;">
```

```
  <a href="{{ url_for('zarzadzanie.lista_praktyk') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>
```

```
  <button type="submit" class="przycisk przycisk--glowny">Utwórz praktykę</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\formularz_studenta.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}{% 'Edytuj' if uzytkownik else 'Nowy' %} student{% endblock %}

{% block naglowek %}{% 'Edytuj studenta' if uzytkownik else 'Dodaj studenta' %}{% endblock %}

{% block naglowek_akcje %}

    <a href="{% url_for('zarzadzanie.lista_uzytkownikow') %}" class="przycisk przycisk--drugorzeczny">← Powrót</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 640px; margin: 0 auto;">

    <div class="karta">

        <div class="karta__naglowek">

            <h2 class="karta__tytul">Dane konta studenckiego</h2>

            {% if not uzytkownik %}

                <span style="font-size: 0.825rem; color: var(--kolor-tekst-drugor); background: #faeeda; padding: 0.25rem 0.75rem;
border-radius: 999px; font-weight: 500;">

                    Logowanie przez Microsoft – e-mail musi być kontem w domenie uczelni

                </span>

            {% endif %}

        </div>

        <div class="karta__tresc">

            <form method="POST" novalidate class="formularz-panel" aria-label="Formularz studenta">

                {% form.hidden_tag() %}

                <div class="formularz-panel__siatka" style="margin-bottom: 1.5rem;">

                    <div class="pole-formularza {% if form.imie.errors %}pole-formularza--blad{% endif %}">

                        <label for="imie" class="pole-formularza__etykieta">Imię <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

                        <input type="text" id="imie" name="imie" class="pole-formularza__input"

                            value="{% form.imie.data or '' %}" required aria-required="true"

                            {% if form.imie.errors %}aria-invalid="true" aria-describedby="imie-blad"{% endif %}>

                        {% if form.imie.errors %}<span id="imie-blad" class="pole-formularza__blad" role="alert">{% form.imie.errors[0]
%}</span>{% endif %}

                    </div>

                    <div class="pole-formularza {% if form.nazwisko.errors %}pole-formularza--blad{% endif %}">

                        <label for="nazwisko" class="pole-formularza__etykieta">Nazwisko <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>
```

```

<input type="text" id="nazwisko" name="nazwisko" class="pole-formularza__input"

value="{{ form.nazwisko.data or '' }}" required aria-required="true"

{% if form.nazwisko.errors %}aria-invalid="true" aria-describedby="nazwisko-blad"{% endif %}>

    {% if form.nazwisko.errors %}<span id="nazwisko-blad" class="pole-formularza__blad" role="alert">{{
form.nazwisko.errors[0] }}</span>{% endif %}

</div>

</div>

<div class="formularz-panel__siatka" style="margin-bottom: 1.5rem;">

    <div class="pole-formularza {% if form.email.errors %}pole-formularza--blad{% endif %}">

        <label for="email" class="pole-formularza__etykieta">E-mail uczelniany <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

        <input type="email" id="email" name="email" class="pole-formularza__input"

value="{{ form.email.data or '' }}" required aria-required="true" autocomplete="off"

{% if form.email.errors %}aria-invalid="true" aria-describedby="email-blad"{% endif %}>

            {% if form.email.errors %}<span id="email-blad" class="pole-formularza__blad" role="alert">{{
form.email.errors[0] }}</span>{% endif %}

        </div>

        <div class="pole-formularza {% if form.numer_albumu.errors %}pole-formularza--blad{% endif %}">

            <label for="numer_albumu" class="pole-formularza__etykieta">Nr albumu <span class="pole-formularza__wymagane"
aria-label="wymagane">*</span></label>

            <input type="text" id="numer_albumu" name="numer_albumu" class="pole-formularza__input"

value="{{ form.numer_albumu.data or '' }}" required aria-required="true"

placeholder="np. 12345" maxlength="20"

{% if form.numer_albumu.errors %}aria-invalid="true" aria-describedby="album-blad"{% endif %}>

                {% if form.numer_albumu.errors %}<span id="album-blad" class="pole-formularza__blad" role="alert">{{
form.numer_albumu.errors[0] }}</span>{% endif %}

                {% if not uzytkownik %}<span style="font-size: 0.775rem; color: var(--kolor-tekst-trzeciorzed);">Będzie użyty
jako hasło tymczasowe</span>{% endif %}

            </div>

        <div class="pole-formularza">

            <label for="plec" class="pole-formularza__etykieta">Płeć</label>

            <select id="plec" name="plec" class="filtr-select" style="width:100%;">

                <option value="" {% if not form.plec.data %}selected{% endif %}>--- Wybierz ---</option>

                <option value="M" {% if form.plec.data == 'M' %}selected{% endif %}>Mężczyzna</option>

                <option value="K" {% if form.plec.data == 'K' %}selected{% endif %}>Kobieta</option>

            </select>

```

```
</div>
```

```
</div>
```

```
<div style="display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 1rem; margin-bottom: 1.5rem;">
```

```
<div class="pole-formularza">
```

```
<label for="kierunek" class="pole-formularza__etykieta">Kierunek studiów</label>
```

```
<input type="text" id="kierunek" name="kierunek" class="pole-formularza__input"
```

```
value="{{ form.kierunek.data or '' }}" placeholder="np. Informatyka">
```

```
</div>
```

```
<div class="pole-formularza">
```

```
<label for="specjalnosc" class="pole-formularza__etykieta">Specjalność</label>
```

```
<input type="text" id="specjalnosc" name="specjalnosc" class="pole-formularza__input"
```

```
value="{{ form.specjalnosc.data or '' }}" placeholder="np. PBDiOU">
```

```
</div>
```

```
<div class="pole-formularza">
```

```
<label for="tryb_studiow" class="pole-formularza__etykieta">Tryb studiów</label>
```

```
<select id="tryb_studiow" name="tryb_studiow" class="filtr-select" style="width:100%;">
```

```
<option value="">--- Wybierz ---</option>
```

```
<option value="stacjonarne" {% if form.tryb_studiow.data == 'stacjonarne' %}selected{% endif %}>Stacjonarne</option>
```

```
<option value="niestacjonarne" {% if form.tryb_studiow.data == 'niestacjonarne' %}selected{% endif %}>Niestacjonarne</option>
```

```
</select>
```

```
</div>
```

```
</div>
```

```
<div class="pole-formularza {% if form.uopz_id.errors %}pole-formularza--blad{% endif %}" style="margin-bottom: 1.5rem;">
```

```
<label for="uopz_id" class="pole-formularza__etykieta">Opiekun uczelniany (UOPZ) <span class="pole-formularza__wymagane" aria-label="wymagane">*</span></label>
```

```
<select id="uopz_id" name="uopz_id" class="filtr-select" style="width:100%;" required>
```

```
<option value="">— wybierz opiekuna —</option>
```

```
{% for val, label in form.uopz_id.choices %}
```

```
<option value="{{ val }}" {% if form.uopz_id.data == val %}selected{% endif %}>{{ label }}</option>
```

```
{% endfor %}
```

```
</select>
```

```
        {% if form.uopz_id.errors %}<span class="pole-formularza__blad" role="alert">{{ form.uopz_id.errors[0]
    }}</span>{% endif %}

    </div>

    {% if not uzytkownik %}

        <div style="background: #f4f6f9; border-radius: var(--promien); padding: 0.875rem 1rem; font-size: 0.875rem; color:
    var(--kolor-tekst-drugor); margin-bottom: 1.5rem;">

            Student zaloguje się e-mailem i hasłem tymczasowym (nr albumu), a następnie zostanie poproszony o ustawienie
            własnego hasła.

        </div>

    {% endif %}

    <div style="display: flex; gap: 0.75rem; flex-wrap: wrap; margin-top: 0.5rem; justify-content: flex-end;">

        <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

        <button type="submit" class="przycisk przycisk--glowny">{{ 'Zapisz zmiany' if uzytkownik else 'Utwórz konto'
    }}</button>

    </div>

</form>

</div>

</div>

</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\formularz_uzytkownika.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}{% 'Edytuj użytkownika' if uzytkownik else 'Nowy użytkownik' %}{% endblock %}

{% block naglowek %}{% 'Edytuj użytkownika' if uzytkownik else 'Nowy użytkownik' %}{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.lista_uzytkownikow') %}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 480px; margin: 0 auto;">

    <div class="karta">

        <div class="karta__tresc">

            <form method="POST" class="formularz-panel" novalidate>

                {% form.hidden_tag() %}

            <div class="formularz-panel__siatka">

                <div class="pole-formularza {% if form.imie.errors %}pole-formularza--blad{% endif %}">

                    <label for="imie" class="pole-formularza__etykieta">Imię</label>

                    {% form.imie(class="pole-formularza__input", placeholder="np. Jan") %}

                    {% for error in form.imie.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

                </div>

                <div class="pole-formularza {% if form.nazwisko.errors %}pole-formularza--blad{% endif %}">

                    <label for="nazwisko" class="pole-formularza__etykieta">Nazwisko</label>

                    {% form.nazwisko(class="pole-formularza__input", placeholder="np. Kowalski") %}

                    {% for error in form.nazwisko.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

                </div>

                <div class="pole-formularza {% if form.email.errors %}pole-formularza--blad{% endif %}">

                    <label for="email" class="pole-formularza__etykieta">Adres e-mail</label>

                    {% form.email(class="pole-formularza__input", placeholder="jan.kowalski@ans.elblag.edu.pl") %}

                </div>

            </div>

        </div>

    </div>

</div>
```

```

    {% for error in form.email.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

</div>

<div class="formularz-panel__siatka">

    <div class="pole-formularza {% if form.rola.errors %}pole-formularza--blad{% endif %}">

        <label for="rola" class="pole-formularza__etykieta">Rola w systemie</label>

        {{ form.rola(class="filtr-select", style="width: 100%;") }}

        {% for error in form.rola.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

    </div>

    <div id="kontener-albumu" class="pole-formularza {% if form.numer_albumu.errors %}pole-formularza--blad{% endif %}" style="display: {% if form.rola.data == 'STUDENT' %}block{% else %}none{% endif %};">

        <label for="numer_albumu" class="pole-formularza__etykieta">Numer albumu</label>

        {{ form.numer_albumu(class="pole-formularza__input", placeholder="np. 12345") }}

        {% for error in form.numer_albumu.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

    </div>

</div>

<div class="formularz-panel__sekcja">

    <h3 class="formularz-panel__sekcja-tytul">Bezpieczeństwo</h3>

    <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1rem;">

        {% if uzytkownik %}Pozostaw puste, aby nie zmieniać hasła.{% else %}Hasło musi mieć minimum 8 znaków.{% endif %}

    </p>

    <div class="formularz-panel__siatka">

        <div class="pole-formularza {% if form.haslo.errors %}pole-formularza--blad{% endif %}">

            <label for="haslo" class="pole-formularza__etykieta">Nowe hasło</label>

            {{ form.haslo(class="pole-formularza__input") }}

            {% for error in form.haslo.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

        </div>

        <div class="pole-formularza {% if form.haslo_potwierdzenie.errors %}pole-formularza--blad{% endif %}">

            <label for="haslo_potwierdzenie" class="pole-formularza__etykieta">Powtórz hasło</label>

            {{ form.haslo_potwierdzenie(class="pole-formularza__input") }}

```

```

        {% for error in form.haslo_potwierdzenie.errors %}<span class="pole-formularza__blad">{{ error }}</span>{%
endfor %}

        </div>

    </div>

</div>

    <div class="formularz-panel__akcje" style="margin-top: 1rem; padding-top: 1.5rem; border-top: 1px solid
var(--kolor-ramka); display:flex; gap:0.5rem; justify-content:flex-end;">

        <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzedy">Anuluj</a>

        <button type="submit" class="przycisk przycisk--glowny">{{ 'Zapisz zmiany' if uzytkownik else 'Utwórz
użytkownika' }}</button>

    </div>

</form>

</div>

</div>

</div>

{% endblock %}

{% block scripts_extra %}

<script>

    document.getElementById('rola').addEventListener('change', function() {

        const kontener = document.getElementById('kontener-albumu');

        if (this.value === 'STUDENT') {

            kontener.style.display = 'block';

        } else {

            kontener.style.display = 'none';

            document.getElementById('numer_albumu').value = '';

        }

    });

</script>

{% endblock %}

```


PLIK: app_admin\templates\zarzadzanie\formularz_zakladu.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}{% 'Edytuj zakład' if zaklad else 'Nowy zakład pracy' %}{% endblock %}

{% block naglowek %}{% 'Edytuj zakład' if zaklad else 'Nowy zakład pracy' %}{% endblock %}

{% block naglowek_akcje %}

<a href="{% url_for('zarzadzanie.lista_zakladow') %}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}

{% block tresc %}

<div class="karta" style="max-width: 800px; margin: 0 auto;">

    <div class="karta__tresc">

        <form method="POST" class="formularz-panel" novalidate>

            {% form.hidden_tag() %}

            <div class="pole-formularza {% if form.nazwa.errors %}pole-formularza--blad{% endif %}">

                <label for="nazwa" class="pole-formularza__etykieta">Pełna nazwa zakładu</label>

                {% form.nazwa(class="pole-formularza__input", placeholder="np. Software House Sp. z o.o.") %}

                {% for error in form.nazwa.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

            </div>

            <div class="pole-formularza {% if form.adres.errors %}pole-formularza--blad{% endif %}">

                <label for="adres" class="pole-formularza__etykieta">Adres (ulica i numer)</label>

                {% form.adres(class="pole-formularza__input", placeholder="np. ul. Grunwaldzka 137") %}

                {% for error in form.adres.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

            </div>

            <div class="formularz-panel__siatka">

                <div class="pole-formularza {% if form.kod_pocztowy.errors %}pole-formularza--blad{% endif %}">

                    <label for="kod_pocztowy" class="pole-formularza__etykieta">Kod pocztowy</label>

                    {% form.kod_pocztowy(class="pole-formularza__input", placeholder="np. 82-300") %}

                    {% for error in form.kod_pocztowy.errors %}<span class="pole-formularza__blad">{% error %}</span>{% endfor %}

                </div>

            </div>

        </form>

    </div>

</div>
```

```

<div class="pole-formularza {% if form.miasto.errors %}pole-formularza--blad{% endif %}">

  <label for="miasto" class="pole-formularza__etykieta">Miasto</label>

  {{ form.miasto(class="pole-formularza__input", placeholder="np. Elbląg") }}

  {% for error in form.miasto.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

</div>

</div>

<div class="formularz-panel__siatka">

  <div class="pole-formularza {% if form.nip_krs.errors %}pole-formularza--blad{% endif %}">

    <label for="nip_krs" class="pole-formularza__etykieta">NIP / KRS</label>

    {{ form.nip_krs(class="pole-formularza__input", placeholder="NIP lub numer rejestrowy") }}

    {% for error in form.nip_krs.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

  </div>

  <div class="pole-formularza {% if form.przedstawiciel_prawny.errors %}pole-formularza--blad{% endif %}">

    <label for="przedstawiciel_prawny" class="pole-formularza__etykieta">Przedstawiciel prawny</label>

    {{ form.przedstawiciel_prawny(class="pole-formularza__input", placeholder="Imię i nazwisko (np. Prezes Zarządu)") }}

    {% for error in form.przedstawiciel_prawny.errors %}<span class="pole-formularza__blad">{{ error }}</span>{% endfor %}

  </div>

</div>

<div class="formularz-panel__akcje" style="margin-top: 1.5rem; padding-top: 1.5rem; border-top: 1px solid var(--kolor-ramka);">

  <button type="submit" class="przycisk przycisk--glowny">

    {{ 'Zapisz zmiany' if zaklad else 'Dodaj zakład pracy' }}

  </button>

</div>

</form>

</div>

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\import_csv.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Import studentów{% endblock %}

{% block naglowek %}Import studentów z CSV{% endblock %}

{% block naglowek_akcje %}

    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzeczny">← Powrót</a>

{% endblock %}

{% block tresc %}

<div style="max-width: 560px; display: flex; flex-direction: column; gap: 1.25rem; margin: 0 auto;">

    <div class="karta">

        <div class="karta__naglowek"><h2 class="karta__tytul">Wgraj plik CSV</h2></div>

        <div class="karta__tresc">

            <form method="POST" enctype="multipart/form-data" novalidate class="formularz-panel">

                {{ form.hidden_tag() }}

                <div class="pole-formularza">

                    <label for="plik" class="pole-formularza__etykieta">

                        Plik CSV <span class="pole-formularza__wymagane" aria-label="wymagane">*</span>

                    </label>

                    <label for="plik" style="display:flex; align-items:center; gap:0.75rem; padding:0.75rem 1rem;

                        border:2px dashed var(--kolor-ramka); border-radius:var(--promien);

                        cursor:pointer; background:var(--kolor-tlo); transition:border-color 0.15s;"

                        onmouseover="this.style.borderColor='var(--kolor-glowny)'"

                        onmouseout="this.style.borderColor='var(--kolor-ramka)'">

                            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" width="22" height="22"

                                style="flex-shrink:0;color:var(--kolor-tekst-drugor)"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline

                                    points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>

                            <span id="plik-label" style="font-size:0.875rem; color:var(--kolor-tekst-drugor);">Wybierz plik CSV lub

                                przeciągnij tutaj</span>

                        </label>

                        <input type="file" id="plik" name="plik" accept=".csv" required

                            style="position:absolute;width:1px;height:1px;opacity:0;overflow:hidden;"

                            onchange="document.getElementById('plik-label').textContent = this.files[0]?.name || 'Wybierz plik

                                CSV'">

                    {% if form.plik.errors %}
```

```
<span class="pole-formularza__blad" role="alert">{{ form.plik.errors[0] }}</span>
```

```
{% endif %}
```

```
</div>
```

```
<div class="pole-formularza">
```

```
<label for="uopz_id" class="pole-formularza__etykieta">
```

```
Przypisz opiekuna (UOPZ) <span class="pole-formularza__wymagane" aria-label="wymagane">*</span>
```

```
</label>
```

```
<select id="uopz_id" name="uopz_id" class="pole-formularza__input" required>
```

```
{% for val, label in form.uopz_id.choices %}
```

```
<option value="{{ val }}" {% if form.uopz_id.data == val %>selected{% endif %}>{{ label }}</option>
```

```
{% endfor %}
```

```
</select>
```

```
{% if form.uopz_id.errors %}
```

```
<span class="pole-formularza__blad" role="alert">{{ form.uopz_id.errors[0] }}</span>
```

```
{% endif %}
```

```
</div>
```

```
<button type="submit" class="przycisk przycisk--glowny" style="width:100%; margin-top:0.75rem;">
```

```
<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" width="16" height="16" style="vertical-align:middle;margin-right:0.4rem"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
```

```
Importuj studentów
```

```
</button>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
<div class="karta">
```

```
<div class="karta__naglowek"><h2 class="karta__tytul">Format pliku</h2></div>
```

```
<div class="karta__tresc" style="display:flex; flex-direction:column; gap:0.75rem;">
```

```
<p style="font-size:0.85rem; color:var(--kolor-tekst-drugor);">
```

```
Pierwsza linia musi zawierać nagłówki. Wszystkie kolumny są wymagane.
```

```
Kolumna email musi zawierać konto Microsoft w domenie uczelni. Kodowanie: UTF-8 lub UTF-8 z BOM.
```

```
</p>
```

```
        <code style="display:block; background:var(--kolor-tlo); border:1px solid var(--kolor-ramka);
border-radius:var(--promien); padding:0.75rem 1rem; font-size:0.8rem; line-height:2; white-space:pre;
overflow-x:auto;">imie,nazwisko,email,numer_albumu,plec,kierunek,tryb_studiow
```

Jan,Kowalski,54321@ans-elblag.pl,54321,M,Informatyka,stacjonarne

Anna,Nowak,67890@ans-elblag.pl,67890,K,Informatyka,niestacjonarne</code>

</div>

</div>

</div>

{% if wyniki %}

<div class="karta" style="margin-top: 1.25rem; max-width:560px;">

<div class="karta__naglowek">

<h2 class="karta__tytul">Wyniki importu</h2>

</div>

<div class="karta__tresc">

<div style="display: flex; gap: 2rem; margin-bottom: 1rem;">

<div style="text-align: center;">

<p style="font-size: 2rem; font-weight: 700; color: var(--kolor-sukces);">{{ wyniki.utworzono }}</p>

<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Kont utworzonych</p>

</div>

<div style="text-align: center;">

<p style="font-size: 2rem; font-weight: 700; color: var(--kolor-blad);">{{ wyniki.pominieto }}</p>

<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Pominiętych</p>

</div>

</div>

{% if wyniki.bledy %}

<details>

<summary style="cursor: pointer; font-size: 0.875rem; font-weight: 500; color: var(--kolor-blad);">

Pokaż błędy ({{ wyniki.bledy|length }})

</summary>

<ul style="margin-top: 0.5rem; font-size: 0.825rem; color: var(--kolor-tekst-drugor); list-style: none; padding: 0;">

{% for blad in wyniki.bledy %}

<li style="padding: 0.25rem 0; border-bottom: 1px solid var(--kolor-ramka);">{{ blad }}

{% endfor %}

</details>

{% endif %}

</div>

</div>

{% endif %}

{% endblock %}

PLIK: app_admin\templates\zarzadzanie\index.html

<h1>Management</h1>


```

        <span class="status status--zakonczona">Aktywna</span>

    {% else %}

        <span class="status status--szkic">Nieaktywna</span>

    {% endif %}

</td>

<td style="white-space: nowrap;">

    <div style="display: flex; gap: 0.375rem;">

        <form method="POST"

            action="{% url_for('zarzadzanie.przelacz_aktywnosc_praktyki', id=p.id) %}"

            style="display: inline;">

            {{ csrf_form.hidden_tag() }}

            <button type="submit"

                class="przycisk {% if p.status.value == 'ACTIVE' %}przycisk--ostrzezenie{% else
                %}przycisk--drugorzedny{% endif %} przycisk--maly"

                onclick="return confirm('Zmienić status praktyki?')">

                {{ 'Dezaktywuj' if p.status.value == 'ACTIVE' else 'Aktywuj' }}

            </button>

        </form>

        <form method="POST"

            action="{% url_for('zarzadzanie.usun_praktyke', id=p.id) %}"

            style="display: inline;">

            {{ csrf_form.hidden_tag() }}

            <button type="submit"

                class="przycisk przycisk--niebezpieczny przycisk--maly"

                onclick="return confirm('Czy na pewno chcesz usunąć tę praktykę? Usuną się też wszystkie
                zapisy.')">

                Usun

            </button>

        </form>

    </div>

</td>

</tr>

```

```

        {% else %}

        <tr>

            <td colspan="6" class="tabela--pusta">Brak praktyk. Utwórz pierwszą.</td>

        </tr>

        {% endfor %}

    </tbody>

</table>

</div>


{% if praktyki.pages > 1 %}

<nav class="paginacja" aria-label="Paginacja">

    <span class="paginacja__info">

        Strona {{ praktyki.page }} z {{ praktyki.pages }}

    </span>

    <div class="paginacja__przyciski">

        {% for nr in praktyki.iter_pages(left_edge=1, right_edge=1, left_current=2, right_current=2) %}

            {% if nr %}

                <a href="{{ url_for('zarzadzanie.lista_praktyk', strona=nr) }}"

                    class="paginacja__przycisk {% if nr == praktyki.page %}paginacja__przycisk--aktywny{% endif %}"

                    {% if nr == praktyki.page %}aria-current="page"{% endif %}>{{ nr }}</a>

            {% else %}

                <span class="paginacja__przycisk" aria-hidden="true">...</span>

            {% endif %}

        {% endfor %}

    </div>

</nav>

{% endif %}

</div>


<style>.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip:
rect(0,0,0,0); white-space: nowrap; border: 0; }</style>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\szczegoly_praktyki.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Szczegóły praktyki{% endblock %}

{% block naglowek %}Szczegóły praktyki{% endblock %}


{% block naglowek_akcje %}

<div class="akcje-panelu">

    <a href="{% url_for('zarzadzanie.lista_praktyk') %}" class="przycisk przycisk--drugorzeczny">Powrót do listy</a>

    {% if current_user.rola.value in ['ADMIN', 'UOPZ'] %}

        <a href="{% url_for('evaluation.ocen_praktyke', id=praktyka.id) %}" class="przycisk przycisk--glowny">Oceń / Egzaminuj</a>

        <a href="{% url_for('evaluation.podglad_sprawozdania', id=praktyka.id) %}" class="przycisk przycisk--drugorzeczny">Zobacz Sprawozdanie</a>

        <a href="{% url_for('journal.podglad', id=praktyka.id) %}" class="przycisk przycisk--drugorzeczny">Zobacz Dziennik</a>

        <a href="{% url_for('documents.panel', id=praktyka.id) %}" class="przycisk przycisk--drugorzeczny" style="border-color: #8e44ad; color: #8e44ad;"> Dokumenty (PDF)</a>

    {% endif %}

</div>

{% endblock %}


{% block tresc %}

<div class="uklad-szczegolow" style="display: grid; grid-template-columns: 2fr 1fr; gap: 2rem;">

    <div class="lewa-kolumna" style="display: flex; flex-direction: column; gap: 2rem;">

        <!-- Dane podstawowe -->

        <section class="karta">

            <header class="karta__naglowek">

                <h2 class="karta__tytul">Informacje o praktyce</h2>

                <span class="status status--{% praktyka.status.value|lower|replace('_', '-') %}">{% tlumacz_status(praktyka.status.value) %}</span>

            </header>

            <div class="karta__tresc">

                <dl style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem;">

                    <div>
```

```

        <dt style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Student</dt>

        <dd style="font-weight: 600;">{{ praktyka.student.imie }} {{ praktyka.student.nazwisko }}</dd>

        <dd style="font-size: 0.9rem;">Album: {{ praktyka.student.numer_albumu or '-' }}</dd>

    </div>

    <div>

        <dt style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Zakład pracy</dt>

        <dd style="font-weight: 600;">{{ praktyka.zaklad.nazwa }}</dd>

        <dd style="font-size: 0.9rem;">{{ praktyka.zaklad.miasto }}</dd>

    </div>

    <div>

        <dt style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Termin</dt>

        <dd>{{ praktyka.data_rozpoczecia }} – {{ praktyka.data_zakonczenia or '...' }}</dd>

    </div>

    <div>

        <dt style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Ścieżka</dt>

        <dd>{{ praktyka.typ_sciezki.value }}</dd>

    </div>

</dl>

</div>

</section>

<!-- Postęp godzin -->

<section class="karta">

    <header class="karta__naglowek">

        <h2 class="karta__tytul">Realizacja godzinowa</h2>

    </header>

    <div class="karta__tresc" style="padding: 2rem;">

        <div style="display: flex; justify-content: space-between; margin-bottom: 1rem; align-items: flex-end;">

            <span style="font-size: 2rem; font-weight: 700; color: var(--kolor-glowny);">{{ praktyka.laczenie_godzin }}
            <small style="font-size: 1rem; color: var(--kolor-tekst-drugor);" / 160 h</small></span>

            <span style="font-weight: 600; color: var(--kolor-sukces);">{{ (praktyka.laczenie_godzin / 1.6)|round|int
            }}%</span>

        </div>

        <div class="pasek-postep" style="height: 12px;">

            <div class="pasek-postep__wypelnienie {{ 'pasek-postep__wypelnienie--zakonczone' if praktyka.laczenie_godzin >=
            160 else '' }}"

```

```

        style="width: {% if praktyka.lacznie_godzin >= 160 %}100{% else %}{ { praktyka.lacznie_godzin / 1.6 } }{%
endif %}%;"></div>

</div>

</div>

</section>

</div>

<div class="prawa-kolumna" style="display: flex; flex-direction: column; gap: 2rem;">

<!-- Opiekunowie -->

<section class="karta">

    <header class="karta__naglowek">

        <h2 class="karta__tytul">Opiekunowie</h2>

    </header>

    <div class="karta__tresc">

        <div style="margin-bottom: 1.5rem;">

            <div style="font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em; color:
var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">Uczelniany (UOPZ)</div>

            <div style="font-weight: 600;">{{ praktyka.uopz.imie }} {{ praktyka.uopz.nazwisko }}</div>

        </div>

        <div>

            <div style="font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em; color:
var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">Zakładowy (ZOPZ)</div>

            <div style="font-weight: 600;">{{ praktyka.zopz.imie }} {{ praktyka.zopz.nazwisko }}</div>

        </div>

    </div>

</section>

<!-- Oceny (jeśli są) -->

{% if praktyka.ocena_sprawozdania or praktyka.ocena_uopz %}

<section class="karta">

    <header class="karta__naglowek">

        <h2 class="karta__tytul">Wyniki oceny</h2>

    </header>

    <div class="karta__tresc">

```

```
<dl style="display: flex; flex-direction: column; gap: 1rem;">

  <div style="display: flex; justify-content: space-between;">

    <dt>Sprawozdanie (S):</dt>

    <dd style="font-weight: 700; color: var(--kolor-glowny);">{{ praktyka.ocena_sprawozdania or '-' }}</dd>

  </div>

  <div style="display: flex; justify-content: space-between;">

    <dt>Ocena UOPZ (U):</dt>

    <dd style="font-weight: 700; color: var(--kolor-glowny);">{{ praktyka.ocena_uopz or '-' }}</dd>

  </div>

  <div style="display: flex; justify-content: space-between;">

    <dt>Ocena ZOPZ (Z):</dt>

    <dd style="font-weight: 700; color: var(--kolor-glowny);">{{ praktyka.ocena_zopz or '-' }}</dd>

  </div>

</dl>

</div>

</section>

{% endif %}
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\uzytkownicy.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Użytkownicy{% endblock %}

{% block naglowek %}Zarządzanie użytkownikami{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    <a href="{{ url_for('zarzadzanie.nowy_student') }}" class="przycisk przycisk--glowny">

        + Dodaj studenta

    </a>

    <a href="{{ url_for('zarzadzanie.nowy_pracownik') }}" class="przycisk przycisk--glowny">

        + Dodaj pracownika

    </a>

    <a href="{{ url_for('zarzadzanie.import_csv') }}" class="przycisk przycisk--drugorzedny">

        Import z CSV

    </a>

</div>

{% endblock %}


{% block tresc %}

<div class="karta">

    <div class="karta__naglowek">

        <form method="GET" class="pasek-filtrow" role="search" aria-label="Filtruj użytkowników">

            <div class="filtr-szukaj">

                <span class="filtr-szukaj__ikona" aria-hidden="true">

                    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round">

                        <circle cx="11" cy="11" r="8"></circle>

                        <path d="m21 21-4.3-4.3"></path>

                    </svg>

                </span>

                <input

                    type="search"

                    name="szukaj"

                />

            </div>

        </form>

    </div>

</div>

{% endblock %}
```

```

        id="szukaj"

        class="filtr-szukaj__input"

        value="{{ request.args.get('szukaj', '') }}"

        placeholder="Szukaj po imieniu, nazwisku, e-mailu..."

        aria-label="Szukaj użytkowników"

    >

</div>

<label for="rola" class="sr-only">Filtruj po roli</label>

<select name="rola" id="rola" class="filtr-select" onchange="this.form.submit()">

    <option value="">Wszystkie role</option>

    <option value="STUDENT" {% if request.args.get('rola') == 'STUDENT' %}>selected{% endif %}>Studenci</option>

        <option value="UOPZ"          {% if request.args.get('rola') == 'UOPZ'          %}>selected{% endif %}>Opiekunowie
uczelni</option>

            <option value="ADMIN"          {% if request.args.get('rola') == 'ADMIN'          %}>selected{% endif
%}>Administratorzy</option>

</select>

<button type="submit" class="przycisk przycisk--drugorzecny">Szukaj</button>

{% if request.args.get('szukaj') or request.args.get('rola') %}

    <a href="{{ url_for('zarzadzanie.lista_uzytkownikow') }}" class="przycisk przycisk--drugorzecny">

        Wyczyść filtry

    </a>

{% endif %}

</form>


<span style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

    {{ uzytkownicy.total }} użytkowników

</span>

</div>


<div class="tabela-wrapper">

    <table class="tabela" aria-label="Lista użytkowników systemu">

        <thead>

            <tr>

                <th scope="col">Użytkownik</th>

                <th scope="col">Rola</th>

            </tr>

        </thead>

        <tbody>

            <tr>

                <td>{{ user.username }}</td>

                <td>{{ user.role }}</td>

            </tr>

        </tbody>

    </table>

</div>

```



```

<th scope="col">Nr albumu</th>

<th scope="col">Status</th>

<th scope="col">Data rejestracji</th>

<th scope="col"><span class="sr-only">Akcje</span></th>

</tr>

</thead>

<tbody>

{% for u in uzytkownicy.items %}

<tr>

<td>

<div style="display: flex; flex-direction: column; gap: 0.15rem;">

<span style="font-weight: 500;">{{ u.first_name }} {{ u.last_name }}</span>

<span style="font-size: 0.8rem; color: var(--kolor-tekst-trzeciorzed);">{{ u.email }}</span>

</div>

</td>

<td>

{% set mapa_rol = {'ADMIN': 'admin', 'UOPZ': 'uopz', 'STUDENT': 'student'} %}

{% set klasa = mapa_rol.get(u.role.value, 'student') %}

<span class="rola-odznaka rola-odznaka--{{ klasa }}">{{ u.role.value }}</span>

</td>

<td>{{ u.album_number or '-' }}</td>

<td>

{% if u.is_active %}

<span class="status status--zakonczone">Aktywny</span>

{% else %}

<span class="status status--szkic">Nieaktywny</span>

{% endif %}

</td>

<td style="white-space: nowrap; font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

{{ u.created_at.strftime('%d.%m.%Y') }}

</td>

<td style="white-space: nowrap;">

<div style="display: flex; gap: 0.375rem;">

```

```

{% if u.role.name == 'STUDENT' %}

    <a href="{{ url_for('zarzadzanie.edytuj_studenta', id=u.id) }}"

        class="przycisk przycisk--drugorzeczny przycisk--maly"

        aria-label="Edytuj studenta {{ u.first_name }} {{ u.last_name }}">

            Edytuj

    </a>

{% endif %}


{% if u.id != current_user.id %}

    <form method="POST"

        action="{{ url_for('zarzadzanie.przelacz_aktywnosc', id=u.id) }}"

        style="display: inline;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit"

            class="przycisk {% if u.is_active %}przycisk--ostrzezenie{% else %}przycisk--drugorzeczny{%
endif %} przycisk--maly"

            aria-label="{{ 'Dezaktywuj' if u.is_active else 'Aktywuj' }}" konto {{ u.first_name }} {{
u.last_name }}"

            onclick="return confirm('Czy na pewno chcesz zmienić status tego konta?')">

            {{ 'Dezaktywuj' if u.is_active else 'Aktywuj' }}

        </button>

    </form>


    <form method="POST"

        action="{{ url_for('zarzadzanie.usun_uzytkownika', id=u.id) }}"

        style="display: inline;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit"

            class="przycisk przycisk--niebezpieczny przycisk--maly"

            aria-label="Usuń konto {{ u.first_name }} {{ u.last_name }}"

            onclick="return confirm('Czy na pewno chcesz trwale usunąć konto {{ u.first_name }} {{
u.last_name }}? Tej operacji nie można cofnąć.')">

            Usuń

        </button>

    </form>

{% endif %}

```

```

        </div>

    </td>

</tr>

{% else %}

<tr>

    <td colspan="6" class="tabela--pusta">

        {% if request.args.get('szukaj') %}

            Nie znaleziono użytkowników pasujących do zapytania „{{ request.args.get('szukaj') }}”.

        {% else %}

            Brak użytkowników w systemie.

        {% endif %}

    </td>

</tr>

{% endfor %}

</tbody>

</table>

</div>


{% if uzytkownicy.pages > 1 %}

<nav class="paginacja" aria-label="Paginacja listy użytkowników">

    <span class="paginacja__info">

        Wyświetlanie {{ uzytkownicy.per_page * (uzytkownicy.page - 1) + 1 }}-{{ [uzytkownicy.per_page * uzytkownicy.page,
uzytkownicy.total] | min }}

        z {{ uzytkownicy.total }}

    </span>

    <div class="paginacja__przyciski" role="list">

        {% if uzytkownicy.has_prev %}

            <a href="{{ url_for('zarzadzanie.lista_uzytkownikow', strona=uzytkownicy.prev_num, **request.args) }}"

                class="paginacja__przycisk" aria-label="Poprzednia strona"><</a>

        {% endif %}

        {% for nr in uzytkownicy.iter_pages(left_edge=1, right_edge=1, left_current=2, right_current=2) %}

            {% if nr %}

                <a href="{{ url_for('zarzadzanie.lista_uzytkownikow', strona=nr, **request.args) }}"

                    class="paginacja__przycisk {% if nr == uzytkownicy.page %}paginacja__przycisk--aktywny{% endif %}"

```

```

        {% if nr == uzytkownicy.page %}aria-current="page"{% endif %}>

        {{ nr }}

    </a>

    {% else %}

        <span class="paginacja__przycisk" aria-hidden="true">...</span>

    {% endif %}

{% endfor %}

{% if uzytkownicy.has_next %}

    <a href="{% url_for('zarzadzanie.lista_uzytkownikow', strona=uzytkownicy.next_num, **request.args) %}"

        class="paginacja__przycisk" aria-label="Następna strona"></a>

    {% endif %}

</div>

</nav>

{% endif %}

</div>

<style>

.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip: rect(0,0,0,0);
white-space: nowrap; border: 0; }

.przycisk--ostrzezenie {

    background: #f59e0b;

    color: white;

    border: 1px solid #f59e0b;

}

.przycisk--ostrzezenie:hover {

    background: #d97706;

    border-color: #d97706;

}

</style>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\zaklady.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Zakłady pracy{% endblock %}

{% block naglowek %}Zakłady pracy{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    {% if current_user.rola.value in ['ADMIN', 'UOPZ'] %}

    <a href="{{ url_for('zarzadzanie.nowy_zaklad') }}" class="przycisk przycisk--glowny">

        <span aria-hidden="true">+</span> Dodaj zakład

    </a>

    {% endif %}

</div>

{% endblock %}


{% block tresc %}

<div class="karta">

    <div class="karta__naglowek">

        <form class="pasek-filtrow" method="GET" action="{{ url_for('zarzadzanie.lista_zakladow') }}">

            <div class="filtr-szukaj">

                <span class="filtr-szukaj__ikona" aria-hidden="true"><svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="11" cy="11"
r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg></span>

                <input type="text" name="szukaj" value="{{ request.args.get('szukaj', '') }}" class="filtr-szukaj__input"
placeholder="Nazwa zakładu lub miasto..." aria-label="Wyszukaj zakład">

            </div>

            <button type="submit" class="przycisk przycisk--drugorzecny">Szukaj</button>

        </form>

    </div>


    <div class="karta__tresc">

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>
```

```

<tr>

    <th scope="col">Nazwa zakładu</th>

    <th scope="col">Miasto</th>

    <th scope="col">NIP / KRS</th>

    <th scope="col">Akcje</th>

</tr>

</thead>

<tbody>

    {% for z in zaklady.items %}

    <tr>

        <td>

            <div style="font-weight: 600;">{{ z.nazwa }}</div>

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ z.adres }}</div>

        </td>

        <td>{{ z.miasto or '-' }}</td>

        <td>{{ z.nip_krs or '-' }}</td>

        <td>

            <div class="akcje-tabeli">

                <a href="{{ url_for('zarzadzanie.edytuj_zaklad', id=z.id) }}" class="przycisk przycisk--drugorzeczny przycisk--maly">Edytuj</a>

            </div>

        </td>

    </tr>

    {% else %}

    <tr>

        <td colspan="4" class="tabela--pusta">Brak zarejestrowanych zakładów pracy.</td>

    </tr>

    {% endfor %}

</tbody>

</table>

</div>

```

```

{% if zaklady.pages > 1 %}

```

```

<nav class="paginacja" aria-label="Nawigacja po stronach">

```

```

    <span class="paginacja__info">Strona {{ zaklady.page }} z {{ zaklady.pages }}</span>

```

```
<div class="paginacja__przyciski">

    {% for strona in zaklady.iter_pages() %}

        {% if strona %}

            <a href="{% url_for('zarzadzanie.lista_zakladow', strona=strona, szukaj=request.args.get('szukaj', '')) %}"
class="paginacja__przycisk {% 'paginacja__przycisk--aktywny' if strona == zaklady.page else '' %}">

                {{ strona }}

            </a>

        {% else %}

            <span class="paginacja__separator">...</span>

        {% endif %}

    {% endfor %}

</div>

</nav>

{% endif %}

</div>

</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\dziekan\decyzja.html

```
{% extends "layouts/base_panel.html" %}
```

```
{% block tytul %}Decyzja dziekana{% endblock %}
```

```
{% block naglowek %}Decyzja dziekana w sprawie wniosku{% endblock %}
```

```
{% block naglowek_akcje %}
```

```
<div style="display: flex; gap: 0.5rem;">
```

```
<a href="{{ url_for('zarzadzanie.dziekan_lista') }}" class="przycisk przycisk--drugorzeczny">← Powrót do listy</a>
```

</div>

```
{% endblock %}
```

```
{% block tresp %}
```

```
<div class="grid-2">
```

```
<!-- Lewa kolumna: Dane wniosku -->
```

<div>

```
<div class="karta" style="margin-bottom: 2rem;">
```

```
<div class="karta_naglowek">
```

Informacje o wniosku

</div>

```
<div class="karta tresc">
```

```
<div class="info-grid">
```

<div class="info-item">

<label>Student</label>

<div>{{ zapis.student.first name }} {{ zapis.student.last name }}</div>

</div>

<div class="info-item">

<label>Nr albumu</label>

<div>{{ zapis.student.album number }}</div>

</div>

```
<div class="info-item">
```

<label>Ścieżka</label>


```

<div>

    {% if zapis.track_type.value == 'EMPLOYMENT' %}

    <span class="badge badge--info">Praca zawodowa</span>

    {% elif zapis.track_type.value == 'OWN_BUSINESS' %}

    <span class="badge badge--warning">Działalność gospodarcza</span>

    {% endif %}

</div>

</div>

<div class="info-item">

    <label>Data zgłoszenia</label>

    <div>{{ zapis.enrolled_at.strftime('%d.%m.%Y %H:%M') }}</div>

</div>

</div>

{% if zapis.firma_nazwa %}

<div style="border-top: 1px solid #e5e7eb; margin-top: 1.5rem; padding-top: 1.5rem;">

    <h3 style="font-size: 1rem; margin-bottom: 1rem;">Miejsce pracy/działalności</h3>

    <div class="info-grid">

        <div class="info-item">

            <label>Nazwa</label>

            <div><strong>{{ zapis.firma_nazwa }}</strong></div>

        </div>

        <div class="info-item">

            <label>Miasto</label>

            <div>{{ zapis.firma_miasto or 'Nie podano' }}</div>

        </div>

        {% if zapis.firma_nip_krs %}

        <div class="info-item">

            <label>NIP/KRS</label>

            <div>{{ zapis.firma_nip_krs }}</div>

        </div>

        {% endif %}

    </div>

</div>

</div>

```

```
{% endif %}
```

```
{% if zapis.uzasadnienie_sciezki %}
```

```
<div style="border-top: 1px solid #e5e7eb; margin-top: 1.5rem; padding-top: 1.5rem;">
```

```
  <h3 style="font-size: 1rem; margin-bottom: 1rem;">Uzasadnienie studenta</h3>
```

```
  <div class="uzasadnienie-box">
```

```
    {{ zapis.uzasadnienie_sciezki }}
```

```
  </div>
```

```
</div>
```

```
{% endif %}
```

```
<!-- Decyzja komisji -->
```

```
{% if zapis.decyzja_komisji %}
```

```
<div style="border-top: 1px solid #e5e7eb; margin-top: 1.5rem; padding-top: 1.5rem;">
```

```
  <h3 style="font-size: 1rem; margin-bottom: 1rem;">Decyzja komisji weryfikującej</h3>
```

```
  <div class="komisja-decyzja">
```

```
    <div class="decyzja-header">
```

```
      <span class="status status--zakonczona">Zatwierdzone</span>
```

```
      <span class="decyzja-data">
```

```
        {{ zapis.decyzja_komisji_o.strftime('%d.%m.%Y %H:%M') if zapis.decyzja_komisji_o }}
```

```
      </span>
```

```
    </div>
```

```
    {% if zapis.komentarze_komisji %}
```

```
      <div class="decyzja-komentarz">
```

```
        <strong>Komentarz komisji:</strong><br>
```

```
        {{ zapis.komentarze_komisji }}
```

```
      </div>
```

```
    {% endif %}
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```

<!-- Prawa kolumna: Formularz decyzji dziekana -->

<div>

  <div class="karta">

    <div class="karta__naglowek">

      <h2>Decyzja dziekana</h2>

      <span class="karta__opis">

        Podejmij ostateczną decyzję w sprawie wniosku o zaliczenie praktyki na podstawie pracy zawodowej/działalności
        gospodarczej

      </span>

    </div>

    <div class="karta__tresc">

      <form method="POST">

        {{ form.hidden_tag() }}

        <div class="pole-formularza">

          {{ form.decyzja.label(class="pole-formularza__etykieta") }}

          {{ form.decyzja(class="pole-formularza__select") }}

          {% for error in form.decyzja.errors %}

            <span class="pole-formularza__blad">{{ error }}</span>

          {% endfor %}

        </div>

        <div class="pole-formularza">

          {{ form.komentarz.label(class="pole-formularza__etykieta") }}

          {{ form.komentarz(class="pole-formularza__textarea", rows="8", placeholder="Uzasadnienie decyzji dziekana
          (opcjonalne)...") }}

          {% for error in form.komentarz.errors %}

            <span class="pole-formularza__blad">{{ error }}</span>

          {% endfor %}

          <div class="pole-formularza__pomoc">

            Komentarz będzie widoczny dla studenta i w dokumentacji

          </div>

        </div>

      </div>

    </div>

```

```

<div class="akcje-formularza">

    {{ form.submit(class="przycisk przycisk--glowny") }}

    <a href="{{ url_for('zarzadzanie.dziekan_lista') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

</div>

</form>

</div>

</div>

<!-- Generowanie dokumentów -->

<div class="karta" style="margin-top: 2rem;">

    <div class="karta__naglowek">

        <h2>Dokumenty praktyki</h2>

        <span class="karta__opis">Generuj dokumenty związane z wnioskiem studenta</span>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">

            {% if zapis.track_type.value == 'EMPLOYMENT' %}

            <!-- Ścieżka B - Praca zawodowa -->

            <div class="document-card">

                <div class="document-title">ZAL_7 - Wniosek o zaliczenie</div>

                <div class="document-desc">Wniosek o zaliczenie praktyki na podstawie pracy zawodowej</div>

                <a href="{{ url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_7') }}"

                    class="przycisk przycisk--maly przycisk--glowny" target="_blank">

                    Generuj PDF

                </a>

            </div>

            <div class="document-card">

                <div class="document-title">ZAL_4a - Efekty uczenia</div>

                <div class="document-desc">Zestawienie efektów uczenia się dla pracy zawodowej</div>

                <a href="{{ url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_4a') }}"

                    class="przycisk przycisk--maly przycisk--drugorzeczny" target="_blank">

                    Generuj PDF

                </a>

            </div>

        </div>

    </div>

</div>

```

```

{% elif zapis.track_type.value == 'OWN_BUSINESS' %}

<!-- Ścieżka C - Działalność gospodarcza -->

<div class="document-card">

    <div class="document-title">ZAL_4b - Wniosek działalność</div>

    <div class="document-desc">Wniosek o zaliczenie na podstawie działalności gospodarczej</div>

    <a href="{% url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_4b') %}"

        class="przycisk przycisk--mały przycisk--główny" target="_blank">

        Generuj PDF

    </a>

</div>

<div class="document-card">

    <div class="document-title">ZAL_8 - Protokół egzaminacyjny</div>

    <div class="document-desc">Protokół z egzaminu praktycznego</div>

    <a href="{% url_for('documents.generuj_auto', id=zapis.id, doc_type='ZAL_8') %}"

        class="przycisk przycisk--mały przycisk--drugorzędny" target="_blank">

        Generuj PDF

    </a>

</div>

{% endif %}

</div>

</div>

</div>

<!-- Informacje dodatkowe -->

<div class="karta" style="margin-top: 2rem;">

    <div class="karta__naglowek">

        <h2>Dokumenty załączone przez studenta</h2>

    </div>

    <div class="karta__tresc">

        <div style="text-align: center; color: var(--kolor-tekst-drugor);">

            <p>Liczba załączonych dokumentów: <strong>{% zapis.uploaded_documents | length %}</strong></p>

            <p style="font-size: 0.9rem;">

                Szczegóły dokumentów dostępne w panelu komisji weryfikującej

            </p>

        </div>

    </div>

</div>

```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<style>
```

```
.grid-2 {
```

```
  display: grid;
```

```
  grid-template-columns: 1fr 1fr;
```

```
  gap: 2rem;
```

```
}
```

```
.info-grid {
```

```
  display: grid;
```

```
  grid-template-columns: 1fr 1fr;
```

```
  gap: 1rem;
```

```
}
```

```
.info-item {
```

```
  display: flex;
```

```
  flex-direction: column;
```

```
  gap: 0.25rem;
```

```
}
```

```
.info-item label {
```

```
  font-weight: 500;
```

```
  font-size: 0.85rem;
```

```
  color: var(--kolor-tekst-dlugor);
```

```
}
```

```
.uzasadnienie-box {
```

```
background: #f8fafc;

border: 1px solid #e2e8f0;

border-left: 4px solid var(--kolor-glowny);

border-radius: 0.5rem;

padding: 1rem;

line-height: 1.6;

}
```

```
.komisja-decyzja {

background: #f0fdf4;

border: 1px solid #bbf7d0;

border-radius: 0.5rem;

padding: 1rem;

}
```

```
.decyzja-header {

display: flex;

justify-content: space-between;

align-items: center;

margin-bottom: 0.5rem;

}
```

```
.decyzja-data {

font-size: 0.85rem;

color: var(--kolor-tekst-drugor);

}
```

```
.decyzja-komentarz {

margin-top: 0.75rem;

padding: 0.75rem;

background: white;

border-radius: 0.375rem;

font-size: 0.9rem;

line-height: 1.5;
```

```
}
```

```
.document-card {  
  
  padding: 1rem;  
  
  border: 1px solid #e5e7eb;  
  
  border-radius: 0.5rem;  
  
  background: #f9fafb;  
  
  display: flex;  
  
  flex-direction: column;  
  
  gap: 0.5rem;  
  
}
```

```
.document-title {  
  
  font-weight: 600;  
  
  font-size: 0.9rem;  
  
  color: var(--kolor-glowny);  
  
}
```

```
.document-desc {  
  
  font-size: 0.8rem;  
  
  color: var(--kolor-tekst-drugor);  
  
  flex-grow: 1;  
  
}
```

```
@media (max-width: 768px) {  
  
  .grid-2 {  
  
    grid-template-columns: 1fr;  
  
    gap: 1rem;  
  
  }  
  
  .info-grid {  
  
    grid-template-columns: 1fr;  
  
  }  
  
}
```


</style>

{% endblock %}

PLIK: app_admin\templates\zarzadzanie\dziekan\lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Decyzje dziekana{% endblock %}

{% block naglowek %}Wnioski oczekujące na decyzję dziekana{% endblock %}

{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">

    {% if wnioski.items %}

    <div class="karta">

        <div class="karta__tresc">

            <div class="tabela-wrapper">

                <table class="tabela">

                    <thead>

                        <tr>

                            <th style="width: 25%;">Student</th>

                            <th style="width: 15%;">Ścieżka</th>

                            <th style="width: 22%;">Firma / Działalność</th>

                            <th style="width: 18%;">Decyzja komisji</th>

                            <th style="width: 10%;">Data komisji</th>

                            <th style="width: 10%; text-align: center;">Akcje</th>

                        </tr>

                    </thead>

                    <tbody>

                        {% for wniosek in wnioski.items %}

                        <tr>

                            <td>

                                <div style="font-weight: 600;">{{ wniosek.student.first_name }} {{ wniosek.student.last_name }}</div>

                                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ wniosek.student.email or 'Brak email' }}</div>

                                {% if wniosek.student.album_number %}

                                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ wniosek.student.album_number }}</div>

                                {% endif %}

                            </td>

                        </tr>

                        </tbody>

                    </table>

                </div>

            </div>

        </div>

    {% endif %}

    </div>

</div>
```

```

</td>

<td>

    <span class="status

        {%- if wniosek.track_type.value == 'EMPLOYMENT' %} status--w-trakcie

        {%- elif wniosek.track_type.value == 'OWN_BUSINESS' %} status--szkic

        {%- endif %}">

        {% if wniosek.track_type.value == 'EMPLOYMENT' %}Praca zawodowa

        {% elif wniosek.track_type.value == 'OWN_BUSINESS' %}Działalność gospodarcza

        {% endif %}

    </span>

</td>

<td>

    <div style="font-weight: 600;">{{ wniosek.firma_nazwa or 'Nie podano' }}</div>

    {% if wniosek.firma_miasto %}

    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ wniosek.firma_miasto }}</div>

    {% endif %}

</td>

<td>

    <span class="status status--completed">Zatwierdzone przez komisję</span>

    {% if wniosek.komentarze_komisji %}

    <div style="margin-top: 0.4rem; font-size: 0.78rem; color: var(--kolor-tekst-drugor); line-height: 1.4;">

        {{ wniosek.komentarze_komisji[:60] }}{% if wniosek.komentarze_komisji|length > 60 %}...{% endif %}

    </div>

    {% endif %}

</td>

<td style="font-size: 0.85rem;">

    {% if wniosek.decyzja_komisji_o %}

    {{ wniosek.decyzja_komisji_o.strftime('%d.%m.%Y') }}

    <div style="font-size: 0.7rem; color: var(--kolor-tekst-drugor);">

        {{ wniosek.decyzja_komisji_o.strftime('%H:%M') }}

    </div>

    {% else %}—{% endif %}

</td>

<td style="text-align: center;">

```

```

        <a href="{{ url_for('zarzadzanie.dziekan_decyzja', id=wniosek.id) }}"

            class="przycisk przycisk--maly przycisk--glowny">

                Podejmij decyzję

            </a>

        </td>

    </tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

</div>

{% if wnioski.pages > 1 %}

<div class="paginacja">

    {% if wnioski.has_prev %}

        <a href="{{ url_for('zarzadzanie.dziekan_lista', page=wnioski.prev_num) }}" class="paginacja__link"><
Poprzednia</a>

    {% endif %}

    {% for page_num in wnioski.iter_pages() %}

        {% if page_num %}

            {% if page_num != wnioski.page %}

                <a href="{{ url_for('zarzadzanie.dziekan_lista', page=page_num) }}" class="paginacja__link">{{ page_num }}</a>

            {% else %}

                <span class="paginacja__link paginacja__link--aktywny">{{ page_num }}</span>

            {% endif %}

        {% else %}

            <span class="paginacja__kropki">...</span>

        {% endif %}

    {% endfor %}

    {% if wnioski.has_next %}

        <a href="{{ url_for('zarzadzanie.dziekan_lista', page=wnioski.next_num) }}" class="paginacja__link">Następna →</a>

    {% endif %}

</div>

{% endif %}

```

```
{% else %}

<div class="karta">

    <div class="karta__tresc" style="text-align: center; padding: 3rem 2rem;">

        <svg width="64" height="64" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.5" style="color:
var(--kolor-tekst-drugor); margin-bottom: 1rem;">

            <path d="M9 12l2 2 4-4"/>

            <path d="M21 12c0 4.97-4.03 9-9 9s-9-4.03-9-9 4.03-9 9-9c.83 0 1.63.11 2.38.32"/>

        </svg>

        <h3 style="margin-bottom: 0.5rem;">Brak wniosków oczekujących na decyzję</h3>

        <p style="color: var(--kolor-tekst-drugor); margin-bottom: 0;">

            Wszystkie wnioski zostały już przetworzone lub nie ma nowych zgłoszeń zatwierdzonych przez komisję.

        </p>

    </div>

</div>

{% endif %}

</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\enrollments\list.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Zgłoszenia studentów{% endblock %}

{% block naglowek %}Zarządzanie zgłoszeniami praktyk{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Powrót</a>

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">

    <!-- Filtry statusu -->

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__tresc">

            <div style="display: flex; gap: 1rem; align-items: center;">

                <span style="font-weight: 600;">Filtruj według statusu:</span>

                <a href="{{ url_for('zarzadzanie.lista_zgloszen') }}"

                    class="przycisk przycisk--{% if not request.args.get('status') %}glowny{% else %}drugorzeczny{% endif %} przycisk--maly">

                    Wszystkie

                </a>

                <a href="{{ url_for('zarzadzanie.lista_zgloszen', status='PENDING') }}"

                    class="przycisk przycisk--{% if request.args.get('status') == 'PENDING' %}glowny{% else %}drugorzeczny{% endif %} przycisk--maly">

                    Oczekujące

                </a>

                <a href="{{ url_for('zarzadzanie.lista_zgloszen', status='IN_PROGRESS') }}"

                    class="przycisk przycisk--{% if request.args.get('status') == 'IN_PROGRESS' %}glowny{% else %}drugorzeczny{% endif %} przycisk--maly">

                    Zatwierdzone

                </a>

            </div>

        </div>

    </div>

</div>

</div>
```

```

<a href="{{ url_for('zarzadzanie.lista_zgloszen', status='AWAITING_APPROVAL') }}"

        class="przycisk przycisk--{% if request.args.get('status') == 'AWAITING_APPROVAL' %}glowny{% else
%}drugorzeczny{% endif %} przycisk--maly">

        Nowe zgłoszenia

</a>

</div>

</div>

</div>

<!-- Lista zgłoszeń -->

<div class="karta">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">Lista zgłoszeń ({{ zgloszenia.total }})</h2>

    </div>

    <div class="karta__tresc">

        <div class="tabela-wrapper">

            <table class="tabela">

                <thead>

                    <tr>

                        <th style="width: 25%;">Student</th>

                        <th style="width: 20%;">Praktyka</th>

                        <th style="width: 20%;">Firma</th>

                        <th style="width: 15%;">Status</th>

                        <th style="width: 12%;">Data zgłoszenia</th>

                        <th style="width: 8%; text-align: center;">Akcje</th>

                    </tr>

                </thead>

                <tbody>

                    {% for z in zgloszenia.items %}

                        <tr>

                            <td>

                                <div style="font-weight: 600;">{{ z.student.first_name }} {{ z.student.last_name }}</div>

                                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ z.student.email }}</div>

                                {% if z.student.album_number %}

                                    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ z.student.album_number

```


Szczegóły


```
{% if not z.uopz and current_user.role.value == 'ADMIN' and z.track_type.value == 'STANDARD' %}
```

```
<a href="{{ url_for('zarzadzanie.przypisz_uopz', id=z.id) }}"
```

```
class="przycisk przycisk--maly"
```

```
style="min-width: 80px;">
```

Przypisz UOPZ


```
{% endif %}
```

</div>

</td>

</tr>

```
{% else %}
```

<tr>

<td colspan="6" class="tabela--pusta">

Brak zgłoszeń{% if request.args.get('status') %} o wybranym statusie{% endif %}.

</td>

</tr>

```
{% endfor %}
```

</tbody>

</table>

</div>

<!-- Paginacja -->

```
{% if zgloszenia.pages > 1 %}
```

```
<div style="display: flex; justify-content: center; margin-top: 2rem; gap: 0.5rem;">
```

```
{% if zgloszenia.has_prev %}
```

```
<a href="{{ url_for('zarzadzanie.lista_zgloszen', page=zgloszenia.prev_num, **request.args) }}"
```

```
class="przycisk przycisk--drugorzecny przycisk--maly">← Poprzednia</a>
```

```
{% endif %}
```

```
{% for page_num in zgloszenia.iter_pages() %}
```

```
{% if page_num %}
```

```

{% if page_num != zgloszenia.page %}

<a href="{{ url_for('zarzadzanie.lista_zgloszen', page=page_num, **request.args) }}"

    class="przycisk przycisk--drugorzeczny przycisk--maly">{{ page_num }}</a>

{% else %}

<span class="przycisk przycisk--glowny przycisk--maly">{{ page_num }}</span>

{% endif %}

{% else %}

    <span>...</span>

{% endif %}

{% endfor %}


{% if zgloszenia.has_next %}

<a href="{{ url_for('zarzadzanie.lista_zgloszen', page=zgloszenia.next_num, **request.args) }}"

    class="przycisk przycisk--drugorzeczny przycisk--maly">Następna →</a>

{% endif %}

</div>

{% endif %}


</div>

</div>

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\enrollments\moje_lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Moje zgłoszenia - UOPZ{% endblock %}

{% block naglowek %}Zgłoszenia moich studentów{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Dashboard</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">

    <!-- Info dla UOPZ -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-info);">

        <div class="karta__tresc">

            <div>

                <div style="font-weight: 600; margin-bottom: 0.5rem;">Panel opiekuna uczelnianego (UOPZ)</div>

                <div style="color: var(--kolor-tekst-drugor);">

                    Tutaj znajdziesz wszystkie zgłoszenia studentów, których masz pod opieką.

                    Możesz przeglądać szczegółowe dane wypełnione przez studentów i podejmować decyzje o zatwierdzeniu lub
                    odrzuceniu zgłoszenia.

                </div>

            </div>

        </div>

    </div>

    <!-- Filtry statusu -->

    <div class="karta" style="margin-bottom: 2rem;">

        <div class="karta__tresc">

            <div style="display: flex; gap: 1rem; align-items: center; flex-wrap: wrap;">

                <span style="font-weight: 600;">Filtruj według statusu:</span>

            </div>

        </div>

    </div>

</div>
```

[illegible]

Dziś ostatni dzień na ocenę

{% else %}

Pozostało {{ item.dni_do_deadline }} dni do deadline ({{ item.deadline.strftime('%d.%m.%Y') }})

{% endif %}

</div>

</div>

<div>

<a href="{{ url_for('evaluation.ocen_praktyke', id=item.zapis.id) }}"

class="przycisk przycisk--glowny przycisk--maly">

Wystaw ocenę

</div>

</div>

</div>

{% endfor %}

</div>

</div>

{% endif %}

<!-- Lista zgłoszeń -->

<div class="karta">

<div class="karta__naglowek">

<h2 class="karta__tytul">

Moje zgłoszenia

{% if zgloszenia.total > 0 %}

{{ zgloszenia.total }}

{% endif %}

</h2>

</div>

<div class="karta__tresc">

{% if zgloszenia.items %}

<div class="tabela-wrapper">

<table class="tabela">

<thead>

```

<tr>

    <th style="width: 25%;">Student</th>

    <th style="width: 20%;">Praktyka</th>

    <th style="width: 20%;">Firma</th>

    <th style="width: 15%;">Status</th>

    <th style="width: 12%;">Zgłoszono</th>

    <th style="width: 8%; text-align: center;">Akcje</th>

</tr>

</thead>

<tbody>

    {% for z in zgloszenia.items %}

    <tr>

        <td>

            <div style="font-weight: 600;">{{ z.student.first_name }} {{ z.student.last_name }}</div>

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ z.student.email }}</div>

            {% if z.student.album_number %}

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ z.student.album_number
    }}</div>

            {% endif %}

        </td>

        <td>

            <div style="font-weight: 600;">{{ z.praktyka.rok_uczelnianny }}</div>

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ z.praktyka.semestr }}</div>

            {% if z.termin_od and z.termin_do %}

                <div style="font-size: 0.75rem; color: var(--kolor-tekst-drugor); margin-top: 0.25rem;">

                    {{ z.termin_od.strftime('%d.%m') }} - {{ z.termin_do.strftime('%d.%m.%Y') }}

                </div>

            {% endif %}

        </td>

        <td>

            <div style="font-weight: 600;">{{ z.firma_nazwa or 'Nie podano' }}</div>

            {% if z.firma_miasto %}

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ z.firma_miasto }}</div>

            {% endif %}

            {% if z.track_type.value != 'STANDARD' %}

```

```

<div style="font-size: 0.75rem; color: var(--kolor-info); margin-top: 0.25rem;">

    {% if z.track_type.value == 'EMPLOYMENT' %}Na podstawie zatrudnienia

    {% elif z.track_type.value == 'OWN_BUSINESS' %}Własna działalność

    {% endif %}

</div>

{% endif %}

</td>

<td>

    <span class="status status--{{ z.status.value|lower|replace('_', '-') }}">

        {{ tlumacz_status(z.status.value) }}

    </span>

    {% if z.komentarze_admina or z.komentarze_uopz %}

    <div style="font-size: 0.7rem; color: var(--kolor-info); margin-top: 0.25rem;">

        Są komentarze

    </div>

    {% endif %}

</td>

<td style="font-size: 0.85rem;">

    {{ z.enrolled_at.strftime('%d.%m.%Y') if z.enrolled_at else '-' }}

    <div style="font-size: 0.7rem; color: var(--kolor-tekst-drugor);">

        {{ z.enrolled_at.strftime('%H:%M') if z.enrolled_at else '' }}

    </div>

</td>

<td style="text-align: center;">

    <div style="display: flex; flex-direction: column; gap: 0.5rem; align-items: center;">

        <a href="{{ url_for('zarzadzanie.szczegoly_zgloszenia', id=z.id) }}"

            class="przycisk przycisk--{% if z.status.value == 'AWAITING_APPROVAL' %}glowny{% else
%}drugorzeczny{% endif %} przycisk--maly"

            style="min-width: 80px;">

            {% if z.status.value == 'AWAITING_APPROVAL' %}Oceń{% else %}Szczegóły{% endif %}

        </a>

        {% if z.status.value == 'IN_PROGRESS' %}

        <a href="{{ url_for('evaluation.ocen_praktyke', id=z.id) }}"

            class="przycisk przycisk--maly"

```

```

        style="min-width: 80px; ">

        Ocena UOPZ

    </a>

    {% endif %}

</div>

</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

<!-- Paginacja -->

{% if zgloszenia.pages > 1 %}

<div style="display: flex; justify-content: center; margin-top: 2rem; gap: 0.5rem;">

    {% if zgloszenia.has_prev %}

    <a href="{{ url_for('zarzadzanie.moje_zgloszenia', page=zgloszenia.prev_num, **request.args) }}"

        class="przycisk przycisk--drugorzecny przycisk--maly">← Poprzednia</a>

    {% endif %}

    {% for page_num in zgloszenia.iter_pages() %}

        {% if page_num %}

            {% if page_num != zgloszenia.page %}

                <a href="{{ url_for('zarzadzanie.moje_zgloszenia', page=page_num, **request.args) }}"

                    class="przycisk przycisk--drugorzecny przycisk--maly">{{ page_num }}</a>

            {% else %}

                <span class="przycisk przycisk--glowny przycisk--maly">{{ page_num }}</span>

            {% endif %}

        {% else %}

            <span>...</span>

        {% endif %}

    {% endfor %}

    {% if zgloszenia.has_next %}

```



```

<a href="{{ url_for('zarzadzanie.moje_zgloszenia', page=zgloszenia.next_num, **request.args) }}"

    class="przycisk przycisk--drugorzeczny przycisk--maly">Następna →</a>

{% endif %}

</div>

{% endif %}


{% else %}

<!-- Pusty stan -->

<div style="text-align: center; padding: 4rem 2rem;">

    <div style="font-size: 3rem; margin-bottom: 1rem; opacity: 0.3;"></div>

    <div style="font-size: 1.2rem; font-weight: 600; margin-bottom: 1rem; color: var(--kolor-tekst-drugor);">

        Brak zgłoszeń

    </div>

    <div style="color: var(--kolor-tekst-drugor);">

        {% if request.args.get('status') %}

            Nie masz zgłoszeń o wybranym statusie.

        {% else %}

            Nie zostały Ci jeszcze przypisani studenci do opieki.

        {% endif %}

    </div>

    {% if request.args.get('status') %}

        <div style="margin-top: 2rem;">

            <a href="{{ url_for('zarzadzanie.moje_zgloszenia') }}" class="przycisk przycisk--drugorzeczny">

                Pokaż wszystkie zgłoszenia

            </a>

        </div>

        {% endif %}

    </div>

    {% endif %}

</div>

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\enrollments\przypisz_uopz.html

```
{% extends "layouts/base.html" %}

{% block tytul %}Przypisz UOPZ{% endblock %}

{% block tresc %}

<h1>Przypisz opiekuna uczelnianego</h1>

<div class="karta">

    <form method="post">

        {{ form.hidden_tag() }}

        <div class="pole-form">

            <label>{{ form.uopz_id.label }}</label>

            {{ form.uopz_id(class_='select') }}

        </div>

        <div style="margin-top:1rem;">

            <button type="submit" class="przycisk przycisk--glowny">Zapisz</button>

            <a href="{{ url_for('zarzadzanie.lista_zgloszen') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

        </div>

    </form>

</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\enrollments\szczegoly.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Szczegóły zgłoszenia – {{ zapis.student.first_name }} {{ zapis.student.last_name }}{% endblock %}

{% block naglowek %}Szczegóły zgłoszenia{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    {% if current_user.role.value == 'ADMIN' %}

        <a href="{{ url_for('zarzadzanie.lista_zgloszen') }}" class="przycisk przycisk--drugorzeczny">← Powrót do listy</a>

    {% else %}

        <a href="{{ url_for('zarzadzanie.moje_zgloszenia') }}" class="przycisk przycisk--drugorzeczny">← Moje zgłoszenia</a>

    {% endif %}

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto;">

    {# — Karta identyfikacyjna zgłoszenia — #}

    {% set status_val = zapis.status.value %}

    {% if status_val == 'PENDING' %}

        {% set border_color = 'var(--kolor-akcent)' %}

        {% set status_cls = 'status--in-progress' %}

        {% set status_label = 'Oczekuje na zatwierdzenie' %}

    {% elif status_val == 'AWAITING_APPROVAL' %}

        {% set border_color = '#3b82f6' %}

        {% set status_cls = 'status--planned' %}

        {% set status_label = 'Oczekuje na akceptację UOPZ' %}

    {% elif status_val == 'IN_PROGRESS' %}

        {% set border_color = 'var(--kolor-sukces)' %}

        {% set status_cls = 'status--completed' %}

        {% set status_label = 'Zatwierdzona – w realizacji' %}

    {% elif status_val == 'COMPLETED' %}
```

```

{% set border_color = 'var(--kolor-glowny)' %}

{% set status_cls = 'status--completed' %}

{% set status_label = 'Zakończona' %}

{% else %}

{% set border_color = 'var(--kolor-ramka)' %}

{% set status_cls = 'status--draft' %}

{% set status_label = status_val %}

{% endif %}

<div style="background: var(--kolor-biale); border-radius: var(--promien-duzy); border-top: 4px solid {{ border_color
}}; box-shadow: var(--cien-karta); margin-bottom: 2rem; overflow: hidden;">

    <div style="padding: 1.75rem 2rem; display: grid; grid-template-columns: auto 1fr auto; gap: 2rem; align-items:
center;">

        {# Awatar inicjałowy #}

        <div style="width: 56px; height: 56px; border-radius: 50%; background: var(--kolor-glowny); color: #fff; display:
flex; align-items: center; justify-content: center; font-size: 1.25rem; font-weight: 700; flex-shrink: 0; letter-spacing:
0.5px;">

            {{ zapis.student.first_name[0] }}{{ zapis.student.last_name[0] }}

        </div>

        {# Dane studenta i praktyki #}

        <div style="display: grid; grid-template-columns: repeat(4, 1fr); gap: 1.5rem; min-width: 0;">

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Student</div>

                <div style="font-weight: 700; font-size: 1rem; color: var(--kolor-tekst);">{{ zapis.student.first_name }} {{
zapis.student.last_name }}</div>

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-top: 0.15rem;">{{ zapis.student.email
}}</div>

                {% if zapis.student.album_number %}

                    <div style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed);">nr alb. {{ zapis.student.album_number
}}</div>

                {% endif %}

            </div>

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Rok akademicki</div>

                <div style="font-weight: 600; color: var(--kolor-tekst);">{{ zapis.praktyka.rok_uczelnianny }}</div>

```

```

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ zapis.praktyka.semestr }}</div>

    </div>

    <div>

        <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Ścieżka</div>

        <div style="font-weight: 600; color: var(--kolor-tekst);">

            {% if zapis.track_type.value == 'STANDARD' %}Standardowa

            {% elif zapis.track_type.value == 'EMPLOYMENT' %}Na podst. zatrudnienia

            {% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność

            {% else %}{{ zapis.track_type.value }}{% endif %}

        </div>

    </div>

    <div>

        <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Data zgłoszenia</div>

        <div style="font-weight: 600; color: var(--kolor-tekst);">{{ zapis.enrolled_at.strftime('%d.%m.%Y') if
zapis.enrolled_at else '-' }}</div>

        <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ zapis.enrolled_at.strftime('%H:%M') if
zapis.enrolled_at else '' }}</div>

    </div>

</div>

    {# Odznaka statusu #}

    <div style="text-align: right; flex-shrink: 0;">

        <span class="status {{ status_cls }}" style="font-size: 0.75rem; padding: 0.4rem 1rem;">{{ status_label }}</span>

        {% if zapis.uopz %}

            <div style="margin-top: 0.75rem; font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); text-align: right;">

                UOPZ: <strong style="color: var(--kolor-tekst-drugor);">{{ zapis.uopz.first_name }} {{ zapis.uopz.last_name
                }}</strong>

            </div>

            {% endif %}

        </div>

</div>

</div>

```

```
{# — Główna siatka treści — #}
```

```
<div style="display: grid; grid-template-columns: 1fr 480px; gap: 1.5rem; align-items: start;">
```

```
{# == LEWA KOLUMNA == #}
```

```
<div style="display: flex; flex-direction: column; gap: 1.5rem;">
```

```
{# Dane podstawowe #}
```

```
<div class="karta">
```

```
<div class="karta__naglowek">
```

```
<h3 class="karta__tytul">Dane podstawowe</h3>
```

```
</div>
```

```
<div class="karta__tresc">
```

```
<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">
```

```
<div class="pole-info">
```

```
<div class="pole-info__etykieta">Ścieżka praktyki</div>
```

```
<div class="pole-info__wartosc">
```

```
{% if zapis.track_type.value == 'STANDARD' %}Standardowa
```

```
{% elif zapis.track_type.value == 'EMPLOYMENT' %}Na podstawie zatrudnienia
```

```
{% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność gospodarcza
```

```
{% else %}{{ zapis.track_type.value }}{% endif %}
```

```
</div>
```

```
</div>
```

```
<div class="pole-info">
```

```
<div class="pole-info__etykieta">Specjalność</div>
```

```
<div class="pole-info__wartosc">{{ zapis.specjalnosc or '-' }}</div>
```

```
</div>
```

```
{% if zapis.track_type.value == 'STANDARD' %}
```

```
<div class="pole-info">
```

```
<div class="pole-info__etykieta">Termin praktyki</div>
```

```
<div class="pole-info__wartosc">
```

```
{{ zapis.termin_od.strftime('%d.%m.%Y') if zapis.termin_od else 'Brak' }}
```

```
<span style="color: var(--kolor-tekst-trzeciorzed); margin: 0 0.25rem;"></span>
```

```
{{ zapis.termin_do.strftime('%d.%m.%Y') if zapis.termin_do else 'Brak' }}
```

```
</div>
```

```

</div>

<div class="pole-info">

  <div class="pole-info__etykieta">Ubezpieczenie NW</div>

  <div class="pole-info__wartosc">

    {% if zapis.ubezpieczenie_nw %}

      <span style="color: var(--kolor-sukces); display: inline-flex; align-items: center; gap: 0.35rem;">

        <span style="width: 8px; height: 8px; border-radius: 50%; background: currentColor; display: inline-block;"></span>

        Potwierdzone

      </span>

    {% else %}

      <span style="color: var(--kolor-blad); display: inline-flex; align-items: center; gap: 0.35rem;">

        <span style="width: 8px; height: 8px; border-radius: 50%; background: currentColor; display: inline-block;"></span>

        Nie potwierdzone

      </span>

    {% endif %}

  </div>

</div>

{% endif %}

</div>

</div>

```

```

{% if zapis.track_type.value == 'STANDARD' %}

```

```

{# Dane zakładu pracy #}

```

```

<div class="karta">

  <div class="karta__naglowek">

    <h3 class="karta__tytul">Zakład pracy</h3>

  </div>

  <div class="karta__tresc">

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

      <div class="pole-info">

        <div class="pole-info__etykieta">Nazwa firmy</div>

```

```

        <div class="pole-info__wartosc" style="font-weight: 600;">{{ zapis.firma_nazwa or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">NIP / KRS</div>

    <div class="pole-info__wartosc">{{ zapis.firma_nip_krs or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">Adres</div>

    <div class="pole-info__wartosc">{{ zapis.firma_adres or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">Miasto</div>

    <div class="pole-info__wartosc">{{ zapis.firma_miasto or '-' }}</div>

</div>

<div class="pole-info" style="grid-column: 1 / -1;">

    <div class="pole-info__etykieta">Przedstawiciel firmy</div>

    <div class="pole-info__wartosc">

        {{ zapis.firma_upowazniony_osoba or '-' }}

        {% if zapis.firma_upowazniony_stanowisko %}

            <span style="color: var(--kolor-tekst-drugor); font-weight: 400;"> · {{
zapis.firma_upowazniony_stanowisko }}</span>

            {% endif %}

        </div>

    </div>

</div>

</div>

</div>

{{# Opiekun zakładowy (ZOPZ) #}}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Opiekun zakładowy (ZOPZ)</h3>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

```



```

<div class="pole-info">

    <div class="pole-info__etykieta">Imię i nazwisko</div>

    <div class="pole-info__wartosc" style="font-weight: 600;">{{ zapis.zopz_imie_nazwisko or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">Stanowisko</div>

    <div class="pole-info__wartosc">{{ zapis.zopz_stanowisko or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">Telefon</div>

    <div class="pole-info__wartosc">{{ zapis.zopz_telefon or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">E-mail</div>

    <div class="pole-info__wartosc">{{ zapis.zopz_email or '-' }}</div>

</div>

</div>

</div>

</div>

```

```
{# Harmonogram #}
```

```
<div class="karta">
```

```
    <div class="karta__naglowek" style="display: flex; justify-content: space-between; align-items: center;">
```

```
        <h3 class="karta__tytul">Harmonogram praktyki</h3>
```

```
        {% set suma_dni = namespace(wartosc=0) %}
```

```
        {% for e in efekty %}
```

```
            {% set h = harmonogram_dict.get(e.id) %}
```

```
            {% if h and h.liczba_dni > 0 %}{% set suma_dni.wartosc = suma_dni.wartosc + h.liczba_dni %}{% endif %}
```

```
        {% endfor %}
```

```

                                <span class="status" {% if suma_dni.wartosc > 120 %}status--draft"
style="background:#fee2e2;color:var(--kolor-blad){% elif suma_dni.wartosc < 100 %}status--in-progress{% else
%}status--completed{% endif %}">

```

```
                                {{ suma_dni.wartosc }} / 120 dni
```

```
                                </span>
```

```
    </div>
```

```
</div>
```

```
<div class="karta__tresc" style="padding: 0;">
```

```
<div class="tabela-wrapper">
```

```
<table class="tabela">
```

```
<thead>
```

```
<tr>
```

```
<th style="width: 4%; text-align: center;">Nr</th>
```

```
<th style="width: 38%;">Efekt uczenia się</th>
```

```
<th style="width: 25%;">Dział / Jednostka</th>
```

```
<th style="width: 25%;">Przykładowe zadania</th>
```

```
<th style="width: 8%; text-align: center;">Dni</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{% for e in efekty %}
```

```
{% set h = harmonogram_dict.get(e.id) %}
```

```
{% set brak = not h or h.liczba_dni == 0 %}
```

```
<tr {% if brak %}style="opacity: 0.55;"{% endif %}>
```

```
<td style="text-align: center; font-weight: 600; color: var(--kolor-tekst-trzeciorzed); font-size: 0.8rem;">{{ '%02d'|format(e.id) }}</td>
```

```
<td style="font-size: 0.82rem; line-height: 1.45;">{{ e.description }}</td>
```

```
<td style="font-size: 0.85rem;">
```

```
{% if h and h.nazwa_dzialu %}{{ h.nazwa_dzialu }}
```

```
{% else %}<span style="color: var(--kolor-tekst-trzeciorzed); font-style: italic;">Nie przypisano</span>{% endif %}
```

```
</td>
```

```
<td style="font-size: 0.85rem;">
```

```
{% if h and h.przykladowe_prace %}{{ h.przykladowe_prace }}
```

```
{% else %}<span style="color: var(--kolor-tekst-trzeciorzed); font-style: italic;">Nie podano</span>{% endif %}
```

```
</td>
```

```
<td style="text-align: center; font-weight: 700; font-size: 0.95rem;
```

```
color: {% if brak %}var(--kolor-tekst-trzeciorzed){% else %}var(--kolor-glowny){% endif %};">
```

```
{{ h.liczba_dni if h else 0 }}
```

```
</td>
```

```
</tr>
```

```
{% endfor %}
```

```

</tbody>

<tfoot>

    <tr style="background: var(--kolor-tlo);">

        <td colspan="4" style="text-align: right; font-weight: 600; padding-right: 1rem; font-size: 0.85rem;
color: var(--kolor-tekst-drugor);">Suma dni:</td>

        <td style="text-align: center; font-weight: 700; font-size: 1.05rem;

            color: {% if suma_dni.wartosc > 120 %}var(--kolor-blad){% elif suma_dni.wartosc < 100 %}#92400e{%
else %}var(--kolor-sukces){% endif %};">

            {{ suma_dni.wartosc }}

        </td>

    </tr>

</tfoot>

</table>

</div>

{% if suma_dni.wartosc > 120 %}

    <div style="margin: 1rem 1.5rem; padding: 0.75rem 1rem; background: #fee2e2; border-radius: var(--promien);
color: var(--kolor-blad); font-size: 0.85rem; display: flex; gap: 0.5rem; align-items: flex-start;">

        <strong>Przekroczono limit 120 dni.</strong> Harmonogram wymaga korekty przed zatwierdzeniem.

    </div>

    {% elif suma_dni.wartosc < 100 %}

        <div style="margin: 1rem 1.5rem; padding: 0.75rem 1rem; background: #fef3c7; border-radius: var(--promien);
color: #92400e; font-size: 0.85rem; display: flex; gap: 0.5rem; align-items: flex-start;">

            <strong>Niewystarczająca liczba dni (min. 100).</strong> Harmonogram wymaga uzupełnienia.

        </div>

    {% endif %}

</div>

</div>

{% else %}

{# — Ścieżka B/C — #}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Dane pracodawcy / działalności</h3>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

```

```

<div class="pole-info">

    <div class="pole-info__etykieta">Nazwa firmy / działalności</div>

    <div class="pole-info__wartosc" style="font-weight: 600;">{{ zapis.firma_nazwa or '-' }}</div>

</div>

<div class="pole-info">

    <div class="pole-info__etykieta">Stanowisko / zakres obowiązków</div>

    <div class="pole-info__wartosc">{{ zapis.zopz_stanowisko or '-' }}</div>

</div>

<div class="pole-info" style="grid-column: 1 / -1;">

    <div class="pole-info__etykieta">Adres</div>

    <div class="pole-info__wartosc">{{ zapis.firma_adres or '-' }}{% if zapis.firma_miasto %}, {{
zapis.firma_miasto }}{% endif %}</div>

</div>

</div>

</div>

{% if zapis.uzasadnienie_sciezki %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Uzasadnienie wniosku</h3>

    </div>

    <div class="karta__tresc">

        <p style="white-space: pre-line; font-size: 0.9rem; line-height: 1.7; color: var(--kolor-tekst);">{{
zapis.uzasadnienie_sciezki }}</p>

    </div>

</div>

{% endif %}

{% if zapis.decyzja_komisji or zapis.decyzja_dziekana %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Decyzje</h3>

    </div>

    <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 1rem;">

```

```

{% if zapis.decyzja_komisji %}

    <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo);">

        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.5rem;">

            <span style="font-size: 0.75rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em;
color: var(--kolor-tekst-trzeciorzed);">Komisja ds. praktyk</span>

            <span class="status {% if zapis.decyzja_komisji == 'APPROVED' %}status--completed{% elif
zapis.decyzja_komisji == 'REJECTED' %}status--draft" style="background:#fee2e2;color:var(--kolor-blad){% else
%}status--in-progress{% endif %}">

                {% if zapis.decyzja_komisji == 'APPROVED' %}Zatwierdzono

                {% elif zapis.decyzja_komisji == 'PARTIALLY_APPROVED' %}Częściowo zatwierdzono

                {% else %}Odrzucono{% endif %}

            </span>

        </div>

        {% if zapis.komentarze_komisji %}<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); line-height:
1.5; margin: 0;">{{ zapis.komentarze_komisji }}</p>{% endif %}

    </div>

{% endif %}

{% if zapis.decyzja_dziekana %}

    <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo);">

        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.5rem;">

            <span style="font-size: 0.75rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em;
color: var(--kolor-tekst-trzeciorzed);">Dyrektor IIS</span>

            <span class="status {% if zapis.decyzja_dziekana == 'APPROVED' %}status--completed{% else %}status--draft"
style="background:#fee2e2;color:var(--kolor-blad){% endif %}">

                {{ 'Zatwierdzono' if zapis.decyzja_dziekana == 'APPROVED' else 'Odrzucono' }}

            </span>

        </div>

        {% if zapis.komentarze_dziekana %}<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);
line-height: 1.5; margin: 0;">{{ zapis.komentarze_dziekana }}</p>{% endif %}

    </div>

{% endif %}

</div>

{% endif %}

{% endif %}

{# Załączniki studenta #}

```

```

{% if uploaded_docs %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Załączniki studenta</h3>

        <span style="font-size: 0.75rem; font-weight: 400; color: var(--kolor-tekst-trzeciorzed);">{{
uploaded_docs|length }} plik{% if uploaded_docs|length != 1 %}i{% endif %}</span>

    </div>

    <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 0.5rem;">

        {% for dok in uploaded_docs %}

            <div style="display: flex; justify-content: space-between; align-items: center; padding: 0.75rem 1rem; border:
1px solid var(--kolor-ramka); border-radius: var(--promien); background: var(--kolor-tlo); transition: border-color
0.15s;">

                <div style="min-width: 0;">

                    <div style="font-weight: 600; font-size: 0.875rem; white-space: nowrap; overflow: hidden; text-overflow:
ellipsis;">{{ dok.original_filename }}</div>

                    <div style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); margin-top: 0.15rem;">{{
dok.document_type }} · {{ dok.uploaded_at.strftime('%d.%m.%Y, %H:%M') }}</div>

                </div>

                <a href="{{ url_for('uploads.download_document', document_id=dok.id) }}"

                    class="przycisk przycisk--maly przycisk--drugorzeczny" style="flex-shrink: 0; margin-left:
1rem;">Pobierz</a>

            </div>

        {% endfor %}

    </div>

</div>

{% endif %}

</div>

{# == PRAWA KOLUMNA == #}

<div style="display: flex; flex-direction: column; gap: 1.5rem; position: sticky; top: 1.5rem;">

    {# Panel decyzji #}

    {% if zapis.status.value in ['PENDING', 'AWAITING_APPROVAL'] %}

    <div class="karta">

        <div class="karta__naglowek">

```

```

    <h3 class="karta__tytul">Decyzja</h3>

</div>

<div class="karta__tresc">

    <form method="POST">

        {{ form.hidden_tag() }}

        <div style="margin-bottom: 1.25rem;">

            <label class="pole-formularza__etykieta" style="display: block; margin-bottom: 0.4rem;">Komentarz do
studenta <span style="font-weight: 400; color: var(--kolor-tekst-trzeciorzed);">(opcjonalny)</span></label>

            {{ form.komentarz(class="pole-formularza__textarea", rows="7", placeholder="Uwagi widoczne dla studenta w
panelu praktyk...", style="width: 100%; box-sizing: border-box;") }}

        </div>

        <div style="border-top: 1px solid var(--kolor-ramka); padding-top: 1.25rem; display: flex; flex-direction:
column; gap: 0.5rem;">

            {{ form.zatwierdz(class="przycisk przycisk--glowny", style="width: 100%; justify-content: center;") }}

            {{ form.odrzuc(class="przycisk przycisk--drugorzecny", style="width: 100%; justify-content: center;") }}

        </div>

    </form>

</div>

</div>

{% else %}

{# Stan po zatwierdzeniu #}

<div class="karta">

    <div class="karta__tresc" style="text-align: center; padding: 2rem 1.5rem;">

        <div style="width: 48px; height: 48px; border-radius: 50%; background: #d1fae5; display: flex; align-items:
center; justify-content: center; margin: 0 auto 1rem; font-size: 1.35rem; color: var(--kolor-sukces);"></div>

        <div style="font-weight: 700; color: var(--kolor-sukces); font-size: 1rem; margin-bottom: 0.4rem;">Zgłoszenie
zatwierdzone</div>

        <div style="color: var(--kolor-tekst-drugor); font-size: 0.875rem; line-height: 1.5;">Praktyka jest w trakcie
realizacji przez studenta.</div>

    </div>

</div>

{% endif %}

```

```

    {# Blok przypisania UOPZ #}

    {% if not zapis.uopz and current_user.role.value == 'ADMIN' and zapis.track_type.value == 'STANDARD' %}

        <div style="padding: 1rem; border: 1px dashed var(--kolor-ramka); border-radius: var(--promien); text-align:
center;">

            <div style="font-size: 0.8rem; color: var(--kolor-tekst-trzeciorzed); margin-bottom: 0.75rem;">Brak przypisanego
opiekuna uczelnianego</div>

            <a href="{{ url_for('zarzadzanie.przypisz_uopz', id=zapis.id) }}" class="przycisk przycisk--maly" style="width:
100%;">Przypisz UOPZ</a>

        </div>

    {% endif %}

</div>

</div>

</div>

</div>

</div>

<style>

.pole-info__etykieta {

    font-size: 0.7rem;

    font-weight: 600;

    text-transform: uppercase;

    letter-spacing: 0.06em;

    color: var(--kolor-tekst-trzeciorzed);

    margin-bottom: 0.3rem;

}

.pole-info__wartosc {

    font-size: 0.9rem;

    color: var(--kolor-tekst);

    line-height: 1.45;

}

</style>

{% endblock %}

```


PLIK: app_admin\templates\zarzadzanie\firmy\formularz.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}{% if tryb == 'dodaj' %}Dodaj firmę{% else %}Edytuj firmę{% endif %}{% endblock %}

{% block naglowek %}{% if tryb == 'dodaj' %}Dodawanie nowej firmy{% else %}Edycja firmy{% endif %}{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('zarzadzanie.lista_firm') }}" class="przycisk przycisk--drugorzeczny"><- Powrót do listy</a>

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">

    <div class="karta">

        <div class="karta__naglowek">

            <h2 class="karta__tytul">

                {% if tryb == 'dodaj' %}

                Dane nowej firmy

                {% else %}

                Edycja danych firmy: {{ firma.nazwa }}

                {% endif %}

            </h2>

        </div>

        <div class="karta__tresc">

            <form method="POST" novalidate>

                {{ form.hidden_tag() }}

                <div style="display: grid; grid-template-columns: 1fr; gap: 1.5rem; margin-bottom: 2rem;">

                    <!-- Nazwa firmy -->

                    <div class="pole-formularza {% if form.nazwa.errors %}pole-formularza--blad{% endif %}">

                        {{ form.nazwa.label(class="pole-formularza__etykieta") }}
```

```

    {{ form.nazwa(class="pole-formularza__input") }}

    {% for e in form.nazwa.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

</div>


<!-- Adres -->

<div class="pole-formularza {% if form.adres.errors %}pole-formularza--blad{% endif %}">

    {{ form.adres.label(class="pole-formularza__etykieta") }}

    {{ form.adres(class="pole-formularza__input") }}

    {% for e in form.adres.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

    <small style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">

        Pełny adres, np. "ul. Kolejowa 15/3"

    </small>

</div>


<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem;">

    <!-- Miasto -->

    <div class="pole-formularza {% if form.miasto.errors %}pole-formularza--blad{% endif %}">

        {{ form.miasto.label(class="pole-formularza__etykieta") }}

        {{ form.miasto(class="pole-formularza__input") }}

        {% for e in form.miasto.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

    </div>


    <!-- NIP/KRS -->

    <div class="pole-formularza {% if form.nip_krs.errors %}pole-formularza--blad{% endif %}">

        {{ form.nip_krs.label(class="pole-formularza__etykieta") }}

        {{ form.nip_krs(class="pole-formularza__input") }}

        {% for e in form.nip_krs.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

        <small style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">

            NIP lub numer KRS

        </small>

    </div>

</div>

</div>

```

```
<div style="margin-bottom: 1.5rem; padding: 1rem; background: #f0f9ff; border-radius: 0.5rem; border-left: 4px solid #0ea5e9;">

  <div style="font-weight: 500; color: #0c4a6e; margin-bottom: 0.5rem; display: flex; align-items: center; gap: 0.5rem;">

    <svg viewBox="0 0 24 24" style="width: 1rem; height: 1rem; fill: currentColor;">

      <circle cx="12" cy="12" r="10"/>

      <line x1="12" y1="16" x2="12" y2="12"/>

      <line x1="12" y1="8" x2="12.01" y2="8"/>

    </svg>

    Informacja o firmach w systemie

  </div>

  <div style="color: #0c4a6e; font-size: 0.9rem; line-height: 1.4;">

    Wszystkie firmy dodawane do systemu mają <strong>stałą umowę z uczelnią</strong>. Studenci którzy wybierają firmy z tej bazy nie muszą generować Załącznika 3 (Porozumienia). Studenci mogą też podać własne firmy podczas zapisów na praktyki.

  </div>

</div>

<div style="display: flex; justify-content: space-between; align-items: center;">

  <a href="{ url_for('zarzadzanie.lista_firm') }}" class="przycisk przycisk--drugorzeczny">

    Anuluj

  </a>

  <button type="submit" class="przycisk przycisk--glowny">

    {% if tryb == 'dodaj' %}

      Dodaj firmę

    {% else %}

      Zapisz zmiany

    {% endif %}

  </button>

</div>

</form>

</div>

</div>
```

</div>

{% endblock %}

PLIK: app_admin\templates\zarzadzanie\firmy\lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Zarządzanie firmami{% endblock %}

{% block naglowek %}Firmy w systemie{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('zarzadzanie.dodaj_firme') }}" class="przycisk przycisk--glowny">+ Dodaj firmę</a>

    <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Dashboard</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">

    <!-- Info o systemie -->

    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-info);">

        <div class="karta__tresc">

            <div>

                <div style="font-weight: 600; margin-bottom: 0.5rem;">System zarządzania firmami</div>

                <div style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

                    Firma z flagą <strong>Stała umowa</strong> nie wymagają generowania Załącznika 3 (Porozumienia).

                    Studenci mogą wybierać firmy z bazy podczas rejestracji na praktykę.

                </div>

            </div>

        </div>

    </div>

</div>


<!-- Lista firm -->

<div class="karta">

    <div class="karta__naglowek">

        <form method="GET" class="pasek-filtrow" role="search" aria-label="Filtruj firmy">

            <div class="filtr-szukaj">
```

```

<span class="filtr-szukaj__ikona" aria-hidden="true">

    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round">

        <circle cx="11" cy="11" r="8"></circle>

        <path d="m21 21-4.3-4.3"></path>

    </svg>

</span>

<input

    type="search"

    name="szukaj"

    id="szukaj"

    class="filtr-szukaj__input"

    value="{{ request.args.get('szukaj', '') }}"

    placeholder="Szukaj po nazwie, adresie, NIP/KRS..."

    aria-label="Szukaj firmy"

    >

</div>

<label for="status" class="sr-only">Filtruj po statusie</label>

<select name="status" id="status" class="filtr-select" onchange="this.form.submit()">

    <option value="wszystkie" {% if request.args.get('status', 'wszystkie') == 'wszystkie' %}>selected{% endif
%}>Wszystkie</option>

    <option value="aktywne" {% if request.args.get('status') == 'aktywne' %}>selected{% endif %}>Tylko
aktywne</option>

    <option value="nieaktywne" {% if request.args.get('status') == 'nieaktywne' %}>selected{% endif %}>Tylko
nieaktywne</option>

</select>

<button type="submit" class="przycisk przycisk--drugorzecny">Szukaj</button>

{% if request.args.get('szukaj') or request.args.get('status', 'aktywne') != 'aktywne' %}

    <a href="{{ url_for('zarzadzanie.lista_firm') }}" class="przycisk przycisk--drugorzecny">

        Wyczyść filtry

    </a>

{% endif %}

</form>

<span style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

    {{ firmy.total }} firm

```

```

</span>

</div>

<div class="karta__tresc">

  {% if firmy.items %}

  <div class="tabela-wrapper">

    <table class="tabela">

      <thead>

        <tr>

          <th style="width: 30%;">Nazwa firmy</th>

          <th style="width: 25%;">Adres</th>

          <th style="width: 15%;">NIP/KRS</th>

          <th style="width: 10%; text-align: center;">Status</th>

          <th style="width: 20%; text-align: center;">Akcje</th>

        </tr>

      </thead>

      <tbody>

        {% for firma in firmy.items %}

        <tr>

          <td>

            <div style="font-weight: 600;">{{ firma.nazwa }}</div>

          </td>

          <td>

            <div style="font-size: 0.9rem; color: var(--kolor-tekst-dugor);">

              {% if firma.adres %}

              {{ firma.adres }}

              {% if firma.miasto %}<br>{{ firma.miasto }}{% endif %}

              {% else %}

              Brak adresu

              {% endif %}

            </div>

          </td>

          <td style="font-size: 0.9rem;">

            {{ firma.nip_krs or '-' }}

          </td>

        </tr>

      </tbody>

    </table>

  </div>

  {% endif %}

</div>

```

```

<td style="text-align: center;">

    {% if firma.is_active %}

        <span class="status status--zakonczone">Aktywna</span>

    {% else %}

        <span class="status status--szkie">Nieaktywna</span>

    {% endif %}

</td>

<td style="white-space: nowrap;">

    <a href="{{ url_for('zarzadzanie.edytuj_firme', id=firma.id) }}"

        class="przycisk przycisk--maly przycisk--drugorzeczny">

        Edytuj

    </a>

    <form method="POST" action="{{ url_for('zarzadzanie.przelacz_aktywnosc_firmy', id=firma.id) }}"

        style="display: inline-block; margin-left: 0.25rem;">

        {{ csrf_form.hidden_tag() }}

        {% set text = 'dezaktywować' if firma.is_active else 'aktywować' %}

        <button type="submit"

            class="przycisk {% if firma.is_active %}przycisk--ostrzezenie{% else %}przycisk--drugorzeczny{%
endif %} przycisk--maly"

            onclick="return confirm('Czy na pewno chcesz {{ text }} firmę?')">

            {{ 'Dezaktywuj' if firma.is_active else 'Aktywuj' }}

        </button>

    </form>

    <form method="POST" action="{{ url_for('zarzadzanie.usun_firme', id=firma.id) }}"

        style="display: inline-block; margin-left: 0.25rem;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit"

            class="przycisk przycisk--niebezpieczny przycisk--maly"

            onclick="return confirm('Czy na pewno chcesz trwale usunąć firmę {{ firma.nazwa }}? Tej
operacji nie można cofnąć.')">

            Usuń

        </button>

    </form>

</td>

</tr>

```



```

        {% endfor %}

    </tbody>

</table>

</div>

<!-- Paginacja -->

{% if firmy.pages > 1 %}

<div style="display: flex; justify-content: center; margin-top: 2rem; gap: 0.5rem;">

    {% if firmy.has_prev %}

    <a href="{{ url_for('zarzadzanie.lista_firm', page=firmy.prev_num, **request.args) }}"

        class="przycisk przycisk--drugorzeczny przycisk--maly">← Poprzednia</a>

    {% endif %}

    {% for page_num in firmy.iter_pages() %}

        {% if page_num %}

            {% if page_num != firmy.page %}

                <a href="{{ url_for('zarzadzanie.lista_firm', page=page_num, **request.args) }}"

                    class="przycisk przycisk--drugorzeczny przycisk--maly">{{ page_num }}</a>

            {% else %}

                <span class="przycisk przycisk--glowny przycisk--maly">{{ page_num }}</span>

            {% endif %}

        {% else %}

            <span>...</span>

        {% endif %}

    {% endfor %}

    {% if firmy.has_next %}

    <a href="{{ url_for('zarzadzanie.lista_firm', page=firmy.next_num, **request.args) }}"

        class="przycisk przycisk--drugorzeczny przycisk--maly">Następna →</a>

    {% endif %}

</div>

{% endif %}

{% else %}

```

```

<!-- Pusty stan -->

<div style="text-align: center; padding: 4rem 2rem;">

    <div style="font-size: 3rem; margin-bottom: 1rem; opacity: 0.3;">

        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round" style="width: 3rem; height: 3rem;">

            <rect x="4" y="2" width="16" height="20" rx="2" ry="2"/>

            <line x1="9" y1="6" x2="9" y2="6.01"/>

            <line x1="15" y1="6" x2="15" y2="6.01"/>

            <line x1="9" y1="10" x2="9" y2="10.01"/>

            <line x1="15" y1="10" x2="15" y2="10.01"/>

            <line x1="9" y1="14" x2="9" y2="14.01"/>

            <line x1="15" y1="14" x2="15" y2="14.01"/>

            <path d="M9 18h6"/>

        </svg>

    </div>

    <div style="font-size: 1.2rem; font-weight: 600; margin-bottom: 1rem; color: var(--kolor-tekst-drugor);">

        Brak firm w systemie

    </div>

    <div style="color: var(--kolor-tekst-drugor); margin-bottom: 2rem;">

        Dodaj pierwszą firmę do systemu praktyk.

    </div>

    <a href="{{ url_for('zarzadzanie.dodaj_firme') }}" class="przycisk przycisk--glowny">

        + Dodaj firmę

    </a>

</div>

{% endif %}

</div>

</div>

</div>

{% endblock %}

```

PLIK: app_admin\templates\zarzadzanie\komisja\lista.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Komisja weryfikująca{% endblock %}

{% block naglowek %}Wnioski do weryfikacji przez komisję{% endblock %}

{% block tresc %}

<div style="max-width: 1400px; margin: 0 auto;">

    {% if wnioski.items %}

    <div class="karta">

        <div class="karta__tresc">

            <div class="tabela-wrapper">

                <table class="tabela">

                    <thead>

                        <tr>

                            <th style="width: 25%;">Student</th>

                            <th style="width: 15%;">Ścieżka</th>

                            <th style="width: 25%;">Firma/Działalność</th>

                            <th style="width: 12%;">Data zgłoszenia</th>

                            <th style="width: 10%;">Dokumenty</th>

                            <th style="width: 13%; text-align: center;">Akcje</th>

                        </tr>

                    </thead>

                    <tbody>

                        {% for wniosek in wnioski.items %}

                        <tr>

                            <td>

                                <div style="font-weight: 600;">{{ wniosek.student.first_name }} {{ wniosek.student.last_name }}</div>

                                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ wniosek.student.email or 'Brak email' }}</div>

                                {% if wniosek.student.album_number %}

                                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Album: {{ wniosek.student.album_number }}</div>

                                {% endif %}

                            </td>

                        </tr>

                        </tbody>

                    </table>

                </div>

            </div>

        </div>

    {% endif %}

    </div>

</div>
```

```

</td>

<td>

  <span class="status

    {%- if wniosek.track_type.value == 'EMPLOYMENT' %} status--w-trakcie

    {%- elif wniosek.track_type.value == 'OWN_BUSINESS' %} status--szkic

    {%- endif %}">

    {% if wniosek.track_type.value == 'EMPLOYMENT' %}Praca zawodowa

    {% elif wniosek.track_type.value == 'OWN_BUSINESS' %}Działalność gospodarcza

    {% endif %}

  </span>

</td>

<td>

  <div style="font-weight: 600;">{{ wniosek.firma_nazwa or 'Nie podano' }}</div>

  {% if wniosek.firma_miasto %}

  <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ wniosek.firma_miasto }}</div>

  {% endif %}

  {% if wniosek.track_type.value != 'STANDARD' %}

  <div style="font-size: 0.75rem; color: var(--kolor-info); margin-top: 0.25rem;">

    {% if wniosek.track_type.value == 'EMPLOYMENT' %}Na podstawie zatrudnienia

    {% elif wniosek.track_type.value == 'OWN_BUSINESS' %}Własna działalność

    {% endif %}

  </div>

  {% endif %}

</td>

<td style="font-size: 0.85rem;">

  {{ wniosek.enrolled_at.strftime('%d.%m.%Y') if wniosek.enrolled_at else '-' }}

  <div style="font-size: 0.7rem; color: var(--kolor-tekst-drugor);">

    {{ wniosek.enrolled_at.strftime('%H:%M') if wniosek.enrolled_at else '' }}

  </div>

</td>

<td style="text-align: center;">

  <span class="badge badge--neutral" style="font-size: 0.8rem;">

    {{ wniosek.uploaded_documents | length }} plików

  </span>

```

```

        </td>

        <td style="text-align: center;">

            <a href="{ { url_for('zarzadzanie.komisja_weryfikuj', id=wniosek.id) }}"

                class="przycisk przycisk--maly przycisk--glowny">

                    Weryfikuj

            </a>

        </td>

    </tr>

    {% endfor %}

</tbody>

</table>

</div>

</div>

</div>

<!-- Paginacja -->

{% if wnioski.pages > 1 %}

<div class="paginacja">

    {% if wnioski.has_prev %}

        <a href="{ { url_for('zarzadzanie.komisja_lista', page=wnioski.prev_num) }}" class="paginacja__link">←
Poprzednia</a>

    {% endif %}

    {% for page_num in wnioski.iter_pages() %}

        {% if page_num %}

            {% if page_num != wnioski.page %}

                <a href="{ { url_for('zarzadzanie.komisja_lista', page=page_num) }}" class="paginacja__link">{{ page_num }}</a>

            {% else %}

                <span class="paginacja__link paginacja__link--aktywny">{{ page_num }}</span>

            {% endif %}

        {% else %}

            <span class="paginacja__kropki">...</span>

        {% endif %}

    {% endfor %}

```

```
{% if wnioski.has_next %}

    <a href="{% url_for('zarzadzanie.komisja_lista', page=wnioski.next_num) %}" class="paginacja__link">Następna →</a>

{% endif %}

</div>

{% endif %}


{% else %}

<div class="karta">

    <div class="karta__tresc" style="text-align: center; padding: 3rem 2rem;">

        <svg width="64" height="64" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.5" style="color:
var(--kolor-tekst-drugor); margin-bottom: 1rem;">

            <path d="M9 12l2 4-4"/>

            <path d="M21 12c0 4.97-4.03 9-9 9s-9-4.03-9-9 4.03-9 9-9c.83 0 1.63.11 2.38.32"/>

        </svg>

        <h3 style="margin-bottom: 0.5rem;">Brak wniosków do weryfikacji</h3>

        <p style="color: var(--kolor-tekst-drugor); margin-bottom: 0;">

            Wszystkie wnioski zostały już przetworzone lub nie ma nowych zgłoszeń.

        </p>

    </div>

</div>

{% endif %}


</div>

{% endblock %}
```

PLIK: app_admin\templates\zarzadzanie\komisja\weryfikuj.html

```
{% extends "layouts/base_panel.html" %}

{% block tytul %}Weryfikacja wniosku – {{ zapis.student.first_name }} {{ zapis.student.last_name }}{% endblock %}

{% block naglowek %}Weryfikacja wniosku komisji{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    <a href="{{ url_for('zarzadzanie.komisja_lista') }}" class="przycisk przycisk--drugorzeczny">< Powrót do listy</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto;">

    {# — Karta identyfikacyjna — #}

    {% set status_val = zapis.status.value %}

    {% if status_val == 'COMMISSION_REVIEW' %}

        {% set border_color = '#3b82f6' %}

        {% set status_cls = 'status--planned' %}

        {% set status_label = 'Do rozpatrzenia przez komisję' %}

    {% elif status_val == 'AWAITING_APPROVAL' %}

        {% set border_color = 'var(--kolor-akcent)' %}

        {% set status_cls = 'status--in-progress' %}

        {% set status_label = 'Zwrócone do uzupełnienia' %}

    {% elif status_val == 'DEAN_APPROVAL' %}

        {% set border_color = 'var(--kolor-glowny)' %}

        {% set status_cls = 'status--planned' %}

        {% set status_label = 'Przekazane do dziekana' %}

    {% elif status_val == 'IN_PROGRESS' %}

        {% set border_color = 'var(--kolor-sukces)' %}

        {% set status_cls = 'status--completed' %}

        {% set status_label = 'Zatwierdzona – w realizacji' %}

    {% else %}
```

```

{% set border_color = 'var(--kolor-ramka)' %}

{% set status_cls = 'status--draft' %}

{% set status_label = status_val %}

{% endif %}

<div style="background: var(--kolor-biale); border-radius: var(--promien-duzy); border-top: 4px solid {{ border_color
}}; box-shadow: var(--cien-karta); margin-bottom: 2rem; overflow: hidden;">

    <div style="padding: 1.75rem 2rem; display: grid; grid-template-columns: auto 1fr auto; gap: 2rem; align-items:
center;">

        <div style="width: 56px; height: 56px; border-radius: 50%; background: var(--kolor-glowny); color: #fff; display:
flex; align-items: center; justify-content: center; font-size: 1.25rem; font-weight: 700; flex-shrink: 0; letter-spacing:
0.5px;">

            {{ zapis.student.first_name[0] }}{{ zapis.student.last_name[0] }}

        </div>

        <div style="display: grid; grid-template-columns: repeat(4, 1fr); gap: 1.5rem; min-width: 0;">

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Student</div>

                <div style="font-weight: 700; font-size: 1rem; color: var(--kolor-tekst);">{{ zapis.student.first_name }} {{
zapis.student.last_name }}</div>

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor); margin-top: 0.15rem;">{{ zapis.student.email
}}</div>

                {% if zapis.student.album_number %}

                    <div style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed);">nr alb. {{ zapis.student.album_number
}}</div>

                {% endif %}

            </div>

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Rok akademicki</div>

                <div style="font-weight: 600; color: var(--kolor-tekst);">{{ zapis.praktyka.rok_uczelnianny }}</div>

                <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ zapis.praktyka.semestr }}</div>

            </div>

            <div>

                <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Ścieżka</div>

                <div style="font-weight: 600; color: var(--kolor-tekst);">

```



```

        {% if zapis.track_type.value == 'EMPLOYMENT' %}Na podst. zatrudnienia

        {% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność

        {% else %}Standardowa{% endif %}

    </div>

</div>

<div>

    <div style="font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--kolor-tekst-trzeciorzed); margin-bottom: 0.35rem;">Data zgłoszenia</div>

    <div style="font-weight: 600; color: var(--kolor-tekst);">{{ zapis.enrolled_at.strftime('%d.%m.%Y') if
zapis.enrolled_at else '-' }}</div>

    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">{{ zapis.enrolled_at.strftime('%H:%M') if
zapis.enrolled_at else '' }}</div>

</div>

</div>

<div style="text-align: right; flex-shrink: 0;">

    <span class="status {{ status_cls }}" style="font-size: 0.75rem; padding: 0.4rem 1rem;">{{ status_label }}</span>

    {% if zapis.uopz %}

    <div style="margin-top: 0.75rem; font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); text-align: right;">

        UOPZ: <strong style="color: var(--kolor-tekst-drugor);">{{ zapis.uopz.first_name }} {{ zapis.uopz.last_name
    }}</strong>

    </div>

    {% endif %}

</div>

</div>

</div>

{# — Główna siatka treści — #}

<div style="display: grid; grid-template-columns: 1fr 480px; gap: 1.5rem; align-items: start;">

    {# == LEWA KOLUMNA == #}

    <div style="display: flex; flex-direction: column; gap: 1.5rem;">

        {# Dane podstawowe #}

        <div class="karta">

```

```

<div class="karta__naglowek">

    <h3 class="karta__tytul">Dane podstawowe</h3>

</div>

<div class="karta__tresc">

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

        <div class="pole-info">

            <div class="pole-info__etykieta">Ścieżka praktyki</div>

            <div class="pole-info__wartosc">

                {% if zapis.track_type.value == 'EMPLOYMENT' %}Na podstawie zatrudnienia

                {% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność gospodarcza

                {% else %}Standardowa{% endif %}

            </div>

        </div>

        <div class="pole-info">

            <div class="pole-info__etykieta">Specjalność</div>

            <div class="pole-info__wartosc">{{ zapis.specjalnosc or '-' }}</div>

        </div>

    </div>

</div>

</div>

{# Dane pracodawcy / działalności #}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Dane pracodawcy / działalności</h3>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.25rem;">

            <div class="pole-info">

                <div class="pole-info__etykieta">Nazwa firmy / działalności</div>

                <div class="pole-info__wartosc" style="font-weight: 600;">{{ zapis.firma_nazwa or '-' }}</div>

            </div>

            <div class="pole-info">

                <div class="pole-info__etykieta">Stanowisko / zakres obowiązków</div>

```

```

        <div class="pole-info__wartosc">{{ zapis.zopz_stanowisko or '-' }}</div>

    </div>

    <div class="pole-info" style="grid-column: 1 / -1;">

        <div class="pole-info__etykieta">Adres</div>

        <div class="pole-info__wartosc">{{ zapis.firma_adres or '-' }}{% if zapis.firma_miasto %}, {{
zapis.firma_miasto }}{% endif %}</div>

    </div>

</div>

</div>

</div>

</div>

{# Uzasadnienie studenta #}

{% if zapis.uzasadnienie_sciezki %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Uzasadnienie wniosku</h3>

    </div>

    <div class="karta__tresc">

        <p style="white-space: pre-line; font-size: 0.9rem; line-height: 1.7; color: var(--kolor-tekst); margin: 0;">{{
zapis.uzasadnienie_sciezki }}</p>

    </div>

</div>

{% endif %}

{# Załączniki przesłane przez studenta #}

{% if dokumenty %}

<div class="karta">

    <div class="karta__naglowek">

        <h3 class="karta__tytul">Załączniki studenta</h3>

        <span style="font-size: 0.75rem; font-weight: 400; color: var(--kolor-tekst-trzeciorzed);">{{ dokumenty|length
}} plik{% if dokumenty|length != 1 %}i{% endif %}</span>

    </div>

    <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 0.5rem;">

        {% for dok in dokumenty %}

            <div style="display: flex; justify-content: space-between; align-items: center; padding: 0.75rem 1rem; border:
1px solid var(--kolor-ramka); border-radius: var(--promien); background: var(--kolor-tlo);">

```

```

<div style="min-width: 0;">

    <div style="font-weight: 600; font-size: 0.875rem; white-space: nowrap; overflow: hidden; text-overflow:
ellipsis;">{{ dok.oryginalna_nazwa }}</div>

        <div style="font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); margin-top: 0.15rem;">{{
dok.typ_dokumentu }} · {{ (dok.rozmiar_pliku / 1024)|round(1) }} KB · {{ dok.przeslano_o.strftime('%d.%m.%Y %H:%M')
}}</div>

    </div>

    <a href="{{ url_for('uploads.download_document', document_id=dok.id) }}"

        class="przycisk przycisk--maly przycisk--drugorzeczny" style="flex-shrink: 0; margin-left:
1rem;">Pobierz</a>

    </div>

    {% endfor %}

</div>

</div>

    {% endif %}

</div>

    {{# == PRAWA KOLUMNA == #}}

<div style="display: flex; flex-direction: column; gap: 1.5rem; position: sticky; top: 1.5rem;">

    {{# Formularz decyzji komisji #}}

    <div class="karta">

        <div class="karta__naglowek">

            <h3 class="karta__tytul">Decyzja komisji</h3>

        </div>

        <div class="karta__tresc">

            <form method="POST">

                {{ form.hidden_tag() }}

                <div style="margin-bottom: 1.25rem;">

                    {{ form.decyzja.label(class="pole-formularza__etykieta", style="display: block; margin-bottom: 0.4rem;") }}

                    {{ form.decyzja(class="pole-formularza__select") }}

                    {% for error in form.decyzja.errors %}

                        <span class="pole-formularza__blad">{{ error }}</span>

                    {% endfor %}

                </div>

```

</div>

<div style="margin-bottom: 1.25rem;">

<label class="pole-formularza__etykieta" style="display: block; margin-bottom: 0.4rem;">Komentarz (opcjonalny)</label>

{{ form.komentarz(class="pole-formularza__textarea", rows="7", placeholder="Uzasadnienie decyzji widoczne dla studenta i dziekana...", style="width: 100%; box-sizing: border-box;") }}

{% for error in form.komentarz.errors %}

{{ error }}

{% endfor %}

</div>

<div style="border-top: 1px solid var(--kolor-ramka); padding-top: 1.25rem; display: flex; flex-direction: column; gap: 0.5rem;">

{{ form.submit(class="przycisk przycisk--glowny", style="width: 100%; justify-content: center;") }}

Anuluj

</div>

</form>

</div>

</div>

{# Historia decyzji #}

{% if zapis.decyzja_komisji or zapis.decyzja_dziekana %}

<div class="karta">

<div class="karta__naglowek">

<h3 class="karta__tytul">Historia decyzji</h3>

</div>

<div class="karta__tresc" style="display: flex; flex-direction: column; gap: 1rem;">

{% if zapis.decyzja_komisji %}

<div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background: var(--kolor-tlo);">

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.5rem;">

Komisja ds. praktyk


```

        {% if zapis.decyzja_komisji == 'APPROVED' %}Zatwierdzone

        {% elif zapis.decyzja_komisji == 'PARTIALLY_APPROVED' %}Częściowo zatwierdzone

        {% else %}Odrzucone{% endif %}

    </span>

</div>

    {% if zapis.komentarze_komisji %}

        <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); line-height: 1.5; margin: 0;">{{
zapis.komentarze_komisji }}</p>

        {% endif %}

    </div>

    {% endif %}

    {% if zapis.decyzja_dziekana %}

        <div style="padding: 1rem; border: 1px solid var(--kolor-ramka); border-radius: var(--promien); background:
var(--kolor-tlo);">

            <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.5rem;">

                <span style="font-size: 0.75rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em;
color: var(--kolor-tekst-trzeciorzed);">Dyrektor IIS</span>

                <span class="status {% if zapis.decyzja_dziekana == 'APPROVED' %}status--completed{% else %}status--draft{%
endif %}">

                    {{ 'Zatwierdzone' if zapis.decyzja_dziekana == 'APPROVED' else 'Odrzucone' }}

                </span>

            </div>

            <div>

                {% if zapis.komentarze_dziekana %}

                    <p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); line-height: 1.5; margin: 0;">{{
zapis.komentarze_dziekana }}</p>

                    {% endif %}

                </div>

                {% endif %}

            </div>

        </div>

    {% endif %}

    {# Poprzedni komentarz komisji #}

    {% if zapis.komentarze_uopz %}

        <div class="karta">

            <div class="karta__naglowek">

                <h3 class="karta__tytul">Poprzedni komentarz komisji</h3>

```

```
</div>

<div class="karta__tresc">

    <div style="font-size: 0.875rem; color: var(--kolor-tekst); line-height: 1.55; white-space: pre-wrap;">{{
zapis.komentarze_uopz }}</div>

    </div>

</div>

{% endif %}


</div>


</div>


</div>


<style>

.pole-info__etykieta {

    font-size: 0.7rem;

    font-weight: 600;

    text-transform: uppercase;

    letter-spacing: 0.06em;

    color: var(--kolor-tekst-trzeciorzed);

    margin-bottom: 0.3rem;

}

.pole-info__wartosc {

    font-size: 0.9rem;

    color: var(--kolor-tekst);

    line-height: 1.45;

}

</style>

{% endblock %}
```

PLIK: app_student\config.py

```
import os

from datetime import timedelta


def _require(name: str) -> str:

    v = os.environ.get(name)

    if not v:

        raise RuntimeError(

            f"Zmienna środowiskowa {name!r} nie jest ustawiona. "

            "Uzupełnij plik .env lub zmienne środowiskowe kontenera."

        )

    return v


class Config:

    # Osobne nazwy ciasteczek dla admina i studenta

    SESSION_COOKIE_NAME = 'student_session'


    SECRET_KEY = _require('SECRET_KEY')

    SQLALCHEMY_DATABASE_URI = _require('DATABASE_URL')

    SQLALCHEMY_TRACK_MODIFICATIONS = False


    # Microsoft Entra ID (Azure AD) – OAuth 2.0

    AZURE_CLIENT_ID = _require('AZURE_CLIENT_ID')

    AZURE_CLIENT_SECRET = _require('AZURE_CLIENT_SECRET')

    AZURE_TENANT_ID = _require('AZURE_TENANT_ID')


    # Session

    PERMANENT_SESSION_LIFETIME = timedelta(hours=8)


    # External services

    TEX_SERVICE_URL = os.environ.get('TEX_SERVICE_URL', 'http://tex-service:5002')
```



```
# Rate limiting (flask-limiter + Redis)

RATELIMIT_STORAGE_URI = os.environ.get('CELERY_BROKER_URL', os.environ.get('REDIS_URL', 'redis://localhost:6379/0'))


# Security

WTF_CSRF_ENABLED = True

WTF_CSRF_TIME_LIMIT = None


class DevelopmentConfig(Config):

    DEBUG = True

    ENV = 'development'


class ProductionConfig(Config):

    DEBUG = False

    ENV = 'production'

    SESSION_COOKIE_SECURE = True

    SESSION_COOKIE_HTTPONLY = True

    SESSION_COOKIE_SAMESITE = 'Lax'


config_dict = {

    'development': DevelopmentConfig,

    'production': ProductionConfig,

    'default': DevelopmentConfig

}
```

PLIK: app_student\Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \

    PYTHONUNBUFFERED=1 \

    PYTHONPATH=/app

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \

    curl \

    libmagic1 \

    && rm -rf /var/lib/apt/lists/*

COPY app_student/requirements.txt requirements.txt

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Bezpieczeństwo: dedykowany nieuprzywilejowany użytkownik (nie root)

RUN addgroup --system appgroup \

    && adduser --system --ingroup appgroup --no-create-home appuser \

    && chown -R appuser:appgroup /app

USER appuser

EXPOSE 5001

CMD ["gunicorn", "--bind", "0.0.0.0:5001", "--workers", "2", "app_student:create_app()"]
```

PLIK: app_student__init__.py

```
from flask import Flask

from jinja2 import select_autoescape

from core.extensions import db, login_manager, csrf, limiter

from core.error_handlers import register_error_handlers

from app_student.config import config_dict


def create_app():

    from pathlib import Path as _Path

    from flask import Blueprint as _Blueprint

    app = Flask(__name__)

    # — Core static assets (CSS design system + templates) —
    core_bp = _Blueprint(

        'core_static', __name__,

        static_folder=str(_Path(__file__).parent.parent / 'core' / 'static'),

        static_url_path='/core/static',

        template_folder=str(_Path(__file__).parent.parent / 'core' / 'templates'),

    )

    app.register_blueprint(core_bp)

    app.jinja_options = app.jinja_options.copy()

    app.jinja_options.update(dict(

        autoescape=select_autoescape(['html', 'xml'])

    ))

    env = __import__('os').environ.get('FLASK_ENV', 'development')

    app.config.from_object(config_dict.get(env, config_dict['default']))


    db.init_app(app)

    login_manager.init_app(app)

    csrf.init_app(app)

    limiter.init_app(app)

    register_error_handlers(app)
```

```

login_manager.login_view = 'auth.logowanie'

login_manager.login_message = 'Zaloguj się, aby uzyskać dostęp.'


with app.app_context():

    from core.autoryzacja import stworz_blueprint_auth

    from core.pliki import stworz_blueprint_pliki

    from core.modele import UserRole

    from app_student.routes.pulpit import dashboard_bp

    from app_student.routes.praktyki import praktyki_bp

    from app_student.routes.dziennik import dziennik_bp

    from app_student.routes.sprawozdania import sprawozdania_bp

    from app_student.routes.dokumenty import documents_bp


    auth_bp = stworz_blueprint_auth(

        dozwolone_role=[UserRole.STUDENT],

        template_logowania='auth/logowanie.html',

        template_zmiany_hasla='auth/zmien_haslo.html',

    )

    uploads_bp = stworz_blueprint_pliki()


    app.register_blueprint(auth_bp)

    app.register_blueprint(dashboard_bp, url_prefix='/panel')

    app.register_blueprint(praktyki_bp, url_prefix='/praktyki')

    app.register_blueprint(dziennik_bp, url_prefix='/dziennik')

    app.register_blueprint(sprawozdania_bp, url_prefix='/sprawozdanie')

    app.register_blueprint(documents_bp, url_prefix='/dokumenty')

    app.register_blueprint(uploads_bp, url_prefix='/uploads')


# Kontekst globalny: informacje o aktywnym zapisie studenta

@app.context_processor

def inject_aktywny_zapis():

    from flask_login import current_user

    from core.modele import EnrollmentStatus

    from core.repozytoria import RepozytoriumZapisow

```

```

info = {'ma_aktywny': False, 'sciezka': None, 'status': None}

wymaga_uwagi = False

try:

    if current_user.is_authenticated:

        _repo = RepozytoriumZapisow()

        z = _repo.aktywny_dla_studenta(current_user.id, [

            EnrollmentStatus.IN_PROGRESS, EnrollmentStatus.COMPLETED,

            EnrollmentStatus.COMMISSION_REVIEW, EnrollmentStatus.DEAN_APPROVAL,

            EnrollmentStatus.AWAITING_APPROVAL,

        ])

        if z:

            info = {

                'ma_aktywny': True,

                'sciezka': z.track_type.value if z.track_type else 'STANDARD',

                'status': z.status.value,

            }

            zapisy_do_sprawdzenia = _repo.aktywne_dla_studenta(current_user.id, [

                EnrollmentStatus.PENDING,

                EnrollmentStatus.AWAITING_APPROVAL,

            ])

            for z in zapisy_do_sprawdzenia:

                if z.status == EnrollmentStatus.PENDING and (z.admin_comments or z.supervisor_comments):

                    wymaga_uwagi = True

                    break

                if z.status == EnrollmentStatus.AWAITING_APPROVAL and z.supervisor_comments:

                    wymaga_uwagi = True

                    break

except Exception:

    pass

return {'aktywny_zapis_info': info, 'nav_wymaga_uwagi': wymaga_uwagi}

```

Dodaj funkcje pomocnicze do szablonów

@app.template_global()

def tlumacz_status(status_value):

```
translations = {

    'PENDING': 'Szkic',

    'AWAITING_APPROVAL': 'Oczekuje na zatwierdzenie',

    'COMMISSION_REVIEW': 'Weryfikacja komisji',

    'DEAN_APPROVAL': 'Oczekuje na dziekana',

    'IN_PROGRESS': 'W trakcie',

    'COMPLETED': 'Zakończona',

    'REJECTED': 'Odrzucony'

}

return translations.get(status_value, status_value)
```

```
@app.before_request
```

```
def sprawdz_studenta():
```

```
    from flask import request, redirect, url_for, abort
```

```
    from flask_login import current_user
```

```
    from core.modele import UserRole
```

```
    if not current_user.is_authenticated:
```

```
        return
```

```
    if getattr(current_user.role, 'value', current_user.role) != 'STUDENT':
```

```
        abort(403)
```

```
return app
```

```
@login_manager.user_loader
```

```
def wczytaj_uzytkownika(user_id):
```

```
    from core.modele import User
```

```
    return db.session.get(User, user_id)
```

PLIK: app_student\routes\dokumenty.py

```
"""
app_student/routes/dokumenty.py
"""

import io

import logging

import httpx

from flask import (Blueprint, abort, send_file, jsonify, request,
                   current_app, flash, redirect, url_for, make_response,
                   render_template)

from flask_login import login_required, current_user

from core.extensions import db, limiter

from core.modele import InternshipEnrollment, EnrollmentStatus

from core.uslugi.dokumenty import (
    DOC_CONFIG, STATIC_TEMPLATES, rozwiaz_dokumenty, buduj_kontekst,
)

from core.repozytoria import RepozytoriumZapisow

_repo_zapisow = RepozytoriumZapisow()

logger = logging.getLogger(__name__)

documents_bp = Blueprint('documents', __name__)

def _get_tex_url() -> str:
    return current_app.config.get('TEX_SERVICE_URL', 'http://tex-service:5002')

# — Lista dokumentów studenta —————

@documents_bp.route('/moje')
```

```

@login_required

def moje_dokumenty():

    zapisy = _repo_zapisow.aktywne_dla_studenta(

        current_user.id,

        [EnrollmentStatus.AWAITING_APPROVAL, EnrollmentStatus.IN_PROGRESS, EnrollmentStatus.COMPLETED],

    )

    dokumenty_list = [

        {

            'zapis': z,

            'sciezka': z.track_type.value if z.track_type else 'STANDARD',

            'docs': rozwiadz_dokumenty(z),

        }

        for z in zapisy

    ]

    return render_template('dokumenty/moje_dokumenty.html', dokumenty_list=dokumenty_list)

```

— Dokumenty statyczne

```

@documents_bp.route('/staly/<doc_key>')

@login_required

def pobierz_staly(doc_key):

    if doc_key not in STATIC_TEMPLATES:

        abort(404)

    template_name, filename = STATIC_TEMPLATES[doc_key]

    try:

        response = httpx.post(

            f"_{get_tex_url()}/generuj",

            json={'template': template_name, 'context': {}, 'filename': filename},

            timeout=30,

        )

        if response.status_code == 200:

```



```

        pdf_response = make_response(response.content)

        pdf_response.headers['Content-Type'] = 'application/pdf'

        pdf_response.headers['Content-Disposition'] = f'attachment; filename="{filename}"'

        return pdf_response

    flash('Błąd generowania dokumentu.', 'error')

except Exception as e:

    flash(f'Błąd połączenia z serwisem PDF: {str(e)}', 'error')

return redirect(url_for('documents.moje_dokumenty'))

```

— Dokumenty dynamiczne —————

```

@documents_bp.route('/dynamiczny/<uuid:zapis_id>/<typ>')

@login_required

def pobierz_dynamiczny(zapis_id, typ):

    if typ not in DOC_CONFIG:

        abort(404)

    zapis = db.session.get(InternshipEnrollment, zapis_id)

    if not zapis or zapis.student_id != current_user.id:

        abort(404)

    template_name, filename = DOC_CONFIG[typ]

    ctx = buduj_kontekst(zapis, typ)

    try:

        response = httpx.post(

            f"{_get_tex_url()}/generuj",

            json={'template': template_name, 'context': ctx, 'filename': filename},

            timeout=60,

        )

        if response.status_code == 200:

            import unicodedata

            safe = (

```

```

        unicodedata.normalize('NFKD', zapis.student.last_name or '')

        .encode('ascii', 'ignore').decode('ascii') or 'student'

    )

    pdf_name = template_name.replace('.tex.j2', '')

    pdf_response = make_response(response.content)

    pdf_response.headers['Content-Type'] = 'application/pdf'

    pdf_response.headers['Content-Disposition'] = f'attachment; filename="{pdf_name}_{safe}.pdf"'

    return pdf_response

    flash(f'Błąd generowania dokumentu: {response.text[:200]}', 'error')

except Exception as e:

    flash(f'Błąd połączenia z serwisem PDF: {str(e)}', 'error')

return redirect(url_for('documents.moje_dokumenty'))


@documents_bp.route('/generuj/<doc_type>', methods=['POST'])

@login_required

@limiter.limit("30 per hour")

def generuj(doc_type: str):

    if doc_type not in DOC_CONFIG:

        abort(403)

    force = request.args.get('force') == 'true'

    zapis = (

        _repo_zapisow.aktywny_dla_studenta(current_user.id, [EnrollmentStatus.IN_PROGRESS])

        or _repo_zapisow.ostatni_dla_studenta(current_user.id)

    )

    if not zapis:

        return jsonify({'error': 'Nie znaleziono zapisu na praktykę.'}), 404

    # Walidacja kompletności danych

    warnings = []

    if not (zapis.firma_nazwa or (zapis.firma and zapis.firma.nazwa)):

```

```

        warnings.append('Nazwa firmy')

    if not (zapis.firma_adres or (zapis.firma and zapis.firma.adres)):

        warnings.append('Adres firmy')

    if doc_type == 'ZAL_6' and not getattr(zapis, 'wpisy_dziennika', None):

        warnings.append('Brak wpisów w dzienniku')


    if warnings and not force:

        return jsonify({

            'requires_confirmation': True,

            'warnings': warnings,

            'message': 'Niektóre dane są puste. Dokument będzie niekompletny.',

        }), 200


template_name, filename = DOC_CONFIG[doc_type]

try:

    context = buduj_kontekst(zapis, doc_type)

    resp = httpx.post(

        f"{_get_tex_url()}/generuj",

        json={'template': template_name, 'context': context, 'filename': filename},

        timeout=30.0,

    )

    if resp.status_code != 200:

        err = resp.json() if 'application/json' in resp.headers.get('content-type', '') else {}

        return jsonify({'error': err.get('error', 'Błąd serwisu PDF')}), 500


    return send_file(

        io.BytesIO(resp.content),

        mimetype='application/pdf',

        as_attachment=True,

        download_name=filename,

    )

except Exception as e:

    logger.error("Błąd krytyczny generowania %s: %s", doc_type, e, exc_info=True)

    return jsonify({'error': str(e)}), 500

```

PLIK: app_student\routes\dziennik.py

```
import uuid

from datetime import date


from flask import Blueprint, render_template, redirect, url_for, flash, request

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from wtforms import StringField, SelectMultipleField, TextAreaField

from wtforms.validators import DataRequired, ValidationError

from wtforms.widgets import ListWidget, CheckboxInput


from core.modele import InternshipEnrollment, JournalEntry, LearningOutcome, EnrollmentStatus

from core.extensions import db

from core.repozytoria import RepozytoriumZapisow, RepozytoriumEfektow, RepozytoriumWpisow


_repo_zapisow = RepozytoriumZapisow()

_repo_efektow = RepozytoriumEfektow()

_repo_wpisow = RepozytoriumWpisow()


dziennik_bp = Blueprint('dziennik', __name__)


class FormularzWpisu(FlaskForm):

    data_wpisu = StringField('Data', validators=[DataRequired()])

    liczba_godzin = StringField('Liczba godzin (1-8)', validators=[DataRequired()])

    opis = TextAreaField('Opis wykonanych prac', validators=[DataRequired()])

    efekty_ids = SelectMultipleField(

        'Efekty uczenia się',

        validators=[DataRequired(message='Wybierz co najmniej jeden efekt uczenia się.')],

        widget=ListWidget(prefix_label=False),

        option_widget=CheckboxInput(),

    )

    def validate_liczba_godzin(self, pole):
```

```

try:

    val = int(pole.data)

except (ValueError, TypeError):

    raise ValidationError('Podaj liczbę całkowitą.')

if val < 1 or val > 8:

    raise ValidationError('Maksymalnie 8 godzin dziennie (regulamin ANS).')


def validate_data_wpisu(self, pole):

    try:

        date.fromisoformat(pole.data)

    except ValueError:

        raise ValidationError('Nieprawidłowy format daty.')


def _aktywny_zapis():

    return _repo_zapisow.aktywny_dla_studenta(current_user.id, [

        EnrollmentStatus.IN_PROGRESS, EnrollmentStatus.COMMISSION_REVIEW,

        EnrollmentStatus.DEAN_APPROVAL, EnrollmentStatus.COMPLETED,

    ])


@dziennik_bp.route('/')

@login_required

def index():

    zapis = _aktywny_zapis()

    # Jeśli student nie ma aktywnego zapisu, szukamy jakiegokolwiek innego (np. zakończonego)

    if not zapis:

        jakikolwiek = _repo_zapisow.pierwszy_dla_studenta(current_user.id)

        # Zwracamy puste wpisy, żeby szablon się nie wysypał

        return render_template('dziennik/index.html', zapis=None, wpisy=[], jakikolwiek=jakikolwiek,
csrf_form=FlaskForm())

    # Jeśli jest aktywny zapis, normalnie ładujemy wpisy

    wpisy = _repo_wpisow.dla_zapisu(zapis.id)

```

```
csrf_form = FlaskForm()
```

```
return render_template('dziennik/index.html', zapis=zapis, wpisy=wpisy, jakikolwiek=zapis, csrf_form=csrf_form)
```

```
@dziennik_bp.route('/nowy', methods=['GET', 'POST'])
```

```
@login_required
```

```
def nowy_wpis():
```

```
    zapis = _aktywny_zapis()
```

```
    if not zapis:
```

```
        flash('Nie masz aktywnej praktyki. Skontaktuj się z opiekunem.', 'danger')
```

```
        return redirect(url_for('dziennik.index'))
```

```
    efekty = _repo_efektow.wszystkie()
```

```
    form = FormularzWpisu()
```

```
    form.efekty_ids.choices = [
```

```
        (str(e.id), f'{e.kod}: {e.description[:80]}...' if len(e.description) > 80 else f'{e.kod}: {e.description}')
```

```
        for e in efekty
```

```
    ]
```

```
    if form.validate_on_submit():
```

```
        data = date.fromisoformat(form.data_wpisu.data)
```

```
        duplikat = _repo_wpisow.znajdz_duplikat(zapis.id, data)
```

```
        if duplikat:
```

```
            flash('Wpis na ten dzień już istnieje. Możesz go edytować.', 'danger')
```

```
            return render_template('dziennik/nowy_wpis.html', form=form, zapis=zapis)
```

```
    godziny = int(form.liczba_godzin.data)
```

```
    wybrane_efekty = _repo_efektow.po_ids([int(i) for i in form.efekty_ids.data])
```

```
    wpis = JournalEntry(
```

```
        id                = uuid.uuid4(),
```

```
        enrollment_id     = zapis.id,
```

```
        entry_date        = data,
```

```
        duration_hours    = godziny,
```

```
        description       = form.opis.data.strip(),
```

```

        learning_outcomes = wybrane_efekty,

    )

    db.session.add(wpis)

    db.session.commit()

    flash(f'Wpis z dnia {data.strftime("%d.%m.%Y")} został dodany ({godziny} h).', 'success')

    return redirect(url_for('dziennik.index'))


if request.method == 'GET':

    form.data_wpisu.data = date.today().isoformat()


    return render_template('dziennik/nowy_wpis.html', form=form, zapis=zapis)


@dziennik_bp.route('/edytuj/<uuid:wpis_id>', methods=['GET', 'POST'])
@login_required
def edytuj_wpis(wpis_id):

    wpisu = db.session.get(JournalEntry, wpis_id)

    if not wpisu:

        from flask import abort

        abort(404)


    zapis = db.session.get(InternshipEnrollment, wpisu.enrollment_id)

    if not zapis or zapis.student_id != current_user.id:

        from flask import abort

        abort(403)


    efekty = _repo_efektow.wszystkie()

    form = FormularzWpisu()

    form.efekty_ids.choices = [

        (str(e.id), f'{e.kod}: {e.description[:80]}...' if len(e.description) > 80 else f'{e.kod}: {e.description}')

        for e in efekty

    ]


    if form.validate_on_submit():

```

```

    wpis.entry_date      = date.fromisoformat(form.data_wpisu.data)

    wpis.duration_hours = int(form.liczba_godzin.data)

    wpis.description     = form.opis.data.strip()

    wpis.efekty_uczenia = _repo_efektow.po_ids([int(i) for i in form.efekty_ids.data])

    db.session.commit()

    flash('Wpis został zaktualizowany.', 'success')

    return redirect(url_for('dziennik.index'))

if request.method == 'GET':

    form.data_wpisu.data      = wpis.entry_date.isoformat()

    form.liczba_godzin.data = str(wpis.duration_hours)

    form.opis.data           = wpis.description

    form.efekty_ids.data     = [str(e.id) for e in wpis.efekty_uczenia]

    return render_template('dziennik/nowy_wpis.html', form=form, zapis=zapis, edycja=True, wpis=wpis)

@dziennik_bp.route('/usun/<uuid:wpis_id>', methods=['POST'])
@login_required
def usun_wpis(wpis_id):

    wpis = db.session.get(JournalEntry, wpis_id)

    if not wpis:

        from flask import abort

        abort(404)

    zapis = db.session.get(InternshipEnrollment, wpis.enrollment_id)

    if not zapis or zapis.student_id != current_user.id:

        from flask import abort

        abort(403)

    db.session.delete(wpis)

    db.session.commit()

    flash('Wpis został usunięty.', 'success')

    return redirect(url_for('dziennik.index'))

```


PLIK: app_student\routes\praktyki.py

```
import uuid

import datetime

from flask import Blueprint, render_template, redirect, url_for, flash, request, abort, make_response

from flask_wtf import FlaskForm

from wtforms import StringField, SelectField, DateField, BooleanField, TextAreaField

from wtforms.validators import DataRequired, Optional, Length, Email, ValidationError

import re

from flask_login import login_required, current_user

from core.extensions import db

import httpx

from core.modele import (Internship, InternshipEnrollment, InternshipStatus, EnrollmentStatus,
                          InternshipPath, LearningOutcome, InternshipSchedule, Company,
                          IndividualProgram, DocumentStatus, UploadedDocument,
                          WorkplaceDetails, PathJustification)

from core.repozytoria import (RepozytoriumPraktyk, RepozytoriumZapisow,
                              RepozytoriumEfektow, RepozytoriumFirm,
                              RepozytoriumDokumentowStudenta)


_repo_praktyk = RepozytoriumPraktyk()

_repo_zapisow = RepozytoriumZapisow()

_repo_efektow = RepozytoriumEfektow()

_repo_firm = RepozytoriumFirm()

_repo_docs = RepozytoriumDokumentowStudenta()


praktyki_bp = Blueprint('praktyki', __name__)


# =====

# NÓWE FORMULARZE KREATORA

# =====


class FormularzSciezka(FlaskForm):

    """Krok 1: Tylko wybór ścieżki."""
```

```

track_type = SelectField('Ścieżka praktyki', choices=[

    ('STANDARD', 'A – Standardowa praktyka'),

    ('EMPLOYMENT', 'B – Praca etatowa / staż'),

    ('OWN_BUSINESS', 'C – Własna działalność gospodarcza'),

], validators=[DataRequired(message='Wybierz ścieżkę.')])

```

```

class FormularzDaneFirmy(FlaskForm):

```

```

    """Krok 2A: Dane zakładu pracy + ZOPZ + terminy (tylko ścieżka A)."""

```

```

    # Terminy i dane podstawowe

```

```

    termin_od = DateField('Data rozpoczęcia', validators=[DataRequired(message='Podaj datę.')])

```

```

    termin_do = DateField('Data zakończenia', validators=[DataRequired(message='Podaj datę.')])

```

```

    ubezpieczenie_nw = BooleanField('Posiadam ubezpieczenie NW na czas trwania praktyki')

```

```

    # Tryb znalezienia miejsca

```

```

    firma_typ = SelectField('Jak znalazłeś/-aś miejsce praktyki?', choices=[

```

```

        ('database', 'Uczelnia kieruje do zakładu (firma ma umowę z ANS)'),

```

```

        ('custom', 'Sam/a znalazłem/-am miejsce (wymaga Zał. 9 i Zał. 1)'),

```

```

    ], validators=[DataRequired()])

```

```

    firma_id = SelectField('Wybierz firmę z listy', choices=[], validators=[Optional()])

```

```

    firma_nazwa = StringField('Nazwa zakładu pracy', validators=[Optional(), Length(max=255)])

```

```

    firma_adres = StringField('Adres (ulica, nr)', validators=[Optional(), Length(max=255)])

```

```

    firma_miasto = StringField('Miasto i kod pocztowy', validators=[Optional(), Length(max=100)])

```

```

    firma_nip_krs = StringField('NIP / KRS', validators=[Optional(), Length(max=50)])

```

```

    firma_upowazniony_osoba = StringField('Osoba upoważniona do podpisania porozumienia', validators=[Optional(), Length(max=255)])

```

```

    firma_upowazniony_stanowisko = StringField('Stanowisko osoby upoważnionej', validators=[Optional(), Length(max=255)])

```

```

    zopz_imie_nazwisko = StringField('Opiekun zakładowy (ZOPZ) – imię i nazwisko', validators=[Optional(), Length(max=255)])

```

```

    zopz_stanowisko = StringField('Stanowisko ZOPZ', validators=[Optional(), Length(max=255)])

```

```

    zopz_telefon = StringField('Telefon ZOPZ', validators=[Optional(), Length(max=50)])

```

```

    zopz_email = StringField('E-mail ZOPZ', validators=[Optional(), Email(message='Nieprawidłowy email.')])

```

```
def validate_firma_miasto(self, field):

    if not field.data:

        return

    if re.fullmatch(r'\d{2}-\d{3}', field.data.strip()):

        raise ValidationError('Podaj nazwę miasta, nie kod pocztowy. Kod pocztowy możesz dołączyć do adresu.')
```

```
def validate_zopz_imie_nazwisko(self, field):

    if not field.data:

        return

    parts = field.data.strip().split()

    if len(parts) < 2:

        raise ValidationError('Podaj imię i nazwisko (co najmniej dwa wyrazy).')

    if any(char.isdigit() for char in field.data):

        raise ValidationError('Imię i nazwisko nie może zawierać cyfr.')
```

```
def validate_firma_upowazniony_osoba(self, field):

    if not field.data:

        return

    parts = field.data.strip().split()

    if len(parts) < 2:

        raise ValidationError('Podaj imię i nazwisko osoby upoważnionej (co najmniej dwa wyrazy).')

    if any(char.isdigit() for char in field.data):

        raise ValidationError('Imię i nazwisko nie może zawierać cyfr.')
```

```
class FormularzWnioszek(FlaskForm):
```

```
    """Krok 2B/C: Wniosek dla ścieżek B i C."""
```

```
    pracodawca_nazwa      = StringField('Nazwa pracodawcy / firmy', validators=[DataRequired(message='Podaj nazwę.'),
Length(max=255)])
```

```
    pracodawca_adres      = StringField('Adres', validators=[Optional(), Length(max=255)])
```

```
    pracodawca_miasto     = StringField('Miasto', validators=[Optional(), Length(max=100)])
```

```
    stanowisko            = StringField('Stanowisko / zakres działalności', validators=[DataRequired(message='Podaj
stanowisko.'), Length(max=255)])
```

```
    uzasadnienie          = TextAreaField('Uzasadnienie wniosku', validators=[DataRequired(message='Napisz uzasadnienie.'),
Length(max=2000)])
```

```

# =====

# NOWE ROUTE KREATORA

# =====

@praktyki_bp.route('/<uuid:id>/kreator/sciezka', methods=['GET', 'POST'])
@login_required
def kreator_sciezka(id):
    """Krok 1: Wybór ścieżki."""

    praktyka = db.session.get(Internship, id)

    if not praktyka:
        flash('Praktyka niedostępna.', 'danger')
        return redirect(url_for('praktyki.lista'))

    istniejacy = _repo_zapisow.pending_dla_studenta_i_praktyki(current_user.id, id)

    form = FormularzSciezka()

    if form.validate_on_submit():
        if istniejacy:
            zapis = istniejacy
        else:
            zapis = InternshipEnrollment(id=uuid.uuid4(), internship_id=id,
                                          student_id=current_user.id, status=EnrollmentStatus.PENDING,
                                          supervisor_id=getattr(current_user, 'supervisor_id', None))
            db.session.add(zapis)

        zapis.track_type = InternshipPath(form.track_type.data)
        db.session.commit()

    if form.track_type.data == 'STANDARD':
        return redirect(url_for('praktyki.kreator_firma', zapis_id=zapis.id))
    else:
        return redirect(url_for('praktyki.kreator_wniosek', zapis_id=zapis.id))

```

```

if istniejacy and request.method == 'GET':

    form.track_type.data = istniejacy.track_type.value if istniejacy.track_type else 'STANDARD'

return render_template('kreator/krok1_sciezka.html', form=form, praktyka=praktyka, istniejacy=istniejacy)


@praktyki_bp.route('/zgloszenie/<uuid:zapis_id>/kreator/firma', methods=['GET', 'POST'])
@login_required
def kreator_firma(zapis_id):

    """Krok 2A: Dane zakładu + ZOPZ (ścieżka A)."""

    zapis = db.session.get(InternshipEnrollment, zapis_id)

    if not zapis or zapis.student_id != current_user.id or zapis.track_type != InternshipPath.STANDARD:

        abort(404)

    form = FormularzDaneFirmy()

    firmy_list = _repo_firm.aktywne()

    form.firma_id.choices = [('', '--- Wybierz firmę ---')] + [(str(f.id), f.name) for f in firmy_list]

    if form.validate_on_submit():

        if not form.ubezpieczenie_nw.data:

            flash('Ubezpieczenie NW jest wymagane.', 'danger')

            return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

        zapis.start_date = form.termin_od.data

        zapis.end_date = form.termin_do.data

        zapis.accident_insurance = True

        zapis.specialization = getattr(current_user, 'specialization', '') or ''

        dm = zapis.workplace_details or WorkplaceDetails(enrollment_id=zapis.id)

        if dm not in db.session:

            db.session.add(dm)

        if form.firma_typ.data == 'database':

```

```

if not form.firma_id.data:

    flash('Wybierz firmę z listy.', 'danger')

    return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

zapis.company_id = form.firma_id.data

dm.company_name = dm.company_address = dm.company_city = None

dm.company_tax_id = dm.authorized_person = dm.authorized_person_position = None

else:

    if not form.firma_nazwa.data or not form.firma_adres.data or not form.firma_miasto.data:

        flash('Podaj nazwę, adres i miasto firmy.', 'danger')

        return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

    zapis.company_id = None

    dm.company_name = form.firma_nazwa.data

    dm.company_address = form.firma_adres.data

    dm.company_city = form.firma_miasto.data

    dm.company_tax_id = form.firma_nip_krs.data

    dm.authorized_person = form.firma_upowazniony_osoba.data

    dm.authorized_person_position = form.firma_upowazniony_stanowisko.data

if not form.zopz_imie_nazwisko.data or not form.zopz_email.data:

    flash('Podaj imię/nazwisko i email opiekuna zakładowego (ZOPZ).', 'danger')

    return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

dm.workplace_mentor_name = form.zopz_imie_nazwisko.data

dm.workplace_mentor_position = form.zopz_stanowisko.data

dm.workplace_mentor_phone = form.zopz_telefon.data

dm.workplace_mentor_email = form.zopz_email.data

db.session.commit()

if zapis.status == EnrollmentStatus.AWAITING_APPROVAL:

    from core.uslugi.workflow import ZapisFSM

    ZapisFSM(zapis).wyslij_do_komisji()

    db.session.commit()

    flash('Dane zaktualizowane i zgłoszenie odesłane do komisji.', 'success')

    return redirect(url_for('praktyki.szczegoly_zgloszenia', id=zapis.id))

return redirect(url_for('praktyki.zapisz_krok2', id=zapis.id))

```

```

if request.method == 'GET':

    dm = zapis.workplace_details

    form.termin_od.data      = zapis.start_date

    form.termin_do.data      = zapis.end_date

    form.ubezpieczenie_nw.data = zapis.accident_insurance

    form.firma_typ.data = 'database' if zapis.company_id else 'custom'

    form.firma_id.data  = str(zapis.company_id) if zapis.company_id else ''

    form.firma_nazwa.data      = dm.company_name          if dm else None

    form.firma_adres.data      = dm.company_address        if dm else None

    form.firma_miasto.data     = dm.company_city           if dm else None

    form.firma_nip_krs.data    = dm.company_tax_id         if dm else None

    form.firma_upowazniony_osoba.data = dm.authorized_person    if dm else None

    form.firma_upowazniony_stanowisko.data = dm.authorized_person_position    if dm else None

    form.zopz_imie_nazwisko.data = dm.workplace_mentor_name    if dm else None

    form.zopz_stanowisko.data    = dm.workplace_mentor_position if dm else None

    form.zopz_telefon.data       = dm.workplace_mentor_phone    if dm else None

    form.zopz_email.data         = dm.workplace_mentor_email    if dm else None


return render_template('kreator/krok2a_firma.html', form=form, zapis=zapis, firmy_list=firmy_list)

```

```

@praktyki_bp.route('/zgloszenie/<uuid:zapis_id>/kreator/wniosek', methods=['GET', 'POST'])

```

```

@login_required

```

```

def kreator_wniosek(zapis_id):

```

```

    """Krok 2B/C: Wniosek dla ścieżek B i C."""

```

```

    zapis = db.session.get(InternshipEnrollment, zapis_id)

```

```

    if not zapis or zapis.student_id != current_user.id:

```

```

        abort(404)

```

```

    if zapis.track_type == InternshipPath.STANDARD:

```

```

        return redirect(url_for('praktyki.kreator_firma', zapis_id=zapis_id))

```

```

    form = FormularzWniosek()

```

```

if form.validate_on_submit():

    dm = zapis.workplace_details or WorkplaceDetails(enrollment_id=zapis.id)

    if dm not in db.session:

        db.session.add(dm)

    dm.company_name          = form.pracodawca_nazwa.data

    dm.company_address       = form.pracodawca_adres.data

    dm.company_city          = form.pracodawca_miasto.data

    dm.workplace_mentor_position = form.stanowisko.data


    uz = zapis.path_justification or PathJustification(enrollment_id=zapis.id)

    if uz not in db.session:

        db.session.add(uz)

    uz.justification = form.uzasadnienie.data


    byl_awaiting = zapis.status == EnrollmentStatus.AWAITING_APPROVAL

    db.session.commit()

    if byl_awaiting:

        from core.uslugi.workflow import ZapisFSM

        ZapisFSM(zapis).wyslij_do_komisji()

        db.session.commit()

        flash('Dane zaktualizowane i wniosek odesłany do komisji.', 'success')

        return redirect(url_for('praktyki.szczegoly_zgloszenia', id=zapis.id))

    return redirect(url_for('praktyki.potwierdz_wyslanie', id=zapis.id))


if request.method == 'GET':

    dm = zapis.workplace_details

    uz = zapis.path_justification

    form.pracodawca_nazwa.data = dm.company_name          if dm else None

    form.pracodawca_adres.data = dm.company_address       if dm else None

    form.pracodawca_miasto.data = dm.company_city         if dm else None

    form.stanowisko.data       = dm.workplace_mentor_position if dm else None

    form.uzasadnienie.data     = uz.justification         if uz else None


return render_template('kreator/krok2bc_wniosek.html', form=form, zapis=zapis)

```



```

@praktyki_bp.route('/')

@login_required

def lista():

    dostepne = _repo_praktyk.aktywne()

    zapisy_data = {}

    for z in _repo_zapisow.dla_studenta(current_user.id):

        komentarz_admina = z.admin_comments

        komentarz_uopz = z.supervisor_comments

        sciezka = z.path_type.value if z.path_type else None

        # Ścieżka A: admin zwraca do PENDING z komentarzem

        zwrocone_a = (z.status == EnrollmentStatus.PENDING and bool(komentarz_admina or komentarz_uopz))

        # Ścieżka B/C: komisja zwraca do AWAITING_APPROVAL z komentarzem UOPZ

        zwrocone_bc = (

            z.status == EnrollmentStatus.AWAITING_APPROVAL

            and bool(komentarz_uopz)

            and sciezka in ('EMPLOYMENT', 'OWN_BUSINESS')

        )

        zwrocone = zwrocone_a or zwrocone_bc

        zapisy_data[str(z.internship_id)] = {

            'id': str(z.id),

            'status': z.status.value,

            'sciezka': sciezka,

            'zwrocone': zwrocone,

            'komentarz_zwrotny': komentarz_admina or komentarz_uopz or '',

            'wymaga_uwagi': (

                z.status == EnrollmentStatus.AWAITING_APPROVAL

                and bool(komentarz_uopz)

                and sciezka == 'STANDARD'

            ),


```

```
}
```

```
csrf_form = FlaskForm()
```

```
return render_template('praktyki/lista.html', dostepne=dostepne, zapisy_data=zapisy_data, csrf_form=csrf_form)
```

```
@praktyki_bp.route('/zgloszenie/<uuid:id>/zakonczone', methods=['POST'])
```

```
@login_required
```

```
def zakoncz_praktyke(id):
```

```
    zapis = db.session.get(InternshipEnrollment, id)
```

```
    if not zapis or zapis.student_id != current_user.id:
```

```
        abort(404)
```

```
    if zapis.status != EnrollmentStatus.IN_PROGRESS:
```

```
        flash('Praktykę można zakończyć tylko gdy jest w trakcie realizacji.', 'warning')
```

```
        return redirect(url_for('praktyki.lista'))
```

```
    from core.uslugi.workflow import ZapisFSM
```

```
    ZapisFSM(zapis).zakonczone()
```

```
    db.session.commit()
```

```
    flash('Internship została oznaczona jako zakończona. Dokumenty końcowe są teraz dostępne w zakładce Moje Dokumenty.',  
          'success')
```

```
    return redirect(url_for('praktyki.lista'))
```

```
@praktyki_bp.route('/<uuid:id>/zapisz/krok1')
```

```
@login_required
```

```
def zapisz_krok1(id):
```

```
    """Stara trasa – przekierowanie do nowego kreatora."""
```

```
    return redirect(url_for('praktyki.kreator_sciezka', id=id))
```

```
@praktyki_bp.route('/zgloszenie/<uuid:id>/krok2', methods=['GET', 'POST'])
```

```
@login_required
```

```
def zapisz_krok2(id):
```

```
    zapis = db.session.get(InternshipEnrollment, id)
```

```
if not zapis or zapis.student_id != current_user.id:

    abort(404)

efekty = _repo_efektow.wszystkie()

if request.method == 'POST':

    # Czyszczenie starego jeśli student wraca z jakiegoś powodu

    _repo_zapisow.usun_harmonogram(zapis.id)

    suma_dni = 0

    nowe_wiersze = []

    for e in efekty:

        dz = request.form.get(f'dzial_{e.id}', '')

        pr = request.form.get(f'prace_{e.id}', '')

        dni_str = request.form.get(f'dni_{e.id}', '0')

        try:

            dni = int(dni_str)

        except Exception:

            dni = 0

        if dz.strip() and pr.strip():

            nowe_wiersze.append(InternshipSchedule(

                id=uuid.uuid4(),

                enrollment_id=zapis.id,

                learning_outcome_id=e.id,

                department_name=dz,

                example_tasks=pr,

                days_count=dni

            ))

            suma_dni += dni

    db.session.add_all(nowe_wiersze)

    db.session.commit()
```

```
        return redirect(url_for('praktyki.potwierdz_wyslanie', id=zapis.id))
```

```
    else:
```

```
        # GET request - pobranie istniejących danych harmonogramu
```

```
        istniejace_harmonogramy = {}
```

```
        harmonogramy = _repo_zapisow.harmonogram_dla_zapisu(zapis.id)
```

```
        for h in harmonogramy:
```

```
            istniejace_harmonogramy[str(h.learning_outcome_id)] = {
```

```
                'dzial': h.department_name,
```

```
                'prace': h.example_tasks,
```

```
                'dni': h.days_count
```

```
            }
```

```
    csrf_form = FlaskForm()
```

```
    return render_template('kreator/krok3_harmonogram.html',
```

```
                           zapis=zapis,
```

```
                           efekty=efekty,
```

```
                           csrf_form=csrf_form,
```

```
                           istniejace_harmonogramy=istniejace_harmonogramy)
```

```
@praktyki_bp.route('/zgloszenie/<uuid:id>/potwierdz-wyslanie')
```

```
@login_required
```

```
def potwierdz_wyslanie(id):
```

```
    zapis = db.session.get(InternshipEnrollment, id)
```

```
    if not zapis or zapis.student_id != current_user.id:
```

```
        abort(404)
```

```
    if zapis.status != EnrollmentStatus.PENDING:
```

```
        return redirect(url_for('praktyki.szczegoly_zgloszenia', id=id))
```

```
    from flask_wtf import FlaskForm
```

```
    csrf_form = FlaskForm()
```

```
    return render_template('kreator/potwierdz_wyslanie.html', zapis=zapis, csrf_form=csrf_form)
```

```

@praktyki_bp.route('/zgloszenie/<uuid:id>/wyslij', methods=['POST'])

@login_required

def wyslij_do_zatwierdzenia(id):

    zapis = db.session.get(InternshipEnrollment, id)

    if not zapis or zapis.student_id != current_user.id:

        abort(404)

    if zapis.status != EnrollmentStatus.PENDING:

        flash('Zgłoszenie zostało już wysłane.', 'info')

        return redirect(url_for('praktyki.szczegoly_zgloszenia', id=id))

    # Ścieżka A → czeka na UOPZ; B/C → od razu do komisji

    from core.uslugi.workflow import ZapisFSM

    fsm = ZapisFSM(zapis)

    if zapis.path_type and zapis.path_type.value in ('EMPLOYMENT', 'OWN_BUSINESS'):

        fsm.wyslij_do_komisji()

    else:

        fsm.wyslij_do_akceptacji()

    db.session.commit()

    flash('Zgłoszenie zostało przesłane do zatwierdzenia.', 'success')

    return redirect(url_for('praktyki.szczegoly_zgloszenia', id=id))

```

```

@praktyki_bp.route('/zgloszenie/<uuid:id>/szczegoly')

@login_required

def szczegoly_zgloszenia(id):

    """Szczegóły zgłoszenia studenta wraz z komentarzami UOPZ"""

    zapis = db.session.get(InternshipEnrollment, id)

    if not zapis or zapis.student_id != current_user.id:

        abort(404)

    rows = _repo_docs.dla_zapisu_studenta(id, current_user.id)

    uploaded_docs = [

        {

            'id': str(d.id),

            'original_filename': d.original_filename,

```

```

        'document_type': d.document_type,

        'uploaded_at': d.uploaded_at,

    }

    for d in rows

]

from flask_wtf import FlaskForm

csrf_form = FlaskForm()

    return render_template('praktyki/szczegoly_zgloszenia.html', zapis=zapis, uploaded_docs=uploaded_docs,
csrf_form=csrf_form)

@praktyki_bp.route('/zgloszenie/<uuid:id>/resubmit', methods=['POST'])

@login_required

def resubmit_zgloszenia(id):

    """Student ponownie wysyła zgłoszenie po poprawkach."""

    zapis = db.session.get(InternshipEnrollment, id)

    if not zapis or zapis.student_id != current_user.id:

        abort(404)

    if zapis.status != EnrollmentStatus.AWAITING_APPROVAL:

        flash('Zgłoszenie nie może być ponownie wysłane w tym statusie.', 'warning')

        return redirect(url_for('praktyki.szczegoly_zgloszenia', id=id))

    from core.uslugi.workflow import ZapisFSM

    ZapisFSM(zapis).wyslij_do_komisji()

    db.session.commit()

    flash('Zgłoszenie zostało ponownie wysłane do weryfikacji komisji.', 'success')

    return redirect(url_for('praktyki.szczegoly_zgloszenia', id=id))

```

PLIK: app_student\routes\pulpit.py

```
from flask import Blueprint, render_template

from flask_login import login_required, current_user

from core.modele import InternshipEnrollment, EnrollmentStatus

from core.repozytoria import RepozytoriumZapisow


dashboard_bp = Blueprint('dashboard', __name__)


_repo_zapisow = RepozytoriumZapisow()


@dashboard_bp.route('/')

@login_required

def index():

    zapis = _repo_zapisow.ostatni_dla_studenta(current_user.id)

    return render_template('dashboard/index.html', zapis=zapis)
```

PLIK: app_student\routes\sprawozdania.py

```
import uuid

from flask import Blueprint, render_template, redirect, url_for, flash, request

from flask_login import login_required, current_user

from flask_wtf import FlaskForm

from wtforms import TextAreaField

from wtforms.validators import DataRequired, Optional


from core.extensions import db

from core.modele import InternshipEnrollment, InternshipReport, EnrollmentStatus, InternshipPath

from core.repozytoria import RepozytoriumZapisow


_repo_zapisow = RepozytoriumZapisow()


sprawozdania_bp = Blueprint('sprawozdania', __name__)


class FormularzSprawozdaniaStandard(FlaskForm):

    """Zał. 7 – ścieżka A: standardowa praktyka."""

    charakterystyka = TextAreaField('I. Charakterystyka miejsca odbywania praktyki', validators=[DataRequired()])

    opis = TextAreaField('II. Opis i analiza wykonywanych prac', validators=[DataRequired()])

    wiedza = TextAreaField('III. Wiedza i umiejętności uzyskane w trakcie praktyki', validators=[Optional()])


class FormularzSprawozdaniaPracaZawodowa(FlaskForm):

    """Zał. 7a – ścieżki B/C: praca zawodowa lub działalność gospodarcza."""

    charakterystyka = TextAreaField('I. Charakterystyka miejsca pracy / działalności', validators=[DataRequired()])

    opis = TextAreaField('II. Opis i analiza wykonywanych prac / działalności', validators=[DataRequired()])

    wiedza = TextAreaField('III. Wiedza i umiejętności uzyskane w trakcie pracy zawodowej lub działalności',
validators=[Optional()])


@sprawozdania_bp.route('/', methods=['GET', 'POST'])

@login_required
```



```

def index():

    zapis = _repo_zapisow.aktywny_dla_studenta(current_user.id, [

        EnrollmentStatus.IN_PROGRESS, EnrollmentStatus.COMMISSION_REVIEW,

        EnrollmentStatus.DEAN_APPROVAL, EnrollmentStatus.COMPLETED, EnrollmentStatus.AWAITING_APPROVAL,

    ])

    if not zapis:

        ma_zapis = _repo_zapisow.pierwszy_dla_studenta(current_user.id)

        return render_template('sprawozdania/index.html', zapis=None, ma_zapis=ma_zapis)

# Utwórz sprawozdanie jeśli nie istnieje
if not zapis.sprawozdanie:

    nowe_spr = InternshipReport(

        id=uuid.uuid4(),

        enrollment_id=zapis.id,

        charakterystyka_miejsca='',

        opis_i_analiza=''

    )

    db.session.add(nowe_spr)

    db.session.commit()

    zapis = db.session.get(InternshipEnrollment, zapis.id)

# Wybierz formularz i szablon wg ścieżki
sciezka = zapis.track_type.value if zapis.track_type else 'STANDARD'

is_standard = sciezka == 'STANDARD'

FormClass = FormularzSprawozdaniaStandard if is_standard else FormularzSprawozdaniaPracaZawodowa

form = FormClass()

if form.validate_on_submit():

    zapis.sprawozdanie.charakterystyka_miejsca = form.charakterystyka.data

    zapis.sprawozdanie.opis_i_analiza = form.opis.data

    db.session.commit()

    flash('InternshipReport zostało zapisane.', 'success')

```

```
        return redirect(url_for('sprawozdania.index'))

elif request.method == 'GET':

    form.charakterystyka.data = zapis.sprawozdanie.charakterystyka_miejsca

    form.opis.data = zapis.sprawozdanie.opis_i_analiza


return render_template('sprawozdania/index.html',

                       zapis=zapis, form=form, ma_zapis=zapis,

                       is_standard=is_standard, sciezka=sciezka)
```

PLIK: app_student\static\css\student.css

```
/* =====  
  
student.css – Punkt wejścia arkusza stylów panelu studenta  
  
System Praktyk Zawodowych – ANS Elbląg  
  
Moduły ładowane w kolejności zależności:  
  
1. zmienne – tokeny CSS, reset, dostępność (WCAG 2.1 AA)  
  
2. komunikaty – flash messages i powiadomienia  
  
3. formularze – pola input, checkboxy  
  
4. logowanie – strona logowania i zmiana hasła  
  
5. układ – sidebar, panel-main, nagłówek  
  
6. karty – komponent .karta  
  
7. tabele – tabele danych  
  
8. przyciski – przyciski i grupy akcji  
  
9. statusy – odznaki statusów i ról  
  
10. komponenty – pasek postępu, paginacja, filtry  
  
11. dashboard – statystyki, strona błędu  
  
===== */  
  
@import url('/core/static/css/modul/zmienne.css');  
  
@import url('/core/static/css/modul/komunikaty.css');  
  
@import url('/core/static/css/modul/formularze.css');  
  
@import url('/core/static/css/modul/logowanie.css');  
  
@import url('/core/static/css/modul/uklad.css');  
  
@import url('/core/static/css/modul/karty.css');  
  
@import url('/core/static/css/modul/tabele.css');  
  
@import url('/core/static/css/modul/przyciski.css');  
  
@import url('/core/static/css/modul/statusy.css');  
  
@import url('/core/static/css/modul/komponenty.css');  
  
@import url('/core/static/css/modul/dashboard.css');
```

PLIK: app_student\templates\auth\logowanie.html

```
{% extends "layouts/base_auth.html" %}

{% block tytul %}Logowanie{% endblock %}


{% block body %}

<a href="#logowanie" class="skip-link">Przejdź do logowania</a>


<main class="panel-logowania">


<div class="panel-logowania__branding" aria-hidden="true">

<div class="branding__tresc">

<div class="branding__logo-kontener">



</div>

<div class="branding__separator"></div>

<p class="branding__nazwa-systemu">System Praktyk Zawodowych</p>

<p class="branding__podtytul">Panel studenta</p>

<div class="branding__dekoracja">

<span></span><span></span><span></span></div>

</div>

</div>


<div class="panel-logowania__formularz">

<div class="formularz__kontener" id="logowanie">


<div class="formularz__naglowek">

<h1 class="formularz__tytul">Zaloguj się</h1>

<p class="formularz__opis">Użyj studenckiego konta Microsoft.</p>

</div>


{% with messages = get_flashed_messages(with_categories=true) %}

{% if messages %}
```

```

<div class="komunikaty" aria-live="polite">

    {% for kategoria, wiadomosc in messages %}

    <div class="komunikat komunikat--{{ kategoria }}" role="alert">

        <span class="komunikat__ikona" aria-hidden="true">

            {% if kategoria == 'success' %}{% elif kategoria == 'danger' %}{% else %}i{% endif %}

        </span>

        {{ wiadomosc }}

    </div>

    {% endfor %}

</div>

{% endif %}

{% endwith %}

```

```

<div class="logowanie-microsoft">

    <a

        href="{{ url_for('auth.ms_login', next=request.args.get('next', '')) }}"

        class="przycisk-microsoft"

        aria-label="Zaloguj się przez konto Microsoft"

    >

        <svg class="przycisk-microsoft__logo" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 21 21"
        aria-hidden="true">

            <rect x="1" y="1" width="9" height="9" fill="#f25022"/>

            <rect x="11" y="1" width="9" height="9" fill="#7fba00"/>

            <rect x="1" y="11" width="9" height="9" fill="#00a4ef"/>

            <rect x="11" y="11" width="9" height="9" fill="#ffb900"/>

        </svg>

        <span>Zaloguj się przez Microsoft</span>

    </a>

</div>

```

```

<div class="formularz__stopka">

    <p class="formularz__stopka-tekst">

        Zaloguj się kontem w domenie<br>

        <strong>tulodzieckiall@gmail.onmicrosoft.com</strong>

    </p>

```

</div>

</div>

</div>

</main>

{% endblock %}

PLIK: app_student\templates\dashboard\index.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Pulpit{% endblock %}

{% block naglowek %}Witaj, {{ current_user.first_name }}{% endblock %}


{% block tresc %}


{% if not zapis %}

    <div style="display: flex; justify-content: center; align-items: center; padding-top: 4rem;">

        <div class="karta" style="max-width:560px; text-align:center; padding:3rem 2rem; width: 100%;">

            <div style="color: var(--kolor-glowny); margin-bottom: 1.5rem;">

                <svg xmlns="http://www.w3.org/2000/svg" width="64" height="64" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round">

                    <path d="M14 2H6a2 2 0 0 0-2 2v16c0 1.1.9 2 2 2h12a2 2 0 0 0 2-2V8l-6-6z"/>

                    <path d="M14 3v5h5M16 13H8M16 17H8M10 9H8"/>

                </svg>

            </div>

            <h2 style="font-size:1.25rem; font-weight:600; margin-bottom:0.5rem;">Nie jesteś jeszcze zapisany na praktykę</h2>

            <p style="color:var(--kolor-tekst-drugor); margin-bottom:1.5rem;">Zapisz się do aktywnej praktyki żeby zacząć
prowadzić dziennik.</p>

            <a href="{{ url_for('praktyki.lista') }}" class="przycisk przycisk--glowny">Zobacz dostępne praktyki</a>

        </div>

    </div>

{% else %}

    <section class="siatka-statystyk" aria-label="Twój postęp">

        <div class="karta-stat"><span class="karta-stat__liczba">{{ zapis.total_hours_logged }}</span><span
class="karta-stat__etykieta">Zalogowanych godzin</span></div>

        <div class="karta-stat"><span class="karta-stat__liczba">{{ zapis.praktyka.wymiar_godzin }}</span><span
class="karta-stat__etykieta">Wymaganych godzin</span></div>

        <div class="karta-stat"><span class="karta-stat__liczba">{{ zapis.wpisy_dziennika | length }}</span><span
class="karta-stat__etykieta">Wpisów w dzienniku</span></div>

        <div class="karta-stat">{% set procent = 0 if not zapis.praktyka.wymiar_godzin else [(zapis.total_hours_logged /
zapis.praktyka.wymiar_godzin * 100)|round|int, 100]|min %}<span class="karta-stat__liczba">{{ procent }}%</span><span
class="karta-stat__etykieta">Ukończono</span></div>

    </section>

<div class="karta" style="margin-bottom:2rem;">
```

```

<div class="karta__tresc">

    <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:0.75rem;">

        <span style="font-weight:600;">Postęp praktyki – {{ zapis.praktyka.rok_uczelniany }} ({{ zapis.praktyka.semestr
        }})</span>

    <div style="display:flex; align-items:center; gap:0.75rem;">

        <span class="status

            {%- if zapis.status.value == 'PENDING' %} status--draft

            {%- elif zapis.status.value == 'AWAITING_APPROVAL' %} status--planned

            {%- elif zapis.status.value == 'COMMISSION_REVIEW' %} status--planned

            {%- elif zapis.status.value == 'DEAN_APPROVAL' %} status--planned

            {%- elif zapis.status.value == 'IN_PROGRESS' %} status--w-trakcie

            {%- elif zapis.status.value == 'COMPLETED' %} status--zakonczone

            {%- elif zapis.status.value == 'REJECTED' %} status--draft

            {%- endif %}">

            {% if zapis.status.value == 'PENDING' %}Szkic

            {% elif zapis.status.value == 'AWAITING_APPROVAL' %}Oczekuje na zatwierdzenie

            {% elif zapis.status.value == 'COMMISSION_REVIEW' %}Weryfikacja komisji

            {% elif zapis.status.value == 'DEAN_APPROVAL' %}Oczekuje na dziekana

            {% elif zapis.status.value == 'IN_PROGRESS' %}W trakcie

            {% elif zapis.status.value == 'COMPLETED' %}Zakończona

            {% elif zapis.status.value == 'REJECTED' %}Odrzucony

            {% endif %}

        </span>

        <span style="font-size:0.85rem; color:var(--kolor-tekst-drugor);">{{ zapis.total_hours_logged }} / {{
        zapis.praktyka.wymiar_godzin }} h</span>

    </div>

</div>

<div class="pasek-postep" style="height:12px;">

    <div class="pasek-postep__wypelnienie" style="width:{{ procent }}%;"></div>

</div>

{% if zapis.komentarze_uopz %}

    <div style="margin-top:1rem; padding:0.75rem; background:#fef3cd; border-radius:0.375rem; border-left:4px solid
    #f59e0b;">

        <div style="font-weight:500; color:#92400e; margin-bottom:0.5rem;">Uwagi UOPZ:</div>

        <div style="color:#92400e; font-size:0.9rem; white-space: pre-wrap;">{{ zapis.komentarze_uopz }}</div>

        <div style="margin-top:0.75rem;">

```



```
        <a href="{{ url_for('praktyki.szczegoly_zgloszenia', id=zapis.id) }}" class="przycisk przycisk--maly
przycisk--drugorzeczny">Zobacz szczegóły zgłoszenia</a>
```

```
    </div>
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
<div style="display:flex; gap:1rem; flex-wrap:wrap; justify-content:center; align-items:center; margin-bottom: 2rem;">
```

```
    <a href="{{ url_for('dziennik.nowy_wpis') }}" class="przycisk przycisk--glowny">Dodaj wpis do dziennika</a>
```

```
    <a href="{{ url_for('dziennik.index') }}" class="przycisk przycisk--drugorzeczny">Przeglądaj dziennik</a>
```

```
</div>
```

```
{% endif %}
```

```
{% endblock %}
```

PLIK: app_student\templates\dokumenty\moje_dokumenty.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Moje Dokumenty{% endblock %}

{% block naglowek %}Moje Dokumenty{% endblock %}


{% block tresc %}

<div style="max-width: 900px; margin: 0 auto;">


    {% if not dokumenty_list %}

    <div class="karta">

        <div class="karta__tresc" style="text-align: center; padding: 3rem 2rem;">

            <div style="font-size: 3rem; margin-bottom: 1rem; opacity: 0.3;">

                <svg viewBox="0 0 24 24" style="width: 3rem; height: 3rem; fill: none; stroke: currentColor; stroke-width: 2;">

                    <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>

                    <polyline points="14 2 14 8 20 8"/>

                </svg>

            </div>

            <div style="font-size: 1.1rem; font-weight: 600; margin-bottom: 0.5rem;">Brak dostępnych dokumentów</div>

            <div style="color: var(--kolor-tekst-drugor);">Dokumenty pojawią się tutaj po zatwierdzeniu praktyki przez
            opiekuna.</div>

        </div>

    </div>

    {% endif %}


    {% for wpis in dokumenty_list %}

    <div class="karta" style="margin-bottom: 1.5rem;">

        <div class="karta__naglowek">

            <div>

                <div style="font-weight: 600;">{{ wpis.zapis.praktyka.rok_uczelniiany }} · Semestr {{ wpis.zapis.praktyka.semestr
                }}</div>

                <div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-top: 0.25rem;">

                    {% if wpis.zapis.firma %}{{ wpis.zapis.firma.nazwa }}{% elif wpis.zapis.firma_nazwa %}{{ wpis.zapis.firma_nazwa
                    }}{% endif %}

                    · Ścieżka:

                    {% if wpis.sciezka == 'STANDARD' %}Standardowa
```

```

        {% elif wpis.sciezka == 'EMPLOYMENT' %}Praca etatowa

        {% else %}Działalność gospodarcza{% endif %}

    </div>

</div>

    <span class="status {% if wpis.zapis.status.value == 'COMPLETED' %}status--zakonczone{% else %}status--aktywna{%
endif %}">

        {% if wpis.zapis.status.value == 'COMPLETED' %}Zakończona{% else %}W trakcie{% endif %}

    </span>

</div>

<div class="karta__tresc">

    <div style="display: grid; gap: 0.6rem;">

        {% for doc in wpis.docs %}

            {% if doc.separator is defined and doc.separator %}

                <div style="display: flex; align-items: center; gap: 0.75rem; margin: 0.5rem 0;">

                    <hr style="flex: 1; border: none; border-top: 1px dashed #d1d5db;">

                    <span style="font-size: 0.78rem; color: var(--kolor-tekst-drugor); white-space: nowrap; font-weight: 500;">{{
doc.nazwa }}</span>

                    <hr style="flex: 1; border: none; border-top: 1px dashed #d1d5db;">

                </div>

            {% else %}

                <div style="display: flex; justify-content: space-between; align-items: center; padding: 0.75rem 1rem;

                    background: {% if doc.dostepny %}#f8fafc{% else %}#f3f4f6{% endif %};

                    border: 1px solid {% if doc.dostepny %}#e2e8f0{% else %}#d1d5db{% endif %};

                    border-radius: 0.5rem;">

                    <div>

                        <div style="font-weight: 500; font-size: 0.9rem; {% if not doc.dostepny %}color: var(--kolor-tekst-drugor);{%
endif %}">

                            {{ doc.nazwa }}

                        </div>

                        {% if doc.opis %}

                            <div style="font-size: 0.78rem; color: var(--kolor-tekst-drugor);">{{ doc.opis }}</div>

                        {% endif %}

                        {% if not doc.dostepny and doc.powod %}

                            <div style="font-size: 0.78rem; color: #d97706; margin-top: 0.15rem;">{{ doc.powod }}</div>

                        {% endif %}

                    </div>

```

```
{% if doc.dostepny %}

    {% if doc.staly is defined and doc.staly %}

        <a href="{{ url_for('documents.pobierz_staly', doc_key=doc.klucz_staly) }}"

            class="przycisk przycisk--drugorzeczny przycisk--maly">Pobierz PDF</a>

    {% elif doc.dynamiczny is defined and doc.dynamiczny %}

        <a href="{{ url_for('documents.pobierz_dynamiczny', zapis_id=doc.zapis_id, typ=doc.typ) }}"

            class="przycisk przycisk--drugorzeczny przycisk--maly">Pobierz PDF</a>

    {% endif %}

    {% else %}

        <span style="font-size: 0.78rem; color: var(--kolor-tekst-drugor); padding: 0.3rem 0.625rem; white-space:
nowrap;">Niedostępný</span>

    {% endif %}

</div>

{% endif %}

{% endfor %}

</div>

</div>

</div>

{% endfor %}

</div>

{% endblock %}
```

PLIK: app_student\templates\dokumenty\panel.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Panel Dokumentów{% endblock %}

{% block naglowek %}Panel Dokumentów do Wydrukowania{% endblock %}

{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

  <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Powrót do pulpitu</a>

</div>

{% endblock %}

{% block tresc %}

<div style="max-width: 1200px; margin: 0 auto;">

  <!-- Informacja o systemie -->

  <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-info);">

    <div class="karta__tresc">

      <div>

        <div style="font-weight: 600; margin-bottom: 0.5rem;">System Generowania Dokumentów</div>

        <div style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

          Dokumenty generują się automatycznie na podstawie danych z Twojego zgłoszenia praktyki.

          <strong>Wydrukuj, podpisz i dostarcz do firmy/uczelni przed rozpoczęciem praktyki.</strong>

        </div>

      </div>

    </div>

  </div>

  <div>

    <!-- Pusty stan -->

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

      <div style="font-size: 3rem; margin-bottom: 1rem; opacity: 0.3;"></div>

      <h3 style="margin-bottom: 0.5rem;">Brak praktyk do dokumentacji</h3>

      <p style="color: var(--kolor-tekst-drugor); margin-bottom: 2rem;">

        Najpierw zapisz się na praktykę i uzupełnij harmonogram.

      </p>

    </div>

  </div>

</div>

{% endblock %}
```

</p>

Zobacz dostępne praktyki

</div>

{% else %}

<!-- Lista praktyk z dokumentami -->

{% for item in dokumenty_data %}

<div class="karta" style="margin-bottom: 2rem;">

<div class="karta__naglowek" style="background: var(--kolor-tlo);">

<div style="display: flex; justify-content: space-between; align-items: center;">

<h3 class="karta__tytul">

{{ item.zapis.praktyka.rok_uczelniany }} ({{ item.zapis.praktyka.semestr }})

</h3>

{{ tлумacz_status(item.zapis.status.value) }}

</div>

</div>

<div class="karta__tresc">

<div style="margin-bottom: 1.5rem;">

<div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor);">

Firma: {{ item.zapis.firma_nazwa or 'Nie podano' }} |

Okres:

{% if item.zapis.termin_od and item.zapis.termin_do %}

{{ item.zapis.termin_od.strftime('%d.%m.%Y') }} - {{ item.zapis.termin_do.strftime('%d.%m.%Y') }}

{% else %}

Nie określono

{% endif %}

</div>

</div>

```

{% if not item.harmonogram_gotowy %}

<!-- Harmonogram nie gotowy -->

    <div style="padding: 1rem; background: #fef3cd; border-radius: 0.5rem; border-left: 4px solid #f59e0b;
margin-bottom: 1.5rem;">

        <div style="font-weight: 500; color: #92400e; margin-bottom: 0.5rem;">

            Harmonogram nie został uzupełniony

        </div>

        <div style="color: #92400e; font-size: 0.9rem; margin-bottom: 1rem;">

            Przed generowaniem dokumentów musisz uzupełnić harmonogram praktyki (krok 2).

        </div>

        <a href="{{ url_for('praktyki.zapisz_krok2', id=item.zapis.id) }}" class="przycisk przycisk--maly
przycisk--glowny">

            Uzupełnij harmonogram

        </a>

    </div>

{% elif not item.mozna_generowac %}

<!-- Status nie pozwala na generowanie -->

<div style="padding: 1rem; background: #f3f4f6; border-radius: 0.5rem; margin-bottom: 1.5rem;">

    <div style="color: var(--kolor-tekst-drugor); font-size: 0.9rem;">

        Dokumenty będą dostępne po zatwierdzeniu zgłoszenia przez UOPZ.

    </div>

</div>

{% else %}

<!-- Dokumenty dostępne do generacji -->

<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 1rem;">

    <!-- Załącznik 1 - Podanie -->

    <div class="dokument-karta">

        <div class="dokument-karta__naglowek">

            <h4>Załącznik 1</h4>

            <span class="dokument-status dokument-status--dostepny">Dostępny</span>

        </div>

```

```

<div class="dokument-karta__tresc">

    <div class="dokument-tytul">Podanie o realizację praktyki studenckiej</div>

    <div class="dokument-opis">Dane studenta, firmy, planowane terminy</div>

    <a href="{{ url_for('dokumenty.generuj_dokument', enrollment_id=item.zapis.id, doc_type='ZALACZNIK_1') }}"
        class="przycisk przycisk--maly przycisk--glowny">

        Pobierz PDF

    </a>

</div>

</div>

<!-- Załącznik 2 - Skierowanie -->

<div class="dokument-karta">

    <div class="dokument-karta__naglowek">

        <h4>Załącznik 2</h4>

        <span class="dokument-status dokument-status--dostepny">Dostępny</span>

    </div>

    <div class="dokument-karta__tresc">

        <div class="dokument-tytul">Skierowanie na praktykę studencką</div>

        <div class="dokument-opis">Z automatycznym numerem pisma wychodzącego</div>

        <a href="{{ url_for('dokumenty.generuj_dokument', enrollment_id=item.zapis.id, doc_type='ZALACZNIK_2') }}"
            class="przycisk przycisk--maly przycisk--glowny">

            Pobierz PDF

        </a>

    </div>

</div>

<!-- Załącznik 3 - Porozumienie (warunkowo) -->

{% if not item.firma_ma_umowe %}

<div class="dokument-karta">

    <div class="dokument-karta__naglowek">

        <h4>Załącznik 3</h4>

        <span class="dokument-status dokument-status--dostepny">Dostępny</span>

    </div>

    <div class="dokument-karta__tresc">

```



```

<div class="dokument-tytul">Porozumienie o organizację praktyki</div>

<div class="dokument-opis">Umowa z firmą (2 egzemplarze)</div>

<a href="{{ url_for('dokumenty.generuj_dokument', enrollment_id=item.zapis.id, doc_type='ZALACZNIK_3') }}"
    class="przycisk przycisk--maly przycisk--glowny">

    Pobierz PDF

</a>

</div>

</div>

{% else %}

<div class="dokument-karta dokument-karta--nieaktywny">

    <div class="dokument-karta__naglowek">

        <h4>Załącznik 3</h4>

        <span class="dokument-status dokument-status--niewymagany">Niewymagany</span>

    </div>

    <div class="dokument-karta__tresc">

        <div class="dokument-tytul">Porozumienie o organizację praktyki</div>

        <div class="dokument-opis">Firma ma stałą umowę z uczelnią</div>

    </div>

</div>

{% endif %}

<!-- Załącznik 4 - Indywidualny Program -->

<div class="dokument-karta {% if not item.program_approved %}dokument-karta--oczekuje{% endif %}">

    <div class="dokument-karta__naglowek">

        <h4>Załącznik 4</h4>

        <span class="dokument-status dokument-status--{% if item.program_approved %}dostepny{% else %}oczekuje{%
endif %}">

            {% if item.program_approved %}Zatwierdzony{% else %}Wymaga zatwierdzenia{% endif %}

        </span>

    </div>

    <div class="dokument-karta__tresc">

        <div class="dokument-tytul">Indywidualny Program Praktyk</div>

        <div class="dokument-opis">Na podstawie Twojego harmonogramu</div>

        {% if item.program_approved %}

```

```

        <a href="{{ url_for('dokumenty.generuj_dokument', enrollment_id=item.zapis.id, doc_type='ZALACZNIK_4') }}"
            class="przycisk przycisk--maly przycisk--glowny">

            Pobierz PDF

        </a>

        {% else %}

            <form method="POST" action="{{ url_for('dokumenty.zatwierdz_program', enrollment_id=item.zapis.id) }}"
                style="margin-top: 0.5rem;">

                {{ csrf_form.hidden_tag() }}

                <button type="submit" class="przycisk przycisk--maly przycisk--drugorzedny">

                    Poproś o zatwierdzenie

                </button>

            </form>

        {% endif %}

    </div>

</div>

</div>

{% endif %}

</div>

</div>

{% endfor %}

{% endif %}

</div>

<style>

.dokument-karta {

    border: 1px solid var(--kolor-ramka);

    border-radius: var(--promien);

    overflow: hidden;

    background: white;

}

.dokument-karta--nieaktywny {

    opacity: 0.6;

```

```
background: #f9f9f9;

}

.dokument-karta--oczekuje {

  border-color: var(--kolor-ostrezenie);

}

.dokument-karta__naglowek {

  padding: 0.75rem 1rem;

  background: var(--kolor-tlo);

  display: flex;

  justify-content: space-between;

  align-items: center;

  border-bottom: 1px solid var(--kolor-ramka);

}

.dokument-karta__naglowek h4 {

  margin: 0;

  font-size: 0.9rem;

  color: var(--kolor-glowny);

}

.dokument-karta__tresc {

  padding: 1rem;

}

.dokument-tytul {

  font-weight: 600;

  margin-bottom: 0.5rem;

  font-size: 0.9rem;

}

.dokument-opis {

  font-size: 0.8rem;
```

```
    color: var(--kolor-tekst-drugor);

    margin-bottom: 1rem;

}
```

```
.dokument-status {

    padding: 0.25rem 0.5rem;

    border-radius: 3px;

    font-size: 0.7rem;

    font-weight: 500;

}
```

```
.dokument-status--dostepny {

    background: #d1fae5;

    color: #065f46;

}
```

```
.dokument-status--oczekuje {

    background: #fef3cd;

    color: #92400e;

}
```

```
.dokument-status--niewymagany {

    background: #e5e7eb;

    color: #6b7280;

}
```

```
</style>
```

```
{% endblock %}
```

PLIK: app_student\templates\dziennik\index.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Dziennik praktyki{% endblock %}

{% block naglowek %}Dziennik praktyki{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem; align-items: center;">

    {% if zapis %}

        <a href="{% url_for('dziennik.nowy_wpis') %}" class="przycisk przycisk--glowny">Nowy wpis</a>

        <a href="{% url_for('documents.moje_dokumenty') %}" class="przycisk przycisk--drugorzeczny">Pobierz PDF (Moje Dokumenty)</a>

    {% endif %}

</div>

{% endblock %}


{% block tresc %}


{% if not zapis %}

    <div style="display: flex; justify-content: center; align-items: center; padding-top: 4rem;">

        <div class="karta" style="max-width: 560px; text-align: center; padding: 3rem 2rem; width: 100%;">

            {% if jakikolwiek %}

                <p style="color: var(--kolor-tekst-drugor);">Twój zapis oczekuje na aktywację przez opiekuna.</p>

            {% else %}

                <p style="color: var(--kolor-tekst-drugor);">Najpierw zapisz się na praktykę.</p>

                <a href="{% url_for('praktyki.lista') %}" class="przycisk przycisk--glowny" style="margin-top: 1rem;">Zobacz praktyki</a>

            {% endif %}

        </div>

    </div>

{% else %}

    <div class="karta" style="margin-bottom: 1.5rem; border-left: 4px solid var(--kolor-glowny);">

        <div class="karta__tresc">

            <div style="display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 1.5rem;">
```

```

<div>

    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Praktyka</div>

    <div style="font-weight: 600;">{{ zapis.praktyka.rok_uczelnianny }}</div>

    <div style="font-size: 0.8rem; color: var(--kolor-tekst-drugor);">Semestr {{ zapis.praktyka.semestr }}</div>

</div>

<div>

    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Wpisy</div>

    <div style="font-weight: 600; font-size: 1.2rem;">{{ wpisy | length }}</div>

</div>

<div>

    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Godziny</div>

    <div style="font-weight: 700; font-size: 1.2rem; color: var(--kolor-glowny);">{{ zapis.total_hours_logged }} / {{ zapis.praktyka.wymiar_godzin }} h</div>

    <div class="pasek-postep" style="margin-top: 0.5rem; margin-bottom: 1rem;">

        {% set proc_h = 0 %}

        {% if zapis and zapis.praktyka and zapis.praktyka.wymiar_godzin > 0 %}

            {% set proc_h = [(zapis.total_hours_logged / zapis.praktyka.wymiar_godzin * 100)|int, 100]|min %}

            {% endif %}

        <div class="pasek-postep__wypelnienie{% if proc_h >= 100 %} pasek-postep__wypelnienie--zakonczone{% endif %}" style="width: {{ proc_h }}%;"></div>

    </div>

    <div style="font-size: 0.75rem; text-transform: uppercase; color: var(--kolor-tekst-drugor); margin-bottom: 0.25rem;">Dni</div>

    <div style="font-weight: 700; font-size: 1.2rem; color: var(--kolor-glowny);">{{ wpisy | length }} / 120 dni</div>

    <div class="pasek-postep" style="margin-top: 0.5rem;">

        {% set proc_d = [(wpisy | length / 120 * 100)|int, 100]|min %}

        <div class="pasek-postep__wypelnienie{% if proc_d >= 100 %} pasek-postep__wypelnienie--zakonczone{% endif %}" style="width: {{ proc_d }}%;"></div>

    </div>

</div>

</div>

</div>

```

```

<div class="karta">

  <div class="karta__naglowek"><h2 class="karta__tytul">Wpisy dziennika</h2></div>

  <div class="tabela-wrapper">

    <table class="tabela">

      <thead>

        <tr>

          <th>Data</th>

          <th>Godz.</th>

          <th>Opis czynności</th>

          <th>Efekty</th>

          <th></th>

        </tr>

      </thead>

      <tbody>

        {% for w in wpisy %}

        <tr>

          <td style="white-space: nowrap;"><strong>{{ w.entry_date.strftime('%d.%m.%Y') }}</strong></td>

          <td style="white-space: nowrap;">{{ w.duration_hours }} h</td>

          <td style="font-size: 0.85rem; line-height: 1.5;">{{ w.description }}</td>

          <td style="white-space: nowrap;">

            {% if w.efekty_uczenia %}

            {% for e in w.efekty_uczenia %}

              <span style="font-size:0.75rem; background:var(--kolor-tlo); padding:0.2rem 0.4rem; border-radius:4px; font-weight:600; display:inline-block; margin:1px;">{{ e.kod }}</span>

            {% endfor %}

            {% else %}<!--{% endif %}

          </td>

          <td style="white-space: nowrap;">

            <a href="{{ url_for('dziennik.edytuj_wpis', wpis_id=w.id) }}" class="przycisk przycisk--maly przycisk--drugorzeczny">Edytuj</a>

            <form method="POST" action="{{ url_for('dziennik.usun_wpis', wpis_id=w.id) }}" style="display:inline;">

              {{ csrf_form.hidden_tag() }}

              <button type="submit" class="przycisk przycisk--maly przycisk--niebezpieczny"

                onclick="return confirm('Usunąć ten wpis?')">Usuń</button>

            </td>

          </tr>

        {% endfor %}

      </tbody>

    </table>

  </div>

</div>

```

```
</form>
```

```
</td>
```

```
</tr>
```

```
{% else %}
```

```
<tr><td colspan="5" class="tabela--pusta">Brak wpisów. Dodaj pierwszy wpis.</td></tr>
```

```
{% endfor %}
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
{% endblock %}
```


PLIK: app_student\templates\dziennik\nowy_wpis.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}{% if edycja %}Edytuj wpis{% else %}Nowy wpis{% endif %}{% endblock %}

{% block naglowek %}{% if edycja %}Edytuj wpis z {{ wpis.entry_date.strftime('%d.%m.%Y') }}{% else %}Nowy wpis do
dziennika{% endif %}{% endblock %}


{% block naglowek_akcje %}

<a href="{{ url_for('dziennik.index') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 640px; margin: 0 auto;">


<!-- Info o postępie -->

<div class="karta" style="margin-bottom: 1.5rem; border-left: 4px solid var(--kolor-glowny);">

    <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

        <span style="font-weight: 600;">{{ zapis.praktyka.rok_uczelniany }} – {{ zapis.praktyka.semestr }}</span>

        <span style="font-size: 0.9rem; color: var(--kolor-tekst-drugor);">

            {{ zapis.total_hours_logged }} / {{ zapis.praktyka.wymiar_godzin }} h

        </span>

    </div>

</div>

</div>


<div class="karta">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">Dane wpisu</h2>

    </div>

    <div class="karta__tresc">

        <form method="POST" action="{% if edycja %}{% url_for('dziennik.edytuj_wpis', wpis_id=wpis.id) %}{% else %}{%
url_for('dziennik.nowy_wpis') %}{% endif %}" novalidate>

            {{ form.hidden_tag() }}


            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; margin-bottom: 1rem;">


                <div class="pole-formularza {% if form.data_wpisu.errors %}pole-formularza--blad{% endif %}">
```

```

    {{ form.data_wpisu.label(class="pole-formularza__etykieta") }}

    {{ form.data_wpisu(class="pole-formularza__input", type="date") }}

    {% for e in form.data_wpisu.errors %}

        <span class="pole-formularza__blad">{{ e }}</span>

    {% endfor %}

</div>

```

```

<div class="pole-formularza {% if form.liczba_godzin.errors %}pole-formularza--blad{% endif %}">

    {{ form.liczba_godzin.label(class="pole-formularza__etykieta") }}

    {{ form.liczba_godzin(class="pole-formularza__input", type="number", min="1", max="8", placeholder="1-8") }}

    {% for e in form.liczba_godzin.errors %}

        <span class="pole-formularza__blad">{{ e }}</span>

    {% endfor %}

</div>

```

```

</div>

```

```

    <div class="pole-formularza {% if form.efekty_ids.errors %}pole-formularza--blad{% endif %}"
style="margin-bottom: 1rem;">

```

```

    {{ form.efekty_ids.label(class="pole-formularza__etykieta") }}

```

```

    <div style="border: 1px solid var(--kolor-ramka, #d1d5db); border-radius: 6px; padding: 0.75rem; max-height:
220px; overflow-y: auto; display: flex; flex-direction: column; gap: 0.5rem;">

```

```

    {% for subfield in form.efekty_ids %}

```

```

        <label style="display: flex; align-items: flex-start; gap: 0.5rem; cursor: pointer; font-size: 0.875rem;
line-height: 1.4;">

```

```

            {{ subfield(style="margin-top: 2px; flex-shrink: 0;") }}

```

```

            <span>{{ subfield.label.text }}</span>

```

```

        </label>

```

```

    {% endfor %}

```

```

</div>

```

```

{% for e in form.efekty_ids.errors %}

```

```

    <span class="pole-formularza__blad">{{ e }}</span>

```

```

{% endfor %}

```

```

</div>

```

```

<div class="pole-formularza {% if form.opis.errors %}pole-formularza--blad{% endif %}" style="margin-bottom:

```

```

1.5rem;">

    {{ form.opis.label(class="pole-formularza__etykieta") }}

    {{ form.opis(class="pole-formularza__input", rows="5", placeholder="Opisz szczegółowo wykonane czynności...")
}}

<span style="font-size: 0.78rem; color: var(--kolor-tekst-drugor);">

    Opis powinien być szczegółowy – będzie widoczny w Dzienniku Praktyki (Załącznik 6).

</span>

{% for e in form.opis.errors %}

    <span class="pole-formularza__blad">{{ e }}</span>

{% endfor %}

</div>

<div style="display: flex; gap: 0.5rem; justify-content: center;">

    <button type="submit" class="przycisk przycisk--glowny">{% if edycja %}Zapisz zmiany{% else %}Dodaj wpis{%
endif %}</button>

    <a href="{{ url_for('dziennik.index') }}" class="przycisk przycisk--drugorzeczny">Anuluj</a>

</div>

</form>

</div>

</div>

</div>

{% endblock %}

```

PLIK: app_student\templates\kreator\krok1_sciezka.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Kreator praktyki – Krok 1{% endblock %}

{% block naglowek %}Rejestracja praktyki zawodowej{% endblock %}

{% block naglowek_akcje %}

    <a href="{{ url_for('praktyki.lista') }}" class="przycisk przycisk--drugorzeczny">← Wróć</a>

{% endblock %}


{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">


    <!-- Pasek -->

    <div style="display:flex; gap:0.5rem; margin-bottom:2rem; font-size:0.82rem;">

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#3b82f6; color:white;
border-radius:0.375rem; font-weight:600;">1. Ścieżka</div>

        <div style="color:#d1d5db; align-self:center;"></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af;
border-radius:0.375rem;">2. Miejsce / Wniosek</div>

        <div style="color:#d1d5db; align-self:center;"></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af;
border-radius:0.375rem;">3. Harmonogram</div>

        <div style="color:#d1d5db; align-self:center;"></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af;
border-radius:0.375rem;">4. Realizacja</div>

    </div>


<div class="karta">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">Krok 1 – Wybierz ścieżkę praktyki</h2>

    </div>

    <div class="karta__tresc">

        <form method="POST" id="form-sciezka">

            {{ form.hidden_tag() }}

            <input type="hidden" name="track_type" id="selected_track" value="{{ form.track_type.data or '' }}">


            <div style="display:grid; grid-template-columns:1fr 1fr 1fr; gap:1rem; margin-bottom:1.5rem;">
```

<!-- Kafelek A -->

<div class="kafelek-sciezki" data-value="STANDARD" onclick="selectSciezka(this)"

style="border:2px solid {% if form.track_type.data == 'STANDARD' %}#3b82f6{% else %}#e2e8f0{% endif %};

background:{% if form.track_type.data == 'STANDARD' %}#eff6ff{% else %}#fafafa{% endif %};

border-radius:0.75rem; padding:1.25rem; cursor:pointer; transition:all 0.15s;">

<div style="font-weight:700; font-size:0.95rem; margin-bottom:0.4rem;">[A] Standardowa praktyka</div>

<div style="font-size:0.8rem; color:var(--kolor-tekst-drugor); line-height:1.4;">

Odbędę praktykę w firmie lub instytucji na podstawie porozumienia z uczelnią.

</div>

<div style="margin-top:0.75rem; font-size:0.78rem; color:#6b7280;">

4 dokumenty · 120 dni roboczych

</div>

</div>

<!-- Kafelek B -->

<div class="kafelek-sciezki" data-value="EMPLOYMENT" onclick="selectSciezka(this)"

style="border:2px solid {% if form.track_type.data == 'EMPLOYMENT' %}#3b82f6{% else %}#e2e8f0{% endif %};

background:{% if form.track_type.data == 'EMPLOYMENT' %}#eff6ff{% else %}#fafafa{% endif %};

border-radius:0.75rem; padding:1.25rem; cursor:pointer; transition:all 0.15s;">

<div style="font-weight:700; font-size:0.95rem; margin-bottom:0.4rem;">[B] Praca etatowa / staż</div>

<div style="font-size:0.8rem; color:var(--kolor-tekst-drugor); line-height:1.4;">

Pracuję lub pracowałem/am w branży IT w ciągu ostatnich 3 lat i chcę zaliczyć to jako praktykę.

</div>

<div style="margin-top:0.75rem; font-size:0.78rem; color:#6b7280;">

Wniosek + decyzja komisji

</div>

</div>

<!-- Kafelek C -->

<div class="kafelek-sciezki" data-value="OWN_BUSINESS" onclick="selectSciezka(this)"

style="border:2px solid {% if form.track_type.data == 'OWN_BUSINESS' %}#3b82f6{% else %}#e2e8f0{% endif

%};

background:{% if form.track_type.data == 'OWN_BUSINESS' %}#eff6ff{% else %}#fafafa{% endif %};

border-radius:0.75rem; padding:1.25rem; cursor:pointer; transition:all 0.15s;">

```
<div style="font-weight:700; font-size:0.95rem; margin-bottom:0.4rem;">[C] Własna działalność</div>
```

```
<div style="font-size:0.8rem; color:var(--kolor-tekst-drugor); line-height:1.4;">
```

```
    Prowadzę firmę informatyczną (CEIDG/KRS) i chcę zaliczyć działalność jako praktykę.
```

```
</div>
```

```
<div style="margin-top:0.75rem; font-size:0.78rem; color:#6b7280;">
```

```
    Wniosek + decyzja Dyrektora IIS
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<!-- Podgląd wybranej ścieżki -->
```

```
    <div id="podglad-sciezki" style="display:none; padding:1rem; border-radius:0.5rem; background:#f0fdf4; border:1px solid #a7f3d0; margin-bottom:1.5rem; font-size:0.875rem;">
```

```
</div>
```

```
{% if form.track_type.errors %}
```

```
<div style="color:#dc2626; font-size:0.85rem; margin-bottom:1rem;">Wybierz ścieżkę praktyki.</div>
```

```
{% endif %}
```

```
<div style="display:flex; justify-content:flex-end;">
```

```
    <button type="submit" class="przycisk przycisk--glowny" id="btn-dalej" {% if not form.track_type.data %}disabled style="opacity:0.5;"{% endif %}>
```

```
        Dalej →
```

```
</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
<!-- Info praktyki -->
```

```
<div style="margin-top:1rem; font-size:0.82rem; color:var(--kolor-tekst-drugor); text-align:center;">
```

```
    Praktyka: <strong>{{ praktyka.rok_uczelniany }}</strong> · Semestr {{ praktyka.semestr }} · {{ praktyka.wymiar_godzin }} h
```

```
</div>
```

```
</div>
```

```

<style>

.kafelek-sciezki:hover { border-color:#3b82f6 !important; background:#eff6ff !important; }

.kafelek-sciezki.aktywny { border-color:#2563eb !important; background:#dbeafe !important; }

</style>


<script>

const podglady = {

    STANDARD:      'Odbędziesz 120 dni roboczych w zakładzie pracy. Potrzebujesz: Porozumienie (zał.1), Program (zał.2), Indywidualny Program (zał.2a), Kartę praktyki (zał.3). Po zakończeniu: Dziennik, Sprawozdanie, Potwierdzenie efektów.',

    EMPLOYMENT:     'Złożysz wniosek o zaliczenie zatrudnienia (zał.4b) z dokumentami od pracodawcy. Komisja podejmie decyzję. Potrzebujesz: wniosek + skan umowy/zakres obowiązków.',

    OWN_BUSINESS:   'Złożysz wniosek o zaliczenie działalności (zał.4b) z wypisem z CEIDG/KRS. Dyrektor IIS podejmie decyzję.'

};


function selectSciezka(el) {

    document.querySelectorAll('.kafelek-sciezki').forEach(k => {

        k.style.borderColor = '#e2e8f0';

        k.style.background = '#fafafa';

    });

    el.style.borderColor = '#2563eb';

    el.style.background = '#dbeafe';


    const val = el.dataset.value;

    document.getElementById('selected_track').value = val;

    const box = document.getElementById('podglad-sciezki');

    box.textContent = podglady[val];

    box.style.display = 'block';


    const btn = document.getElementById('btn-dalej');

    btn.disabled = false;

    btn.style.opacity = '1';

}

// Inicjalizacja jeśli jest wybrana ścieżka

```

```
const cur = document.getElementById('selected_track').value;

if (cur) {

    const el = document.querySelector(`[data-value="${cur}"]`);

    if (el) selectSciezka(el);

}

</script>

{% endblock %}
```


PLIK: app_student\templates\kreator\krok2a_firma.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Kreator – Krok 2: Miejsce praktyki{% endblock %}

{% block naglowek %}Rejestracja praktyki zawodowej{% endblock %}


{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">


    <!-- Pasek -->

    <div style="display:flex; gap:0.5rem; margin-bottom:2rem; font-size:0.82rem;">

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#d1fae5; color:#065f46; border-radius:0.375rem;">Ścieżka A</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#3b82f6; color:white; border-radius:0.375rem; font-weight:600;">2. Miejsce praktyki</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem;">3. Harmonogram</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem;">4. Realizacja</div>

    </div>


<form method="POST" novalidate>

    {{ form.hidden_tag() }}


    <!-- === SEKCJA 1: Terminy i UOPZ === -->

    <div class="karta" style="margin-bottom:1rem;">

        <div class="karta__naglowek"><h2 class="karta__tytul">Terminy i opiekun uczelniany</h2></div>

        <div class="karta__tresc">

            <div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem; margin-bottom:1rem;">

                <div class="pole-formularza {% if form.termin_od.errors %}pole-formularza--blad{% endif %}">

                    {{ form.termin_od.label(class="pole-formularza__etykieta") }}

                    {{ form.termin_od(class="pole-formularza__input", type="date") }}

                    {% for e in form.termin_od.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

                </div>

            </div>

        </div>

    </div>

</form>
```

```

<div class="pole-formularza {% if form.termin_do.errors %}pole-formularza--blad{% endif %}">

    {{ form.termin_do.label(class="pole-formularza__etykieta") }}

    {{ form.termin_do(class="pole-formularza__input", type="date") }}

    {% for e in form.termin_do.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

</div>

</div>

<!-- Ubezpieczenie -->

        <div style="background:#fef3cd; border:1px solid #fde68a; border-radius:0.5rem; padding:0.875rem;
margin-top:1rem;">

            <label style="display:flex; gap:0.75rem; align-items:flex-start; cursor:pointer;">

                {{ form.ubezpieczenie_nw(style="margin-top:0.15rem; width:16px; height:16px; flex-shrink:0;") }}

                <span style="font-weight:500; font-size:0.9rem;">{{ form.ubezpieczenie_nw.label.text }}</span>

            </label>

            <div style="font-size:0.78rem; color:#92400e; margin-top:0.3rem; margin-left:22px;">Wymagane przed rozpoczęciem
praktyki (art. 304 KP)</div>

        </div>

</div>

</div>

</div>

<!-- === SEKCJA 2: Tryb znalezienia miejsca === -->

<div class="karta" style="margin-bottom:1rem;">

    <div class="karta__naglowek"><h2 class="karta__tytul">Zakład pracy</h2></div>

    <div class="karta__tresc">

        <div class="pole-formularza" style="margin-bottom:1rem;">

            {{ form.firma_typ.label(class="pole-formularza__etykieta") }}

            {{ form.firma_typ(class="filtr-select", style="width:100%; ", id="firma_typ") }}

        </div>

        <!-- Tryb: firma z bazy -->

        <div id="blok-baza">

            <div class="pole-formularza">

                {{ form.firma_id.label(class="pole-formularza__etykieta") }}

                {{ form.firma_id(class="filtr-select", style="width:100%; ", id="firma_id_select") }}

            </div>

            <div id="firma-podglad" style="display:none; margin-top:0.5rem; padding:0.75rem; background:#f0fdf4; border:1px
solid #a7f3d0; border-radius:0.5rem; font-size:0.85rem;"></div>

```

```
</div>
```

```
<!-- Tryb: własna firma -->
```

```
<div id="blok-custom" style="display:none;">
```

```
    <div style="background:#fef3cd; border:1px solid #fde68a; border-radius:0.5rem; padding:0.75rem; margin-bottom:1rem; font-size:0.82rem; color:#92400e;">
```

```
        Własna firma wymaga wygenerowania <strong>Zał. 9</strong> (oświadczenie instytucji) i <strong>Zał. 1</strong> (porozumienie). Możesz je wygenerować po wypełnieniu poniższych danych.
```

```
</div>
```

```
<div class="pole-formularza">
```

```
    {{ form.firma_nazwa.label(class="pole-formularza__etykieta") }}
```

```
    {{ form.firma_nazwa(class="pole-formularza__input") }}
```

```
</div>
```

```
<div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem;">
```

```
    <div class="pole-formularza">
```

```
        {{ form.firma_adres.label(class="pole-formularza__etykieta") }}
```

```
        {{ form.firma_adres(class="pole-formularza__input", placeholder="ul. Przykładowa 1") }}
```

```
    </div>
```

```
<div class="pole-formularza {% if form.firma_miasto.errors %}pole-formularza--blad{% endif %}">
```

```
    {{ form.firma_miasto.label(class="pole-formularza__etykieta") }}
```

```
    {{ form.firma_miasto(class="pole-formularza__input", placeholder="Elbląg") }}
```

```
    {% for e in form.firma_miasto.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}
```

```
</div>
```

```
</div>
```

```
<div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem;">
```

```
    <div class="pole-formularza">
```

```
        {{ form.firma_nip_krs.label(class="pole-formularza__etykieta") }}
```

```
        {{ form.firma_nip_krs(class="pole-formularza__input") }}
```

```
    </div>
```

```
<div class="pole-formularza {% if form.firma_upowazniony_osoba.errors %}pole-formularza--blad{% endif %}">
```

```
    {{ form.firma_upowazniony_osoba.label(class="pole-formularza__etykieta") }}
```

```
    {{ form.firma_upowazniony_osoba(class="pole-formularza__input", placeholder="Jan Kowalski") }}
```

```
    {% for e in form.firma_upowazniony_osoba.errors %}<span class="pole-formularza__blad">{{ e }}</span>{%
```

```
endfor %}
```

```
</div>
```

```
</div>
```

```

<div class="pole-formularza">

    {{ form.firma_upowazniony_stanowisko.label(class="pole-formularza__etykieta") }}

    {{ form.firma_upowazniony_stanowisko(class="pole-formularza__input") }}

</div>

</div>

</div>

</div>

<!-- === SEKCJA 3: ZOPZ === -->

<div class="karta" style="margin-bottom:1.5rem;">

    <div class="karta__naglowek"><h2 class="karta__tytul">Opiekun zakładowy (ZOPZ)</h2></div>

    <div class="karta__tresc">

        <div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem;">

            <div class="pole-formularza {% if form.zopz_imie_nazwisko.errors %}pole-formularza--blad{% endif %}">

                {{ form.zopz_imie_nazwisko.label(class="pole-formularza__etykieta") }}

                {{ form.zopz_imie_nazwisko(class="pole-formularza__input", placeholder="Jan Kowalski") }}

                {% for e in form.zopz_imie_nazwisko.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

            </div>

            <div class="pole-formularza">

                {{ form.zopz_stanowisko.label(class="pole-formularza__etykieta") }}

                {{ form.zopz_stanowisko(class="pole-formularza__input") }}

            </div>

        </div>

        <div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem;">

            <div class="pole-formularza">

                {{ form.zopz_telefon.label(class="pole-formularza__etykieta") }}

                {{ form.zopz_telefon(class="pole-formularza__input") }}

            </div>

            <div class="pole-formularza">

                {{ form.zopz_email.label(class="pole-formularza__etykieta") }}

                {{ form.zopz_email(class="pole-formularza__input", type="email") }}

            </div>

        </div>

    </div>

</div>

</div>

```

```
</div>
```

```
<div style="display:flex; justify-content:space-between;">
```

```
    <a href="{{ url_for('praktyki.kreator_sciezka', id=zapis.internship_id) }}" class="przycisk przycisk--drugorzeczny">← Wróć</a>
```

```
    <button type="submit" class="przycisk przycisk--glowny">Dalej – Harmonogram →</button>
```

```
</div>
```

```
</form>
```

```
<!-- Dane firm do podpowiedzi -->
```

```
<script>
```

```
const firmy = {
```

```
    {% for f in firmy_list %}
```

```
        "{{ f.id }}": { nazwa: "{{ f.nazwa }}", adres: "{{ f.adres or '' }}", miasto: "{{ f.miasto or '' }}", nip: "{{ f.nip_krs or '' }}" }{% if not loop.last %},{% endif %}
```

```
    {% endfor %}
```

```
};
```

```
const typSel = document.getElementById('firma_typ');
```

```
const blokBaza = document.getElementById('blok-baza');
```

```
const blokCustom = document.getElementById('blok-custom');
```

```
const firmaSel = document.getElementById('firma_id_select');
```

```
const podglad = document.getElementById('firma-podglad');
```

```
function toggleFirma() {
```

```
    if (typSel.value === 'database') {
```

```
        blokBaza.style.display = 'block'; blokCustom.style.display = 'none';
```

```
    } else {
```

```
        blokBaza.style.display = 'none'; blokCustom.style.display = 'block';
```

```
    }
```

```
}
```

```
firmaSel.addEventListener('change', function() {
```

```
    const f = firmy[this.value];
```

```
    if (f) {
```

```
        podglad.innerHTML = `${f.nazwa}</strong><br>${f.adres}, ${f.miasto}${f.nip ? ' · NIP: ' + f.nip : ''}`;

        podglad.style.display = 'block';

    } else {

        podglad.style.display = 'none';

    }

});

typSel.addEventListener('change', toggleFirma);

toggleFirma();

// Pokaż podgląd jeśli firma już wybrana
if (firmaSel.value) firmaSel.dispatchEvent(new Event('change'));

</script>

</div>

{% endblock %}
```

PLIK: app_student\templates\kreator\krok2bc_wniosek.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Kreator praktyki – Wniosek{% endblock %}

{% block naglowek %}Rejestracja praktyki{% endblock %}


{% block tresc %}

<div style="max-width: 700px; margin: 0 auto;">


    <!-- Pasek postępu -->

    <div style="display: flex; gap: 0.5rem; margin-bottom: 2rem; align-items: center;">

        <div style="flex:1; text-align:center; padding:0.5rem; background:#d1fae5; color:#065f46; border-radius:0.375rem; font-size:0.85rem;">Ścieżka</div>

        <div style="color:#d1d5db;"><</div>

        <div style="flex:1; text-align:center; padding:0.5rem; background:#3b82f6; color:white; border-radius:0.375rem; font-weight:600; font-size:0.85rem;">2. Wniosek</div>

        <div style="color:#d1d5db;"><</div>

        <div style="flex:1; text-align:center; padding:0.5rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem; font-size:0.85rem;">3. Decyzja komisji</div>

        <div style="color:#d1d5db;"><</div>

        <div style="flex:1; text-align:center; padding:0.5rem; background:#f3f4f6; color:#9ca3af; border-radius:0.375rem; font-size:0.85rem;">4. Sprawozdanie</div>

    </div>


<div class="karta">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">

            {% if zapis.track_type.value == 'EMPLOYMENT' %}

                Krok 2: Wniosek – Praca etatowa / staż

            {% else %}

                Krok 2: Wniosek – Własna działalność gospodarcza

            {% endif %}

        </h2>

    </div>

    <div class="karta__tresc">


        <!-- Info o wymaganych załącznikach -->

    </div>

</div>
```



```

<select id="upload-type" class="pole-formularza__input" style="margin-bottom:0.5rem;">

    {% if zapis.track_type.value == 'EMPLOYMENT' %}

    <option value="umowa_pracy">Umowa o pracę</option>

    <option value="zakres_obowiazkow">Zakres obowiązków</option>

    <option value="zaswiadczenie_zatrudnienie">Zaświadczenie o zatrudnieniu</option>

    <option value="inne">Inne</option>

    {% else %}

    <option value="ceidg">Wpis z CEIDG</option>

    <option value="krs">Odpis z KRS</option>

    <option value="inne">Inne</option>

    {% endif %}

</select>

<button type="button" onclick="uploadZalacznik()" class="przycisk przycisk--drugorzeczny przycisk--maly">

    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" width="14" height="14"
style="vertical-align:middle;margin-right:0.3rem"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline
points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>

    Dodaj plik

</button>

<div id="upload-status" style="font-size:0.8rem; margin-top:0.5rem;"></div>

<div id="upload-lista" style="margin-top:0.75rem;"></div>

</div>

<form method="POST" novalidate>

    {{ form.hidden_tag() }}

    <div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem; margin-bottom:1rem;">

        <div class="pole-formularza {% if form.pracodawca_nazwa.errors %}pole-formularza--blad{% endif %}">

            {{ form.pracodawca_nazwa.label(class="pole-formularza__etykieta") }}

            {{ form.pracodawca_nazwa(class="pole-formularza__input") }}

            {% for e in form.pracodawca_nazwa.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

        </div>

        <div class="pole-formularza {% if form.stanowisko.errors %}pole-formularza--blad{% endif %}">

            {{ form.stanowisko.label(class="pole-formularza__etykieta") }}

            {{ form.stanowisko(class="pole-formularza__input") }}

            {% for e in form.stanowisko.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

```

```
</div>
```

```
</div>
```

```
<div style="display:grid; grid-template-columns:1fr 1fr; gap:1rem; margin-bottom:1rem;">
```

```
<div class="pole-formularza">
```

```
  {{ form.pracodawca_adres.label(class="pole-formularza__etykieta") }}
```

```
  {{ form.pracodawca_adres(class="pole-formularza__input") }}
```

```
</div>
```

```
<div class="pole-formularza">
```

```
  {{ form.pracodawca_miasto.label(class="pole-formularza__etykieta") }}
```

```
  {{ form.pracodawca_miasto(class="pole-formularza__input") }}
```

```
</div>
```

```
</div>
```

```
      <div class="pole-formularza {% if form.uzasadnienie.errors %}pole-formularza--blad{% endif %}"
style="margin-bottom:1.5rem;">
```

```
    {{ form.uzasadnienie.label(class="pole-formularza__etykieta") }}
```

```
    {{ form.uzasadnienie(class="pole-formularza__input", rows=5, style="height:auto;", placeholder="Opisz zakres
obowiązków/działalności i dlaczego spełniają wymagania praktyki zawodowej...") }}
```

```
    {% for e in form.uzasadnienie.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}
```

```
</div>
```

```
<div style="display:flex; justify-content:space-between; gap:1rem;">
```

```
      <a href="{{ url_for('praktyki.kreator_sciezka', id=zapis.internship_id) }}" class="przycisk
przycisk--drugorzeczny">← Wróć</a>
```

```
      <button type="submit" class="przycisk przycisk--glowny">Złóż wniosek →</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
const uploadUrl = "{{ url_for('uploads.upload_document', enrollment_id=zapis.id) }}";
```

```
const listUrl = "{{ url_for('uploads.list_documents', enrollment_id=zapis.id) }}";
```

```
let uploadedFiles = [];
```

```
async function loadExistingFiles() {  
  try {  
    const r = await fetch(listUrl);  
    if (!r.ok) return;  
    const docs = await r.json();  
    uploadedFiles = docs.map(d => ({  
      id: d.id,  
      name: d.original_filename,  
      type: d.document_type,  
    }));  
    renderLista();  
  } catch(e) { /* cicho ignoruj */ }  
}  
loadExistingFiles();
```

```
async function uploadZalacznik() {  
  const fileInput = document.getElementById('upload-file');  
  const typeSelect = document.getElementById('upload-type');  
  const status = document.getElementById('upload-status');  
  
  if (!fileInput.files.length) {  
    status.textContent = 'Wybierz plik.';  
    status.style.color = '#dc2626';  
    return;  
  }  
  
  const fd = new FormData();  
  fd.append('file', fileInput.files[0]);  
  fd.append('document_type', typeSelect.value);  
  fd.append('csrf_token', document.querySelector('[name=csrf_token]').value);  
  
  status.textContent = 'Wysyłanie...';  
  status.style.color = '#6b7280';
```

```

try {

    const r = await fetch(uploadUrl, { method: 'POST', body: fd });

    const data = await r.json();

    if (r.ok) {

        status.textContent = 'Plik dodany pomyślnie.';

        status.style.color = '#059669';

        uploadedFiles.push({ id: data.document_id, name: fileInput.files[0].name, type:
typeSelect.options[typeSelect.selectedIndex].text });

        renderLista();

        fileInput.value = '';

        document.getElementById('upload-file-label').textContent = 'Wybierz plik PDF lub przeciągnij tutaj';

    } else {

        status.textContent = 'Błąd: ' + (data.error || 'Nie udało się przesłać.');
```

```

        status.style.color = '#dc2626';
    }
} catch(e) {
    status.textContent = 'Błąd połączenia.';
    status.style.color = '#dc2626';
}
}

```

```

async function usunPlik(id) {

    const csrfToken = document.querySelector('[name=csrf_token]').value;

    const fd = new FormData();

    fd.append('csrf_token', csrfToken);

    try {

        const r = await fetch(`/uploads/document/${id}/delete`, {

            method: 'POST',

            headers: { 'X-CSRFToken': csrfToken },

            body: fd

        });

        const text = await r.text();

        if (r.ok) {

            uploadedFiles = uploadedFiles.filter(f => f.id !== id);

```

```

        renderLista();

    } else {

        alert('Błąd usuwania (status ' + r.status + '): ' + text.substring(0, 200));

    }

} catch(e) {

    alert('Błąd połączenia: ' + e);

}

}

function renderLista() {

    const el = document.getElementById('upload-lista');

    el.innerHTML = uploadedFiles.map(f =>

        `
```

PLIK: app_student\templates\kreator\krok3_harmonogram.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Kreator – Krok 3: Harmonogram{% endblock %}

{% block naglowek %}Rejestracja praktyki zawodowej{% endblock %}


{% block tresc %}

<div style="max-width: 900px; margin: 0 auto;">


    <!-- Pasek kroków -->

    <div style="display:flex; gap:0.5rem; margin-bottom:2rem; font-size:0.82rem;">

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#d1fae5; color:#065f46;
border-radius:0.375rem;">Ścieżka A</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#d1fae5; color:#065f46;
border-radius:0.375rem;">2. Miejsce praktyki</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#3b82f6; color:white;
border-radius:0.375rem; font-weight:600;">3. Harmonogram</div>

        <div style="color:#d1d5db; align-self:center;">></div>

        <div style="flex:1; text-align:center; padding:0.5rem 0.25rem; background:#f3f4f6; color:#9ca3af;
border-radius:0.375rem;">4. Realizacja</div>

    </div>


<form method="POST" novalidate>

    {{ csrf_form.hidden_tag() }}


    <div class="karta" style="margin-bottom:1.5rem;">

        <div class="karta__naglowek">

            <h2 class="karta__tytul">Harmonogram praktyki – efekty uczenia się</h2>

        </div>

        <div style="padding:0.6rem 1rem; background:#fef3cd; border-bottom:1px solid #fde68a; font-size:0.8rem;
color:#92400e;">

            Dla każdego efektu podaj dział zakładu pracy, przykładowe czynności i planowaną liczbę dni. Wiersze bez działu
            lub czynności zostaną pominięte.

        </div>

        {% for e in efekty %}
```

```

{% set h = istnieje_harmonogramy.get(e.id|string, {}) %}

<div style="border-bottom:1px solid var(--kolor-ramka); padding:0.6rem 1rem;">

    <div style="font-size:0.78rem; color:var(--kolor-tekst-drugor); margin-bottom:0.35rem;">

        {{ e.opis }}

    </div>

    <div style="display:grid; grid-template-columns:1fr 1fr auto; gap:0.5rem; align-items:center;">

        <input type="text" name="dzial_{{ e.id }}" value="{{ h.get('dzial', '') }}"

            class="pole-formularza__input" style="font-size:0.8rem; padding:0.3rem 0.5rem;" placeholder="Dział
zakładu pracy">

        <input type="text" name="prace_{{ e.id }}" value="{{ h.get('prace', '') }}"

            class="pole-formularza__input" style="font-size:0.8rem; padding:0.3rem 0.5rem;" placeholder="Przykładowe
czynności">

        <div style="display:flex; align-items:center; gap:0.4rem;">

            <input type="number" name="dni_{{ e.id }}" value="{{ h.get('dni', '') }}"

                class="pole-formularza__input" style="width:4rem; font-size:0.8rem; padding:0.3rem 0.5rem;
text-align:center;" min="0" max="60" placeholder="0">

            <span style="font-size:0.8rem; color:var(--kolor-tekst-drugor); white-space:nowrap;">dni</span>

        </div>

    </div>

</div>

{% endfor %}

</div>

<div style="display:flex; justify-content:space-between;">

    <a href="{{ url_for('praktyki.kreator_firma', zapis_id=zapis.id) }}" class="przycisk przycisk--drugorzeczny">←
Wróć</a>

    <button type="submit" class="przycisk przycisk--glowny">Zapisz harmonogram →</button>

</div>

</form>

</div>

{% endblock %}

```

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Potwierdzenie wysłania zgłoszenia{% endblock %}

{% block naglowek %}Rejestracja praktyki zawodowej{% endblock %}

{% block tresc %}

<div style="max-width: 600px; margin: 3rem auto; text-align: center;">

    <div class="karta" style="padding: 2.5rem 2rem;">

        <div style="margin-bottom: 1rem;">

            <svg xmlns="http://www.w3.org/2000/svg" width="52" height="52" viewBox="0 0 24 24" fill="none"
                stroke="var(--kolor-glowny)" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round">

                <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>

                <polyline points="14 2 14 8 20 8"/>

                <polyline points="9 15 11 17 15 13"/>

            </svg>

        </div>

        <h2 style="font-size: 1.25rem; font-weight: 700; margin-bottom: 0.5rem;">Harmonogram zapisany</h2>

        <p style="color: var(--kolor-tekst-drugor); margin-bottom: 2rem; font-size: 0.9rem;">

            Czy chcesz teraz przesłać zgłoszenie do zatwierdzenia przez opiekuna?<br>

            Po wysłaniu nie będzie można edytować danych bez kontaktu z opiekunem.

        </p>

        <div style="display: flex; flex-direction: column; gap: 0.75rem; align-items: center;">

            <form method="POST" action="{% url_for('praktyki.wyslij_do_zatwierdzenia', id=zapis.id) %}">

                {% csrf_form.hidden_tag() %}

                <button type="submit" class="przycisk przycisk--glowny" style="min-width: 220px; justify-content: center;">

                    Tak, prześlij zgłoszenie

                </button>

            </form>

        </div>

    </div>

{% if zapis.track_type.value == 'STANDARD' %}
```



```
<a href="{ { url_for('praktyki.zapisz_krok2', id=zapis.id) }}"
```

```
class="przycisk przycisk--drugorzecny" style="min-width: 220px; justify-content: center;">
```

```
Wróć i popraw
```

```
</a>
```

```
{% else %}
```

```
<a href="{ { url_for('praktyki.kreator_wniosek', zapis_id=zapis.id) }}"
```

```
class="przycisk przycisk--drugorzecny" style="min-width: 220px; justify-content: center;">
```

```
Wróć i popraw
```

```
</a>
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_student\templates\layouts\base_student.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-scale=1.0">

    {% set _csrf_token = csrf_token() %}

    <meta name="csrf-token" content="{{ _csrf_token }}">

    <title>{% block tytuł %}Panel studenta{% endblock %} – ANS Elbląg</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='css/student.css') }}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono&display=swap"
rel="stylesheet">

</head>

<body class="panel-body">


    <a href="#main-content" class="skip-link">Przejdź do głównej treści</a>


    <nav class="sidebar" aria-label="Nawigacja główna panelu">

        <div class="sidebar__naglowek">

            <div class="branding__logo-kontener" style="max-width: 140px; margin-bottom: 10px;">

                

            </div>

            <span class="sidebar__system-label">Panel studenta</span>

        </div>


        <div class="sidebar__uzytkownik" aria-label="Zalogowany użytkownik">

            <div class="sidebar__uzytkownik-avatar" aria-hidden="true">

                {% if current_user.is_authenticated and current_user.first_name and current_user.last_name %}

                    {{ current_user.first_name[0] }}{{ current_user.last_name[0] }}

                {% else %}

                    –

                {% endif %}

            </div>

            <div class="sidebar__uzytkownik-info">
```

```

{% if current_user.is_authenticated %}

    <span class="sidebar__uzytkownik-imie">{{ current_user.first_name }} {{ current_user.last_name }}</span>

    <span class="sidebar__uzytkownik-rola">Nr alb. {{ current_user.album_number or '-' }}</span>

{% else %}

    <span class="sidebar__uzytkownik-imie">Gość</span>

    <span class="sidebar__uzytkownik-rola">—</span>

{% endif %}

</div>

</div>

<ul class="sidebar__menu" role="list">

    <li class="sidebar__menu-pozycja">

        <a href="{{ url_for('dashboard.index') }}" class="sidebar__link {% if request.endpoint == 'dashboard.index' %}sidebar__link--aktywny{% endif %}" {% if request.endpoint == 'dashboard.index' %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="m3 9 9-7 9 7v11a2 2 0 0 1-2 2H5a2 2 0 0 1-2 2z"/><polyline points="9,22 9,12 15,12 15,22"/></svg></span> Pulpit

        </a>

    </li>

    <li class="sidebar__menu-sekcja" aria-hidden="true">Praktyki</li>

    <li class="sidebar__menu-pozycja">

        <a href="{{ url_for('praktyki.lista') }}" class="sidebar__link {% if request.blueprint == 'praktyki' %}sidebar__link--aktywny{% endif %}" {% if request.blueprint == 'praktyki' %}aria-current="page"{% endif %}>

            <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><rect x="2" y="3" width="20" height="14" rx="2" ry="2"/><line x1="8" y1="21" x2="16" y2="21"/><line x1="12" y1="17" x2="12" y2="21"/></svg></span> Dostępne praktyki{% if nav_wymaga_uwagi %} <span style="display:inline-flex;align-items:center;justify-content:center;background:#ef4444;color:white;border-radius:9999px;width:1.1rem;height:1.1rem;font-size:0.65rem;font-weight:700;margin-left:0.35rem;">!</span>{% endif %}

        </a>

    </li>

    {% if aktywny_zapis_info.ma_aktywny and aktywny_zapis_info.sciezka == 'STANDARD' and aktywny_zapis_info.status in ('IN_PROGRESS', 'COMPLETED') %}

        <li class="sidebar__menu-pozycja">

            <a href="{{ url_for('dziennik.index') }}" class="sidebar__link {% if request.blueprint == 'dziennik' %}sidebar__link--aktywny{% endif %}" {% if request.blueprint == 'dziennik' %}aria-current="page"{% endif %}>

                <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 21l3.09 6.26L22 9.27l-5 4.87 1.18 6.88L12 17.77l-6.18 3.25L7 14.14 2 9.27l6.91-1.01L12 2z"/></svg></span> Dziennik praktyki

            </a>

        </li>

```

```

{% endif %}

{% if aktywny_zapis_info.ma_aktywny and aktywny_zapis_info.status in ('IN_PROGRESS', 'COMPLETED') %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('sprawozdania.index') }}" class="sidebar__link {% if request.blueprint == 'sprawozdania' %}sidebar__link--aktywny{% endif %}" {% if request.blueprint == 'sprawozdania' %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="18" y1="20" x2="18" y2="10"/><line x1="12" y1="20" x2="12" y2="4"/><line x1="6" y1="20" x2="6" y2="14"/></svg></span> Sprawozdanie

    </a>

</li>

{% endif %}

<li class="sidebar__menu-pozycja">

    <a href="{{ url_for('documents.moje_dokumenty') }}" class="sidebar__link {% if request.endpoint == 'documents.moje_dokumenty' %}sidebar__link--aktywny{% endif %}" {% if request.endpoint == 'documents.moje_dokumenty' %}aria-current="page"{% endif %}>

        <span class="sidebar__ikona" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16" y1="17" x2="8" y2="17"/><polyline points="10 9 9 9 8 9"/></svg></span> Moje Dokumenty

    </a>

</li>

</ul>

<div class="sidebar__stopka">

    <form method="POST" action="{{ url_for('auth.wylogowanie') }}">

        <input type="hidden" name="csrf_token" value="{{ _csrf_token }}">

        <button type="submit" class="sidebar__wylogowanie">

            <svg viewBox="0 0 24 24"><path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4"/><polyline points="16 17 21 12 16 7"/><line x1="21" y1="12" x2="9" y2="12"/></svg> Wyloguj się

        </button>

    </form>

</div>

</nav>

<div class="panel-main">

    <header class="panel-main__naglowek">

        <div class="panel-main__naglowek-tresc">

            <h1 class="panel-main__tytul">{% block naglowek %}{% endblock %}</h1>

            {% block naglowek_akcje %}{% endblock %}


```

```

    </div>

</header>

<main id="main-content" class="panel-main__tresc" tabindex="-1">

    {% block tresc %}{% endblock %}

</main>

</div>

{% with wiadomosci = get_flashed_messages(with_categories=True) %}

    {% if wiadomosci %}

        <div class="komunikaty komunikaty--fixed" role="status" aria-live="polite">

            {% for kategoria, wiadomosc in wiadomosci %}

                <div class="komunikat komunikat--{{ kategoria }} komunikat--minimal" role="alert">

                    <span class="komunikat__ikona" aria-hidden="true">

                        {% if kategoria == 'danger' %}{% elif kategoria == 'success' %}{% else %}{% endif %}

                    </span>

                    <span class="komunikat__tekst">{{ wiadomosc }}</span>

                        <button type="button" class="komunikat__zamknij" aria-label="Zamknij powiadomienie"
onclick="this.parentElement.remove()"></button>

                    </div>

                {% endfor %}

            </div>

        {% endif %}

    {% endwith %}

    <script src="{{ url_for('static', filename='js/pdf_generator.js') }}"></script>

</body>

</html>

```

PLIK: app_student\templates\praktyki\lista.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Dostępne praktyki{% endblock %}

{% block naglowek %}Dostępne praktyki{% endblock %}


{% block tresc %}

<div style="max-width: 700px; margin: 0 auto;">


    {% if not dostepne %}

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

        <p style="color: var(--kolor-tekst-drugor);">Brak aktywnych praktyk. Skontaktuj się z opiekunem uczelni.</p>

    </div>


    {% else %}

    <div style="display: flex; flex-direction: column; gap: 1rem;">

        {% for p in dostepne %}

        {% set zapis_info = zapisy_data.get(p.id|string) %}

        {% set jest_zwrocone = zapis_info and zapis_info.zwrocone %}

        <div class="karta" style="{% if jest_zwrocone %}border-left: 4px solid #dc2626;{% endif %}">

            <div class="karta__tresc" style="display: flex; flex-direction: column; gap: 0.75rem;">


                {# Wiersz główny: nazwa + akcje #}

                <div style="display: flex; align-items: center; justify-content: space-between; gap: 1rem;">

                    <div>

                        <div style="font-weight: 600; font-size: 1.05rem;">{{ p.rok_uczelnianny }}</div>

                        <div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-top: 0.25rem;">Semestr {{ p.semestr }}
                        · {{ p.wymiar_godzin }} h</div>

                    </div>


                    {% if zapis_info %}

                    <div style="display: flex; align-items: center; gap: 0.75rem;">

                        {% if jest_zwrocone %}

                        <span style="display: inline-flex; align-items: center; gap: 0.35rem; background: #fef2f2; color: #dc2626; border: 1px solid #fecaca; border-radius: 0.375rem; padding: 0.25rem 0.6rem; font-size: 0.8rem; font-weight: 700;">
```

Wymaga poprawek

{% if zapis_info.sciezka == 'STANDARD' %}

Popraw i wyślij ponownie

{% elif zapis_info.sciezka in ('EMPLOYMENT', 'OWN_BUSINESS') %}

Popraw i wyślij ponownie

{% else %}

Popraw i wyślij ponownie

{% endif %}

{% elif zapis_info.status == 'PENDING' %}

Nieukończone

{% if zapis_info.sciezka == 'STANDARD' %}

Kontynuuj zgłoszenie

{% elif zapis_info.sciezka in ('EMPLOYMENT', 'OWN_BUSINESS') %}

Kontynuuj zgłoszenie

{% else %}

Kontynuuj zgłoszenie

{% endif %}

{% elif zapis_info.wymaga_uwagi %}

! Wymaga uzupełnienia

Szczegóły

{% else %}

{{ tlumacz_status(zapis_info.status) }}

Szczegóły

{% if zapis_info.status == 'IN_PROGRESS' %}

```

        <form method="POST" action="{ { url_for('praktyki.zakoncz_praktyke', id=zapis_info.id) }}"
style="display:inline;">

        {{ csrf_form.hidden_tag() }}

        <button type="submit" class="przycisk przycisk--maly przycisk--glowny"

                onclick="return confirm('Czy na pewno chcesz oznaczyć praktykę jako zakończoną? Po zakończeniu
zostaną udostępnione dokumenty końcowe.')">

                Zakończ praktykę

        </button>

</form>

{% endif %}

{% endif %}

</div>

{% else %}

        <a href="{ { url_for('praktyki.kreator_sciezka', id=p.id) }}" class="przycisk przycisk--glowny"
style="text-decoration:none;">Zapisz się</a>

        {% endif %}

</div>

{# Wiadomość zwrotna od admina/UOPZ #}

{% if jest_zwrocone and zapis_info.komentarz_zwrotny %}

<div style="background:#fef2f2; border:1px solid #fecaca; border-radius:var(--promien); padding:0.75rem 1rem;">

        <div style="font-size:0.7rem; font-weight:700; text-transform:uppercase; letter-spacing:0.06em; color:#dc2626;
margin-bottom:0.3rem;">Uwaga opiekuna</div>

        <div style="font-size:0.875rem; color:#7f1d1d; line-height:1.55; white-space:pre-wrap;">{{
zapis_info.komentarz_zwrotny }}</div>

</div>

        {% endif %}

</div>

</div>

{% endfor %}

</div>

{% endif %}

</div>

{% endblock %}

```


PLIK: app_student\templates\praktyki\szczegoly_zgloszenia.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Szczegóły zgłoszenia{% endblock %}

{% block naglowek %}Szczegóły zgłoszenia praktyki{% endblock %}


{% block naglowek_akcje %}

<div style="display: flex; gap: 0.5rem;">

    <a href="{{ url_for('dashboard.index') }}" class="przycisk przycisk--drugorzeczny">← Powrót do pulpitu</a>

</div>

{% endblock %}


{% block tresc %}

<div style="max-width: 800px; margin: 0 auto;">


    <!-- Szkic: tylko przycisk powrotu do edycji -->

    {% if zapis.status.value == 'PENDING' %}

    <div class="karta" style="text-align:center; padding:3rem 2rem;">

        <div style="font-size:1.1rem; font-weight:600; margin-bottom:0.5rem;">Zgłoszenie nie zostało jeszcze wysłane</div>

        <div style="color:var(--kolor-tekst-drugor); margin-bottom:1.5rem;">Wróć do edycji, aby uzupełnić dane i przesłać zgłoszenie.</div>

        {% if zapis.track_type.value == 'STANDARD' %}

        <a href="{{ url_for('praktyki.kreator_firma', zapis_id=zapis.id) }}" class="przycisk przycisk--glowny">← Powrót do edycji</a>

        {% else %}

        <a href="{{ url_for('praktyki.kreator_wniosek', zapis_id=zapis.id) }}" class="przycisk przycisk--glowny">← Powrót do edycji</a>

        {% endif %}

    </div>


    {% else %}


    <!-- Baner akcji – wymaga uzupełnienia -->

    {% if zapis.status.value == 'AWAITING_APPROVAL' and zapis.komentarze_uopz %}

    <div style="background:#ffffbeb; border:1px solid #f59e0b; border-left:4px solid #d97706; border-radius:0.5rem; padding:1rem 1.25rem; margin-bottom:1.5rem; display:flex; align-items:center; justify-content:space-between; gap:1rem; flex-wrap:wrap;">
```

```

<div>

    <div style="font-weight:600; color:#92400e; margin-bottom:0.25rem;">Zgłoszenie wymaga uzupełnienia</div>

    <div style="font-size:0.85rem; color:#92400e;">{{ zapis.komentarze_uopz }}</div>

</div>

<div style="display:flex; gap:0.5rem; flex-shrink:0; flex-wrap:wrap;">

    {% if zapis.track_type.value == 'STANDARD' %}

        <a href="{{ url_for('praktyki.kreator_firma', zapis_id=zapis.id) }}" class="przycisk przycisk--glowny przycisk--maly">Edytuj i odeślij →</a>

    {% else %}

        <a href="{{ url_for('praktyki.kreator_wniosek', zapis_id=zapis.id) }}" class="przycisk przycisk--glowny przycisk--maly">Edytuj i odeślij →</a>

    {% endif %}

    <form method="POST" action="{{ url_for('praktyki.resubmit_zgloszenia', id=zapis.id) }}" style="margin:0;">

        <input type="hidden" name="csrf_token" id="resubmit-csrf">

        <button type="submit" class="przycisk przycisk--drugorzeczny przycisk--maly">Odeślij bez zmian</button>

    </form>

</div>

</div>

        <script>document.getElementById('resubmit-csrf').value
document.querySelector('meta[name="csrf-token"]')?.getAttribute('content')|'|';</script>

    {% endif %}

<!-- Status zgłoszenia -->

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

        <h2>Status zgłoszenia</h2>

    </div>

    <div class="karta__tresc">

        <div style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">

                    Praktyka: {{ zapis.praktyka.rok_uczelnianny }} ({{ zapis.praktyka.semestr }})

                </div>

                {% if zapis.status.value != 'PENDING' %}

                    <div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor);">

                        Wysłano: {{ zapis.enrolled_at.strftime('%d.%m.%Y %H:%M') }}

                    </div>

                {% endif %}

            </div>

            <div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor); margin-bottom: 0.5rem;">

                Wysłano: {{ zapis.enrolled_at.strftime('%d.%m.%Y %H:%M') }}

            </div>

        </div>

    </div>

</div>

```

```

</div>

{% endif %}

</div>

<span class="status

{%- if zapis.status.value == 'PENDING' %} status--draft

{%- elif zapis.status.value == 'AWAITING_APPROVAL' %} status--planned

{%- elif zapis.status.value == 'COMMISSION_REVIEW' %} status--planned

{%- elif zapis.status.value == 'DEAN_APPROVAL' %} status--planned

{%- elif zapis.status.value == 'IN_PROGRESS' %} status--w-trakcie

{%- elif zapis.status.value == 'COMPLETED' %} status--zakonczone

{%- elif zapis.status.value == 'REJECTED' %} status--draft

{%- endif %}">

{% if zapis.status.value == 'PENDING' %}Szkic

{% elif zapis.status.value == 'AWAITING_APPROVAL' %}Oczekuje na zatwierdzenie

{% elif zapis.status.value == 'COMMISSION_REVIEW' %}Weryfikacja komisji

{% elif zapis.status.value == 'DEAN_APPROVAL' %}Oczekuje na dziekana

{% elif zapis.status.value == 'IN_PROGRESS' %}Zatwierdzone - w trakcie

{% elif zapis.status.value == 'COMPLETED' %}Zakończone

{% elif zapis.status.value == 'REJECTED' %}Odrzucony

{% endif %}

</span>

</div>

</div>

</div>

```

```

<!-- Uploadowane załączniki studenta -->

{% if uploaded_docs %}

<div class="karta" style="margin-bottom: 2rem;">

  <div class="karta__naglowek">

    <h2>Twoje załączniki ({{ uploaded_docs|length }})</h2>

  </div>

  <div class="karta__tresc">

    <div style="display:flex; flex-direction:column; gap:0.5rem;">

```

```

    {% for dok in uploaded_docs %}

        <div style="display:flex; justify-content:space-between; align-items:center; padding:0.625rem 0.875rem;
background:#f8fafc; border:1px solid #e2e8f0; border-radius:0.375rem;">

            <div>

                <div style="font-weight:500; font-size:0.875rem;">{{ dok.original_filename }}</div>

                <div style="font-size:0.75rem; color:var(--kolor-tekst-drugor);">{{ dok.document_type }}{% if dok.uploaded_at
%} · {{ dok.uploaded_at.strftime('%d.%m.%Y') if dok.uploaded_at is not string else dok.uploaded_at[:10] }}{% endif
%}</div>

            </div>

            <a href="{{ url_for('uploads.download_document', document_id=dok.id) }}"

                class="przycisk przycisk--maly przycisk--drugorzeczny">Pobierz</a>

        </div>

    {% endfor %}

</div>

</div>

</div>

{% endif %}

<!-- Dane zgłoszenia -->

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

        <h2>Dane zgłoszenia</h2>

    </div>

    <div class="karta__tresc">

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; margin-bottom: 2rem;">

            <div>

                <label style="font-weight: 500; margin-bottom: 0.5rem; display: block;">Ścieżka praktyki</label>

                <div style="color: var(--kolor-tekst-drugor);">

                    {% if zapis.track_type.value == 'STANDARD' %}Standardowa

                    {% elif zapis.track_type.value == 'EMPLOYMENT' %}Praca etatowa

                    {% elif zapis.track_type.value == 'OWN_BUSINESS' %}Własna działalność gospodarcza

                    {% endif %}

                </div>

            </div>

        </div>

```

```

<div>

  <label style="font-weight: 500; margin-bottom: 0.5rem; display: block;">Okres praktyki</label>

  <div style="color: var(--kolor-tekst-drugor);">

    {% if zapis.termin_od and zapis.termin_do %}

      {{ zapis.termin_od.strftime('%d.%m.%Y') }} - {{ zapis.termin_do.strftime('%d.%m.%Y') }}

    {% else %}

      Nie określono

    {% endif %}

  </div>

</div>

</div>

{% if zapis.firma_nazwa %}

<div style="border-top: 1px solid #e5e7eb; padding-top: 1.5rem; margin-top: 1.5rem;">

  <h3 style="font-size: 1rem; font-weight: 600; margin-bottom: 1rem;">Zakład pracy</h3>

  <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">

    <div>

      <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Nazwa</label>

      <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.firma_nazwa }}</div>

    </div>

    <div>

      <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">NIP/KRS</label>

      <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.firma_nip_krs or 'Nie podano'
    }}</div>

    </div>

    <div>

      <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Adres</label>

      <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.firma_adres }}</div>

    </div>

    <div>

      <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Miasto</label>

      <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.firma_miasto }}</div>

    </div>

  </div>

</div>

</div>

```

```

{% endif %}

{% if zapis.zopz_imie_nazwisko %}

<div style="border-top: 1px solid #e5e7eb; padding-top: 1.5rem; margin-top: 1.5rem;">

  <h3 style="font-size: 1rem; font-weight: 600; margin-bottom: 1rem;">Opiekun Zakładowy (ZOPZ)</h3>

  <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">

    <div>

      <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Imię i nazwisko</label>

      <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.zopz_imie_nazwisko }}</div>

    </div>

    <div>

      <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Stanowisko</label>

      <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.zopz_stanowisko }}</div>

    </div>

    <div>

      <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">Telefon</label>

      <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.zopz_telefon }}</div>

    </div>

    <div>

      <label style="font-weight: 500; margin-bottom: 0.25rem; display: block; font-size: 0.85rem;">E-mail</label>

      <div style="color: var(--kolor-tekst-drugor); margin-bottom: 0.75rem;">{{ zapis.zopz_email }}</div>

    </div>

  </div>

</div>

{% endif %}

{% if zapis.uopz %}

<div style="border-top: 1px solid #e5e7eb; padding-top: 1.5rem; margin-top: 1.5rem;">

  <h3 style="font-size: 1rem; font-weight: 600; margin-bottom: 1rem;">Opiekun Uczelniany (UOPZ)</h3>

  <div style="color: var(--kolor-tekst-drugor);">

    {{ zapis.uopz.first_name }} {{ zapis.uopz.last_name }}

    {% if zapis.uopz.email %}

      <br><a href="mailto:{{ zapis.uopz.email }}" style="color: var(--kolor-glowny);">{{ zapis.uopz.email }}</a>

    {% endif %}

  </div>

</div>

{% endif %}

```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
{% if zapis.status.value in ['IN_PROGRESS', 'COMPLETED'] and zapis.track_type.value == 'STANDARD' %}
```

```
<div class="karta">
```

```
<div class="karta__tresc" style="display: flex; align-items: center; justify-content: space-between;">
```

```
<div>
```

```
<div style="font-weight: 600; margin-bottom: 0.25rem;">Dokumenty praktyki gotowe do pobrania</div>
```

```
<div style="font-size: 0.85rem; color: var(--kolor-tekst-drugor);">Załącz. 1 (Porozumienie), Załącz. 2 (Program), Załącz. 2a (Indywidualny Program), Załącz. 3 (Karta ze Skierowaniem) – dostępne w zakładce Moje Dokumenty.</div>
```

```
</div>
```

```
<a href="{% url_for('documents.moje_dokumenty') %}" class="przycisk przycisk--glowny">
```

```
Przejdź do Moich Dokumentów →
```

```
</a>
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
{% endif %}{# end: not PENDING #}
```

```
</div>
```

```
{% endblock %}
```

PLIK: app_student\templates\sprawozdania\index.html

```
{% extends "layouts/base_student.html" %}

{% block tytul %}Sprawozdanie z praktyki{% endblock %}

{% block naglowek %}Sprawozdanie z przebiegu praktyki{% endblock %}


{% block tresc %}

<div style="max-width: 900px; margin: 0 auto; padding-bottom: 3rem;">


    {% if not ma_zapis %}

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

        <p style="color: var(--kolor-tekst-drugor);">Najpierw zapisz się na praktykę.</p>

        <a href="{{ url_for('praktyki.lista') }}" class="przycisk przycisk--glowny" style="margin-top: 1rem;">Dostępne
praktyki</a>

    </div>

    {% elif not zapis %}

    <div class="karta" style="text-align: center; padding: 3rem 2rem;">

        <p style="color: var(--kolor-tekst-drugor);">Twój zapis oczekuje na akceptację przez UOPZ, dlatego projekt
sprawozdania jest jeszcze niedostępny.</p>

    </div>

    {% else %}


    <div class="karta" style="margin-bottom: 2rem; border-left: 4px solid var(--kolor-glowny);">

        <div class="karta__tresc" style="display: flex; justify-content: space-between; align-items: center;">

            <div>

                <div style="font-weight: 600; font-size: 1.1rem;">

                    {% if is_standard %}Załącznik 7 – Sprawozdanie z praktyki{% else %}Załącznik 7a – Sprawozdanie z pracy
zawodowej{% endif %}

                </div>

                <div style="font-size: 0.9rem; color: var(--kolor-tekst-drugor);">{{ zapis.praktyka.rok_ucelniiany }} – Semestr
{{ zapis.praktyka.semestr }}</div>

            </div>

            <a href="{{ url_for('documents.moje_dokumenty') }}" class="przycisk przycisk--drugorzeczny">Pobierz PDF (Moje
Dokumenty)</a>

        </div>

    </div>

</div>
```



```
<form method="POST" action="{ { url_for('sprawozdania.index') } }" novalidate>
```

```
{ { form.hidden_tag() } }
```

```
<div class="karta" style="margin-bottom: 2rem;">
```

```
<div class="karta__naglowek">
```

```
<h2 class="karta__tytul">{ { form.charakterystyka.label.text } }</h2>
```

```
</div>
```

```
<div class="karta__tresc">
```

```
<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1rem;">
```

```
{ % if is_standard % }
```

Opisz zwięźle zakład pracy (branża, wielkość, używane technologie) oraz konkretny dział/komórkę organizacyjną, w której realizowana była praktyka.

```
{ % else % }
```

Opisz zwięźle miejsce pracy / działalność (branża, wielkość firmy, technologie IT, zakres działalności).

```
{ % endif % }
```

```
</p>
```

```
<div class="pole-formularza { % if form.charakterystyka.errors % }pole-formularza--blad{ % endif % }">
```

```
{ { form.charakterystyka(class="pole-formularza__input", rows="8", placeholder="Tutaj wpisz sekcję I...") } }
```

```
{ % for e in form.charakterystyka.errors % }<span class="pole-formularza__blad">{ { e } }</span>{ % endfor % }
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="karta" style="margin-bottom: 2rem;">
```

```
<div class="karta__naglowek">
```

```
<h2 class="karta__tytul">{ { form.opis.label.text } }</h2>
```

```
</div>
```

```
<div class="karta__tresc">
```

```
<p style="font-size: 0.85rem; color: var(--kolor-tekst-drugor); margin-bottom: 1rem;">
```

```
{ % if is_standard % }
```

Przedstaw szczegółowy opis i analizę wykonanych prac w ramach zrealizowanego harmonogramu.

```
{ % else % }
```

Opisz syntetycznie jakie zadania wykonywałeś/aś w trakcie pracy zawodowej lub działalności gospodarczej (najważniejsze z punktu widzenia efektów uczenia się).

```
{ % endif % }
```

```

</p>

<div class="pole-formularza {% if form.opis.errors %}pole-formularza--blad{% endif %}">

    {{ form.opis(class="pole-formularza__input", rows="12", placeholder="Tutaj wpisz sekcję II...") }}

    {% for e in form.opis.errors %}<span class="pole-formularza__blad">{{ e }}</span>{% endfor %}

</div>

</div>

</div>

{% if is_standard %}

<div class="karta" style="margin-bottom: 2rem;">

    <div class="karta__naglowek">

        <h2 class="karta__tytul">Sekcja III: Wykonane zadania (Zestawienie efektów)</h2>

    </div>

    <div class="karta__tresc py-3">

        <div style="background: var(--kolor-tlo); border-radius: 8px; padding: 1.5rem; border: 1px dashed
var(--kolor-ramka);">

            <p style="font-size: 0.9rem; color: var(--kolor-tekst);">

                <strong>Uwaga:</strong> Sekcja III zawierająca szczegółowe zestawienie godzinowych nakładów pracy dla każdego
zdefiniowanego efektu uczenia się <strong style="color: var(--kolor-glowny);">jest generowana w 100%
automatycznie</strong> na podstawie Twojego Dziennika Praktyk.

            </p>

            <ul style="margin-top: 1rem; padding-left: 1.5rem; font-size: 0.85rem; color: var(--kolor-tekst-drugor);">

                <li>Nie musisz nic tu wpisywać. Wystarczy regularnie uzupełniać dziennik.</li>

                <li>System sumuje godziny i przypisuje je do właściwych komórek tabeli z załącznika.</li>

            </ul>

        </div>

    </div>

</div>

{% endif %}

<div style="display: flex; justify-content: flex-end; gap: 1rem;">

    <button type="submit" class="przycisk przycisk--glowny" style="padding: 1rem 3rem; font-size: 1.1rem; box-shadow: 0
4px 14px rgba(26, 58, 92, 0.3);">

        Zapisz robocze sprawozdanie

    </button>

</div>

```

</form>

{% endif %}

</div>

{% endblock %}

PLIK: core\autoryzacja.py

"""

core/autoryzacja.py

Kanoniczny moduł autoryzacji – Microsoft Entra ID (OAuth 2.0 / OIDC).

Rejestracja w aplikacji:

```
from core.autoryzacja import stworz_blueprint_auth

app.register_blueprint(stworz_blueprint_auth(

    dozwolone_role=[RolaUzytkownika.STUDENT],

    template_logowania='auth/logowanie.html',

    template_zmiany_hasla='auth/zmien_haslo.html',

))
```

"""

import os

import secrets

from functools import wraps

```
from flask import (Blueprint, render_template, redirect, url_for,

                   flash, request, abort, current_app, session)
```

```
from flask_login import login_user, logout_user, login_required, current_user
```

```
from flask_wtf import FlaskForm
```

```
from wtforms import PasswordField
```

```
from wtforms.validators import DataRequired, Length, EqualTo
```

```
from sqlalchemy.exc import SQLAlchemyError
```

```
from core.modele import User, UserRole
```

```
from core.extensions import db, limiter
```

— Formularz zmiany hasła (zachowany dla kont technicznych) —————

```
class FormularzZmianyHasla(FlaskForm):
```

```
    nowe_haslo      = PasswordField('Nowe hasło', validators=[

        DataRequired(), Length(min=8, max=128),
```

```

])

potwierdzenie = PasswordField('Powtórz hasło', validators=[

    DataRequired(),

    EqualTo('nowe_haslo', message='Hasła muszą być identyczne.'),

])

```

— Dekorator RBAC —————

```

def wymaga_rol(*dozwolone_role):

    """Sprawdza czy zalogowany użytkownik ma jedną z podanych ról."""

    def dekorator(func):

        @wraps(func)

        @login_required

        def wrapper(*args, **kwargs):

            if current_user.role not in dozwolone_role:

                abort(403)

            return func(*args, **kwargs)

        return wrapper

    return dekorator

```

— Pomocnicze — MSAL —————

```

def _msal_app():

    """Buduje instancję MSAL ConfidentialClientApplication."""

    import msal

    return msal.ConfidentialClientApplication(

        client_id = current_app.config['AZURE_CLIENT_ID'],

        client_credential = current_app.config['AZURE_CLIENT_SECRET'],

        authority = (

            f"https://login.microsoftonline.com/"

            f"{current_app.config['AZURE_TENANT_ID']}"

        ),

```

)

```
_MS_SCOPES = ['User.Read']
```

```
# — Fabryka blueprintu —————
```

```
def stworz_blueprint_auth(
    dozwolone_role: list | None = None,
    template_logowania: str = 'auth/logowanie.html',
    template_zmiany_hasla: str = 'auth/zmien_haslo.html',
) -> Blueprint:
    auth_bp = Blueprint('auth', __name__)
```

```
# — Strona logowania – wyświetla przycisk "Zaloguj przez Microsoft" ————
```

```
@auth_bp.route('/', methods=['GET'])
@auth_bp.route('/logowanie', methods=['GET'])
def logowanie():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard.index'))
    return render_template(template_logowania)
```

```
# — Krok 1: przekieruj do Microsoft —————
```

```
@auth_bp.route('/ms-login')
@limiter.limit("20 per minute")
def ms_login():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard.index'))

    state = secrets.token_urlsafe(32)
    session['oauth_state'] = state
```

```

# zachowaj next po zalogowaniu

next_page = request.args.get('next', '')

if next_page.startswith('/'):

    session['oauth_next'] = next_page

try:

    auth_url = _msal_app().get_authorization_request_url(

        scopes      = _MS_SCOPES,

        state       = state,

        redirect_uri = url_for('auth.ms_callback', _external=True),

    )

except Exception:

    current_app.logger.exception('Błąd budowania URL autoryzacji Microsoft')

    flash('Błąd konfiguracji logowania. Skontaktuj się z administratorem.', 'danger')

    return redirect(url_for('auth.logowanie'))

return redirect(auth_url)

```

— Krok 2: callback z Microsoft —————

```

@auth_bp.route('/ms-callback')

@limiter.limit("20 per minute")

def ms_callback():

    # Weryfikacja stanu CSRF

    expected_state = session.pop('oauth_state', None)

    if not expected_state or request.args.get('state') != expected_state:

        current_app.logger.warning('OAuth state mismatch – możliwy atak CSRF')

        flash('Błąd bezpieczeństwa logowania. Spróbuj ponownie.', 'danger')

        return redirect(url_for('auth.logowanie'))

    # Microsoft zwrócił błąd (np. użytkownik anulował)

    if 'error' in request.args:

        error_desc = request.args.get('error_description', request.args['error'])

        current_app.logger.warning('Microsoft OAuth error: %s', error_desc)

```

```

        flash('Logowanie przez Microsoft nie powiodło się.', 'danger')

        return redirect(url_for('auth.logowanie'))

code = request.args.get('code')

if not code:

    flash('Brak kodu autoryzacyjnego od Microsoft.', 'danger')

    return redirect(url_for('auth.logowanie'))

# Wymiana kodu na token

try:

    result = _msal_app().acquire_token_by_authorization_code(

        code            = code,

        scopes          = _MS_SCOPES,

        redirect_uri     = url_for('auth.ms_callback', _external=True),

    )

except Exception:

    current_app.logger.exception('Błąd wymiany kodu OAuth na token')

    flash('Błąd komunikacji z Microsoft. Spróbuj ponownie.', 'danger')

    return redirect(url_for('auth.logowanie'))

if 'error' in result:

    current_app.logger.warning('Token error: %s - %s',

                               result.get('error'),

                               result.get('error_description'))

    flash('Nie udało się uzyskać tokenu od Microsoft.', 'danger')

    return redirect(url_for('auth.logowanie'))

claims = result.get('id_token_claims', {})

ms_email = (claims.get('preferred_username') or '').lower().strip()

if not ms_email:

    flash('Nie udało się odczytać adresu e-mail z konta Microsoft.', 'danger')

    return redirect(url_for('auth.logowanie'))

```



```
# Znajdź użytkownika w bazie po e-mailu
```

```
try:
```

```
    uzytkownik = db.session.query(User).filter_by(email=ms_email).first()
```

```
except SQLAlchemyError:
```

```
    current_app.logger.exception('Błąd bazy danych podczas logowania MS')
```

```
    flash('Błąd połączenia z bazą danych.', 'danger')
```

```
    return redirect(url_for('auth.logowanie'))
```

```
if not uzytkownik or not uzytkownik.is_active:
```

```
    current_app.logger.info('Próba logowania nieznanego konta MS: %s', ms_email)
```

```
    flash(
```

```
        'Twoje konto Microsoft nie jest zarejestrowane w systemie. '
```

```
        'Skontaktuj się z administratorem.',
```

```
        'danger',
```

```
    )
```

```
    return redirect(url_for('auth.logowanie'))
```

```
# Weryfikacja roli
```

```
if dozwolone_role and uzytkownik.role not in dozwolone_role:
```

```
    flash('Twoje konto nie ma dostępu do tego panelu.', 'danger')
```

```
    return redirect(url_for('auth.logowanie'))
```

```
login_user(uzytkownik, remember=False)
```

```
current_app.logger.info('Zalogowano przez Microsoft: %s (rola: %s)',
```

```
                        ms_email, uzytkownik.role)
```

```
next_page = session.pop('oauth_next', None)
```

```
return redirect(
```

```
    next_page if next_page and next_page.startswith('/') else url_for('dashboard.index')
```

```
)
```

```
# — Wylogowanie —————
```

```
@auth_bp.route('/wylogowanie', methods=['GET', 'POST'])
```

```

@login_required

def wylogowanie():

    logout_user()

    session.clear()

    # Wylogowanie globalne z Microsoft (single sign-out)

    tenant = current_app.config.get('AZURE_TENANT_ID', 'common')

    post_logout = url_for('auth.logowanie', _external=True)

    ms_logout_url = (

        f"https://login.microsoftonline.com/{tenant}/oauth2/v2.0/logout"

        f"?post_logout_redirect_uri={post_logout}"

    )

    return redirect(ms_logout_url)


# — Zmiana hasła (dla kont technicznych / awaryjnych) —————

@auth_bp.route('/zmien-haslo', methods=['GET', 'POST'])

@login_required

def zmien_haslo():

    from werkzeug.security import generate_password_hash

    form = FormularzZmianyHasla()

    if form.validate_on_submit():

        current_user.password_hash = generate_password_hash(form.nowe_haslo.data)

        current_user.require_password_change = False

        db.session.commit()

        flash('Hasło zostało zmienione.', 'success')

        return redirect(url_for('dashboard.index'))

    return render_template(template_zmiany_hasla, form=form)


return auth_bp

```

PLIK: core\error_handlers.py

```
# core/error_handlers.py

"""

Centralizowana obsługa błędów HTTP dla całej aplikacji.

Rejestruje handlersy 403, 404, 500 z wspólnymi szablonami.

"""


from flask import render_template


def register_error_handlers(app):

    """

    Rejestruje wszystkie error handlersy dla aplikacji.

    Użycie:

        from core.error_handlers import register_error_handlers

        app = Flask(__name__)

        register_error_handlers(app)

    """


@app.errorhandler(403)

def blad_403(e):

    return render_template('errors/blad.html',

                           kod='403',

                           tytul='Brak dostępu',

                           opis='Nie masz uprawnień do wyświetlenia tej strony.',

                           tekst_przycisku='Wróć do pulpitu'), 403


@app.errorhandler(404)

def blad_404(e):

    return render_template('errors/blad.html',

                           kod='404',

                           tytul='Nie znaleziono strony',

                           opis='Strona, której szukasz nie istnieje lub została przeniesiona.',
```

```
tekst_przycisku='Wróć do pulpitu'), 404
```

```
@app.errorhandler(500)
```

```
def blad_500(e):
```

```
    app.logger.exception('Unhandled Exception:')
```

```
    return render_template('errors/blad.html',
```

```
        kod='500',
```

```
        tytul='Błąd serwera',
```

```
        opis='Wystąpił nieoczekiwany błąd. Administratorzy zostali powiadomieni.',
```

```
        tekst_przycisku='Wróć do pulpitu'), 500
```

PLIK: core\extensions.py

```
from flask_sqlalchemy import SQLAlchemy

from flask_login import LoginManager

from flask_wtf.csrf import CSRFProtect

from flask_limiter import Limiter

from flask_limiter.util import get_remote_address


db = SQLAlchemy()

login_manager = LoginManager()

csrf = CSRFProtect()

limiter = Limiter(key_func=get_remote_address, default_limits=[])
```

PLIK: core\pliki.py

"""

core/pliki.py

Kanoniczny moduł obsługi przesyłania plików – jeden dla całej platformy SOP.

Rejestracja w aplikacji:

```
from core.pliki import stworz_blueprint_pliki, SprawdzaczDostepuPliku

app.register_blueprint(stworz_blueprint_pliki(

    sprawdzacz=SprawdzaczDostepuPliku.tylko_student,

    url_prefix='/uploads',

))
```

Centralny punkt wszystkich zabezpieczeń: walidacja MIME, rozszerzeń,
rozmiaru, secure_filename oraz ochrona przed Path Traversal.

"""

import os

import uuid

import httpx

from pathlib import Path

from werkzeug.utils import secure_filename

from flask import Blueprint, request, jsonify, Response, abort, url_for

from flask_login import login_required, current_user

from core.modele import InternshipEnrollment, UserRole, UploadedDocument

from core.extensions import db, limiter

from core.szyfrowanie import zaszyfruj, odszyfruj_strumieniowo

— Fileserver —————

FILESERVER_URL = os.environ.get('FILESERVER_URL', 'http://fileserver:5003')

FILESERVER_KEY = os.environ.get('FILESERVER_API_KEY', '')

```

def _fs_headers():

    return {'X-API-Key': FILESERVER_KEY}


def _fs_put(filename: str, data: bytes) -> None:

    r = httpx.put(f'{FILESERVER_URL}/files/{filename}', content=data, headers=_fs_headers(), timeout=30)

    r.raise_for_status()


def _fs_get(filename: str) -> bytes:

    r = httpx.get(f'{FILESERVER_URL}/files/{filename}', headers=_fs_headers(), timeout=30)

    r.raise_for_status()

    return r.content


def _fs_get_stream(filename: str):

    """Zwraca otwarty kontekst httpx do strumieniowego pobrania zaszyfrowanego pliku."""

    return httpx.stream('GET', f'{FILESERVER_URL}/files/{filename}',

                        headers=_fs_headers(), timeout=30)


def _fs_delete(filename: str) -> None:

    r = httpx.delete(f'{FILESERVER_URL}/files/{filename}', headers=_fs_headers(), timeout=10)

    if r.status_code != 404:

        r.raise_for_status()


MAX_FILE_SIZE    = 10 * 1024 * 1024 # 10 MB


# Dozwolone rozszerzenia – wyłącznie te

ALLOWED_EXTENSIONS = frozenset({

    '.pdf', '.doc', '.docx',

    '.jpg', '.jpeg', '.png',

    '.zip', '.rar',

})

```

Dozwolone typy MIME deklarowane przez klienta (pierwsza warstwa)

```
ALLOWED_MIME_TYPES = frozenset({

    'application/pdf',

    'application/msword',

    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',

    'image/jpeg',

    'image/png',

    'application/zip',

    'application/x-rar-compressed',

    'application/vnd.rar',

})
```

Magic bytes → akceptowane typy rzeczywiste (druga warstwa, niefałszowalna)

python-magic zwraca te wartości dla danych binarnych odczytanych z pliku.

```
_MAGIC_ALLOWED: frozenset[str] = frozenset({

    'application/pdf',

    'application/msword',

    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',

    'application/zip',          # .docx/.xlsx są ZIP-em wewnątrznie – OK

    'image/jpeg',

    'image/png',

    'application/x-rar-compressed',

    'application/vnd.rar',

    'application/x-rar',

})
```

def _dozwolony_plik(filename: str, content_type: str) -> bool:

"""Warstwa 1: walidacja rozszerzenia + nagłówek HTTP Content-Type."""

if not filename:

return False

ext = Path(filename).suffix.lower()

return ext in ALLOWED_EXTENSIONS and content_type in ALLOWED_MIME_TYPES


```

class MagicBytesError(RuntimeError):

    """Rzucany gdy weryfikacja magic bytes jest niemożliwa do przeprowadzenia."""

def _weryfikuj_magic_bytes(raw_bytes: bytes) -> bool:

    """Warstwa 2: weryfikacja rzeczywistego formatu przez analizę magic bytes.

    Używa python-magic (libmagic), która analizuje sygnaturę binarną pliku
    niezależnie od rozszerzenia i nagłówka HTTP – odporna na spoofing.

    Fail-Closed: jeśli libmagic jest niedostępna lub rzuci błąd, funkcja
    propaguje wyjątek zamiast przepuszczać plik. Upload jest wtedy odrzucany
    z HTTP 500, co zapobiega przemycaaniu plików przez awarię walidatora.

    """

    try:

        import magic

    except ImportError as exc:

        raise MagicBytesError(

            "python-magic (libmagic) jest niedostępna – weryfikacja binarna niemożliwa."

        ) from exc

    try:

        detected = magic.from_buffer(raw_bytes[:4096], mime=True)

    except Exception as exc:

        raise MagicBytesError(

            f"Błąd analizy magic bytes: {exc}"

        ) from exc

    return detected in _MAGIC_ALLOWED

def _sprawdz_dostep_do_zapisu(zapis: InternshipEnrollment) -> bool:

```

```
"""Domyślna kontrola dostępu: student widzi tylko swoje, admin/uopz wszystko."""
```

```
if current_user.role == UserRole.STUDENT:
```

```
    return zapis.student_id == current_user.id
```

```
if current_user.role == UserRole.UOPZ:
```

```
    return zapis.supervisor_id == current_user.id
```

```
return current_user.role == UserRole.ADMIN
```

```
# — Fabryka blueprintu —————
```

```
def stworz_blueprint_pliki(
```

```
    sprawdzacz_dostepu=None,
```

```
) -> Blueprint:
```

```
    """
```

```
    Tworzy blueprint 'uploads' ze scentralizowaną logiką bezpieczeństwa.
```

```
    sprawdzacz_dostepu – callable(zapis) → bool; None = domyślna logika RBAC.
```

```
    """
```

```
    uploads_bp = Blueprint('uploads', __name__)
```

```
    _sprawdz = sprawdzacz_dostepu or _sprawdz_dostep_do_zapisu
```

```
@uploads_bp.route('/enrollment/<uuid:enrollment_id>/upload', methods=['POST'])
```

```
@login_required
```

```
@limiter.limit("30 per hour")
```

```
def upload_document(enrollment_id):
```

```
    zapis = db.session.get(InternshipEnrollment, enrollment_id)
```

```
    if not zapis or not _sprawdz(zapis):
```

```
        abort(403)
```

```
    if 'file' not in request.files:
```

```
        return jsonify({'error': 'Brak pliku'}), 400
```

```
    file = request.files['file']
```

```
    document_type = request.form.get('document_type', '').strip()
```

```
if not file.filename or not document_type:

    return jsonify({'error': 'Brak pliku lub typu dokumentu'}), 400


# — Walidacja rozmiaru —————

file.seek(0, os.SEEK_END)

file_size = file.tell()

file.seek(0)


if file_size > MAX_FILE_SIZE:

    return jsonify({'error': f'Plik zbyt duży (max {MAX_FILE_SIZE // (1024*1024)} MB)'}), 400


# — Walidacja MIME + rozszerzenia —————

content_type = file.content_type

if not _dozwolony_plik(file.filename, content_type):

    return jsonify({'error': 'Niedozwolony typ pliku'}), 400


try:

    original_filename = secure_filename(file.filename)

    if not original_filename:

        return jsonify({'error': 'Nieprawidłowa nazwa pliku'}), 400


    file_ext = Path(original_filename).suffix.lower()

    stored_filename = f"{uuid.uuid4().hex}{file_ext}"


    # Szyfruj i wyślij do fileservera

    raw_bytes = file.read()


    # — Warstwa 2: magic bytes (niefałszowalna weryfikacja formatu) —

    # Fail-Closed: MagicBytesError (błąd biblioteki) → 503, nie 200.

    try:

        magic_ok = _weryfikuj_magic_bytes(raw_bytes)

    except MagicBytesError as exc:

        import logging
```

```

        logging.getLogger(__name__).error("Magic bytes check failed: %s", exc)

        return jsonify({'error': 'Weryfikacja formatu pliku chwilowo niedostępna.'}), 503

    if not magic_ok:

        return jsonify({'error': 'Niedozwolony format pliku (weryfikacja binarna)'}), 400

    encrypted_bytes = zaszyfruj(raw_bytes)

    _fs_put(stored_filename, encrypted_bytes)

    doc = UploadedDocument(

        enrollment_id      = enrollment_id,

        document_type      = document_type,

        original_filename  = original_filename,

        stored_filename    = stored_filename,

        file_path          = stored_filename,    # filename only, not a local path

        file_size          = file_size,

        mime_type          = content_type,

        uploaded_by_id    = current_user.id,

    )

    db.session.add(doc)

    db.session.commit()

    return jsonify({

        'success':      True,

        'document_id': str(doc.id),

        'filename':     original_filename,

        'size':         file_size,

    })

except Exception as e:

    db.session.rollback()

    return jsonify({'error': f'Błąd podczas zapisywania: {str(e)}'}), 500

@uploads_bp.route('/document/<uuid:document_id>/download')

@login_required

```

```

def download_document(document_id):

    doc = db.session.get(UploadedDocument, document_id)

    if not doc:

        abort(404)

    zapis = doc.enrollment

    if not zapis or not _sprawdz(zapis):

        abort(403)

    # Streaming: pobieramy cały zaszyfrowany plik z fileservera i deszyfrujemy
    # w pamięci – eliminuje wyciek kontekstu przy strumieniowaniu przez generator.

    try:

        encrypted = _fs_get(doc.file_path)

    except httpx.HTTPStatusError as e:

        abort(404 if e.response.status_code == 404 else 500)

    except Exception:

        abort(500)

    plain = b''.join(odszyfruj_strumieniowo(iter([encrypted])))

    return Response(

        plain,

        mimetype=doc.mime_type,

        headers={

            'Content-Disposition': f'attachment; filename="{doc.original_filename}"',

            'X-Content-Type-Options': 'nosniff',

        },

    )

@uploads_bp.route('/document/<uuid:document_id>/delete', methods=['POST'])
@login_required
def delete_document(document_id):

    doc = db.session.get(UploadedDocument, document_id)

    if not doc:

        return jsonify({'error': 'Nie znaleziono dokumentu'}), 404

```

```

zapis = doc.enrollment

if not zapis or not _sprawdz(zapis):

    abort(403)

try:

    _fs_delete(doc.file_path)

    doc.is_deleted = True

    db.session.commit()

    return jsonify({'success': True})

except Exception as e:

    db.session.rollback()

    return jsonify({'error': str(e)}), 500


@uploads_bp.route('/enrollment/<uuid:enrollment_id>/documents')
@login_required
def list_documents(enrollment_id):

    zapis = db.session.get(InternshipEnrollment, enrollment_id)

    if not zapis or not _sprawdz(zapis):

        abort(403)

    docs = (

        db.session.execute(

            db.select(UploadedDocument)

            .filter_by(enrollment_id=enrollment_id)

            .order_by(UploadedDocument.uploaded_at.desc())

        ).scalars().all()

    )

    return jsonify([
        {
            'id': str(d.id),
            'document_type': d.document_type,
            'original_filename': d.original_filename,
            'file_size': d.file_size,

```

```
        'mime_type':          d.mime_type,
        'uploaded_at':        d.uploaded_at.isoformat(),
        'uploaded_by':        (f"{d.uploaded_by.first_name} {d.uploaded_by.last_name}"
                               if d.uploaded_by else None),
        'download_url':       url_for('uploads.download_document', document_id=d.id),
        'delete_url':         url_for('uploads.delete_document',    document_id=d.id),
    } for d in docs])

return uploads_bp
```

PLIK: core\szyfrowanie.py

```
"""core/szyfrowanie.py
```

Szyfrowanie plików przesyłanych przez studentów.

Format v2 (chunked AES-256-GCM) – obsługuje streaming deszyfrowania:

Nagłówek: b'SOP1' + 4 bajty (chunk_size, big-endian uint32)

Dla każdego chunku:

4 bajty (długość zaszyfrowanego chunku, big-endian uint32)

12 bajtów nonce (losowe, unikalne dla każdego chunku)

N bajtów zaszyfrowanych danych + 16 bajtów tag GCM

Format v1 (Fernet, legacy) – rozpoznawany po braku nagłówka b'SOP1'.

Używany do odczytu istniejących plików; nowe pliki zawsze zapisywane jako v2.

Klucz pochodzi ze zmiennej środowiskowej FILE_ENCRYPTION_KEY (base64url, 32B).

```
"""
```

```
from __future__ import annotations
```

```
import os
```

```
import struct
```

```
import logging
```

```
from typing import Iterator
```

```
log = logging.getLogger(__name__)
```

```
_MAGIC = b'SOP1'
```

```
_CHUNK_SIZE = 64 * 1024 # 64 KB plaintext per chunk
```

```
_NONCE_SIZE = 12
```

```
_TAG_SIZE = 16
```

```
# — Klucz —————
```



```

def _get_key() -> bytes:

    """Zwraca 32-bajtowy klucz AES z FILE_ENCRYPTION_KEY (base64url)."""

    import base64

    raw = os.environ.get('FILE_ENCRYPTION_KEY', '').strip()

    if raw:

        try:

            key = base64.urlsafe_b64decode(raw + '==')

            if len(key) == 32:

                return key

            # Fernet używa 32B klucza + 32B HMAC = 32B po base64 → weź pierwsze 32

            if len(key) >= 32:

                return key[:32]

        except Exception:

            pass

    raise RuntimeError(

        'FILE_ENCRYPTION_KEY nie jest ustawiony lub ma nieprawidłową wartość. '

        'Ustaw zmienną środowiskową z kluczem base64url (32 bajty).'

    )

```

`_KEY: bytes | None = None`

```

def _key() -> bytes:

    global _KEY

    if _KEY is None:

        _KEY = _get_key()

    return _KEY

```

```

def _cipher(nonce: bytes):

    from cryptography.hazmat.primitives.ciphers.aead import AESGCM

    return AESGCM(_key())

```

— Szyfrowanie (v2 chunked) —————

```
def zaszyfruj(dane: bytes) -> bytes:

    """Szyfruje bajty – zwraca pełny token v2 (w pamięci, dla małych plików)."""

    return b''.join(_zaszyfruj_chunki(iter([dane])))
```

```
def zaszyfruj_strumieniowo(source: Iterator[bytes]) -> Iterator[bytes]:

    """Generator: szyfruje dane strumieniowo chunk po chunk.

    source – iterator bajtów (np. file.read(CHUNK_SIZE) w pętli).

    Yields zaszyfrowane bajty gotowe do zapisu / przesłania.

    """

    yield from _zaszyfruj_chunki(source)
```

```
def _zaszyfruj_chunki(source: Iterator[bytes]) -> Iterator[bytes]:

    from cryptography.hazmat.primitives.ciphers.aead import AESGCM

    aesgcm = AESGCM(_key())

    yield _MAGIC

    yield struct.pack('>I', _CHUNK_SIZE)

    buf = bytearray()

    for fragment in source:

        buf.extend(fragment)

        while len(buf) >= _CHUNK_SIZE:

            chunk, buf = buf[:_CHUNK_SIZE], buf[_CHUNK_SIZE:]

            yield _encrypt_chunk(aesgcm, bytes(chunk))

    if buf:

        yield _encrypt_chunk(aesgcm, bytes(buf))
```

```
def _encrypt_chunk(aesgcm, plaintext: bytes) -> bytes:

    nonce = os.urandom(_NONCE_SIZE)

    ct    = aesgcm.encrypt(nonce, plaintext, None)  # ct includes 16B tag

    blob  = nonce + ct

    return struct.pack('>I', len(blob)) + blob
```

— Deszyfrowanie —————

```
def odszyfruj(token: bytes) -> bytes:

    """Odszyfrowuje token v2 lub v1 (Fernet) – zwraca plaintext w pamięci."""

    return b''.join(odszyfruj_strumieniowo(iter([token])))
```

```
def odszyfruj_strumieniowo(source: Iterator[bytes]) -> Iterator[bytes]:

    """Generator: odszyfrowuje strumieniowo.

    source – iterator zaszyfrowanych bajtów (np. chunki z httpx streaming).

    Yields odszyfrowane bajty.

    Obsługuje formaty v2 (SOP1) i v1 (Fernet) automatycznie.

    """

    buf = bytearray()
```

```
def _read(n: int) -> bytes | None:

    while len(buf) < n:

        try:

            buf.extend(next(source))

        except StopIteration:

            return None

    chunk = bytes(buf[:n])

    del buf[:n]

    return chunk
```

```

# Odczytaj pierwsze 4 bajty – magic lub Fernet

magic = _read(4)

if magic is None:

    return

if magic != _MAGIC:

    # v1 Fernet – wczytaj resztę i odszyfruj blokowo

    rest = bytearray(magic)

    for fragment in source:

        rest.extend(fragment)

    yield _odszyfruj_fernet(bytes(rest))

    return

# v2: odczytaj chunk_size (nieużywany przy deszyfracji, ale w nagłówku)

_read(4) # chunk_size – zapisany dla przyszłych weryfikacji

from cryptography.hazmat.primitives.ciphers.aead import AESGCM

aesgcm = AESGCM(_key())

while True:

    len_bytes = _read(4)

    if len_bytes is None:

        break

    blob_len = struct.unpack('>I', len_bytes)[0]

    blob = _read(blob_len)

    if blob is None:

        raise ValueError('Urwany strumień zaszyfrowanych danych')

    nonce = blob[:_NONCE_SIZE]

    ct_tag = blob[_NONCE_SIZE:]

    plaintext = aesgcm.decrypt(nonce, ct_tag, None)

    yield plaintext

def _odszyfruj_fernet(token: bytes) -> bytes:

```

```
"""Fernet (v1) – blokowe deszyfrowanie dla starszych plików."""

from cryptography.fernet import Fernet

import base64

raw = os.environ.get('FILE_ENCRYPTION_KEY', '').strip()

if raw:

    try:

        return Fernet(raw.encode()).decrypt(token)

    except Exception:

        pass

raise ValueError('Nie można odszyfrować pliku v1 – nieprawidłowy klucz Fernet')
```

PLIK: core\modele\dokumenty.py

```
"""core/modele/dokumenty.py
```

```
Domain models: Uploaded documents, audit log.
```

```
"""
```

```
import uuid
```

```
import enum
```

```
from sqlalchemy.dialects.postgresql import UUID
```

```
from core.extensions import db
```

```
class DocumentStatus(enum.Enum):
```

```
    DRAFT = 'DRAFT'
```

```
    AWAITING_APPROVAL = 'AWAITING_APPROVAL'
```

```
    APPROVED = 'APPROVED'
```

```
    REJECTED = 'REJECTED'
```

```
class DocumentAuditLog(db.Model):
```

```
    """Audit trail for PDF document operations."""
```

```
    __tablename__ = 'document_audit_logs'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    user_id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='SET NULL'), nullable=True)
```

```
    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='SET NULL'), nullable=True)
```

```
    document_type = db.Column(db.String(50), nullable=False)
```

```
    action = db.Column(db.String(50), nullable=False)
```

```
    details = db.Column(db.Text, nullable=True)
```

```
    created_at = db.Column(db.DateTime, server_default=db.func.now())
```

```
    user = db.relationship('User')
```

```
    enrollment = db.relationship('InternshipEnrollment')
```

```
# Backward-compat shims
```

```
@property
```

```
def uzytkownik_id(self):
```

```
    return self.user_id
```

```
@property
```

```
def zapis_id(self):
```

```
    return self.enrollment_id
```

```
@property
```

```
def typ_dokumentu(self):
```

```
    return self.document_type
```

```
@property
```

```
def akcja(self):
```

```
    return self.action
```

```
@property
```

```
def szczegoly(self):
```

```
    return self.details
```

```
@property
```

```
def wykonano_o(self):
```

```
    return self.created_at
```

```
class UploadedDocument(db.Model):
```

```
    """File uploaded by a student or staff member (contracts, certificates, etc.).
```

```
Soft-delete pattern: documents are never physically removed – only marked
```

```
is_deleted=True. The FK to internship_enrollments uses SET NULL so that
```

```
archival documents survive even after the enrollment record is deleted.
```

```
"""
```

```

__tablename__ = 'uploaded_documents'

id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='SET NULL'),
nullable=True)

document_type = db.Column(db.String(50), nullable=False)

original_filename = db.Column(db.String(255), nullable=False)

stored_filename = db.Column(db.String(255), nullable=False)

file_path = db.Column(db.String(500), nullable=False)

file_size = db.Column(db.Integer, nullable=False)

mime_type = db.Column(db.String(100), nullable=False)

uploaded_at = db.Column(db.DateTime, server_default=db.func.now())

uploaded_by_id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='SET NULL'), nullable=True)

is_deleted = db.Column(db.Boolean, nullable=False, default=False, server_default='false')

enrollment = db.relationship('InternshipEnrollment', backref=db.backref('uploaded_documents',
passive_deletes=True))

uploaded_by = db.relationship('User')

# Backward-compat shims

@property
def zapis_id(self):
    return self.enrollment_id

@property
def typ_dokumentu(self):
    return self.document_type

@property
def oryginalna_nazwa(self):
    return self.original_filename

@property
def zapisana_nazwa(self):
    return self.stored_filename

```



```
@property

def sciezka_pliku(self):

    return self.file_path
```

```
@property

def rozmiar_pliku(self):

    return self.file_size
```

```
@property

def typ_mime(self):

    return self.mime_type
```

```
@property

def przeslano_o(self):

    return self.uploaded_at
```

```
@property

def przeslane_przez_id(self):

    return self.uploaded_by_id
```

```
# Backward-compatible aliases
```

```
StatusDokumentu      = DocumentStatus
```

```
LogAudytuDokumentow  = DocumentAuditLog
```

```
DokumentPrzeslany    = UploadedDocument
```

PLIK: core\modele\dziennik.py

```
"""core/modele/dziennik.py

Domain models: Internship journal, learning outcomes, assessments.

"""

import uuid

import enum

from sqlalchemy.dialects.postgresql import UUID

from core.extensions import db


class AssessmentResult(enum.Enum):

    ACHIEVED = 'ACHIEVED'

    PARTIALLY_ACHIEVED = 'PARTIALLY_ACHIEVED'

    NOT_ACHIEVED = 'NOT_ACHIEVED'


class LearningOutcome(db.Model):

    """Learning outcomes – reference dictionary table."""

    __tablename__ = 'learning_outcomes'

    id = db.Column(db.Integer, primary_key=True)

    description = db.Column(db.String(500), nullable=False)

    @property
    def kod(self) -> str:

        return str(self.id).zfill(2)

    # Backward-compatible

    @property
    def opis(self):

        return self.description
```

```
# Junction table: journal_entry ↔ learning_outcomes
```

```
entry_outcomes = db.Table(
    'entry_outcomes',
    db.Column(
        'entry_id',
        UUID(as_uuid=True),
        db.ForeignKey('journal_entries.id', ondelete='CASCADE'),
        primary_key=True,
    ),
    db.Column(
        'outcome_id',
        db.Integer,
        db.ForeignKey('learning_outcomes.id'),
        primary_key=True,
    ),
)
```

```
class JournalEntry(db.Model):
```

```
    """A single journal entry for a student's internship."""
```

```
    __tablename__ = 'journal_entries'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),
    nullable=False)
```

```
    entry_date = db.Column(db.Date, nullable=False)
```

```
    duration_hours = db.Column(db.Integer, nullable=False)
```

```
    description = db.Column(db.Text, nullable=False)
```

```
    learning_outcomes = db.relationship('LearningOutcome', secondary=entry_outcomes, lazy='subquery')
```

```
# Backward-compat shims
```

```
@property
```

```
    def zapis_id(self):
```

```
    return self.enrollment_id
```

```
@property
```

```
def data_wpisu(self):
```

```
    return self.entry_date
```

```
@property
```

```
def liczba_godzin(self):
```

```
    return self.duration_hours
```

```
@property
```

```
def opis(self):
```

```
    return self.description
```

```
@property
```

```
def efekty_uczenia(self):
```

```
    return self.learning_outcomes
```

```
class OutcomeAssessment(db.Model):
```

```
    """Assessment of a single learning outcome for one enrollment."""
```

```
    __tablename__ = 'outcome_assessments'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),  
nullable=False)
```

```
    learning_outcome_id = db.Column(db.Integer, db.ForeignKey('learning_outcomes.id'), nullable=False)
```

```
    result = db.Column(  
        db.Enum(AssessmentResult, name='assessment_result', values_callable=lambda e: [x.value for x in e]),  
        nullable=False,  
    )
```

```
    notes = db.Column(db.Text, nullable=True)
```

```
    learning_outcome = db.relationship('LearningOutcome', lazy='select')
```

```
# Backward-compat shims
```

```
@property
```

```
def zapis_id(self):
```

```
    return self.enrollment_id
```

```
@property
```

```
def efekt_id(self):
```

```
    return self.learning_outcome_id
```

```
@property
```

```
def wynik(self):
```

```
    return self.result
```

```
@wynik.setter
```

```
def wynik(self, v):
```

```
    self.result = v
```

```
@property
```

```
def uwagi(self):
```

```
    return self.notes
```

```
@property
```

```
def efekt(self):
```

```
    return self.learning_outcome
```

```
# Backward-compat aliases
```

```
WynikOceny = AssessmentResult
```

```
EfektUczenia = LearningOutcome
```

```
WpisDziennika = JournalEntry
```

```
OcenaPraktyki = OutcomeAssessment
```

```
wpisy_efekty = entry_outcomes
```

PLIK: core\modele\firmy.py

```
"""core/modele/firmy.py

Domain models: Workplace companies (internship partners).

"""

import uuid

from sqlalchemy.dialects.postgresql import UUID

from core.extensions import db


class Company(db.Model):

    """Workplace – a partner company with a standing agreement with ANS."""

    __tablename__ = 'companies'

    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    name = db.Column(db.String(255), nullable=False)

    address = db.Column(db.String(255), nullable=True)

    city = db.Column(db.String(100), nullable=True)

    tax_id = db.Column(db.String(50), nullable=True)

    has_standing_agreement = db.Column(db.Boolean, default=True)

    is_active = db.Column(db.Boolean, default=True)

    created_at = db.Column(db.DateTime, server_default=db.func.now())

    # Backward-compatible shims

    @property
    def nazwa(self):

        return self.name

    @property
    def adres(self):

        return self.address

    @property
    def miasto(self):
```

```
    return self.city
```

```
@property
```

```
def nip_krs(self):
```

```
    return self.tax_id
```

```
@property
```

```
def ma_stala_umowe(self):
```

```
    return self.has_standing_agreement
```

```
def __repr__(self) -> str:
```

```
    return f'<Company {self.name}>'
```

```
# Backward-compatible alias
```

```
Firma = Company
```

PLIK: core\modele\praktyki.py

```
"""core/modele/praktyki.py
```

```
Domain models: Internships, enrollments, and satellite entities.
```

```
"""
```

```
import uuid
```

```
import enum
```

```
from sqlalchemy.dialects.postgresql import UUID
```

```
from sqlalchemy.ext.hybrid import hybrid_property
```

```
from core.extensions import db
```

```
# — Enums —————
```

```
class InternshipStatus(enum.Enum):
```

```
    ACTIVE = 'ACTIVE'
```

```
    INACTIVE = 'INACTIVE'
```

```
class EnrollmentStatus(enum.Enum):
```

```
    PENDING = 'PENDING'
```

```
    AWAITING_APPROVAL = 'AWAITING_APPROVAL'
```

```
    REVISION_REQUIRED = 'REVISION_REQUIRED' # komisja: wymaga uzupełnień
```

```
    IN_PROGRESS = 'IN_PROGRESS'
```

```
    COMPLETED = 'COMPLETED'
```

```
    REJECTED = 'REJECTED'
```

```
    COMMISSION_REVIEW = 'COMMISSION_REVIEW'
```

```
    DEAN_APPROVAL = 'DEAN_APPROVAL'
```

```
class InternshipPath(enum.Enum):
```

```
    STANDARD = 'STANDARD'
```


EMPLOYMENT = 'EMPLOYMENT'

OWN_BUSINESS = 'OWN_BUSINESS'

class EventType(enum.Enum):

ADMIN_COMMENT = 'ADMIN_KOMENTARZ'

SUPERVISOR_COMMENT = 'UOPZ_KOMENTARZ'

STUDENT_NOTIFICATION = 'POWIADOMIENIE_STUDENTA'

COMMITTEE_DECISION = 'KOMISJA_DECYZJA'

DEAN_DECISION = 'DZIEKAN_DECYZJA'

Backward-compat aliases for enums (remove after all routes migrated)

StatusPraktyki = InternshipStatus

StatusPraktyki.AKTYWNA = InternshipStatus.ACTIVE

StatusPraktyki.NIEAKTYWNA = InternshipStatus.INACTIVE

StatusZapisu = EnrollmentStatus

StatusZapisu.OCZEKUJACY = EnrollmentStatus.PENDING

StatusZapisu.OCZEKUJE_NA_AKCEPT = EnrollmentStatus.AWAITING_APPROVAL

StatusZapisu.WYMAGA_POPRAWEK = EnrollmentStatus.REVISION_REQUIRED

StatusZapisu.W_REALIZACJI = EnrollmentStatus.IN_PROGRESS

StatusZapisu.ZAKONCZONA = EnrollmentStatus.COMPLETED

StatusZapisu.ODRZUCONA = EnrollmentStatus.REJECTED

StatusZapisu.WERYFIKACJA_KOMISJI = EnrollmentStatus.COMMISSION_REVIEW

StatusZapisu.AKCEPTACJA_DZIEKANA = EnrollmentStatus.DEAN_APPROVAL

SciezkaPraktyki = InternshipPath

SciezkaPraktyki.STANDARDOWA = InternshipPath.STANDARD

SciezkaPraktyki.ZATRUDNIENIE = InternshipPath.EMPLOYMENT

SciezkaPraktyki.WLASNA_DZIALALNOSC = InternshipPath.OWN_BUSINESS

TypZdarzenia = EventType

TypZdarzenia.ADMIN_KOMENTARZ = EventType.ADMIN_COMMENT

```

TypZdarzenia.UOPZ_KOMENTARZ          = EventType.SUPERVISOR_COMMENT

TypZdarzenia.POWIADOMIENIE_STUDENTA = EventType.STUDENT_NOTIFICATION

TypZdarzenia.KOMISJA_DECYZJA          = EventType.COMMITTEE_DECISION

TypZdarzenia.DZIEKAN_DECYZJA          = EventType.DEAN_DECISION

```

— Core models —————

```

class Internship(db.Model):

    """Internship edition – academic year and semester."""

    __tablename__ = 'internships'

    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    academic_year = db.Column(db.String(9), nullable=False)

    semester = db.Column(db.String(10), nullable=False)

    required_hours = db.Column(db.Integer, nullable=False, default=160)

    status = db.Column(

        db.Enum(InternshipStatus, name='internship_status', values_callable=lambda e: [x.value for x in e]),

        nullable=False, default=InternshipStatus.INACTIVE,

    )

    created_at = db.Column(db.DateTime, server_default=db.func.now())

    enrollments = db.relationship(

        'InternshipEnrollment', backref='internship',

        lazy='select', cascade='all, delete-orphan', passive_deletes=True,

    )

    # Backward-compatible shims

    @property
    def rok_uczelniany(self):

        return self.academic_year

    @property
    def wymiar_godzin(self):

```

```
return self.required_hours
```

```
@property
```

```
def semestr(self):
```

```
    return self.semester
```

```
class InternshipEnrollment(db.Model):
```

```
    """Core student enrollment record."""
```

```
    __tablename__ = 'internship_enrollments'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    internship_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internships.id', ondelete='CASCADE'), nullable=False)
```

```
    student_id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
```

```
    supervisor_id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='SET NULL'), nullable=True)
```

```
    company_id = db.Column(UUID(as_uuid=True), db.ForeignKey('companies.id', ondelete='SET NULL'), nullable=True)
```

```
    status = db.Column(
        db.Enum(EnrollmentStatus, name='enrollment_status', values_callable=lambda e: [x.value for x in e]),
        nullable=False, default=EnrollmentStatus.PENDING,
```

```
    )
```

```
    path_type = db.Column(
        db.Enum(InternshipPath, name='internship_path', values_callable=lambda e: [x.value for x in e]),
        nullable=False, default=InternshipPath.STANDARD,
```

```
    )
```

```
    start_date = db.Column(db.Date, nullable=True)
```

```
    end_date = db.Column(db.Date, nullable=True)
```

```
    specialization = db.Column(db.String(255), nullable=True)
```

```
    accident_insurance = db.Column(db.Boolean, default=False)
```

```
    total_hours_logged = db.Column(db.Integer, default=0)
```

```
    enrolled_at = db.Column(db.DateTime, server_default=db.func.now())
```

```
# — Relationships —————
```

```

student          = db.relationship('User',                foreign_keys=[student_id], lazy='select')

uopz             = db.relationship('User',                foreign_keys=[supervisor_id], lazy='select')

firma            = db.relationship('Company',             foreign_keys=[company_id], lazy='select')

workplace_details = db.relationship('WorkplaceDetails', back_populates='enrollment', uselist=False, cascade='all,
delete-orphan')

path_justification = db.relationship('PathJustification', back_populates='enrollment', uselist=False, cascade='all,
delete-orphan')

examination      = db.relationship('Examination',        back_populates='enrollment', uselist=False, cascade='all,
delete-orphan')

final_grades     = db.relationship('FinalGrades',        back_populates='enrollment', uselist=False, cascade='all,
delete-orphan')

process_events   = db.relationship('ProcessEvent',       back_populates='enrollment', lazy='select', cascade='all,
delete-orphan', order_by='ProcessEvent.executed_at')

journal_entries  = db.relationship('JournalEntry',       backref='enrollment',        lazy='select', cascade='all,
delete-orphan')

outcome_assessments = db.relationship('OutcomeAssessment', backref='enrollment',        lazy='select', cascade='all,
delete-orphan')

schedule         = db.relationship('InternshipSchedule', backref='enrollment',        lazy='select', cascade='all,
delete-orphan')

report           = db.relationship('InternshipReport',   backref='enrollment',        uselist=False, lazy='select',
cascade='all, delete-orphan')

```

— Backward-compat property shims —————

@property

def praktyka_id(self):

return self.internship_id

@property

def praktyka(self):

return self.internship

@property

def uopz_id(self):

return self.supervisor_id

@uopz_id.setter

def uopz_id(self, v):

self.supervisor_id = v

```
@property

def firma_id(self):

    return self.company_id
```

```
@firma_id.setter

def firma_id(self, v):

    self.company_id = v
```

```
@property

def sciezka(self):

    return self.path_type
```

```
@sciezka.setter

def sciezka(self, v):

    self.path_type = v
```

```
@property

def track_type(self):

    return self.path_type
```

```
@track_type.setter

def track_type(self, v):

    self.path_type = v
```

```
@property

def termin_od(self):

    return self.start_date
```

```
@termin_od.setter

def termin_od(self, v):

    self.start_date = v
```

```
@property
```

```
def termin_do(self):

    return self.end_date


@termin_do.setter

def termin_do(self, v):

    self.end_date = v


@property

def specjalnosc(self):

    return self.specialization


@specjalnosc.setter

def specjalnosc(self, v):

    self.specialization = v


@property

def ubezpieczenie_nw(self):

    return self.accident_insurance


@ubezpieczenie_nw.setter

def ubezpieczenie_nw(self, v):

    self.accident_insurance = v


@property

def laczenie_godzin(self):

    return self.total_hours_logged


@property

def zapisano_o(self):

    return self.enrolled_at


@property

def dane_miejsca(self):

    return self.workplace_details
```

```
@property  
  
def uzasadnienie(self):  
  
    return self.path_justification
```

```
@property  
  
def sprawdzian(self):  
  
    return self.examination
```

```
@property  
  
def oceny_koncowe(self):  
  
    return self.final_grades
```

```
@property  
  
def zdarzenia(self):  
  
    return self.process_events
```

```
@property  
  
def wpisy_dziennika(self):  
  
    return self.journal_entries
```

```
@property  
  
def oceny(self):  
  
    return self.outcome_assessments
```

```
@property  
  
def harmonogram(self):  
  
    return self.schedule
```

```
@property  
  
def sprawozdanie(self):  
  
    return self.report
```

@property

```
def firma_nazwa(self):  
  
    return self.workplace_details.company_name if self.workplace_details else None
```

@property

```
def firma_adres(self):  
  
    return self.workplace_details.company_address if self.workplace_details else None
```

@property

```
def firma_miasto(self):  
  
    return self.workplace_details.company_city if self.workplace_details else None
```

@property

```
def firma_nip_krs(self):  
  
    return self.workplace_details.company_tax_id if self.workplace_details else None
```

@property

```
def firma_upowazniony_osoba(self):  
  
    return self.workplace_details.authorized_person if self.workplace_details else None
```

@property

```
def firma_upowazniony_stanowisko(self):  
  
    return self.workplace_details.authorized_person_position if self.workplace_details else None
```

@property

```
def zopz_imie_nazwisko(self):  
  
    return self.workplace_details.workplace_mentor_name if self.workplace_details else None
```

@property

```
def zopz_stanowisko(self):  
  
    return self.workplace_details.workplace_mentor_position if self.workplace_details else None
```

@property


```
def zopz_telefon(self):

    return self.workplace_details.workplace_mentor_phone if self.workplace_details else None


@property

def zopz_email(self):

    return self.workplace_details.workplace_mentor_email if self.workplace_details else None


@property

def uzasadnienie_sciezki(self):

    return self.path_justification.justification if self.path_justification else None


@property

def zalaczniki_sciezki(self):

    return self.path_justification.attachments if self.path_justification else None


@property

def ocena_sprawozdania(self):

    return self.final_grades.report_grade if self.final_grades else None


@property

def ocena_uopz(self):

    return self.final_grades.supervisor_grade if self.final_grades else None


@property

def ocena_zopz(self):

    return self.final_grades.workplace_grade if self.final_grades else None


@property

def ocena_opisowa_uopz(self):

    return self.final_grades.supervisor_grade_description if self.final_grades else None


@property

def ocena_opisowa_zopz(self):

    return self.final_grades.workplace_grade_description if self.final_grades else None
```

```
@property

def sprawdzian_pytanie_1(self):

    return self.examination.question_1 if self.examination else None
```

```
@property

def sprawdzian_ocena_1(self):

    return self.examination.grade_1 if self.examination else None
```

```
@property

def sprawdzian_pytanie_2(self):

    return self.examination.question_2 if self.examination else None
```

```
@property

def sprawdzian_ocena_2(self):

    return self.examination.grade_2 if self.examination else None
```

```
@property

def sprawdzian_pytanie_3(self):

    return self.examination.question_3 if self.examination else None
```

```
@property

def sprawdzian_ocena_3(self):

    return self.examination.grade_3 if self.examination else None
```

```
# — Process event helpers —————
```

```
def _last_event(self, event_type: EventType):

    return next(

        (e for e in reversed(self.process_events) if e.event_type == event_type),

        None,

    )
```

```
@property
```

```
def committee_decision(self):

    e = self._last_event(EventType.COMMITTEE_DECISION)

    return e.decision if e else None


@property

def committee_comment(self):

    e = self._last_event(EventType.COMMITTEE_DECISION)

    return e.comment if e else None


@property

def committee_decision_at(self):

    e = self._last_event(EventType.COMMITTEE_DECISION)

    return e.executed_at if e else None


@property

def dean_decision(self):

    e = self._last_event(EventType.DEAN_DECISION)

    return e.decision if e else None


@property

def dean_comment(self):

    e = self._last_event(EventType.DEAN_DECISION)

    return e.comment if e else None


@property

def dean_decision_at(self):

    e = self._last_event(EventType.DEAN_DECISION)

    return e.executed_at if e else None


@property

def admin_comments(self):

    e = self._last_event(EventType.ADMIN_COMMENT)

    return e.comment if e else None
```

```
@property

def supervisor_comments(self):

    e = self._last_event(EventType.SUPERVISOR_COMMENT)

    return e.comment if e else None
```

```
# Backward-compatible event property names
```

```
@property

def komentarze_admina(self):

    return self.admin_comments
```

```
@property

def komentarze_uopz(self):

    return self.supervisor_comments
```

```
@property

def decyzja_komisji(self):

    return self.committee_decision
```

```
@property

def komentarze_komisji(self):

    return self.committee_comment
```

```
@property

def decyzja_komisji_o(self):

    return self.committee_decision_at
```

```
@property

def decyzja_dziekana(self):

    return self.dean_decision
```

```
@property

def komentarze_dziekana(self):

    return self.dean_comment
```

```
@property
```

```
def decyzja_dziekana_o(self):  
    return self.dean_decision_at
```

```
# — Grade computations —————
```

```
@hybrid_property
```

```
def exam_grade(self):  
    """Average of examination scores."""  
    if self.examination is None:  
        return None  
    grades = [  
        float(v) for v in (  
            self.examination.grade_1,  
            self.examination.grade_2,  
            self.examination.grade_3,  
        ) if v is not None  
    ]  
    return round(sum(grades) / len(grades), 2) if grades else None
```

```
@hybrid_property
```

```
def final_grade(self):  
    """Final grade: 0.4E + 0.1S + 0.2U + 0.3Z."""  
    if self.final_grades is None:  
        return None  
    e = self.exam_grade  
    s = float(self.final_grades.report_grade) if self.final_grades.report_grade is not None else None  
    u = float(self.final_grades.supervisor_grade) if self.final_grades.supervisor_grade is not None else None  
    z = float(self.final_grades.workplace_grade) if self.final_grades.workplace_grade is not None else None  
    if None in (e, s, u, z):  
        return None  
    return round(0.4 * e + 0.1 * s + 0.2 * u + 0.3 * z, 2)
```

```
# Legacy computed grade aliases
```

```

@hybrid_property

def ocena_e(self):

    return self.exam_grade

```

```

@hybrid_property

def ocena_k(self):

    return self.final_grade

```

— Satellite entities —————

```

class WorkplaceDetails(db.Model):

    """Snapshot of workplace and mentor data copied from the enrollment form."""

    __tablename__ = 'workplace_details'

    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),
    nullable=False, unique=True)

    company_name = db.Column(db.String(255), nullable=True)

    company_address = db.Column(db.String(255), nullable=True)

    company_city = db.Column(db.String(255), nullable=True)

    company_tax_id = db.Column(db.String(50), nullable=True)

    authorized_person = db.Column('company_authorized_person', db.String(255), nullable=True)

    authorized_person_position = db.Column('company_authorized_position', db.String(255), nullable=True)

    workplace_mentor_name = db.Column('workplace_supervisor_name', db.String(255), nullable=True)

    workplace_mentor_position = db.Column('workplace_supervisor_position', db.String(255), nullable=True)

    workplace_mentor_phone = db.Column('workplace_supervisor_phone', db.String(50), nullable=True)

    workplace_mentor_email = db.Column('workplace_supervisor_email', db.String(255), nullable=True)

    enrollment = db.relationship('InternshipEnrollment', back_populates='workplace_details')

# Backward-compatible shims

@property

```

```
def zapis_id(self):

    return self.enrollment_id


@property

def firma_nazwa(self):

    return self.company_name


@firma_nazwa.setter

def firma_nazwa(self, v):

    self.company_name = v


@property

def firma_adres(self):

    return self.company_address


@firma_adres.setter

def firma_adres(self, v):

    self.company_address = v


@property

def firma_miasto(self):

    return self.company_city


@firma_miasto.setter

def firma_miasto(self, v):

    self.company_city = v


@property

def firma_nip_krs(self):

    return self.company_tax_id


@firma_nip_krs.setter

def firma_nip_krs(self, v):

    self.company_tax_id = v
```

```
@property
```

```
def firma_upowazniony_osoba(self):  
  
    return self.authorized_person
```

```
@firma_upowazniony_osoba.setter
```

```
def firma_upowazniony_osoba(self, v):  
  
    self.authorized_person = v
```

```
@property
```

```
def firma_upowazniony_stanowisko(self):  
  
    return self.authorized_person_position
```

```
@firma_upowazniony_stanowisko.setter
```

```
def firma_upowazniony_stanowisko(self, v):  
  
    self.authorized_person_position = v
```

```
@property
```

```
def zopz_imie_nazwisko(self):  
  
    return self.workplace_mentor_name
```

```
@zopz_imie_nazwisko.setter
```

```
def zopz_imie_nazwisko(self, v):  
  
    self.workplace_mentor_name = v
```

```
@property
```

```
def zopz_stanowisko(self):  
  
    return self.workplace_mentor_position
```

```
@zopz_stanowisko.setter
```

```
def zopz_stanowisko(self, v):  
  
    self.workplace_mentor_position = v
```

```
@property
```



```
def zopz_telefon(self):  
    return self.workplace_mentor_phone
```

```
@zopz_telefon.setter
```

```
def zopz_telefon(self, v):  
    self.workplace_mentor_phone = v
```

```
@property
```

```
def zopz_email(self):  
    return self.workplace_mentor_email
```

```
@zopz_email.setter
```

```
def zopz_email(self, v):  
    self.workplace_mentor_email = v
```

```
class PathJustification(db.Model):
```

```
    """Justification for choosing path B or C (optional)."""
```

```
    __tablename__ = 'path_justifications'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),  
nullable=False, unique=True)
```

```
    justification = db.Column(db.Text, nullable=True)
```

```
    attachments = db.Column(db.Text, nullable=True)
```

```
    enrollment = db.relationship('InternshipEnrollment', back_populates='path_justification')
```

```
# Backward-compat
```

```
@property
```

```
def zapis_id(self):  
    return self.enrollment_id
```

```
@property
```

```
def uzasadnienie(self):
```

```
    return self.justification
```

```
@uzasadnienie.setter
```

```
def uzasadnienie(self, v):
```

```
    self.justification = v
```

```
@property
```

```
def zalaczniki(self):
```

```
    return self.attachments
```

```
@zalaczniki.setter
```

```
def zalaczniki(self, v):
```

```
    self.attachments = v
```

```
class Examination(db.Model):
```

```
    """Three examination questions with grades (issued by supervisor)."""
```

```
    __tablename__ = 'examinations'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),  
nullable=False, unique=True)
```

```
    question_1 = db.Column(db.Text, nullable=True)
```

```
    grade_1 = db.Column(db.Numeric(3, 1), nullable=True)
```

```
    question_2 = db.Column(db.Text, nullable=True)
```

```
    grade_2 = db.Column(db.Numeric(3, 1), nullable=True)
```

```
    question_3 = db.Column(db.Text, nullable=True)
```

```
    grade_3 = db.Column(db.Numeric(3, 1), nullable=True)
```

```
    enrollment = db.relationship('InternshipEnrollment', back_populates='examination')
```

```
# Backward-compat
```

```
@property
```

```
def zapis_id(self):
```

```
return self.enrollment_id
```

@property

```
def pytanie_1(self):
```

```
return self.question_1
```

@property

```
def ocena_1(self):
```

```
return self.grade_1
```

@property

```
def pytanie_2(self):
```

```
return self.question_2
```

@property

```
def ocena_2(self):
```

```
return self.grade_2
```

@property

```
def pytanie_3(self):
```

```
return self.question_3
```

@property

```
def ocena_3(self):
```

```
return self.grade_3
```

```
class FinalGrades(db.Model):
```

""Final component grades for an internship.""

```
__tablename__ = 'final_grades'
```

```
id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),
nullable=False, unique=True)
```

```
report_grade            = db.Column(db.Numeric(3, 1), nullable=True)

supervisor_grade        = db.Column(db.Numeric(3, 1), nullable=True)

workplace_grade         = db.Column(db.Numeric(3, 1), nullable=True)

supervisor_grade_description = db.Column(db.Text,          nullable=True)

workplace_grade_description = db.Column(db.Text,          nullable=True)


enrollment = db.relationship('InternshipEnrollment', back_populates='final_grades')
```

```
# Backward-compat
```

```
@property
```

```
def zapis_id(self):
```

```
    return self.enrollment_id
```

```
@property
```

```
def ocena_sprawozdania(self):
```

```
    return self.report_grade
```

```
@property
```

```
def ocena_uopz(self):
```

```
    return self.supervisor_grade
```

```
@property
```

```
def ocena_zopz(self):
```

```
    return self.workplace_grade
```

```
@property
```

```
def ocena_opisowa_uopz(self):
```

```
    return self.supervisor_grade_description
```

```
@property
```

```
def ocena_opisowa_zopz(self):
```

```
    return self.workplace_grade_description
```

```

class ProcessEvent(db.Model):

    """Workflow event log: comments, committee and dean decisions."""

    __tablename__ = 'process_events'

    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),
    nullable=False)

    event_type = db.Column(

        db.Enum(EventType, name='event_type', values_callable=lambda e: [x.value for x in e]),

        nullable=False,

    )

    decision = db.Column(db.String(20), nullable=True) # 'APPROVED' / 'REJECTED'

    comment = db.Column(db.Text, nullable=True)

    executed_by_id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='SET NULL'), nullable=True)

    executed_at = db.Column(db.DateTime, server_default=db.func.now())

    enrollment = db.relationship('InternshipEnrollment', back_populates='process_events')

    executed_by = db.relationship('User', foreign_keys=[executed_by_id], lazy='select')


# Backward-compat

@property

def zapis_id(self):

    return self.enrollment_id


@property

def typ(self):

    return self.event_type


@property

def decyzja(self):

    return self.decision


@property

def komentarz(self):

    return self.comment

```

```
@property
```

```
def wykonane_przez_id(self):  
  
    return self.executed_by_id
```

```
@property
```

```
def wykonano_o(self):  
  
    return self.executed_at
```

```
@property
```

```
def wykonane_przez(self):  
  
    return self.executed_by
```

```
class InternshipSchedule(db.Model):
```

```
    """Schedule of learning outcome completion for one enrollment."""
```

```
    __tablename__ = 'internship_schedules'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),  
nullable=False)
```

```
    learning_outcome_id = db.Column(db.Integer, db.ForeignKey('learning_outcomes.id'), nullable=False)
```

```
    department_name = db.Column(db.String(255), nullable=False)
```

```
    example_tasks = db.Column(db.Text, nullable=False)
```

```
    days_count = db.Column(db.Integer, nullable=False, default=0)
```

```
    learning_outcome = db.relationship('LearningOutcome', lazy='select')
```

```
# Backward-compat
```

```
@property
```

```
def zapis_id(self):  
  
    return self.enrollment_id
```

```
@zapis_id.setter
```

```
def zapis_id(self, v):
```

```
self.enrollment_id = v
```

```
@property
```

```
def efekt_id(self):
```

```
    return self.learning_outcome_id
```

```
@efekt_id.setter
```

```
def efekt_id(self, v):
```

```
    self.learning_outcome_id = v
```

```
@property
```

```
def efekt(self):
```

```
    return self.learning_outcome
```

```
@property
```

```
def nazwa_dzialu(self):
```

```
    return self.department_name
```

```
@nazwa_dzialu.setter
```

```
def nazwa_dzialu(self, v):
```

```
    self.department_name = v
```

```
@property
```

```
def przykladowe_prace(self):
```

```
    return self.example_tasks
```

```
@przykladowe_prace.setter
```

```
def przykladowe_prace(self, v):
```

```
    self.example_tasks = v
```

```
@property
```

```
def liczba_dni(self):
```

```
    return self.days_count
```

```

@liczba_dni.setter

def liczba_dni(self, v):

    self.days_count = v


class InternshipReport(db.Model):

    """Student's internship report."""

    __tablename__ = 'internship_reports'

    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),
    nullable=False, unique=True)

    workplace_description = db.Column(db.Text, nullable=True)

    work_analysis = db.Column(db.Text, nullable=True)

    updated_at = db.Column(db.DateTime, server_default=db.func.now(), onupdate=db.func.now())

    # Backward-compat

    @property

    def zapis_id(self):

        return self.enrollment_id

    @property

    def charakterystyka_miejsca(self):

        return self.workplace_description

    @charakterystyka_miejsca.setter

    def charakterystyka_miejsca(self, v):

        self.workplace_description = v

    @property

    def opis_i_analiza(self):

        return self.work_analysis

    @opis_i_analiza.setter

    def opis_i_analiza(self, v):

```



```
self.work_analysis = v
```

```
class IndividualProgram(db.Model):
```

```
    """Individual internship program (optional)."""
```

```
    __tablename__ = 'individual_programs'
```

```
    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id',  
ondelete='CASCADE'), nullable=False, unique=True)
```

```
    status = db.Column(db.String(30), nullable=False, default='DRAFT')
```

```
    approved_by_supervisor = db.Column(db.Boolean, default=False)
```

```
    approved_at = db.Column(db.DateTime, nullable=True)
```

```
    supervisor_comment = db.Column(db.Text, nullable=True)
```

```
    created_at = db.Column(db.DateTime, server_default=db.func.now())
```

```
    enrollment = db.relationship('InternshipEnrollment', backref=db.backref('individual_program', passive_deletes=True))
```

```
    # Backward-compatible
```

```
    @property
```

```
    def zapis_id(self):
```

```
        return self.enrollment_id
```

```
    @property
```

```
    def zatwierdzony_przez_uopz(self):
```

```
        return self.approved_by_supervisor
```

```
    @property
```

```
    def zatwierdzono_o(self):
```

```
        return self.approved_at
```

```
    @property
```

```
    def komentarz_uopz(self):
```

```
        return self.supervisor_comment
```

```

class DocumentNumber(db.Model):

    """Sequential administrative document number."""

    __tablename__ = 'document_numbers'

    id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    enrollment_id = db.Column(UUID(as_uuid=True), db.ForeignKey('internship_enrollments.id', ondelete='CASCADE'),
    nullable=False)

    document_type = db.Column(db.String(50), nullable=False)

    number = db.Column(db.String(100), nullable=False)

    generated_at = db.Column(db.DateTime, server_default=db.func.now())

    enrollment = db.relationship('InternshipEnrollment')

    # Backward-compat

    @property
    def zapis_id(self):

        return self.enrollment_id

    @property
    def typ_dokumentu(self):

        return self.document_type

    @property
    def numer(self):

        return self.number

    @property
    def wygenerowano(self):

        return self.generated_at

    # Backward-compat class aliases (remove after all routes migrated)

    Praktyka = Internship

    ZapisPraktyki = InternshipEnrollment

```

DaneMiejscaPraktyki	= WorkplaceDetails
UzasadnienieSciezki	= PathJustification
Sprawdzian	= Examination
OcenyKoncowe	= FinalGrades
ZdarzenieProces	= ProcessEvent
HarmonogramPraktyki	= InternshipSchedule
Sprawozdanie	= InternshipReport
IndywidualnyProgram	= IndividualProgram
NumerPisma	= DocumentNumber

PLIK: core\modele\uzytkownicy.py

```
"""core/modele/uzytkownicy.py
```

```
Domain models: System users.
```

```
Joined Table Inheritance (JTI): shared `users` table extended by
```

```
Student, Administrator, and UniversityMentor subclasses.
```

```
"""
```

```
import uuid
```

```
import enum
```

```
from flask_login import UserMixin
```

```
from sqlalchemy.dialects.postgresql import UUID
```

```
from core.extensions import db
```

```
# --- Enums
```

```
class UserRole(enum.Enum):
```

```
    STUDENT = 'STUDENT'
```

```
    UOPZ    = 'UOPZ'
```

```
    ADMIN   = 'ADMIN'
```

```
# --- Base model
```

```
class User(UserMixin, db.Model):
```

```
    """Shared table for all system accounts.
```

```
    Discriminator: `role` column - SQLAlchemy automatically returns
```

```
    the correct subclass when querying through the base class.
```

```
    """
```

```
    __tablename__ = 'users'
```

```
    __mapper_args__ = {
```

```

        'polymorphic_on': 'role',

        'polymorphic_identity': None,

    }

id = db.Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

email = db.Column(db.String(255), unique=True, nullable=False)

password_hash = db.Column(db.String(255), nullable=False)

first_name = db.Column(db.String(100), nullable=False)

last_name = db.Column(db.String(100), nullable=False)

role = db.Column(

    db.Enum(UserRole, name='user_role', values_callable=lambda e: [x.value for x in e]),

    nullable=False,

)

is_active = db.Column(db.Boolean, default=True)

require_password_change = db.Column(db.Boolean, default=True)

created_at = db.Column(db.DateTime, server_default=db.func.now())


def get_id(self) -> str:

    return str(self.id)


# Flask-Login requires `is_active` as a property – already a column, works fine.


@property

def imie(self):

    return self.first_name


@property

def nazwisko(self):

    return self.last_name


@property

def rola(self):

    return self.role

```

```
def __repr__(self) -> str:

    return f'<User {self.email} ({self.role})>'
```

— JTI subclasses —————

```
class Student(User):

    """Student-specific data – `students` table."""

    __tablename__ = 'students'

    id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='CASCADE'), primary_key=True)

    album_number = db.Column(db.String(20), nullable=True)

    gender = db.Column(db.String(1), nullable=True) # 'M' or 'F'

    field_of_study = db.Column(db.String(100), nullable=True)

    specialization = db.Column(db.String(100), nullable=True)

    study_mode = db.Column(db.String(20), nullable=True) # 'full-time'/'part-time'

    supervisor_id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='SET NULL'), nullable=True)

    __mapper_args__ = {

        'polymorphic_identity': UserRole.STUDENT,

        'inherit_condition': id == User.__table__.c.id,

    }

    # Backward-compat shims used in routes/templates that haven't been updated yet

    @property

    def numer_albumu(self):

        return self.album_number

    @property

    def plec(self):

        return self.gender

    @property

    def kierunek(self):
```

```
    return self.field_of_study
```

```
@property
```

```
def specjalnosc(self):
```

```
    return self.specialization
```

```
@property
```

```
def tryb_studiow(self):
```

```
    return self.study_mode
```

```
@property
```

```
def uopz_id(self):
```

```
    return self.supervisor_id
```

```
def __repr__(self) -> str:
```

```
    return f'<Student {self.email} album={self.album_number}>'
```

```
class Administrator(User):
```

```
    """Administrator account."""
```

```
    __tablename__ = 'administrators'
```

```
    __mapper_args__ = {'polymorphic_identity': UserRole.ADMIN}
```

```
id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='CASCADE'), primary_key=True)
```

```
def __repr__(self) -> str:
```

```
    return f'<Administrator {self.email}>'
```

```
class UniversityMentor(User):
```

```
    """University Internship Supervisor (UOPZ)."""
```

```
    __tablename__ = 'university_mentors'
```

```
    __mapper_args__ = {'polymorphic_identity': UserRole.UOPZ}
```

```
id = db.Column(UUID(as_uuid=True), db.ForeignKey('users.id', ondelete='CASCADE'), primary_key=True)
```

```
def __repr__(self) -> str:
```

```
    return f'<UniversityMentor {self.email}>'
```

```
# — Backward-compatible aliases (remove after all routes migrated) —————
```

```
Uzytkownik          = User
```

```
RolaUzytkownika     = UserRole
```

```
OpiekunUczelnianny  = UniversityMentor
```


PLIK: core\modele__init__.py

```
"""core/modele/__init__.py
```

Exports all domain models.

Flask-Migrate (Alembic) must see every db.Model class

before it can generate migration scripts – hence the star-style imports.

```
"""
```

```
from core.modele.uzytownicy import (
```

```
    UserRole,
```

```
    User,
```

```
    Student,
```

```
    Administrator,
```

```
    UniversityMentor,
```

```
    # backward-compat aliases
```

```
    RolaUzytkownika,
```

```
    Uzytkownik,
```

```
    OpikunUczelniany,
```

```
)
```

```
from core.modele.firmy import (
```

```
    Company,
```

```
    # backward-compat alias
```

```
    Firma,
```

```
)
```

```
from core.modele.praktyki import (
```

```
    InternshipStatus,
```

```
    EnrollmentStatus,
```

```
    InternshipPath,
```

```
    EventType,
```

```
    Internship,
```

```
    InternshipEnrollment,
```

```
    WorkplaceDetails,
```

```
PathJustification,

Examination,

FinalGrades,

ProcessEvent,

InternshipSchedule,

InternshipReport,

IndividualProgram,

DocumentNumber,

# backward-compatible aliases

StatusPraktyki,

StatusZapisu,

SciezkaPraktyki,

TypZdarzenia,

Praktyka,

ZapisPraktyki,

DaneMiejscPraktyki,

UzasadnienieSciezki,

Sprawdzian,

OcenyKoncowe,

ZdarzenieProces,

HarmonogramPraktyki,

Sprawozdanie,

IndywidualnyProgram,

NumerPisma,

)

from core.modele.dziennik import (

    AssessmentResult,

    LearningOutcome,

    entry_outcomes,

    JournalEntry,

    OutcomeAssessment,

    # backward-compatible aliases

    WynikOceny,
```

```
EfektUczenia,  
  
wpisy_efekty,  
  
WpisDziennika,  
  
OcenaPraktyki,  
  
)
```

```
from core.modele.dokumenty import (  
  
    DocumentStatus,  
  
    DocumentAuditLog,  
  
    UploadedDocument,  
  
    # backward-compat aliases  
  
    StatusDokumentu,  
  
    LogAudytuDokumentow,  
  
    DokumentPrzeslany,  
  
)
```

```
__all__ = [  
  
    # uzytkownicy – English  
  
    'UserRole', 'User', 'Student', 'Administrator', 'UniversityMentor',  
  
    # uzytkownicy – compat  
  
    'RolaUzytkownika', 'Uzytkownik', 'OpikunUczelniany',  
  
    # firmy – English  
  
    'Company',  
  
    # firmy – compat  
  
    'Firma',  
  
    # praktyki – English  
  
    'InternshipStatus', 'EnrollmentStatus', 'InternshipPath', 'EventType',  
  
    'Internship', 'InternshipEnrollment',  
  
    'WorkplaceDetails', 'PathJustification', 'Examination', 'FinalGrades',  
  
    'ProcessEvent', 'InternshipSchedule', 'InternshipReport',  
  
    'IndividualProgram', 'DocumentNumber',  
  
    # praktyki – compat  
  
    'StatusPraktyki', 'StatusZapisu', 'SciezkaPraktyki', 'TypZdarzenia',  
  
    'Praktyka', 'ZapisPraktyki',
```

```
'DaneMiejscaPraktyki', 'UzasadnienieSciezki', 'Sprawdzian', 'OcenyKoncowe',  
  
'ZdarzenieProces', 'HarmonogramPraktyki', 'Sprawozdanie',  
  
'IndywidualnyProgram', 'NumerPisma',  
  
# dziennik – English  
  
'AssessmentResult', 'LearningOutcome', 'entry_outcomes',  
  
'JournalEntry', 'OutcomeAssessment',  
  
# dziennik – compat  
  
'WynikOceny', 'EfektUczenia', 'wpisy_efekty', 'WpisDziennika', 'OcenaPraktyki',  
  
# dokumenty – English  
  
'DocumentStatus', 'DocumentAuditLog', 'UploadedDocument',  
  
# dokumenty – compat  
  
'StatusDokumentu', 'LogAudytuDokumentow', 'DokumentPrzeslany',
```

```
]
```

PLIK: core\repozytoria\dokumenty.py

```
"""core/repozytoria/dokumenty.py – Repozytoria logów audytu i dokumentów przesłanych."""
```

```
from __future__ import annotations
```

```
import uuid
```

```
from core.extensions import db
```

```
from core.modele import DocumentAuditLog, UploadedDocument
```

```
class RepozytoriumLogow:
```

```
    """Dostęp do dziennika audytu operacji na dokumentach."""
```

```
    def ostatnie_dla_zapisu(self, enrollment_id: uuid.UUID,
                            limit: int = 20) -> list[DocumentAuditLog]:
```

```
        return (
            db.session.query(DocumentAuditLog)
                .filter_by(enrollment_id=enrollment_id)
                .order_by(DocumentAuditLog.created_at.desc())
                .limit(limit)
                .all()
        )
```

```
class RepozytoriumDokumentowStudenta:
```

```
    """Dostęp do przesłanych dokumentów powiązanych z zapisem i studentem."""
```

```
    def dla_zapisu_studenta(self, enrollment_id: uuid.UUID,
                             student_id: uuid.UUID) -> list[UploadedDocument]:
```

```
        return (
            db.session.query(UploadedDocument)
                .filter_by(enrollment_id=enrollment_id, uploaded_by_id=student_id)
                .order_by(UploadedDocument.uploaded_at.desc())
                .all()
        )
```


PLIK: core\repozytoria\efekty.py

```
"""core/repozytoria/efekty.py – Repozytorium efektów uczenia się."""

from __future__ import annotations

from core.extensions import db

from core.modele.dziennik import LearningOutcome


class RepozytoriumEfektow:

    """Jedynie miejsce zapytań ORM dotyczących tabeli efektów uczenia się."""

    def wszystkie(self) -> list[LearningOutcome]:

        return db.session.execute(db.select(LearningOutcome).order_by(LearningOutcome.id)).scalars().all()

    def po_ids(self, ids: list[int]) -> list[LearningOutcome]:

        """Pobiera efekty po liście identyfikatorów (np. do przypisania M2M)."""

        if not ids:

            return []

        return db.session.execute(db.select(LearningOutcome).filter(LearningOutcome.id.in_(ids))).scalars().all()
```

PLIK: core\repozytoria\firmy.py

```
"""core/repozytoria/firmy.py – Repozytorium firm / zakładów pracy."""

from __future__ import annotations

import uuid

from typing import Optional

from core.extensions import db

from core.modele import Company, InternshipEnrollment, EnrollmentStatus


class RepozytoriumFirm:

    """Jedynie miejsce zapytań ORM dotyczących tabeli firm."""

    def znajdz_po_id(self, firma_id: uuid.UUID) -> Optional[Company]:

        return db.session.get(Company, firma_id)

    def aktywne(self) -> list[Company]:

        return db.session.query(Company).filter_by(is_active=True).order_by(Company.name).all()

    def lista_strona(self, szukaj: str = '', status: str = 'wszystkie',

                     strona: int = 1, na_strone: int = 25):

        q = db.session.query(Company)

        if status == 'aktywne':

            q = q.filter_by(is_active=True)

        elif status == 'nieaktywne':

            q = q.filter_by(is_active=False)

        if szukaj:

            q = q.filter(db.or_(

                Company.name.ilike(f'%{szukaj}%'),

                Company.address.ilike(f'%{szukaj}%'),

                Company.city.ilike(f'%{szukaj}%'),

                Company.tax_id.ilike(f'%{szukaj}%'),

            ))
```



```
return q.order_by(Company.name).paginate(page=strona, per_page=na_strone, error_out=False)
```

```
def znajdz_po_nazwie_aktywna(self, name: str,
```

```
        pominij_id: Optional[uuid.UUID] = None) -> Optional[Company]:
```

```
    q = db.session.query(Company).filter_by(name=name, is_active=True)
```

```
    if pominij_id is not None:
```

```
        q = q.filter(Company.id != pominij_id)
```

```
    return q.first()
```

```
def znajdz_po_nip_aktywna(self, tax_id: str,
```

```
        pominij_id: Optional[uuid.UUID] = None) -> Optional[Company]:
```

```
    q = db.session.query(Company).filter_by(tax_id=tax_id, is_active=True)
```

```
    if pominij_id is not None:
```

```
        q = q.filter(Company.id != pominij_id)
```

```
    return q.first()
```

```
def liczba_praktyk(self, firma_id: uuid.UUID) -> int:
```

```
    return db.session.query(InternshipEnrollment).filter_by(company_id=firma_id).count()
```

```
def liczba_aktywnych_praktyk(self, firma_id: uuid.UUID,
```

```
        aktywne_statusy: list) -> int:
```

```
    return (
```

```
        db.session.query(InternshipEnrollment)
```

```
        .filter_by(company_id=firma_id)
```

```
        .filter(InternshipEnrollment.status.in_(aktywne_statusy))
```

```
        .count()
```

```
)
```

```
def zapisz(self, firma: Company) -> Company:
```

```
    db.session.add(firma)
```

```
    db.session.flush()
```

```
    return firma
```

```
def usun(self, firma: Company) -> None:
```

```
db.session.delete(firma)
```

PLIK: core\repozytoria\oceny.py

```
"""core/repozytoria/oceny.py – Repozytorium ocen efektów uczenia się praktyk."""

from __future__ import annotations

import uuid

from core.extensions import db

from core.modele import OutcomeAssessment


class RepozytoriumOcen:

    """Jedynie miejsce zapytań ORM dotyczących tabeli ocen efektów uczenia się."""

    def dla_zapisu(self, enrollment_id: uuid.UUID) -> list[OutcomeAssessment]:

        return db.session.execute(

            db.select(OutcomeAssessment).filter_by(enrollment_id=enrollment_id)

        ).scalars().all()

    def zapisz(self, ocena: OutcomeAssessment) -> OutcomeAssessment:

        db.session.add(ocena)

        db.session.flush()

        return ocena
```

PLIK: core\repozytoria\praktyki.py

```
"""core/repozytoria/praktyki.py

Repozytoria edycji praktyk i zapisów studentów.

"""

from __future__ import annotations

from typing import Optional

import uuid

from sqlalchemy import func, case, exists

from core.extensions import db

from core.modele.praktyki import (
    Internship,
    InternshipStatus,
    InternshipEnrollment,
    EnrollmentStatus,
    InternshipSchedule,
    ProcessEvent,
    EventType,
)

class RepozytoriumPraktyk:

    """Dostęp do edycji praktyk (rok akademicki / semestr)."""

    def znajdz_po_id(self, praktyka_id: uuid.UUID) -> Optional[Internship]:

        return db.session.get(Internship, praktyka_id)

    def wszystkie(self) -> list[Internship]:

        return db.session.query(Internship).order_by(Internship.academic_year.desc(), Internship.semester).all()

    def aktywne(self) -> list[Internship]:
```

```
return (  
    db.session.query(Internship)  
        .filter_by(status=InternshipStatus.ACTIVE)  
        .order_by(Internship.academic_year.desc())  
        .all()  
)
```

```
def aktywna_edycja(self) -> Optional[Internship]:  
    """Zwraca pierwszą aktywną edycję praktyk lub None."""  
    return (  
        db.session.query(Internship)  
            .filter_by(status=InternshipStatus.ACTIVE)  
            .order_by(Internship.academic_year.desc())  
            .first()  
    )
```

```
def lista_strona(self, strona: int = 1, na_strone: int = 25):  
    return (  
        db.session.query(Internship)  
            .order_by(Internship.academic_year.desc(), Internship.semester)  
            .paginate(page=strona, per_page=na_strone, error_out=False)  
    )
```

```
def liczba_nieaktywnych(self) -> int:  
    return db.session.query(Internship).filter_by(status=InternshipStatus.INACTIVE).count()
```

```
def zapisz(self, praktyka: Internship) -> Internship:  
    db.session.add(praktyka)  
    db.session.flush()  
    return praktyka
```

```
def usun(self, praktyka: Internship) -> None:  
    db.session.delete(praktyka)
```

```
class RepozytoriumZapisow:

    """Dostęp do zapisów studentów na edycje praktyk."""

    def znajdz_po_id(self, zapis_id: uuid.UUID) -> Optional[InternshipEnrollment]:

        return db.session.get(InternshipEnrollment, zapis_id)

    def wszystkie(self) -> list[InternshipEnrollment]:

        return db.session.query(InternshipEnrollment).order_by(InternshipEnrollment.enrolled_at.desc()).all()

    def dla_studenta(self, student_id: uuid.UUID) -> list[InternshipEnrollment]:

        return (

            db.session.query(InternshipEnrollment)

            .filter_by(student_id=student_id)

            .order_by(InternshipEnrollment.enrolled_at.desc())

            .all()

        )

    def dla_praktyki(self, praktyka_id: uuid.UUID) -> list[InternshipEnrollment]:

        return (

            db.session.query(InternshipEnrollment)

            .filter_by(internship_id=praktyka_id)

            .order_by(InternshipEnrollment.enrolled_at.desc())

            .all()

        )

    def dla_opiekuna(self, uopz_id: uuid.UUID) -> list[InternshipEnrollment]:

        return (

            db.session.query(InternshipEnrollment)

            .filter_by(supervisor_id=uopz_id)

            .order_by(InternshipEnrollment.enrolled_at.desc())

            .all()

        )
```

```

def po_statusie(self, status: EnrollmentStatus) -> list[InternshipEnrollment]:

    return (

        db.session.query(InternshipEnrollment)

        .filter_by(status=status)

        .order_by(InternshipEnrollment.enrolled_at.desc())

        .all()

    )


def pending_dla_studenta_i_praktyki(self, student_id: uuid.UUID,

                                    praktyka_id: uuid.UUID) -> Optional[InternshipEnrollment]:

    """Zwraca PENDING zapis studenta dla danej edycji praktyki lub None."""

    return (

        db.session.query(InternshipEnrollment)

        .filter_by(student_id=student_id, internship_id=praktyka_id,

                    status=EnrollmentStatus.PENDING)

        .first()

    )


def student_ma_aktywny_zapis(self, student_id: uuid.UUID, praktyka_id: uuid.UUID) -> bool:

    """Sprawdza czy student ma już zapis do danej edycji (inny niż ODRZUCONY)."""

    q = (

        db.session.query(InternshipEnrollment)

        .filter_by(student_id=student_id, internship_id=praktyka_id)

        .filter(InternshipEnrollment.status != EnrollmentStatus.REJECTED)

    )

    return db.session.query(q.exists()).scalar()


def aktywny_dla_studenta(self, student_id: uuid.UUID,

                          statusy: list) -> Optional[InternshipEnrollment]:

    """Pierwszy aktywny zapis studenta (spośród podanych statusów)."""

    return (

        db.session.query(InternshipEnrollment)

        .filter(InternshipEnrollment.student_id == student_id,

                InternshipEnrollment.status.in_(statusy))

    )

```

```
        .first()

    )
```

```
def pierwszy_dla_studenta(self, student_id: uuid.UUID) -> Optional[InternshipEnrollment]:

    """Dowolny pierwszy zapis studenta (do sprawdzenia czy w ogóle istnieje)."""

    return db.session.query(InternshipEnrollment).filter_by(student_id=student_id).first()
```

```
def aktywne_dla_studenta(self, student_id: uuid.UUID,

                          statusy: list) -> list[InternshipEnrollment]:

    """Lista zapisów studenta z podanymi statusami, malejąco po dacie."""

    return (

        db.session.query(InternshipEnrollment)

        .filter(InternshipEnrollment.student_id == student_id,

                InternshipEnrollment.status.in_(statusy))

        .order_by(InternshipEnrollment.enrolled_at.desc())

        .all()

    )
```

```
def ostatni_dla_studenta(self, student_id: uuid.UUID) -> Optional[InternshipEnrollment]:

    """Ostatni zapis studenta (dowolny status), malejąco po dacie."""

    return (

        db.session.query(InternshipEnrollment)

        .filter_by(student_id=student_id)

        .order_by(InternshipEnrollment.enrolled_at.desc())

        .first()

    )
```

```
def w_trakcie_strona(self, szukaj: str = '', supervisor_id=None,

                    strona: int = 1, na_strone: int = 20):

    """Lista zapisów IN_PROGRESS z opcjonalnym filtrem UOPZ i wyszukiwaniem."""

    from sqlalchemy.orm import selectinload

    from core.modele.uzytownicy import User

    q = (

        db.session.query(InternshipEnrollment)
```



```

        .options(selectinload(InternshipEnrollment.student))

        .filter(InternshipEnrollment.status == EnrollmentStatus.IN_PROGRESS)

    )

    if supervisor_id is not None:

        q = q.filter_by(supervisor_id=supervisor_id)

    if szukaj:

        wzorzec = f'%{szukaj}%'

        q = q.join(User, InternshipEnrollment.student_id == User.id).filter(

            db.or_(User.first_name.ilike(wzorzec),

                    User.last_name.ilike(wzorzec))

        )

    return q.order_by(InternshipEnrollment.enrolled_at.desc()).paginate(

        page=strona, per_page=na_strone, error_out=False

    )

def aktywne_i_zakonczone(self, supervisor_id=None) -> list[InternshipEnrollment]:

    """Zapisy IN_PROGRESS i COMPLETED, opcjonalnie filtrowane do jednego UOPZ."""

    from core.modele uzytkownicy import User

    q = (

        db.session.query(InternshipEnrollment)

        .filter(InternshipEnrollment.status.in_(

            [EnrollmentStatus.IN_PROGRESS, EnrollmentStatus.COMPLETED]

        ))

    )

    if supervisor_id is not None:

        q = q.filter_by(supervisor_id=supervisor_id)

    return q.join(InternshipEnrollment.student).order_by(

        InternshipEnrollment.status.desc(),

        InternshipEnrollment.enrolled_at.desc()

    ).all()

def zakonczone_dla_uopz(self, supervisor_id) -> list[InternshipEnrollment]:

    return (

        db.session.query(InternshipEnrollment)

```

```

        .filter(InternshipEnrollment.status == EnrollmentStatus.COMPLETED,

                InternshipEnrollment.supervisor_id == supervisor_id)

        .all()

    )

def lista_zgloszen_strona(self, status_filter: str = '',

                           strona: int = 1, na_strone: int = 25):

    """Paginowana lista zgłoszeń ścieżki STANDARD z opcjonalnym filtrem statusu."""

    from sqlalchemy.orm import selectinload

    from core.modele.uzytownicy import User

    q = (

        db.session.query(InternshipEnrollment)

        .options(

            selectinload(InternshipEnrollment.student),

            selectinload(InternshipEnrollment.firma),

            selectinload(InternshipEnrollment.internship),

        )

        .join(User, InternshipEnrollment.student_id == User.id)

        .filter(InternshipEnrollment.path_type == 'STANDARD')

    )

    if status_filter:

        q = q.filter(InternshipEnrollment.status == EnrollmentStatus(status_filter))

    return q.order_by(InternshipEnrollment.enrolled_at.desc()).paginate(

        page=strona, per_page=na_strone, error_out=False

    )

def wnioski_komisja_strona(self, strona: int = 1, na_strone: int = 25):

    """Paginowana lista wniosków wymagających weryfikacji komisji (ścieżki B/C)."""

    from sqlalchemy.orm import selectinload

    from core.modele.uzytownicy import User

    ma_komentarz_uopz = exists().where(

        (ProcessEvent.enrollment_id == InternshipEnrollment.id) &

        (ProcessEvent.event_type == EventType.SUPERVISOR_COMMENT) &

        ProcessEvent.comment.isnot(None)

```

```

)

q = (
    db.session.query(InternshipEnrollment)
        .options(selectinload(InternshipEnrollment.student),
                  selectinload(InternshipEnrollment.firma))
        .join(User, InternshipEnrollment.student_id == User.id)
        .filter(InternshipEnrollment.path_type.in_(['EMPLOYMENT', 'OWN_BUSINESS']))
        .filter(db.or_(
            InternshipEnrollment.status == EnrollmentStatus.COMMISSION_REVIEW,
            InternshipEnrollment.status == EnrollmentStatus.REVISION_REQUIRED,
            (InternshipEnrollment.status == EnrollmentStatus.AWAITING_APPROVAL) & ma_komentarz_uopz,
        ))
)

return q.order_by(InternshipEnrollment.enrolled_at.desc()).paginate(
    page=strona, per_page=na_strone, error_out=False
)

```

```

def wnioski_dziekana_strona(self, strona: int = 1, na_strone: int = 25):
    """Paginowana lista wniosków oczekujących na decyzję dziekana."""
    from core.modele.uzytownicy import User

    q = (
        db.session.query(InternshipEnrollment)
            .join(User, InternshipEnrollment.student_id == User.id)
            .filter(InternshipEnrollment.status == EnrollmentStatus.DEAN_APPROVAL)
            .filter(InternshipEnrollment.path_type.in_(['EMPLOYMENT', 'OWN_BUSINESS']))
    )

    return q.order_by(InternshipEnrollment.enrolled_at.desc()).paginate(
        page=strona, per_page=na_strone, error_out=False
    )

```

```

def dla_uopz_strona(self, supervisor_id, status_filter: str = '',
                    strona: int = 1, na_strone: int = 25):
    """Paginowana lista zgłoszeń przypisanych do UOPZ."""

    q = db.session.query(InternshipEnrollment).filter(

```

```

        InternshipEnrollment.supervisor_id == supervisor_id

    )

    if status_filter:

        q = q.filter(InternshipEnrollment.status == EnrollmentStatus(status_filter))

    return q.order_by(InternshipEnrollment.enrolled_at.desc()).paginate(

        page=strona, per_page=na_strone, error_out=False

    )


def liczniki_dla_uopz(self, supervisor_id) -> dict:

    """Trzy liczniki statusów dla UOPZ – jedno zapytanie SQL."""

    row = (

        db.session.query(

            func.count().label('wszystkie'),

            func.count(case(

                (InternshipEnrollment.status == EnrollmentStatus.AWAITING_APPROVAL, 1)

            )).label('oczekujace'),

            func.count(case(

                (InternshipEnrollment.status == EnrollmentStatus.IN_PROGRESS, 1)

            )).label('zatwierdzone'),

        )

        .filter(InternshipEnrollment.supervisor_id == supervisor_id)

        .one()

    )

    return {

        'wszystkie': row.wszystkie,

        'oczekujace': row.oczekujace,

        'zatwierdzone': row.zatwierdzone,

    }

```

```

def liczniki_nav(self) -> dict:

    """Liczniki statusów dla paska nawigacji (inject_nav_counts) – jedno zapytanie."""

    row = (

        db.session.query(

            func.count(case(

```

```

        (InternshipEnrollment.status == EnrollmentStatus.AWAITING_APPROVAL, 1)

    ).label('oczekujace'),

    func.count(case(

        (InternshipEnrollment.status == EnrollmentStatus.COMMISSION_REVIEW, 1)

    ).label('komisja'),

    func.count(case(

        (InternshipEnrollment.status == EnrollmentStatus.DEAN_APPROVAL, 1)

    ).label('dziekan'),

    )

    .one()

)

return {

    'nav_oczekujace': row.oczekujace,

    'nav_komisja':    row.komisja,

    'nav_dziekan':    row.dziekan,

}

```

```

def liczba_aktywnych_dla_studentow(self, student_ids: list,

                                   aktywne_statusy: list) -> dict:

    """Zwraca {student_id: count} dla listy studentów – eliminuje N+1."""

    if not student_ids:

        return {}

    rows = (

        db.session.query(

            InternshipEnrollment.student_id,

            func.count(InternshipEnrollment.id).label('cnt')

        )

        .filter(InternshipEnrollment.student_id.in_(student_ids),

                InternshipEnrollment.status.in_(aktywne_statusy))

        .group_by(InternshipEnrollment.student_id)

        .all()

    )

    return {r.student_id: r.cnt for r in rows}

```

```

def statystyki_pulpit(self, supervisor_id=None) -> dict:

    """Liczniki dla panelu głównego – dwa liczniki w jednym zapytaniu."""

    q = db.session.query(InternshipEnrollment)

    if supervisor_id is not None:

        q = q.filter(db.or_(

            InternshipEnrollment.supervisor_id == supervisor_id,

            InternshipEnrollment.status == EnrollmentStatus.PENDING,

        ))

    row = q.with_entities(

        func.count(case((InternshipEnrollment.status == EnrollmentStatus.IN_PROGRESS, 1))).label('aktywne'),

        func.count(case((InternshipEnrollment.status == EnrollmentStatus.COMPLETED, 1))).label('zakończone'),

    ).one()

    return {'praktyki_aktywne': row.aktywne, 'oczekujace_oceny': row.zakończone}


def ostatnie(self, supervisor_id=None, limit: int = 8) -> list[InternshipEnrollment]:

    """Ostatnie zapisy (dla widżetu na pulpicie)."""

    q = db.session.query(InternshipEnrollment)

    if supervisor_id is not None:

        q = q.filter(db.or_(

            InternshipEnrollment.supervisor_id == supervisor_id,

            InternshipEnrollment.status == EnrollmentStatus.PENDING,

        ))

    return q.order_by(InternshipEnrollment.enrolled_at.desc()).limit(limit).all()


# — Harmonogramy (powiązane ściśle z zapisami) —————


def harmonogram_dla_zapisu(self, enrollment_id: uuid.UUID) -> list[InternshipSchedule]:

    return (

        db.session.query(InternshipSchedule)

        .filter_by(enrollment_id=enrollment_id)

        .all()

    )


def usun_harmonogram(self, enrollment_id: uuid.UUID) -> None:

```

```
db.session.query(InternshipSchedule).filter_by(enrollment_id=enrollment_id).delete()
```

```
def zapisz(self, zapis: InternshipEnrollment) -> InternshipEnrollment:
```

```
    db.session.add(zapis)
```

```
    db.session.flush()
```

```
    return zapis
```

```
def usun(self, zapis: InternshipEnrollment) -> None:
```

```
    db.session.delete(zapis)
```

PLIK: core\repozytoria\uzytkownicy.py

```
"""core/repozytoria/uzytkownicy.py
```

Repozytorium użytkowników – jedyne miejsce zapytań ORM

dotyczących tabeli `uzytkownicy` i jej podtabel JTII.

```
"""
```

```
from __future__ import annotations
```

```
from typing import Optional
```

```
import uuid
```

```
from core.extensions import db
```

```
from core.modele.uzytkownicy import UserRole, User, Student, Administrator, UniversityMentor
```

```
# Potrzebne do metod pomocniczych
```

```
import uuid as _uuid
```

```
class RepozytoriumUzytkownikow:
```

```
    """Dostęp do danych użytkowników systemu."""
```

```
    # — Wyszukiwanie —————
```

```
    def znajdz_po_id(self, uzytkownik_id: uuid.UUID) -> Optional[User]:
```

```
        return db.session.get(User, uzytkownik_id)
```

```
    def znajdz_po_emailu(self, email: str) -> Optional[User]:
```

```
        return db.session.query(User).filter_by(email=email).first()
```

```
    def wszyscy(self) -> list[User]:
```

```
        return db.session.query(User).order_by(User.last_name, User.first_name).all()
```

```
    def aktywni(self) -> list[User]:
```

```
        return db.session.query(User).filter_by(is_active=True).order_by(User.last_name).all()
```


— Według roli —————

```
def wszyscy_studenci(self) -> list[Student]:  
    return db.session.query(Student).order_by(User.last_name, User.first_name).all()
```

```
def wszyscy_administratorzy(self) -> list[Administrator]:  
    return db.session.query(Administrator).order_by(User.last_name).all()
```

```
def wszyscy_opiekunowie(self) -> list[UniversityMentor]:  
    return db.session.query(UniversityMentor).order_by(User.last_name).all()
```

```
def opiekunowie_aktywni(self) -> list[UniversityMentor]:  
    return (  
        db.session.query(UniversityMentor)  
        .filter(User.is_active.is_(True))  
        .order_by(User.last_name)  
        .all()  
    )
```

— Zapis / usunięcie —————

```
def zapisz(self, uzytkownik: User) -> User:  
    db.session.add(uzytkownik)  
    db.session.flush()  
    return uzytkownik
```

```
def usun(self, uzytkownik: User) -> None:  
    db.session.delete(uzytkownik)
```

```
def aktywni_uopz(self) -> list[User]:  
    """Lista aktywnych opiekunów uczelnianych posortowana po nazwisku."""  
    return (  
        db.session.query(User)
```

```

        .filter_by(role=UserRole.UOPZ, is_active=True)

        .order_by(User.last_name, User.first_name)

        .all()

    )

```

```

def liczba_aktywnych_studentow(self) -> int:

    return db.session.query(User).filter_by(

        role=UserRole.STUDENT, is_active=True

    ).count()

```

```

def szukaj_strona(self, szukaj: str = '', filtr_rola: str = '',

                    strona: int = 1, na_strone: int = 25):

    """Paginowana lista użytkowników z opcjonalnym wyszukiwaniem i filtrem roli."""

    q = db.session.query(User)

    if szukaj:

        studenci_ids = db.session.query(Student.id).filter(

            Student.album_number.ilike(f'%{szukaj}%')

        ).subquery()

        q = q.filter(db.or_(

            User.first_name.ilike(f'%{szukaj}%'),

            User.last_name.ilike(f'%{szukaj}%'),

            User.email.ilike(f'%{szukaj}%'),

            User.id.in_(studenci_ids),

        ))

    if filtr_rola:

        try:

            q = q.filter(User.role == UserRole[filtr_rola])

        except KeyError:

            pass

    return q.order_by(User.last_name, User.first_name).paginate(

        page=strona, per_page=na_strone, error_out=False

    )

```

```

def studenci_strona(self, szukaj: str = '', supervisor_id=None,

```

```

        strona: int = 1, na_strone: int = 25):

    """Paginowana lista studentów z opcjonalnym filtrem UOPZ i wyszukiwaniem."""

    q = db.session.query(Student)

    if supervisor_id is not None:

        q = q.filter(Student.supervisor_id == supervisor_id)

    if szukaj:

        album_sub = db.session.query(Student.id).filter(

            Student.album_number.ilike(f'%{szukaj}%')

        ).subquery()

        q = q.filter(db.or_(

            Student.first_name.ilike(f'%{szukaj}%'),

            Student.last_name.ilike(f'%{szukaj}%'),

            Student.id.in_(album_sub),

        ))

    return q.order_by(Student.last_name, Student.first_name).paginate(

        page=strona, per_page=na_strone, error_out=False

    )

def znajdz_studenta_po_albumie(self, album_number: str) -> Optional[Student]:

    return db.session.query(Student).filter_by(album_number=album_number).first()

def znajdz_istniejace_po_email_lub_albumie(self, emails: list, albums: list):

    """Batch-lookup emaili i numerów albumów – eliminuje N+1 przy imporcie CSV."""

    return (

        db.session.query(User.email, Student.album_number)

        .outerjoin(Student, User.id == Student.id)

        .filter(db.or_(

            User.email.in_(emails),

            Student.album_number.in_(albums)

        ))

        .all()

    )

```

```
def istnieje_email(self, email: str, pominij_id: Optional[uuid.UUID] = None) -> bool:

    q = db.session.query(User).filter_by(email=email)

    if pominij_id is not None:

        q = q.filter(User.id != pominij_id)

    return db.session.query(q.exists()).scalar()
```

PLIK: core\repozytoria\wpisy.py

```
"""core/repozytoria/wpisy.py – Repozytorium wpisów dziennika praktyk."""

from __future__ import annotations

from typing import Optional

from sqlalchemy import func

from core.extensions import db

from core.modele.dziennik import JournalEntry


class RepozytoriumWpisow:

    """Jedynie miejsce zapytań ORM dotyczących tabeli wpisów dziennika."""

    def dla_zapisu(self, enrollment_id, malejaco: bool = True) -> list[JournalEntry]:

        """Wszystkie wpisy dla danego zapisu, posortowane po dacie."""

        order = JournalEntry.entry_date.desc() if malejaco else JournalEntry.entry_date

        return (

            db.session.query(JournalEntry)

                .filter_by(enrollment_id=enrollment_id)

                .order_by(order)

                .all()

        )

    def statystyki_dla_zapisow(self, ids: list) -> dict:

        """Zwraca {enrollment_id: (max_date, count)} dla listy zapisów – jedno zapytanie SQL.

        Eliminuje pętlę N+1 w widoku listy dzienników.

        """

        if not ids:

            return {}

        rows = (

            db.session.query(

                JournalEntry.enrollment_id,
```

```

        func.max(JournalEntry.entry_date).label('ostatni'),

        func.count(JournalEntry.id).label('liczba'),

    )

    .filter(JournalEntry.enrollment_id.in_(ids))

    .group_by(JournalEntry.enrollment_id)

    .all()

)

return {r.enrollment_id: (r.ostatni, r.liczba) for r in rows}

```

```

def znajdz_duplikat(self, enrollment_id, data) -> Optional[JournalEntry]:

```

```

    """Sprawdza czy wpis na dany dzień już istnieje."""

    return (

        db.session.query(JournalEntry)

        .filter_by(enrollment_id=enrollment_id, entry_date=data)

        .first()

    )

```

```

def zapisz(self, wpis: JournalEntry) -> JournalEntry:

```

```

    db.session.add(wpis)

    db.session.flush()

    return wpis

```

PLIK: core\repozytoria__init__.py

"""core/repozytoria – warstwa dostępu do danych (Repository Pattern).

Każda klasa repozytorium jest jedynym miejscem, w którym

konstruowane są zapytania ORM do konkretnej domeny.

Kontrolery i serwisy nigdy nie wywołują db.session.query() bezpośrednio.

"""

```
from core.repozytoria uzytkownicy import RepozytoriumUzytkownikow
```

```
from core.repozytoria.praktyki import RepozytoriumPraktyk, RepozytoriumZapisow
```

```
from core.repozytoria.wpisy import RepozytoriumWpisow
```

```
from core.repozytoria.efekty import RepozytoriumEfektow
```

```
from core.repozytoria.firmy import RepozytoriumFirm
```

```
from core.repozytoria.dokumenty import RepozytoriumLogow, RepozytoriumDokumentowStudenta
```

```
from core.repozytoria.oceny import RepozytoriumOcen
```

```
__all__ = [
```

```
    'RepozytoriumUzytkownikow',
```

```
    'RepozytoriumPraktyk',
```

```
    'RepozytoriumZapisow',
```

```
    'RepozytoriumWpisow',
```

```
    'RepozytoriumEfektow',
```

```
    'RepozytoriumFirm',
```

```
    'RepozytoriumLogow',
```

```
    'RepozytoriumDokumentowStudenta',
```

```
    'RepozytoriumOcen',
```

```
]
```

PLIK: core\static\css\autoryzacja.css

```
/*  
  
autoryzacja.css  
  
Jedyny arkusz stylów dla ekranów autoryzacji (logowanie, zmiana hasła).  
  
Cel: jedno podejście bez rozproszenia stylów po wielu plikach.  
  
*/  
  
:root {  
  
  --kolor-glowny:          #1a3a5c;  
  
  --kolor-glowny-ciemny:  #0f2540;  
  
  --kolor-glowny-jasny:   #2a5080;  
  
  --kolor-akcent:         #c8963e;  
  
  --kolor-akcent-jasny:   #e8b45a;  
  
  
  --kolor-tlo:            #f4f6f9;  
  
  --kolor-biale:          #ffffff;  
  
  --kolor-tekst:          #1a1a2e;  
  
  --kolor-tekst-drugor:   #4a5568;  
  
  --kolor-tekst-trzeciorzed: #718096;  
  
  
  --kolor-ramka:          #d1d9e6;  
  
  --kolor-blad:           #c0392b;  
  
  --kolor-sukces:         #1e6e3a;  
  
  --kolor-info:           #1a5276;  
  
  
  --fokus-kolor:          #c8963e;  
  
  --fokus-grubosc:        3px;  
  
  --fokus-offset:         3px;  
  
  
  --font-glowny:          'IBM Plex Sans', 'Segoe UI', Tahoma, sans-serif;  
  
  
  --promien:              8px;  
  
  --cien-karta:           0 2px 12px rgba(26, 58, 92, 0.12);  
  
}
```



```
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
```

```
html {  
  
    font-size: 100%;  
  
    -webkit-text-size-adjust: 100%;  
  
    scroll-behavior: smooth;  
  
}
```

```
body {  
  
    font-family: var(--font-glowny), serif;  
  
    font-size: 1rem;  
  
    line-height: 1.6;  
  
    color: var(--kolor-tekst);  
  
    background-color: var(--kolor-tlo);  
  
    min-height: 100vh;  
  
}
```

```
:focus-visible {  
  
    outline: var(--fokus-grubosc) solid var(--fokus-kolor);  
  
    outline-offset: var(--fokus-offset);  
  
    border-radius: calc(var(--promien) - 2px);  
  
}
```

```
:focus:not(:focus-visible) { outline: none; }
```

```
.skip-link {  
  
    position: absolute; top: -100px; left: 1rem; z-index: 9999;  
  
    padding: 0.75rem 1.5rem; background: var(--kolor-glowny); color: var(--kolor-biale);  
  
    font-weight: 600; border-radius: var(--promien); text-decoration: none; transition: top 0.2s;  
  
}
```

```
.skip-link:focus { top: 1rem; }
```

```
.strona-logowania { background: var(--kolor-tlo); }
```

```
.panel-logowania { display: grid; grid-template-columns: 1fr 1fr; min-height: 100vh; }
```

```
.panel-logowania__branding { background: var(--kolor-glowny); display: flex; align-items: center; justify-content: center; padding: 3rem 2.5rem; position: relative; overflow: hidden; }

.panel-logowania__branding::before { content: ''; position: absolute; inset: 0; background-image: repeating-linear-gradient(45deg, transparent, transparent 40px, rgba(255,255,255,0.025) 40px, rgba(255,255,255,0.025) 41px); pointer-events: none; }

.panel-logowania__branding::after { content: ''; position: absolute; top: 0; right: 0; width: 6px; height: 100%; background: linear-gradient(to bottom, var(--kolor-akcent) 0%, var(--kolor-akcent-jasny) 50%, var(--kolor-akcent) 100%); }

.branding__tresc { position: relative; z-index: 1; display: flex; flex-direction: column; align-items: flex-start; gap: 1.5rem; max-width: 320px; }

.branding__logo-kontener { background: white; padding: 10px; border-radius: var(--promien); display: flex; align-items: center; justify-content: center; box-shadow: var(--cien-karta); }

.branding__logo { max-width: 200px; height: auto; display: block; }

.branding__separator { width: 48px; height: 3px; background: var(--kolor-akcent); border-radius: 2px; }

.branding__nazwa-systemu { font-size: 1.5rem; font-weight: 600; color: var(--kolor-biale); line-height: 1.3; letter-spacing: -0.01em; }

.branding__podtytul { font-size: 0.9rem; color: rgba(255,255,255,0.65); letter-spacing: 0.08em; text-transform: uppercase; }

.branding__dekoracja { display: flex; gap: 8px; margin-top: 1rem; }

.branding__dekoracja span { display: block; width: 8px; height: 8px; border-radius: 50%; background: var(--kolor-akcent); opacity: 0.6; }

.branding__dekoracja span:first-child { opacity: 1; }

.branding__dekoracja span:last-child { opacity: 0.3; }

.panel-logowania__formularz { display: flex; align-items: center; justify-content: center; padding: 3rem 2rem; background: var(--kolor-biale); }

.formularz__kontener { width: 100%; max-width: 420px; }

.formularz__naglowek { margin-bottom: 2rem; }

.formularz__tytul { font-size: 1.75rem; font-weight: 600; color: var(--kolor-glowny); line-height: 1.2; letter-spacing: -0.02em; margin-bottom: 0.5rem; }

.formularz__opis { font-size: 0.95rem; color: var(--kolor-tekst-dlugor); }

.komunikaty { margin-bottom: 1.5rem; display: flex; flex-direction: column; gap: 0.5rem; }

.komunikat { display: flex; align-items: center; gap: 0.5rem; padding: 0.5rem 0.75rem; border-radius: 10px; font-size: 0.9rem; line-height: 1.2; border-left: 4px solid; }

.komunikat--danger { background: #fdf2f2; border-color: var(--kolor-blad); color: #8b1a1a; }

.komunikat--success { background: #f0faf4; border-color: var(--kolor-sukces); color: #14452a; }

.komunikat--info { background: #f0f6fc; border-color: var(--kolor-info); color: #0f3449; }

.komunikat__ikona { font-size: 0.85rem; font-weight: 700; flex-shrink: 0; }
```

```

.formularz__forma { display: flex; flex-direction: column; gap: 1.25rem; }

.pole-formularza { display: flex; flex-direction: column; gap: 0.375rem; }

.pole-formularza__etykieta { font-size: 0.875rem; font-weight: 500; color: var(--kolor-tekst); }

.pole-formularza__wymagane { color: var(--kolor-blad); margin-left: 0.2em; font-weight: 700; }


.pole-formularza__input { width: 100%; padding: 0.75rem 1rem; font-family: var(--font-glowny), serif; font-size: 1rem;
color: var(--kolor-tekst); background: var(--kolor-biale); border: 2px solid var(--kolor-ramka); border-radius:
var(--promien); transition: border-color 0.15s, box-shadow 0.15s; }

.pole-formularza__input:hover { border-color: var(--kolor-glowny-jasny); }

.pole-formularza__input:focus { border-color: var(--kolor-glowny); box-shadow: 0 0 0 3px rgba(26,58,92,0.15); }

.pole-formularza--blad .pole-formularza__input { border-color: var(--kolor-blad); }


.pole-formularza__input-wrapper { position: relative; display: flex; align-items: center; }

.pole-formularza__input-wrapper .pole-formularza__input { padding-right: 3rem; }

.pole-formularza__toggle-haslo { position: absolute; right: 0.75rem; background: none; border: none; cursor: pointer;
padding: 0.25rem; color: var(--kolor-tekst-drugor); border-radius: var(--promien); font-size: 1rem; line-height: 1; }

.pole-formularza__toggle-haslo:hover { color: var(--kolor-glowny); }

.pole-formularza__blad { font-size: 0.825rem; color: var(--kolor-blad); font-weight: 500; }


.pole-formularza--checkbox { flex-direction: row; }

.pole-formularza__etykieta-checkbox { display: flex; align-items: center; gap: 0.625rem; cursor: pointer; font-size:
0.9rem; color: var(--kolor-tekst-drugor); }

.pole-formularza__checkbox { width: 18px; height: 18px; flex-shrink: 0; cursor: pointer; accent-color:
var(--kolor-glowny); }


.przycisk-logowania { width: 100%; padding: 0.875rem 1.5rem; margin-top: 0.5rem; font-family: var(--font-glowny), serif;
font-size: 1rem; font-weight: 600; color: var(--kolor-biale); background: var(--kolor-glowny); border: none;
border-radius: var(--promien); cursor: pointer; letter-spacing: 0.02em; transition: background-color 0.15s, transform
0.1s; min-height: 48px; }

.przycisk-logowania:hover { background: var(--kolor-glowny-ciemny); }

.przycisk-logowania:active { transform: translateY(1px); }


.formularz__stopka { margin-top: 2rem; padding-top: 1.5rem; border-top: 1px solid var(--kolor-ramka); display: flex;
flex-direction: column; gap: 0.375rem; }

.formularz__stopka-tekst { font-size: 0.825rem; color: var(--kolor-tekst-trzeciorzed); text-align: center; }

@media (max-width: 900px) {

    .panel-logowania { grid-template-columns: 1fr; }

```

```

.panel-logowania__branding { padding: 2.5rem 2rem; min-height: auto; }

.branding__tresc { flex-direction: row; flex-wrap: wrap; align-items: center; max-width: 100%; gap: 1rem; }

.branding__separator, .branding__podtytul, .branding__dekoracja { display: none; }

.branding__nazwa-systemu { font-size: 1.1rem; }

}

@media (max-width: 480px) {

    .panel-logowania__formularz { padding: 1.5rem 1rem; }

    .formularz__kontener { max-width: 100%; }

    .formularz__tytul { font-size: 1.4rem; }

}

/* — Przycisk logowania przez Microsoft ————— */

.logowanie-microsoft { margin: 1.5rem 0; }

.przycisk-microsoft {

    display: flex;

    align-items: center;

    justify-content: center;

    gap: 0.875rem;

    width: 100%;

    padding: 0.875rem 1.5rem;

    font-family: var(--font-glowny), serif;

    font-size: 1rem;

    font-weight: 600;

    color: #1a1a1a;

    background: #ffffff;

    border: 1.5px solid #d1d9e6;

    border-radius: var(--promien);

    cursor: pointer;

    letter-spacing: 0.01em;

    text-decoration: none;

    transition: background-color 0.15s, border-color 0.15s, box-shadow 0.15s;

    min-height: 48px;

    box-shadow: 0 1px 3px rgba(0,0,0,0.08);

```

```
}
```

```
.przycisk-microsoft:hover {
```

```
    background: #f4f6f9;
```

```
    border-color: #a0aec0;
```

```
    box-shadow: 0 2px 6px rgba(0,0,0,0.12);
```

```
}
```

```
.przycisk-microsoft:active { transform: translateY(1px); }
```

```
.przycisk-microsoft__logo {
```

```
    width: 21px;
```

```
    height: 21px;
```

```
    flex-shrink: 0;
```

```
}
```

PLIK: core\static\css\modul\dashboard.css

```
/* dashboard.css – Statystyki, strona błędu */
```

```
.statystyki-grid, .siatka-statystyk { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 1.5rem; margin-bottom: 2rem; }
```

```
.stat-karta, .karta-stat { background: var(--kolor-biale); padding: 1.5rem; border-radius: var(--promien-duzy); border: 1px solid var(--kolor-ramka); box-shadow: var(--cien-karta); display: flex; flex-direction: column; align-items: center; text-align: center; gap: 0.5rem; }
```

```
.stat-karta__etykieta, .karta-stat__etykieta { font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.08em; color: var(--kolor-tekst-drugor); }
```

```
.stat-karta__wartosc, .karta-stat__liczba { font-size: 2rem; font-weight: 700; color: var(--kolor-glowny); line-height: 1; }
```

```
.karta-stat__ikona { color: var(--kolor-akcent); line-height: 1; }
```

```
.karta-stat__ikona svg { width: 28px; height: 28px; }
```

```
@media (max-width: 768px) { .statystyki-grid { grid-template-columns: 1fr 1fr; } }
```

```
.strona-bledu { display: flex; flex-direction: column; align-items: center; justify-content: center; text-align: center; padding: 4rem 2rem; min-height: 60vh; }
```

```
.strona-bledu__kod { font-size: 5rem; font-weight: 800; color: var(--kolor-ramka); line-height: 1; margin-bottom: 1rem; }
```

```
.strona-bledu__tytul { font-size: 1.5rem; font-weight: 600; color: var(--kolor-tekst); margin-bottom: 0.5rem; }
```

```
.strona-bledu__opis { color: var(--kolor-tekst-drugor); max-width: 400px; margin: 0 auto; }
```

PLIK: core\static\css\modul\formularze.css

```
/* formularze.css – Pola formularzy, inputy, checkboxy */
```

```
.formularz__forma { display: flex; flex-direction: column; gap: 1.25rem; }

.formularz-kontener { display: flex; justify-content: center; }

.formularz-kontener > .karta { width: 100%; max-width: 640px; }

.pole-formularza { display: flex; flex-direction: column; gap: 0.375rem; }

.pole-formularza__naglowek-haslo { display: flex; align-items: center; justify-content: space-between; }

.pole-formularza__etykieta { font-size: 0.875rem; font-weight: 500; color: var(--kolor-tekst); }

.pole-formularza__wymagane { color: var(--kolor-blad); margin-left: 0.2em; font-weight: 700; }

.pole-formularza__input { width: 100%; padding: 0.75rem 1rem; font-family: var(--font-glowny), serif; font-size: 1rem;
color: var(--kolor-tekst); background: var(--kolor-biale); border: 2px solid var(--kolor-ramka); border-radius:
var(--promien); transition: border-color 0.15s, box-shadow 0.15s; }

.pole-formularza__input:hover { border-color: var(--kolor-glowny-jasny); }

.pole-formularza__input:focus { border-color: var(--kolor-glowny); box-shadow: 0 0 0 3px rgba(26,58,92,0.15); outline:
var(--fokus-grubosc) solid var(--fokus-kolor); outline-offset: var(--fokus-offset); }

.pole-formularza--blad .pole-formularza__input { border-color: var(--kolor-blad); }

.pole-formularza--blad .pole-formularza__input:focus { box-shadow: 0 0 0 3px rgba(192,57,43,0.15); }

.pole-formularza__input-wrapper { position: relative; display: flex; align-items: center; }

.pole-formularza__input-wrapper .pole-formularza__input { padding-right: 3rem; }

.pole-formularza__toggle-haslo { position: absolute; right: 0.75rem; background: none; border: none; cursor: pointer;
padding: 0.25rem; color: var(--kolor-tekst-drugor); border-radius: var(--promien); font-size: 1rem; line-height: 1;
transition: color 0.15s; }

.pole-formularza__toggle-haslo:hover { color: var(--kolor-glowny); }

.pole-formularza__blad { font-size: 0.825rem; color: var(--kolor-blad); font-weight: 500; }

.pole-formularza--checkbox { flex-direction: row; }

.pole-formularza__etykieta-checkbox { display: flex; align-items: center; gap: 0.625rem; cursor: pointer; font-size:
0.9rem; color: var(--kolor-tekst-drugor); }

.pole-formularza__checkbox { width: 18px; height: 18px; flex-shrink: 0; cursor: pointer; accent-color:
var(--kolor-glowny); }
```

PLIK: core\static\css\modul\karty.css

```
/* karty.css – Komponent karty (.karta) */
```

```
.karta { background: var(--kolor-biale); border: 1px solid var(--kolor-ramka); border-radius: var(--promien-duzy);  
box-shadow: var(--cien-karta); overflow: hidden; }
```

```
.karta__naglowek { padding: 1rem 1.5rem; border-bottom: 1px solid var(--kolor-ramka); display: flex; align-items: center;  
justify-content: space-between; gap: 1rem; flex-wrap: wrap; }
```

```
.karta__tytul { font-size: 1rem; font-weight: 600; color: var(--kolor-tekst); }
```

```
.karta__tresc { padding: 1.5rem; }
```


PLIK: core\static\css\modul\komponenty.css

/* komponenty.css – Pasek postępu, paginacja, filtry */

```
.pasek-postep { height: 6px; background: var(--kolor-tlo); border-radius: 10px; overflow: hidden; }
```

```
.pasek-postep__wypelnienie { height: 100%; background: var(--kolor-glowny); border-radius: 10px; transition: width 0.4s ease; }
```

```
.pasek-postep__wypelnienie--zakonczone { background: var(--kolor-sukces); }
```

```
.paginacja { display: flex; align-items: center; justify-content: space-between; padding: 1rem 1.5rem; border-top: 1px solid var(--kolor-ramka); }
```

```
.paginacja__info { font-size: 0.8rem; color: var(--kolor-tekst-drugor); }
```

```
.paginacja__przyciski { display: flex; align-items: center; gap: 0.25rem; }
```

```
.paginacja__przycisk { display: inline-flex; align-items: center; justify-content: center; min-width: 32px; height: 32px; padding: 0 0.5rem; font-size: 0.8rem; font-weight: 500; text-decoration: none; color: var(--kolor-tekst-drugor); background: var(--kolor-biale); border: 1px solid var(--kolor-ramka); border-radius: var(--promien); transition: background 0.15s; }
```

```
.paginacja__przycisk:hover { background: var(--kolor-tlo); }
```

```
.paginacja__przycisk--aktywny { background: var(--kolor-glowny); color: var(--kolor-biale); border-color: var(--kolor-glowny); }
```

```
.paginacja__separator { padding: 0 0.25rem; color: var(--kolor-tekst-trzeciorzed); }
```

```
.pasek-filtrow { display: flex; align-items: center; gap: 0.75rem; flex-wrap: wrap; width: 100%; }
```

```
.filtr-szukaj { display: flex; align-items: center; gap: 0.5rem; flex: 1; min-width: 200px; background: var(--kolor-tlo); border: 1px solid var(--kolor-ramka); border-radius: var(--promien); padding: 0 0.75rem; }
```

```
.filtr-szukaj__ikona { color: var(--kolor-tekst-trzeciorzed); font-size: 0.85rem; flex-shrink: 0; }
```

```
.filtr-szukaj__input { flex: 1; border: none; background: transparent; padding: 0.5rem 0; font-family: var(--font-glowny); font-size: 0.85rem; color: var(--kolor-tekst); outline: none; }
```

```
.filtr-select { padding: 0.5rem 0.75rem; font-family: var(--font-glowny); font-size: 0.85rem; color: var(--kolor-tekst); background: var(--kolor-tlo); border: 1px solid var(--kolor-ramka); border-radius: var(--promien); cursor: pointer; }
```

PLIK: core\static\css\modul\komunikaty.css

/* komunikaty.css – Komunikaty flash i powiadomienia */

```
.komunikaty { margin-bottom: 1.5rem; display: flex; flex-direction: column; gap: 0.5rem; }
```

```
.komunikat { display: flex; align-items: center; gap: 0.5rem; padding: 0.5rem 0.75rem; border-radius: 10px; font-size: 0.9rem; line-height: 1.2; border-left: 4px solid; }
```

```
.komunikat--danger { background: #fdf2f2; border-color: var(--kolor-blad); color: #8b1a1a; }
```

```
.komunikat--success { background: #f0faf4; border-color: var(--kolor-sukces); color: #14452a; }
```

```
.komunikat--info { background: #f0f6fc; border-color: var(--kolor-info); color: #0f3449; }
```

```
.komunikat__ikona { font-size: 0.85rem; font-weight: 700; flex-shrink: 0; }
```

```
.komunikat__tekst { flex: 1; }
```

```
.komunikaty--fixed { position: fixed; right: 1rem; bottom: 1rem; z-index: 1200; width: calc(100% - 2rem); max-width: 360px; }
```

```
.komunikat--minimal { box-shadow: 0 4px 16px rgba(10,10,20,0.08); }
```

```
.komunikat__zamknij { margin-left: auto; background: none; border: none; cursor: pointer; font-size: 1.1rem; color: inherit; opacity: 0.5; padding: 0 0.5rem; line-height: 1; }
```

```
.komunikat__zamknij:hover { opacity: 1; }
```

PLIK: core\static\css\modul\logowanie.css

```
/* logowanie.css – Strona logowania i zmiana hasła */
```

```
.strona-logowania { background: var(--kolor-tlo); }
```

```
.panel-logowania { display: grid; grid-template-columns: 1fr 1fr; min-height: 100vh; }
```

```
.panel-logowania__branding { background: var(--kolor-glowny); display: flex; align-items: center; justify-content: center; padding: 3rem 2.5rem; position: relative; overflow: hidden; }
```

```
.panel-logowania__branding::before { content: ''; position: absolute; inset: 0; background-image: repeating-linear-gradient(45deg, transparent, transparent 40px, rgba(255,255,255,0.025) 40px, rgba(255,255,255,0.025) 41px); pointer-events: none; }
```

```
.panel-logowania__branding::after { content: ''; position: absolute; top: 0; right: 0; width: 6px; height: 100%; background: linear-gradient(to bottom, var(--kolor-akcent) 0%, var(--kolor-akcent-jasny) 50%, var(--kolor-akcent) 100%); }
```

```
.branding__tresc { position: relative; z-index: 1; display: flex; flex-direction: column; align-items: flex-start; gap: 1.5rem; max-width: 320px; }
```

```
.branding__logo-kontener { background: white; padding: 10px; border-radius: var(--promien); display: flex; align-items: center; justify-content: center; box-shadow: var(--cien-karta); }
```

```
.branding__logo { max-width: 200px; height: auto; display: block; }
```

```
.branding__separator { width: 48px; height: 3px; background: var(--kolor-akcent); border-radius: 2px; }
```

```
.branding__nazwa-systemu { font-size: 1.5rem; font-weight: 600; color: var(--kolor-biale); line-height: 1.3; letter-spacing: -0.01em; }
```

```
.branding__podtytul { font-size: 0.9rem; color: rgba(255,255,255,0.65); letter-spacing: 0.08em; text-transform: uppercase; }
```

```
.branding__dekoracja { display: flex; gap: 8px; margin-top: 1rem; }
```

```
.branding__dekoracja span { display: block; width: 8px; height: 8px; border-radius: 50%; background: var(--kolor-akcent); opacity: 0.6; }
```

```
.branding__dekoracja span:first-child { opacity: 1; }
```

```
.branding__dekoracja span:last-child { opacity: 0.3; }
```

```
.panel-logowania__formularz { display: flex; align-items: center; justify-content: center; padding: 3rem 2rem; background: var(--kolor-biale); }
```

```
.formularz__kontener { width: 100%; max-width: 420px; }
```

```
.formularz__naglowek { margin-bottom: 2rem; }
```

```
.formularz__tytul { font-size: 1.75rem; font-weight: 600; color: var(--kolor-glowny); line-height: 1.2; letter-spacing: -0.02em; margin-bottom: 0.5rem; }
```

```
.formularz__opis { font-size: 0.95rem; color: var(--kolor-tekst-drugor); }
```

```
.przycisk-logowania { width: 100%; padding: 0.875rem 1.5rem; margin-top: 0.5rem; font-family: var(--font-glowny), serif; }
```

```
font-size: 1rem; font-weight: 600; color: var(--kolor-biale); background: var(--kolor-glowny); border: none;
border-radius: var(--promien); cursor: pointer; letter-spacing: 0.02em; transition: background-color 0.15s, transform
0.1s; min-height: 48px; }
```

```
.przycisk-logowania:hover { background: var(--kolor-glowny-ciemny); }
```

```
.przycisk-logowania:active { transform: translateY(1px); }
```

```
.przycisk-logowania:focus-visible { outline: var(--fokus-grubosc) solid var(--fokus-kolor); outline-offset:
var(--fokus-offset); }
```

```
.formularz__stopka { margin-top: 2rem; padding-top: 1.5rem; border-top: 1px solid var(--kolor-ramka); display: flex;
flex-direction: column; gap: 0.375rem; }
```

```
.formularz__stopka-tekst { font-size: 0.825rem; color: var(--kolor-tekst-trzeciorzed); text-align: center; }
```

```
.formularz__link { color: var(--kolor-glowny-jasny); text-decoration: underline; text-underline-offset: 2px; font-weight:
500; }
```

```
.formularz__link:hover { color: var(--kolor-glowny); }
```

```
@media (max-width: 900px) {
```

```
  .panel-logowania { grid-template-columns: 1fr; }
```

```
  .panel-logowania__branding { padding: 2.5rem 2rem; min-height: auto; }
```

```
  .branding__tresc { flex-direction: row; flex-wrap: wrap; align-items: center; max-width: 100%; gap: 1rem; }
```

```
  .branding__separator, .branding__podtytul, .branding__dekoracja { display: none; }
```

```
  .branding__nazwa-systemu { font-size: 1.1rem; }
```

```
}
```

```
@media (max-width: 480px) {
```

```
  .panel-logowania__formularz { padding: 1.5rem 1rem; }
```

```
  .formularz__kontener { max-width: 100%; }
```

```
  .formularz__tytul { font-size: 1.4rem; }
```

```
}
```

PLIK: core\static\css\modul\przyciski.css

/* przyciski.css – Przyciski i grupy akcji */

```
.przycisk { display: inline-flex; align-items: center; gap: 0.375rem; padding: 0.5rem 1rem; font-family: var(--font-glowny); font-size: 0.85rem; font-weight: 600; text-decoration: none; border: none; border-radius: var(--promien); cursor: pointer; transition: background 0.15s, color 0.15s, box-shadow 0.15s; min-height: 36px; white-space: nowrap; }
```

```
.przycisk--glowny { background: var(--kolor-glowny); color: var(--kolor-biale); }
```

```
.przycisk--glowny:hover { background: var(--kolor-glowny-ciemny); }
```

```
.przycisk--drugorzeczny { background: var(--kolor-tlo); color: var(--kolor-tekst); border: 1px solid var(--kolor-ramka); }
```

```
.przycisk--drugorzeczny:hover { background: var(--kolor-ramka); }
```

```
.przycisk--ostrzezenie { background: #f59e0b; color: white; border: 1px solid #f59e0b; }
```

```
.przycisk--ostrzezenie:hover { background: #d97706; border-color: #d97706; }
```

```
.przycisk--niebezpieczny { background: var(--kolor-blad); color: var(--kolor-biale); }
```

```
.przycisk--niebezpieczny:hover { background: #a93226; }
```

```
.przycisk--maly { padding: 0.3rem 0.625rem; font-size: 0.75rem; min-height: 28px; }
```

```
.akcje-tabeli { display: flex; align-items: center; gap: 0.375rem; }
```

```
.akcje-panelu { display: flex; align-items: center; gap: 0.5rem; }
```

PLIK: core\static\css\modul\statusy.css

```
/* statusy.css – Odznaki statusów i ról */
```

```
.status { display: inline-block; padding: 0.25rem 0.625rem; font-size: 0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.04em; border-radius: 100px; white-space: nowrap; }
```

```
.status--draft, .status--szkic { background: #e2e8f0; color: var(--kolor-tekst-drugor); }
```

```
.status--planned, .status--zaplanowana { background: #dbeafe; color: #1e40af; }
```

```
.status--in-progress, .status--w-trakcie { background: #fef3c7; color: #92400e; }
```

```
.status--pending-evaluation, .status--oczekuje-oceny { background: #fce7f3; color: #9d174d; }
```

```
.status--completed, .status--zakonczone { background: #d1fae5; color: var(--kolor-sukces); }
```

```
.rola-odznaka { display: inline-block; padding: 0.2rem 0.5rem; font-size: 0.7rem; font-weight: 600; border-radius: 100px; text-transform: uppercase; letter-spacing: 0.03em; }
```

```
.rola-odznaka--admin { background: #fef3c7; color: #92400e; }
```

```
.rola-odznaka--uopz { background: #dbeafe; color: #1e40af; }
```

```
.rola-odznaka--zopz { background: #d1fae5; color: var(--kolor-sukces); }
```

```
.rola-odznaka--student { background: #e2e8f0; color: var(--kolor-tekst-drugor); }
```

PLIK: core\static\css\modul\tabele.css

```
/* tabele.css – Tabele danych */
```

```
.tabela-wrapper { overflow-x: auto; }
```

```
.tabela { width: 100%; border-collapse: collapse; font-size: 0.875rem; }
```

```
.tabela thead { background: var(--kolor-tlo); }
```

```
.tabela th { padding: 0.75rem 1rem; text-align: left; font-weight: 600; font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em; color: var(--kolor-tekst-drugor); border-bottom: 2px solid var(--kolor-ramka); white-space: nowrap; }
```

```
.tabela td { padding: 0.875rem 1rem; border-bottom: 1px solid var(--kolor-ramka); vertical-align: middle; color: var(--kolor-tekst); }
```

```
.tabela tbody tr:hover { background: rgba(26,58,92,0.03); }
```

```
.tabela--pusta { text-align: center; color: var(--kolor-tekst-trzeciorzed); padding: 2rem 1rem !important; font-style: italic; }
```

```
.komorka-uzytkownik { display: flex; flex-direction: column; }
```

```
.komorka-uzytkownik__nazwa { font-weight: 600; }
```

```
.komorka-uzytkownik__info { font-size: 0.75rem; color: var(--kolor-tekst-trzeciorzed); }
```

PLIK: core\static\css\modul\uklad.css

/* uklad.css – Layout panelu: sidebar, panel-main, nagłówek */

```
.panel-body { display: flex; min-height: 100vh; background: var(--kolor-tlo); }
```

```
.sidebar {  
    width: 260px; height: 100vh; background: var(--kolor-glowny);  
  
    color: rgba(255,255,255,0.85); display: flex; flex-direction: column;  
  
    flex-shrink: 0; position: sticky; top: 0; overflow-y: auto;
```

```
}
```

```
.sidebar__naglowek { padding: 1.5rem 1.25rem 1rem; border-bottom: 1px solid rgba(255,255,255,0.1); }
```

```
.sidebar__logo { max-width: 100%; height: auto; display: block; }
```

```
.sidebar__system-label { display: block; font-size: 0.7rem; text-transform: uppercase; letter-spacing: 0.12em; color:  
var(--kolor-akcent); font-weight: 600; margin-top: 0.5rem; }
```

```
.sidebar__uzytkownik { display: flex; align-items: center; gap: 0.75rem; padding: 1.25rem; border-bottom: 1px solid  
rgba(255,255,255,0.08); }
```

```
.sidebar__uzytkownik-avatar { width: 38px; height: 38px; border-radius: 50%; background: var(--kolor-akcent); color:  
var(--kolor-glowny-ciemny); display: flex; align-items: center; justify-content: center; font-weight: 700; font-size:  
0.8rem; flex-shrink: 0; }
```

```
.sidebar__uzytkownik-info { display: flex; flex-direction: column; min-width: 0; }
```

```
.sidebar__uzytkownik-imie { font-size: 0.85rem; font-weight: 600; color: #fff; white-space: nowrap; overflow: hidden;  
text-overflow: ellipsis; }
```

```
.sidebar__uzytkownik-rola { font-size: 0.7rem; color: rgba(255,255,255,0.5); text-transform: uppercase; letter-spacing:  
0.05em; }
```

```
.sidebar__menu { list-style: none; padding: 0.75rem 0; flex: 1; }
```

```
.sidebar__menu-sekcja { font-size: 0.65rem; text-transform: uppercase; letter-spacing: 0.1em; color:  
rgba(255,255,255,0.35); padding: 1.25rem 1.25rem 0.375rem; font-weight: 600; }
```

```
.sidebar__menu-pozycja { margin: 1px 0; }
```

```
.sidebar__link { display: flex; align-items: center; gap: 0.75rem; padding: 0.625rem 1.25rem; color:  
rgba(255,255,255,0.75); text-decoration: none; font-size: 0.875rem; font-weight: 500; border-left: 3px solid transparent;  
transition: background 0.15s, color 0.15s, border-color 0.15s; }
```

```
.sidebar__link:hover { background: rgba(255,255,255,0.08); color: #fff; }
```

```
.sidebar__link--aktywny { background: rgba(255,255,255,0.12); color: #fff; border-left-color: var(--kolor-akcent);  
font-weight: 600; }
```

```
.sidebar__ikona { display: inline-flex; align-items: center; justify-content: center; width: 20px; height: 20px;  
flex-shrink: 0; }
```

```
.sidebar__ikona svg { width: 18px; height: 18px; fill: none; stroke: currentColor; stroke-width: 2; stroke-linecap:  
round; stroke-linejoin: round; }
```



```
.sidebar__stopka { padding: 1rem 1.25rem; border-top: 1px solid rgba(255,255,255,0.08); margin-top: auto; }

.sidebar__wylogowanie { display: flex; align-items: center; gap: 0.5rem; width: 100%; padding: 0.625rem 0.75rem;
background: rgba(255,255,255,0.06); border: 1px solid rgba(255,255,255,0.1); border-radius: var(--promien); color:
rgba(255,255,255,0.7); font-family: var(--font-glowny); font-size: 0.85rem; font-weight: 500; cursor: pointer;
transition: background 0.15s, color 0.15s; }

.sidebar__wylogowanie:hover { background: rgba(192,57,43,0.3); color: #fff; }

.sidebar__wylogowanie svg { width: 16px; height: 16px; fill: none; stroke: currentColor; stroke-width: 2; stroke-linecap:
round; stroke-linejoin: round; }

.panel-main { flex: 1; display: flex; flex-direction: column; min-width: 0; }

.panel-main__naglowek { background: var(--kolor-biale); border-bottom: 1px solid var(--kolor-ramka); padding: 1.25rem
2rem; }

.panel-main__naglowek-tresc { display: flex; align-items: center; justify-content: space-between; gap: 1rem; }

.panel-main__tytul { font-size: 1.35rem; font-weight: 600; color: var(--kolor-tekst); letter-spacing: -0.01em; }

.panel-main__tresc { flex: 1; padding: 2rem; }

@media (max-width: 768px) {

    .sidebar { display: none; }

    .panel-main__tresc { padding: 1rem; }

    .panel-main__naglowek { padding: 1rem; }

}
```

PLIK: core\static\css\modul\zmiennie.css

```
/* =====  
  
zmiennie.css – Zmienne CSS, reset, podstawy, dostępność  
  
System Praktyk Zawodowych – ANS Elbląg  
  
WCAG 2.1 AA: kontrast ≥ 4.5:1, fokus widoczny  
  
===== */  
  
:root {  
  
  --kolor-glowny:      #1a3a5c;  
  
  --kolor-glowny-ciemny: #0f2540;  
  
  --kolor-glowny-jasny: #2a5080;  
  
  --kolor-akcent:      #c8963e;  
  
  --kolor-akcent-jasny: #e8b45a;  
  
  
  --kolor-tlo:         #f4f6f9;  
  
  --kolor-biale:        #ffffff;  
  
  --kolor-tekst:        #1a1a2e;  
  
  --kolor-tekst-drugor: #4a5568;  
  
  --kolor-tekst-trzeciorzed: #718096;  
  
  
  --kolor-ramka:        #d1d9e6;  
  
  --kolor-blad:         #c0392b;  
  
  --kolor-sukces:       #1e6e3a;  
  
  --kolor-info:         #1a5276;  
  
  
  --fokus-kolor:        #c8963e;  
  
  --fokus-grubosc:      3px;  
  
  --fokus-offset:       3px;  
  
  
  --font-glowny:        'IBM Plex Sans', 'Segoe UI', Tahoma, sans-serif;  
  
  --font-mono:          'IBM Plex Mono', 'Courier New', monospace;  
  
  
  --promien:            8px;  
  
  --promien-duzy:       16px;
```

```

--cien-karta:    0 2px 12px rgba(26, 58, 92, 0.12);

--cien-glowny:   0 8px 32px rgba(26, 58, 92, 0.18);

}

*, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }

html {

    font-size: 100%;

    -webkit-text-size-adjust: 100%;

    scroll-behavior: smooth;

}

@media (prefers-reduced-motion: reduce) {

    *, ::before, ::after {

        animation-duration: 0.01ms !important;

        transition-duration: 0.01ms !important;

    }

}

body {

    font-family: var(--font-glowny), serif;

    font-size: 1rem;

    line-height: 1.6;

    color: var(--kolor-tekst);

    background-color: var(--kolor-tlo);

    min-height: 100vh;

}

.skip-link {

    position: absolute; top: -100px; left: 1rem; z-index: 9999;

    padding: 0.75rem 1.5rem; background: var(--kolor-glowny); color: var(--kolor-biale);

    font-weight: 600; border-radius: var(--promien); text-decoration: none; transition: top 0.2s;

}

.skip-link:focus { top: 1rem; }

```

```
:focus-visible {  
  
    outline: var(--fokus-grubosc) solid var(--fokus-kolor);  
  
    outline-offset: var(--fokus-offset);  
  
    border-radius: calc(var(--promien) - 2px);  
  
}  
  
:focus:not(:focus-visible) { outline: none; }  
  
  
@media (prefers-contrast: more) {  
  
    :root { --kolor-ramka: #000; --fokus-kolor: #000; --fokus-grubosc: 4px; }  
  
    .pole-formularza__input { border-width: 2px; }  
  
}
```

PLIK: core\templates\auth\zmien_haslo.html

```
{% extends "layouts/base_auth.html" %}

{% block tytul %}Ustaw hasło{% endblock %}


{% block body %}

<div class="panel-logowania" role="main">


    <!-- Lewa kolumna: branding -->

    <aside class="panel-logowania__branding" aria-label="Informacje o instytucji">

        <div class="branding__tresc">

            <div class="branding__logo-kontener">

                

            </div>

            <div class="branding__separator" aria-hidden="true"></div>

            <p class="branding__nazwa-systemu">Pierwsza konfiguracja</p>

            <p class="branding__podtytul">Ustaw własne hasło</p>

        </div>

    </aside>


    <!-- Prawa kolumna: formularz -->

    <main class="panel-logowania__formularz" id="main-content">

        <div class="formularz__kontener">

            <div class="formularz__naglowek">

                <h1 class="formularz__tytul">Ustaw własne hasło</h1>

                <p class="formularz__opis">

                    Twoje konto wymaga ustawienia hasła przed kontynuowaniem.

                    Hasło musi mieć minimum 8 znaków.

                </p>

            </div>

        </div>

    </main>

    {% with messages = get_flashed_messages(with_categories=true) %}

        {% if messages %}
```

```

<div class="komunikaty" aria-live="polite">

    {% for kategoria, wiadomosc in messages %}

    <div class="komunikat komunikat--{{ kategoria }}" role="alert">

        <span class="komunikat__ikona" aria-hidden="true">

            {% if kategoria == 'success' %}{% elif kategoria == 'danger' %}{% else %}i{% endif %}

        </span>

        {{ wiadomosc }}

    </div>

    {% endfor %}

</div>

{% endif %}

{% endwith %}

<form method="POST" class="formularz__forma" novalidate>

    {{ form.hidden_tag() }}

    <div class="pole-formularza {% if form.nowe_haslo.errors %}pole-formularza--blad{% endif %}">

        <label for="nowe_haslo" class="pole-formularza__etykieta">

            Nowe hasło <span class="pole-formularza__wymagane" aria-label="wymagane">*</span>

        </label>

        <div class="pole-formularza__input-wrapper">

            <input type="password" id="nowe_haslo" name="nowe_haslo"

                class="pole-formularza__input" required

                autocomplete="new-password" minlength="8"

                {% if form.nowe_haslo.errors %}aria-invalid="true" aria-describedby="haslo-blad"{% endif %}>

            <button type="button" class="pole-formularza__toggle-haslo"

                aria-label="Pokaż hasło" aria-pressed="false"

                onclick="toggleHaslo(this, 'nowe_haslo')">

                <span aria-hidden="true"></span>

            </button>

        </div>

        {% if form.nowe_haslo.errors %}

            <span id="haslo-blad" class="pole-formularza__blad" role="alert">{{ form.nowe_haslo.errors[0] }}</span>

        {% endif %}

```

```
</div>
```

```
<div class="pole-formularza {% if form.potwierdzenie.errors %}pole-formularza--blad{% endif %}">
```

```
<label for="potwierdzenie" class="pole-formularza__etykieta">
```

```
    Powtórz hasło <span class="pole-formularza__wymagane" aria-label="wymagane">*</span>
```

```
</label>
```

```
<div class="pole-formularza__input-wrapper">
```

```
<input type="password" id="potwierdzenie" name="potwierdzenie"
```

```
    class="pole-formularza__input" required autocomplete="new-password"
```

```
    {% if form.potwierdzenie.errors %}aria-invalid="true" aria-describedby="pow-blad"{% endif %}>
```

```
</div>
```

```
{% if form.potwierdzenie.errors %}
```

```
<span id="pow-blad" class="pole-formularza__blad" role="alert">{{ form.potwierdzenie.errors[0] }}</span>
```

```
{% endif %}
```

```
</div>
```

```
<button type="submit" class="przycisk-logowania">
```

```
    Ustaw hasło i przejdź do systemu
```

```
</button>
```

```
</form>
```

```
</div>
```

```
</main>
```

```
</div>
```

```
{% endblock %}
```

```
{% block scripts_extra %}
```

```
<script>
```

```
function toggleHaslo(btn, id) {
```

```
    const inp = document.getElementById(id);
```

```
    const ukryte = inp.type === 'password';
```

```
    inp.type = ukryte ? 'text' : 'password';
```

```
    btn.setAttribute('aria-pressed', ukryte ? 'true' : 'false');
```

```
    btn.setAttribute('aria-label', ukryte ? 'Ukryj hasło' : 'Pokaż hasło');  
  
}  
  
</script>  
  
{% endblock %}
```


PLIK: core\templates\errors\blad.html

```
{% extends "layouts/base_auth.html" %}
```

```
{% block tytul %}Błąd {{ kod }}{% endblock %}
```

```
{% block body %}
```

```
    {% set url_glowna = url_for('dashboard.index') %}
```

```
    {% include "errors/blad_bazowy.html" %}
```

```
{% endblock %}
```

PLIK: core\templates\errors\blad_bazowy.html

```
{# core/templates/errors/blad_bazowy.html
```

```
    Parametryczny fragment błędu – includowany przez blad.html każdej aplikacji.
```

```
    Zmienne: kod, tytuł, opis, url_glowna, tekst_przycisku #}
```

```
<div class="strona-bledu">
```

```
    <div class="strona-bledu__kod" aria-hidden="true">{{ kod }}</div>
```

```
    <h1 class="strona-bledu__tytul">{{ tytul }}</h1>
```

```
    <p class="strona-bledu__opis">{{ opis }}</p>
```

```
    <div style="margin-top: 1.5rem;">
```

```
        <a href="{{ url_glowna }}" class="przycisk przycisk--glowny">{{ tekst_przycisku }}</a>
```

```
    </div>
```

```
</div>
```

PLIK: core\templates\layouts\base_auth.html

```
<!DOCTYPE html>

<html lang="pl">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>{% block tytuł %}Autoryzacja{% endblock %} – ANS Elbląg</title>

    <link rel="stylesheet" href="{% url_for('core_static.static', filename='css/autoryzacja.css') %}">

    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&display=swap" rel="stylesheet">

    {% block head_extra %}{% endblock %}

</head>

<body class="strona-logowania">

    <a href="#main-content" class="skip-link">Przejdź do treści głównej</a>

    <main id="main-content" class="auth-main">

        {% block body %}{% endblock %}

    </main>

    {% block scripts_extra %}{% endblock %}

</body>

</html>
```

PLIK: core\uslugi\dokumenty.py

```
"""
```

```
core/uslugi/dokumenty.py
```

Warstwa domenowa dokumentów praktyki.

Zawiera dwie odpowiedzialności:

1. Polityka dostępności dokumentów (DocumentEntry + _DOCUMENT_POLICY)
 - zastępuje powielone reguły z app_admin i app_student.
2. Kanoniczny builder kontekstu TeX (buduj_kontekst)
 - jedno miejsce zamiast trzech niezależnych implementacji w:
 - app_admin/routes/zarzadzanie/dokumenty_studentow.py (_build_context)
 - app_student/routes/dokumenty.py (_build_context / _serialize_context)
 - celery_app.py (generate_pdf_dziennik – inline)

Oba kontrolery importują z tego modułu i nie zawierają własnej logiki

budowania kontekstu ani reguł dostępności.

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```
from datetime import date, datetime
```

```
from typing import Callable
```

```
from core.extensions import db
```

```
# -----
```

```
# Mapa szablonów (canonical – jeden słownik dla całej aplikacji)
```

```
# -----
```

```
DOC_CONFIG: dict[str, tuple[str, str]] = {
```

```
    'ZAL_1': ('zal1_porozumienie.tex.j2', 'zal1_porozumienie.pdf'),
```

```
    'ZAL_2': ('zal2_program.tex.j2', 'zal2_program.pdf'),
```

```
    'ZAL_2A': ('zal2a_program.tex.j2', 'zal2a_program.pdf'),
```

```

'ZAL_3': ('zal3_karta.tex.j2', 'zal3_karta.pdf'),
'ZAL_4': ('zal4_efekty.tex.j2', 'zal4_efekty.pdf'),
'ZAL_4A': ('zal4a_komisja.tex.j2', 'zal4a_komisja.pdf'),
'ZAL_4B': ('zal4b_wniosek.tex.j2', 'zal4b_wniosek.pdf'),
'ZAL_6': ('zal6_dziennik.tex.j2', 'zal6_dziennik.pdf'),
'ZAL_7': ('zal7_sprawozdanie.tex.j2', 'zal7_sprawozdanie.pdf'),
'ZAL_7A': ('zal7a_sprawozdanie.tex.j2', 'zal7a_sprawozdanie.pdf'),
'ZAL_8': ('zal8_protokol.tex.j2', 'zal8_protokol.pdf'),
'ZAL_9': ('zal9_oswiadczenie.tex.j2', 'zal9_oswiadczenie.pdf'),
}

```

```

STATIC_TEMPLATES: dict[str, tuple[str, str]] = {
    'ankieta': ('zal5_ankieta.tex.j2', 'zal5_ankieta.pdf'),
}

```

```

# -----
# Polityka dostępności dokumentów (Specification Pattern)
# -----

```

```
@dataclass
```

```

class DocumentEntry:
    """Deklaracja jednego dokumentu i warunków jego dostępności.

    available_when: callable(ctx) → bool
    unavailable_reason: callable(ctx) → str | None

    """
    name: str

    doc_type: str | None = None # None → dokument statyczny
    description: str | None = None
    static_key: str | None = None

    available_when: Callable = field(default=lambda _ctx: True)
    unavailable_reason: Callable = field(default=lambda _ctx: None)

```

```

def resolve(self, ctx: dict) -> dict:

    available = self.available_when(ctx)

    entry = {

        'nazwa':    self.name,

        'opis':     self.description,

        'dostepny': available,

        'powod':    None if available else self.unavailable_reason(ctx),

    }

    if self.static_key:

        entry.update({'staly': True, 'klucz_staly': self.static_key})

    else:

        entry.update({'dynamiczny': True, 'typ': self.doc_type,

                     'zapis_id': ctx['zapis_id']})

    return entry


def _sep(name: str) -> dict:

    return {'separator': True, 'nazwa': name}


# Reguły dostępności – czytelne funkcje zamiast anonimowych lambdy

def _zawsze(ctx: dict) -> bool:            return True                                # noqa: E704

def _w_trakcie_lub_zakonczona(ctx):        return ctx['w_trakcie'] or ctx['zakonczone']    # noqa: E704

def _zakonczone(ctx):                     return ctx['zakonczone']                    # noqa: E704

def _oceniona(ctx):                       return ctx['oceniona']                      # noqa: E704

def _ma_harmonogram(ctx):                  return ctx['harmonogram_count'] > 0          # noqa: E704

def _firma_custom(ctx):                   return ctx['firma_custom']                  # noqa: E704

def _firma_bez_umowy(ctx):                 return ctx['firma_bez_umowy']                # noqa: E704

def _dziekan_lub_zakonczona(ctx):          return ctx['dziekan_zatwierdził'] or ctx['zakonczone']  # noqa: E704


_POWOD_HARMONOGRAM    = lambda _: 'Wymaga wypełnionego harmonogramu'          # noqa: E731

_POWOD_W_TRAKCIE      = lambda _: 'Dostępny po zatwierdzeniu praktyki'        # noqa: E731

_POWOD_ZAKONCZONE     = lambda _: 'Dostępny po zakończeniu praktyki'          # noqa: E731

_POWOD_OCENIONA       = lambda _: 'Dostępny po wystawieniu oceny przez UOPZ'  # noqa: E731

```

_POWOD_DZIEKAN = lambda _: 'Dostępny po decyzji dziekana'

noqa: E731

def _docs_standard() -> list:

return [

DocumentEntry('Zał. 9 – Oświadczenie instytucji', 'ZAL_9',

'Do wypełnienia przez zakład pracy',

available_when=_firma_custom),

DocumentEntry('Zał. 1 – Porozumienie uczelnia ↔ zakład', 'ZAL_1',

'Dla firm bez stałej umowy z ANS',

available_when=_firma_bez_umowy),

DocumentEntry('Zał. 2 – Program praktyki', 'ZAL_2',

'Z danymi studenta i firmy'),

DocumentEntry('Zał. 2a – Indywidualny Program Praktyk', 'ZAL_2A',

'Harmonogram efektów – student + UOPZ + ZOPZ',

available_when=_ma_harmonogram,

unavailable_reason=_POWOD_HARMONOGRAM),

DocumentEntry('Zał. 3 – Karta praktyki / Skierowanie', 'ZAL_3',

'Z danymi studenta, firmy i ZOPZ'),

DocumentEntry('Zał. 6 – Dziennik praktyki', 'ZAL_6',

'Generowany z wpisów dziennika',

available_when=_w_trakcie_lub_zakonczona,

unavailable_reason=_POWOD_W_TRAKCIE),

_sep('Pakiet końcowy'),

DocumentEntry('Zał. 7 – Sprawozdanie końcowe', 'ZAL_7',

'Podpisuje student',

available_when=_zakonczona,

unavailable_reason=_POWOD_ZAKONCZONA),

DocumentEntry('Zał. 4 – Potwierdzenie efektów uczenia się', 'ZAL_4',

'Podpisuje ZOPZ + UOPZ',

available_when=_zakonczona,

unavailable_reason=_POWOD_ZAKONCZONA),

_sep('Po egzaminie komisji'),

DocumentEntry('Zał. 8 – Protokół egzaminu komisji', 'ZAL_8',

```

        available_when=_oceniona,

        unavailable_reason=_POWOD_OCENIONA),

    DocumentEntry('Zał. 5 - Ankieta oceny praktyki', static_key='ankieta',

        description='Formularz anonimowej ankiety'),

]

```

```

def _docs_employment_own_business() -> list:

    return [

        DocumentEntry('Zał. 4b - Wniosek o zaliczenie', 'ZAL_4B',

            'Praca etatowa / własna działalność'),

        DocumentEntry('Zał. 7a - Sprawozdanie z pracy/działalności', 'ZAL_7A',

            'Zatwierdza przełożony/UOPZ',

            available_when=_w_trakcie_lub_zakonczona,

            unavailable_reason=_POWOD_W_TRAKCIE),

        DocumentEntry('Zał. 4a - Potwierdzenie efektów (komisja)', 'ZAL_4A',

            '13 efektów: uzyskał / częściowo / nie',

            available_when=_dziekan_lub_zakonczona,

            unavailable_reason=_POWOD_DZIEKAN),

        _sep('Po egzaminie komisji'),

        DocumentEntry('Zał. 5 - Ankieta', static_key='ankieta'),

        DocumentEntry('Zał. 8 - Protokół egzaminu komisji', 'ZAL_8',

            available_when=_dziekan_lub_zakonczona,

            unavailable_reason=_POWOD_DZIEKAN),

    ]

```

```

_DOCUMENT_POLICY: dict[str, Callable[[[]], list]] = {

    'STANDARD': _docs_standard,

    'EMPLOYMENT': _docs_employment_own_business,

    'OWN_BUSINESS': _docs_employment_own_business,

}

```



```

def buduj_flagi(zapis) -> dict:

    """Oblicza flagi stanu zapisu używane przez politykę dostępności.

    Jedno miejsce w domenie – nie duplikujemy w kontrolerach.

    """

    from core.modele import EnrollmentStatus, InternshipSchedule

    w_trakcie = zapis.status == EnrollmentStatus.IN_PROGRESS

    zakonczona = zapis.status == EnrollmentStatus.COMPLETED

    return {

        'zapis_id':          str(zapis.id),

        'w_trakcie':         w_trakcie,

        'zakonczone':        zakonczona,

        'oceniona':          zakonczona and (zapis.final_grades and zapis.final_grades.supervisor_grade) is not None,

        'dziekan_zatwierdził': w_trakcie or zakonczona,

        'harmonogram_count': db.session.execute(

db.select(db.func.count()).select_from(InternshipSchedule).filter_by(enrollment_id=zapis.id)

                                ).scalar(),

        'firma_bez_umowy':   not zapis.firma or not zapis.firma.has_standing_agreement,

        'firma_custom':      not zapis.company_id,

    }

def rozwiaz_dokumenty(zapis) -> list[dict]:

    """Zwraca listę dokumentów dla danego zapisu zgodnie z polityką ścieżki."""

    sciezka = zapis.track_type.value if zapis.track_type else 'STANDARD'

    factory = _DOCUMENT_POLICY.get(sciezka, _docs_standard)

    ctx = buduj_flagi(zapis)

    result = []

    for entry in factory():

        if isinstance(entry, dict):          # separator

            result.append(entry)

        else:

            result.append(entry.resolve(ctx))

```

```
return result
```

```
# -----
```

```
# Kanoniczny builder kontekstu TeX
```

```
# -----
```

```
def _fmt(value) -> str:
```

```
    """Formatuje datę jako DD.MM.YYYY lub zwraca pusty string."""
```

```
    if value is None:
```

```
        return ''
```

```
    if isinstance(value, (date, datetime)):
```

```
        return value.strftime('%d.%m.%Y')
```

```
    return str(value)
```

```
def _iso(value) -> str:
```

```
    """Formatuje datę jako ISO 8601 lub zwraca pusty string."""
```

```
    if value is None:
```

```
        return ''
```

```
    if isinstance(value, (date, datetime)):
```

```
        return value.isoformat()
```

```
    return str(value)
```

```
def _g(obj, attr: str, default=''):
```

```
    """Bezpieczny getter – zwraca default gdy atrybut nie istnieje lub jest None."""
```

```
    val = getattr(obj, attr, default)
```

```
    return val if val is not None else default
```

```
def buduj_kontekst(zapis, typ: str) -> dict:
```

```
    """Buduje słownik kontekstu dla szablonu TeX danego typu dokumentu.
```

Jedyna implementacja w całej aplikacji – używana przez:

- app_admin/routes/zarzadzanie/dokumenty_studentow.py
- app_student/routes/dokumenty.py
- celery_app.py (generate_pdf_dziennik)

Args:

zapis: instancja ZapisPraktyki (z załadowanymi relacjami)

typ: klucz z DOC_CONFIG, np. 'ZAL_6'

Returns:

Słownik gotowy do serializacji JSON i przekazania do tex-service.

"""

```
from core.modele.dziennik import JournalEntry
```

```
from core.modele import InternshipSchedule
```

```
s = zapis.student
```

```
p = zapis.internship
```

```
uopz = zapis.uopz
```

```
dm = zapis.workplace_details
```

```
firma_nazwa = (_g(zapis.firma, 'name') if zapis.firma else None) or (dm.company_name if dm else None) or ''
```

```
firma_adres = (_g(zapis.firma, 'address') if zapis.firma else None) or (dm.company_address if dm else None) or ''
```

```
firma_miasto = (_g(zapis.firma, 'city') if zapis.firma else None) or (dm.company_city if dm else None) or ''
```

```
firma_nip = (_g(zapis.firma, 'tax_id') if zapis.firma else None) or (dm.company_tax_id if dm else None) or ''
```

```
ctx: dict = {
```

```
    'student': {
```

```
        'first_name': _g(s, 'first_name'),
```

```
        'last_name': _g(s, 'last_name'),
```

```
        'album_number': _g(s, 'album_number'),
```

```
        # Aliasy dla starszych szablonów
```

```
        'imie': _g(s, 'first_name'),
```

```
        'nazwisko': _g(s, 'last_name'),
```

```
        'nr_albumu': _g(s, 'album_number'),
```

```

'numer_albumu': _g(s, 'album_number'),

'plec':          _g(s, 'gender', ''),

'kierunek':      _g(s, 'field_of_study') or 'Informatyka',

'specjalnosc':   _g(s, 'specialization', ''),

'tryb_studiow':  _g(s, 'study_mode', ''),

},

'praktyka': {

    'rok_uczelni': _g(p, 'academic_year') if p else '',

    'semestr':     _g(p, 'semester')      if p else '',

    'wymiar_godzin': _g(p, 'required_hours', 160) if p else 160,

},

'firma': {

    'nazwa':      firma_nazwa,

    'adres':      firma_adres,

    'miasto':     firma_miasto,

    'nip_krs':    firma_nip,

    # Aliasy angielskie

    'name':       firma_nazwa,

    'address':    firma_adres,

    'city':       firma_miasto,

},

'terminy': {

    'od': _fmt(zapis.start_date),

    'do': _fmt(zapis.end_date),

},

'zopz': {

    'imie_nazwisko': dm.workplace_mentor_name if dm else '',

    'stanowisko':    dm.workplace_mentor_position if dm else '',

    'telefon':       dm.workplace_mentor_phone if dm else '',

    'email':         dm.workplace_mentor_email if dm else '',

},

'uopz': {

    'first_name':    _g(uopz, 'first_name') if uopz else '',

    'last_name':     _g(uopz, 'last_name') if uopz else '',

```

```

        'imie_nazwisko': f"{uopz.first_name} {uopz.last_name}" if uopz else '',
    },
    'firma_upowazniony': dm.authorized_person if dm else '',
    'firma_upowazniony_stanowisko': dm.authorized_person_position if dm else '',
    'uzasadnienie': zapis.path_justification.justification if zapis.path_justification else '',
    'specjalnosc': _g(s, 'specialization', '') if s else '',
    'laczenie_godzin': _g(zapis, 'total_hours_logged', 0),
    'data_wniosku': '',
    # Aliasy dla szablonów odwołujących się do zapis.*
    'zapis': {
        'sciezka': zapis.track_type.value if zapis.track_type else 'STANDARD',
        'firma_nazwa': firma_nazwa,
        'firma_adres': firma_adres,
        'firma_miasto': firma_miasto,
    },
}

```

— Rozszerzenia per typ dokumentu —————

```

if typ == 'ZAL_2A':
    harmonogramy = (
        db.session.query(InternshipSchedule)
        .filter_by(enrollment_id=zapis.id)
        .order_by(InternshipSchedule.learning_outcome_id)
        .all()
    )
    ctx['harmonogram'] = [
        {
            'efekt_kod': _g(h.efekt, 'kod') if h.efekt else str(h.learning_outcome_id).zfill(2),
            'efekt_opis': _g(h.efekt, 'opis') if h.efekt else '',
            'dzial': _g(h, 'department_name', ''),
            'prace': _g(h, 'sample_tasks', ''),
            'dni': _g(h, 'days_count', 0),
        }
    ]

```

```

        for h in harmonogramy

    ]

elif typ == 'ZAL_6':

    wpisy = (

        db.session.query(JournalEntry)

        .filter_by(enrollment_id=zapis.id)

        .order_by(JournalEntry.entry_date)

        .all()

    )

    ctx.update({

        'sciezka':          zapis.track_type.value if zapis.track_type else 'STANDARD',

        'data_rozpozecia':  _iso(zapis.start_date),

        'data_zakonczenia': _iso(zapis.end_date),

        'wpisy': [

            {

                'data':      _iso(w.entry_date),

                'opis':      _g(w, 'description', ''),

                'godziny':   _g(w, 'duration_hours', 0),

                'efekt_nr':  ', '.join(f"{e.id:02d}" for e in w.learning_outcomes)

                            if w.learning_outcomes else '--',

            }

            for w in wpisy

        ],

    })

elif typ == 'ZAL_7':

    spr = getattr(zapis, 'sprawozdanie', None)

    ctx.update({

        'charakterystyka_miejsca': _g(spr, 'charakterystyka_miejsca') if spr else '',

        'opis_prac':               _g(spr, 'opis_i_analiza')           if spr else '',

        'efekty_opisy':            [''] * 13,

    })

```

```
elif typ == 'ZAL_4':

    oceny = getattr(zapis, 'oceny', None) or []

    ctx.update({

        'oceny': [

            {

                'efekt_id': o.learning_outcome_id,

                'wynik':    o.result.value if o.result else 'uzyskał/a',

                'uwagi':    _g(o, 'notes', ''),

            }

            for o in sorted(oceny, key=lambda x: x.learning_outcome_id)

        ],

        'uwagi_uopz': _g(zapis, 'supervisor_grade_description', ''),

    })

return ctx
```

PLIK: core\uslugi\ocenie.py

```
"""
```

```
core/uslugi/ocenie.py
```

Usługa oceniania praktyk.

Zawiera całą logikę domenową procesu oceniania – kontrolery tras delegują tutaj i nie zawierają własnej logiki biznesowej.

```
"""
```

```
from __future__ import annotations
```

```
import logging
```

```
from dataclasses import dataclass
```

```
from datetime import date, timedelta
```

```
from typing import Optional
```

```
from core.extensions import db
```

```
from core.modele import InternshipEnrollment, EnrollmentStatus
```

```
log = logging.getLogger(__name__)
```

```
# — DTO —————
```

```
@dataclass
```

```
class GradeFormData:
```

```
    """Transfer Object – dane z formularza ocen, niezależny od HTTP."""
```

```
    report_grade: Optional[float]
```

```
    supervisor_grade: Optional[float]
```

```
    workplace_grade: Optional[float]
```

```
    supervisor_grade_description: Optional[str]
```

```
    workplace_grade_description: Optional[str]
```

```
    exam_question_1: Optional[str]
```

```
    exam_grade_1: Optional[float]
```

```
    exam_question_2: Optional[str]
```

```
    exam_grade_2: Optional[float]
```



```
exam_question_3: Optional[str]

exam_grade_3: Optional[float]

finalize: bool = False      # True → zmień status na COMPLETED
```

```
@dataclass
```

```
class GradeResult:
```

```
    """Wynik operacji zapisu ocen."""
```

```
    success: bool
```

```
    missing_fields: list[str]      # niepuste gdy success=False i finalize=True
```

```
@property
```

```
def error_message(self) -> str | None:
```

```
    if not self.success and self.missing_fields:
```

```
        return f"Nie można zakończyć – brakuje: {'', ' '.join(self.missing_fields)}."
```

```
    return None
```

```
class SerwisOceniania:
```

```
# — Zapis ocen —————
```

```
@staticmethod
```

```
def zapisz_oceny(zapis: InternshipEnrollment, dane: GradeFormData) -> GradeResult:
```

```
    """Zapisuje oceny końcowe i sprawdzian dla zapisu praktyki.
```

```
    Jeśli dane.finalize=True, weryfikuje kompletność przed zamknięciem.
```

```
    Nie wykonuje commit – wywołujący decyduje o transakcji.
```

```
    Returns:
```

```
        GradeResult(success=False, missing_fields=[]) gdy finalize=True
```

```
        i brakuje ocen. W przeciwnym razie GradeResult(success=True).
```

```
    """
```

```
    from core.modele import OcenyKoncowe, Sprawdzian as SprawdzianModel
```

```

ok = zapis.oceny_koncowe or OcenyKoncowe(enrollment_id=zapis.id)

if ok not in db.session:

    db.session.add(ok)


ok.report_grade                = dane.report_grade
ok.supervisor_grade            = dane.supervisor_grade
ok.workplace_grade             = dane.workplace_grade
ok.supervisor_grade_description = dane.supervisor_grade_description
ok.workplace_grade_description = dane.workplace_grade_description


sp = zapis.sprawdzian or SprawdzianModel(enrollment_id=zapis.id)

if sp not in db.session:

    db.session.add(sp)


sp.question_1 = dane.exam_question_1
sp.grade_1    = dane.exam_grade_1
sp.question_2 = dane.exam_question_2
sp.grade_2    = dane.exam_grade_2
sp.question_3 = dane.exam_question_3
sp.grade_3    = dane.exam_grade_3


if dane.finalize:

    missing = SerwisOceny._waliduj_kompletnosc(ok, sp)

    if missing:

        db.session.rollback()

        return GradeResult(success=False, missing_fields=missing)

    from core.uslugi.workflow import ZapisFSM

    ZapisFSM(zapis).zakonczy()


return GradeResult(success=True, missing_fields=[])

```

```

@staticmethod

```

```

def _waliduj_kompletnosc(ok, sp) -> list[str]:

```

```

"""Zwraca listę brakujących ocen. Pusta lista = komplet."""

missing = []

if ok.report_grade is None: missing.append('ocena sprawozdania')

if ok.supervisor_grade is None: missing.append('ocena UOPZ')

if ok.workplace_grade is None: missing.append('ocena ZOPZ')

if None in (sp.grade_1, sp.grade_2, sp.grade_3):

    missing.append('oceny ze sprawdzianu (wszystkie 3 pytania)')

return missing

```

```

@staticmethod

```

```

def get_pilne_oceny(uopz_id=None) -> list[dict]:

    """Zwraca listę praktyk z pilnymi ocenami (termin ≤ 3 dni)."""

    q = db.session.query(InternshipEnrollment).filter_by(status=EnrollmentStatus.COMPLETED)

    if uopz_id:

        q = q.filter_by(supervisor_id=uopz_id)

    pilne = []

    for zapis in q.all():

        if zapis.termin_do:

            deadline = zapis.termin_do + timedelta(days=7)

            dni_do_konca = (deadline - date.today()).days

            if dni_do_konca <= 3:

                pilne.append({

                    'zapis': zapis,

                    'deadline': deadline,

                    'dni_do_deadline': dni_do_konca,

                    'przekroczony': dni_do_konca < 0,

                })

    return sorted(pilne, key=lambda x: x['dni_do_deadline'])

```

```

@staticmethod

```

```

def auto_complete_internships() -> dict:

    """Automatycznie zamyka praktyki z przekroczonym terminem.

```

Warunek zamknięcia: data końcowa minęła ORAZ student zarejestrował co najmniej tyle godzin, ile wymaga edycja praktyki (required_hours). Praktyki bez wymaganego wymiaru godzin są pomijane – pozostają w statusie IN_PROGRESS i wymagają ręcznej interwencji opiekuna.

Returns:

dict z kluczami 'completed' (zamknięte) i 'skipped' (pominięte z powodu niewystarczających godzin).

"""

```
kandydaci = db.session.query(InternshipEnrollment).filter(
    InternshipEnrollment.status == EnrollmentStatus.IN_PROGRESS,
    InternshipEnrollment.end_date < date.today(),
).all()
```

```
completed = skipped = 0
```

```
for p in kandydaci:
```

```
    required = p.internship.required_hours if p.internship else 0
```

```
    if p.total_hours_logged >= required:
```

```
        from core.uslugi.workflow import ZapisFSM
```

```
        ZapisFSM(p).zakoncz()
```

```
        completed += 1
```

```
    else:
```

```
        skipped += 1
```

```
    import logging
```

```
    logging.getLogger(__name__).warning(
```

```
        "auto_complete: pominięto zapis %s – %d/%d h (student: %s %s)",
```

```
        p.id, p.total_hours_logged, required,
```

```
        p.student.first_name if p.student else '?',
```

```
        p.student.last_name if p.student else '?',
```

```
    )
```

```
if completed:
```

```
    db.session.commit()
```

```
return {'completed': completed, 'skipped': skipped}
```

PLIK: core\uslugi\praktyki.py

```
"""core/uslugi/praktyki.py

Internship and enrollment management service.

"""

from __future__ import annotations

import uuid

from datetime import datetime, timezone

from typing import Optional

from core.extensions import db

from core.modele.praktyki import (
    Internship,
    InternshipStatus,
    InternshipEnrollment,
    EnrollmentStatus,
    InternshipPath,
    EventType,
    ProcessEvent,
    InternshipReport,
)

from core.repozytoria.praktyki import RepozytoriumPraktyk, RepozytoriumZapisow


class UsługaPraktyk:

    """Business logic for internship editions and enrollment processing."""

    def __init__(
        self,
        repo_praktyk: Optional[RepozytoriumPraktyk] = None,
        repo_zapisow: Optional[RepozytoriumZapisow] = None,
    ) -> None:

        self._praktyki = repo_praktyk or RepozytoriumPraktyk()
```

```
self._zapisy = repo_zapisow or RepozytoriumZapisow()
```

```
# — Internship editions —————
```

```
def utworz_edycje(self, rok_uczelniany: str, semestr: str, wymiar_godzin: int = 160) -> Internship:
```

```
    praktyka = Internship(
        academic_year=rok_uczelniany,
        semester=semestr,
        required_hours=wymiar_godzin,
        status=InternshipStatus.INACTIVE,
    )
    self._praktyki.zapisz(praktyka)
    db.session.commit()
    return praktyka
```

```
def aktywuj_edycje(self, praktyka: Internship) -> None:
```

```
    praktyka.status = InternshipStatus.ACTIVE
    db.session.commit()
```

```
def dezaktywuj_edycje(self, praktyka: Internship) -> None:
```

```
    praktyka.status = InternshipStatus.INACTIVE
    db.session.commit()
```

```
# — Student enrollments —————
```

```
def zapisz_studenta(
```

```
    self,
    student_id: uuid.UUID,
    praktyka_id: uuid.UUID,
    sciezka: InternshipPath = InternshipPath.STANDARD,
```

```
) -> InternshipEnrollment:
```

```
    if self._zapisy.student_ma_aktywny_zapis(student_id, praktyka_id):
        raise ValueError('Student ma już aktywne zgłoszenie do tej edycji praktyk.')
    zapis = InternshipEnrollment(
```

```

        student_id=student_id,

        internship_id=praktyka_id,

        path_type=sciezka,

        status=EnrollmentStatus.PENDING,

    )

    self._zapisy.zapisz(zapis)

    db.session.commit()

    return zapis

def zmien_status(

    self,

    zapis: InternshipEnrollment,

    nowy_status: EnrollmentStatus,

    komentarz: Optional[str] = None,

    wykonane_przez_id: Optional[uuid.UUID] = None,

) -> None:

    """Changes enrollment status via FSM public methods."""

    from core.uslugi.workflow import ZapisFSM, IllegalTransitionError

    fsm = ZapisFSM(zapis)

    _dispatch = {

        EnrollmentStatus.AWAITING_APPROVAL: fsm.wyslij_do_akceptacji,

        EnrollmentStatus.COMMISSION_REVIEW: fsm.wyslij_do_komisji,

        EnrollmentStatus.IN_PROGRESS:      fsm.zatwierdz_przez_uopz,

        EnrollmentStatus.DEAN_APPROVAL:    fsm.zatwierdz_przez_komisje,

        EnrollmentStatus.COMPLETED:      fsm.zakoncz,

        EnrollmentStatus.REJECTED:        fsm.odrzuc,

    }

    method = _dispatch.get(nowy_status)

    if method is None:

        raise ValueError(f'Nieobsługiwany status docelowy: {nowy_status!r}')

    method()

    if komentarz is not None:

        if nowy_status in (EnrollmentStatus.REJECTED, EnrollmentStatus.AWAITING_APPROVAL):

```



```

        typ = EventType.ADMIN_COMMENT

    elif nowy_status == EnrollmentStatus.COMMISSION_REVIEW:

        typ = EventType.SUPERVISOR_COMMENT

    else:

        typ = EventType.ADMIN_COMMENT

    self._dodaj_zdarzenie(zapis, typ, komentarz=komentarz, wykonane_przez_id=wykonane_przez_id)


db.session.commit()


def przypisz_opiekuna(self, zapis: InternshipEnrollment, uopz_id: uuid.UUID) -> None:

    zapis.supervisor_id = uopz_id

    db.session.commit()


def zatwierdz_przez_komisje(

    self,

    zapis: InternshipEnrollment,

    decyzja: str,

    komentarz: Optional[str] = None,

    wykonane_przez_id: Optional[uuid.UUID] = None,

) -> None:

    from core.uslugi.workflow import ZapisFSM

    self._dodaj_zdarzenie(

        zapis, EventType.COMMITTEE_DECISION,

        decyzja=decyzja, komentarz=komentarz,

        wykonane_przez_id=wykonane_przez_id,

    )

    fsm = ZapisFSM(zapis)

    if decyzja == 'APPROVED':

        fsm.zatwierdz_przez_komisje()

    else:

        fsm.odrzuc()

    db.session.commit()


def zatwierdz_przez_dziekana(

```

```

        self,

        zapis: InternshipEnrollment,

        decyzja: str,

        komentarz: Optional[str] = None,

        wykonane_przez_id: Optional[uuid.UUID] = None,
    ) -> None:

        from core.uslugi.workflow import ZapisFSM

        self._dodaj_zdarzenie(

            zapis, EventType.DEAN_DECISION,

            decyzja=decyzja, komentarz=komentarz,

            wykonane_przez_id=wykonane_przez_id,

        )

        fsm = ZapisFSM(zapis)

        if decyzja == 'APPROVED':

            fsm.zatwierdz_przez_dziekana()

        else:

            fsm.odrzuc()

        db.session.commit()

def powiadom_studenta(

    self,

    zapis: InternshipEnrollment,

    komentarz: Optional[str] = None,

    wykonane_przez_id: Optional[uuid.UUID] = None,
) -> None:

    self._dodaj_zdarzenie(

        zapis, EventType.STUDENT_NOTIFICATION,

        komentarz=komentarz,

        wykonane_przez_id=wykonane_przez_id,

    )

    db.session.commit()

def zakoncz(self, zapis: InternshipEnrollment) -> None:

    from core.uslugi.workflow import ZapisFSM

```

```
ZapisFSM(zapis).zakoncz()
```

```
db.session.commit()
```

```
# — Reports —————
```

```
def pobierz_lub_utworz_sprawozdanie(self, zapis: InternshipEnrollment) -> InternshipReport:
```

```
    if zapis.report is None:
```

```
        report = InternshipReport(enrollment_id=zapis.id)
```

```
        db.session.add(report)
```

```
        db.session.flush()
```

```
        return report
```

```
    return zapis.report
```

```
# — Helpers —————
```

```
def _dodaj_zdarzenie(
```

```
    self,
```

```
    zapis: InternshipEnrollment,
```

```
    typ: EventType,
```

```
    decyzja: Optional[str] = None,
```

```
    komentarz: Optional[str] = None,
```

```
    wykonane_przez_id: Optional[uuid.UUID] = None,
```

```
) -> ProcessEvent:
```

```
    zdarzenie = ProcessEvent(
```

```
        enrollment_id=zapis.id,
```

```
        event_type=typ,
```

```
        decision=decyzja,
```

```
        comment=komentarz,
```

```
        executed_by_id=wykonane_przez_id,
```

```
        executed_at=datetime.now(timezone.utc),
```

```
    )
```

```
    db.session.add(zdarzenie)
```

```
    return zdarzenie
```

— Repository access —————

@property

```
def praktyki(self) -> RepozytoriumPraktyk:  
    return self._praktyki
```

@property

```
def zapisy(self) -> RepozytoriumZapisow:  
    return self._zapisy
```

PLIK: core\uslugi\uzytownicy.py

```
"""core/uslugi/uzytownicy.py

User account management service.

"""

from __future__ import annotations

from typing import Optional

import uuid

from werkzeug.security import generate_password_hash, check_password_hash

from core.extensions import db

from core.modele.uzytownicy import UserRole, User, Student, Administrator, UniversityMentor

from core.repozytoria.uzytownicy import RepozytoriumUzytkownikow


class UsługaUzytkownikow:

    """Business logic for user accounts."""

    def __init__(self, repozytorium: Optional[RepozytoriumUzytkownikow] = None) -> None:

        self._repo = repozytorium or RepozytoriumUzytkownikow()

    # — Authentication —————

    def uwierzytelnij(self, email: str, haslo: str) -> Optional[User]:

        """Returns user if credentials are valid, None otherwise."""

        user = self._repo.znajdz_po_emailu(email)

        if user is None or not user.is_active:

            return None

        if not check_password_hash(user.password_hash, haslo):

            return None

        return user
```

```
def zmien_haslo(self, user: User, nowe_haslo: str) -> None:

    user.password_hash = generate_password_hash(nowe_haslo)

    user.require_password_change = False

    db.session.commit()
```

— Account creation —————

```
def utworz_studenta(

    self,

    email: str,

    haslo: str,

    imie: str,

    nazwisko: str,

    numer_albumu: Optional[str] = None,

    **dane_studenta,

) -> Student:

    if self._repo.istnieje_email(email):

        raise ValueError(f'Konto z adresem {email} już istnieje.')

    student = Student(

        email=email,

        password_hash=generate_password_hash(haslo),

        first_name=imie,

        last_name=nazwisko,

        role=UserRole.STUDENT,

        album_number=numer_albumu,

        **dane_studenta,

    )

    self._repo.zapisz(student)

    db.session.commit()

    return student
```

```
def utworz_administratora(self, email: str, haslo: str, imie: str, nazwisko: str) -> Administrator:

    if self._repo.istnieje_email(email):

        raise ValueError(f'Konto z adresem {email} już istnieje.')
```

```

admin = Administrator(

    email=email,

    password_hash=generate_password_hash(haslo),

    first_name=imie,

    last_name=nazwisko,

    role=UserRole.ADMIN,

)

self._repo.zapisz(admin)

db.session.commit()

return admin

```

```

def utworz_opiekuna(self, email: str, haslo: str, imie: str, nazwisko: str) -> UniversityMentor:

```

```

    if self._repo.istnieje_email(email):

        raise ValueError(f'Konto z adresem {email} już istnieje.')

    mentor = UniversityMentor(

        email=email,

        password_hash=generate_password_hash(haslo),

        first_name=imie,

        last_name=nazwisko,

        role=UserRole.UOPZ,

    )

    self._repo.zapisz(mentor)

    db.session.commit()

    return mentor

```

— Updates —————

```

def aktualizuj(self, user: User, **pola) -> User:

```

```

    """Updates arbitrary model fields and commits."""

    if 'haslo' in pola:

        user.password_hash = generate_password_hash(pola.pop('haslo'))

    for klucz, wartosc in pola.items():

        setattr(user, klucz, wartosc)

    db.session.commit()

```

```
return user
```

```
def dezaktywuj(self, user: User) -> None:
```

```
    user.is_active = False
```

```
    db.session.commit()
```

```
def aktywuj(self, user: User) -> None:
```

```
    user.is_active = True
```

```
    db.session.commit()
```

```
# — Repository access —————
```

```
@property
```

```
def repozytorium(self) -> RepozytoriumUzytkownikow:
```

```
    return self._repo
```


PLIK: core\uslugi\workflow.py

```
"""core/uslugi/workflow.py
```

Maszyna Stanów (FSM) dla ZapisPraktyki.

Każda metoda publiczna reprezentuje jedno legalne przejście między stanami.

Nielegalne przejście rzuca `IllegalTransitionError` – kontrolery łapią ten

wyjątek i tłumaczą na flash/HTTP 400, nigdy nie modyfikując statusu

bezpośrednio.

Diagram przejść:

PENDING

- `wyslij_do_akceptacji()` → `AWAITING_APPROVAL` (ścieżka A)
- `wyslij_do_komisji()` → `COMMISSION_REVIEW` (ścieżki B/C)

AWAITING_APPROVAL

- `zatwierdz_przez_uopz()` → `IN_PROGRESS` (ścieżka A)
- `odrzuc()` → `REJECTED`
- `wyslij_do_komisji()` → `COMMISSION_REVIEW` (ścieżki B/C, po komentarzu UOPZ)

COMMISSION_REVIEW | REVISION_REQUIRED

- `zatwierdz_przez_komisje()` → `DEAN_APPROVAL`
- `zadaj_poprawki()` → `REVISION_REQUIRED`
- `odrzuc()` → `REJECTED`

DEAN_APPROVAL

- `zatwierdz_przez_dziekana()` → `IN_PROGRESS`
- `odrzuc()` → `REJECTED`

IN_PROGRESS

- `zakoncz()` → `COMPLETED`

`COMPLETED` (terminal – brak wyjść)

`REJECTED` (terminal – brak wyjść)

```
"""
```

```
from __future__ import annotations
```

```
from core.modele.praktyki import EnrollmentStatus as S
```

```
# -----
```

```
# Wyjątek dziedziny
```

```
# -----
```

```
class IllegalTransitionError(ValueError):
```

```
    """Próba nielegalnego przejścia między stanami."""
```

```
    def __init__(self, current: S, attempted: S, reason: str = ''):
```

```
        msg = (
```

```
            f"Niedozwolone przejście: {current.value} → {attempted.value}"
```

```
            + (f" ({reason})" if reason else "")
```

```
        )
```

```
        super().__init__(msg)
```

```
        self.current = current
```

```
        self.attempted = attempted
```

```
# -----
```

```
# Dozwolone przejścia (graf)
```

```
# -----
```

```
_ALLOWED: dict[S, set[S]] = {
```

```
    S.PENDING: {
```

```
        S.AWAITING_APPROVAL,
```

```
        S.COMMISSION_REVIEW,
```

```
    },
```

```
    S.AWAITING_APPROVAL: {
```

```
        S.IN_PROGRESS,
```

```
        S.COMMISSION_REVIEW,
```

```
        S.REJECTED,
```

```

    },

    S.COMMISSION_REVIEW: {

        S.DEAN_APPROVAL,

        S.REVISION_REQUIRED,

        S.REJECTED,

    },

    S.REVISION_REQUIRED: {

        S.DEAN_APPROVAL,

        S.REVISION_REQUIRED,    # kolejna runda poprawek

        S.REJECTED,

    },

    S.DEAN_APPROVAL: {

        S.IN_PROGRESS,

        S.REJECTED,

    },

    S.IN_PROGRESS: {

        S.COMPLETED,

    },

    S.COMPLETED: set(),

    S.REJECTED: set(),

}

# -----

# FSM

# -----


class ZapisFSM:

    """Kontroler przejść stanu dla instancji ZapisPraktyki.

    Nie wykonuje commit – wywołujący odpowiada za transakcję.

    Preferowane użycie (z blokadą pesymistyczną):

        with ZapisFSM.lock(enrollment_id) as fsm:

```

```
fsm.zatwierdz_przez_komisje()

db.session.commit()
```

Fallback gdy obiekt jest już załadowany:

```
fsm = ZapisFSM(zapis)

fsm.zatwierdz_przez_uopz()

db.session.commit()

"""
```

```
def __init__(self, zapis) -> None:

    self._zapis = zapis
```

— Blokada pesymistyczna (SELECT ... FOR UPDATE) —————

```
@classmethod
```

```
def lock(cls, enrollment_id) -> 'ZapisFSM':

    """Ładuje ZapisPraktyki z blokadą FOR UPDATE i zwraca FSM.
```

Blokada wierszowa w PostgreSQL gwarantuje, że żaden inny proces

Gunicorn nie zmodyfikuje statusu między odczytem a commitem.

Blokada jest utrzymana do końca transakcji (db.session.commit/rollback).

Użycie jako context manager (zalecane):

```
with ZapisFSM.lock(id) as fsm:

    fsm.zatwierdz_przez_komisje()

    db.session.commit()
```

Użycie bez context managera:

```
fsm = ZapisFSM.lock(id)

fsm.zatwierdz_przez_dziekana()

db.session.commit()

"""

from core.modele.praktyki import InternshipEnrollment

from core.extensions import db
```

```

zapis = (
    db.session.query(InternshipEnrollment)
        .filter_by(id=enrollment_id)
        .with_for_update()
        .one_or_none()
)

if zapis is None:
    from flask import abort
    abort(404)

return cls(zapis)

# Context manager – dla czytelności kodu w kontrolerach

def __enter__(self) -> 'ZapisFSM':
    return self

def __exit__(self, exc_type, exc_val, exc_tb) -> None:
    if exc_type is not None:
        from core.extensions import db
        db.session.rollback()

@property
def zapis(self):
    return self._zapis

# — Wewnętrzna zmiana stanu —————

def _przejdź(self, cel: S, reason: str = '') -> None:
    obecny = self._zapis.status

    if cel not in _ALLOWED.get(obecny, set()):
        raise IllegalTransitionError(obecny, cel, reason)

    self._zapis.status = cel

# — Publiczne przejścia —————

```

```
def wyslij_do_akceptacji(self) -> None:

    """PENDING → AWAITING_APPROVAL (ścieżka standardowa A)."""

    self._przejdź(S.AWAITING_APPROVAL)


def wyslij_do_komisji(self) -> None:

    """PENDING / AWAITING_APPROVAL → COMMISSION_REVIEW (ścieżki B/C)."""

    self._przejdź(S.COMMISSION_REVIEW)


def zatwierdz_przez_uopz(self) -> None:

    """AWAITING_APPROVAL → IN_PROGRESS."""

    self._przejdź(S.IN_PROGRESS, 'decyzja UOPZ')


def zatwierdz_przez_komisje(self) -> None:

    """COMMISSION_REVIEW / REVISION_REQUIRED → DEAN_APPROVAL."""

    self._przejdź(S.DEAN_APPROVAL, 'decyzja komisji')


def zadaj_poprawki(self) -> None:

    """COMMISSION_REVIEW / REVISION_REQUIRED → REVISION_REQUIRED."""

    self._przejdź(S.REVISION_REQUIRED, 'komisja: wymaga uzupełnień')


def zatwierdz_przez_dziekana(self) -> None:

    """DEAN_APPROVAL → IN_PROGRESS."""

    self._przejdź(S.IN_PROGRESS, 'decyzja dziekana')


def zakoncz(self) -> None:

    """IN_PROGRESS → COMPLETED."""

    self._przejdź(S.COMPLETED)


def odrzuc(self) -> None:

    """Dowolny aktywny stan → REJECTED."""

    self._przejdź(S.REJECTED)
```

PLIK: core\uslugi__init__.py

```
"""core/uslugi – warstwa usług biznesowych.
```

```
Usługi zawierają logikę domenową i orkiestrują  
repozytoria. Kontrolery wywołują wyłącznie usługi –  
nie tworzą zapytań ORM ani nie operują bezpośrednio  
na sesjach bazy danych.
```

```
"""
```

```
from core.uslugi.praktyki import UsługaPraktyk  
  
from core.uslugi uzytkownicy import UsługaUzytkownikow  
  
from core.uslugi.ocenianie import SerwisOceniania  
  
from core.uslugi.dokumenty import (  
    DOC_CONFIG, STATIC_TEMPLATES,  
    DocumentEntry, buduj_flagi, rozwiaz_dokumenty, buduj_kontekst,  
)
```

```
__all__ = [  
    'UsługaPraktyk', 'UsługaUzytkownikow', 'SerwisOceniania',  
    'DOC_CONFIG', 'STATIC_TEMPLATES',  
    'DocumentEntry', 'buduj_flagi', 'rozwiaz_dokumenty', 'buduj_kontekst',  
]
```

PLIK: database\docker-compose.yml

version: '3.8'

services:

db:

image: postgres:16-alpine

container_name: ans_postgres

environment:

POSTGRES_USER: \${POSTGRES_USER:-ans_admin}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}

POSTGRES_DB: \${POSTGRES_DB:-ans_praktyki}

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

restart: unless-stopped

redis:

image: redis:7-alpine

container_name: ans_redis

ports:

- "6379:6379"

restart: unless-stopped

worker:

build:

context: ../app_admin

dockerfile: Dockerfile.worker

container_name: ans_celery_worker

environment:

DATABASE_URL: postgresql+psycopg2://\${POSTGRES_USER:-ans_admin}:\${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD in .env}@db:5432/\${POSTGRES_DB:-ans_praktyki}

CELERY_BROKER_URL: redis://redis:6379/0

CELERY_RESULT_BACKEND: redis://redis:6379/0

FLASK_ENV: development

SECRET_KEY: \${SECRET_KEY:?Set SECRET_KEY in .env}

volumes:

- pdf_output:/app/pdf_output

depends_on:

- db
- redis

restart: unless-stopped

volumes:

postgres_data:

pdf_output:

PLIK: fileserver\app.py

```
"""
```

```
fileserver/app.py
```

Prosty serwer plików dostępny tylko na sieci wewnętrznej Docker.

Pliki trafiają tu już zaszyfrowane – serwer nie zna klucza szyfrowania.

API (chronione X-API-Key):

```
PUT    /files/<filename>  – zapisz plik
```

```
GET    /files/<filename>  – pobierz plik
```

```
DELETE /files/<filename>  – usuń plik
```

```
"""
```

```
import os
```

```
from pathlib import Path
```

```
from flask import Flask, request, Response, jsonify, abort
```

```
app = Flask(__name__)
```

```
STORAGE = Path('/data/files')
```

```
STORAGE.mkdir(parents=True, exist_ok=True)
```

```
API_KEY = os.environ.get('FILESERVER_API_KEY', '')
```

```
def _auth():
```

```
    if request.headers.get('X-API-Key') != API_KEY:
```

```
        abort(403)
```

```
@app.route('/health')
```

```
def health():
```

```
    return jsonify({'ok': True})
```

```
def _safe_dest(filename: str) -> Path:

    """Zwraca bezpieczną ścieżkę lub przerywa z HTTP 400 (Path Traversal)."""

    dest = (STORAGE / filename).resolve()

    if not dest.is_relative_to(STORAGE.resolve()):

        abort(400)

    return dest
```

```
@app.route('/files/<path:filename>', methods=['PUT'])
```

```
def upload(filename):

    _auth()

    dest = _safe_dest(filename)

    dest.write_bytes(request.get_data())

    return jsonify({'ok': True}), 201
```

```
@app.route('/files/<path:filename>', methods=['GET'])
```

```
def download(filename):

    _auth()

    dest = _safe_dest(filename)

    if not dest.exists():

        abort(404)

    return Response(dest.read_bytes(), mimetype='application/octet-stream')
```

```
@app.route('/files/<path:filename>', methods=['DELETE'])
```

```
def delete(filename):

    _auth()

    dest = _safe_dest(filename)

    if dest.exists():

        dest.unlink()

    return jsonify({'ok': True})
```

```
if __name__ == '__main__':
```

```
    app.run(host='0.0.0.0', port=5003)
```

PLIK: fileserver\Dockefile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt ./

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py ./

CMD ["gunicorn", "--bind", "0.0.0.0:5003", "--workers", "2", "app:app"]
```

PLIK: tex_service\app.py

```
import io

import logging

import shutil

from pathlib import Path


from flask import Flask, request, send_file, jsonify

from compiler import compile_pdf, render_tex, TexCompilationError


logging.basicConfig(level=logging.INFO)

logger = logging.getLogger(__name__)


app = Flask(__name__)


TEMPLATES_DIR = Path(__file__).parent / "templates"


def _allowed_templates() -> set[str]:

    """Zwraca nazwy plików .tex.j2 z katalogu templates."""

    return {f.name for f in TEMPLATES_DIR.glob("*.tex.j2")}


@app.route("/health")

def health():

    lualatex_ok = bool(shutil.which("lualatex"))

    templates = list(_allowed_templates())

    return jsonify({

        "status": "ok" if lualatex_ok else "degraded",

        "lualatex": lualatex_ok,

        "templates": templates,

    })
```

```

@app.route("/generuj", methods=["POST"])

def generuj():
    """
    Body JSON:

    {
        "template": "zal6_dziennik.tex.j2",
        "context": { ... },
        "filename": "dziennik.pdf"    # opcjonalne
    }

    Odpowiedź: application/pdf (bajty pliku)
    lub          application/json z {"error": "..."} przy błędzie
    """

    data = request.get_json(silent=True)

    if not data:

        return jsonify({"error": "Brak body JSON"}), 400

    template_name = data.get("template", "")
    context = data.get("context", {})
    filename = data.get("filename", "dokument.pdf")

    # Walidacja szablonu

    allowed = _allowed_templates()

    if template_name not in allowed:

        return jsonify({
            "error": f"Nieznany szablon: {template_name!r}",
            "available": sorted(allowed),
        }), 400

    if not isinstance(context, dict):

        return jsonify({"error": "context musi być obiektem JSON"}), 400

    try:

```

```

    pdf_bytes = compile_pdf(template_name, context)

except TexCompilationError as e:

    logger.error("Błąd kompilacji LaTeX dla %s: %s\nLOG:\n%s", template_name, e, e.log)

    return jsonify({"error": "Błąd kompilacji dokumentu PDF."}), 500

except Exception:

    logger.exception("Nieoczekiwany błąd dla %s", template_name)

    return jsonify({"error": "Wewnętrzny błąd serwera."}), 500


return send_file(

    io.BytesIO(pdf_bytes),

    mimetype="application/pdf",

    as_attachment=True,

    download_name=filename,

)


if __name__ == "__main__":

    app.run(host="0.0.0.0", port=5002, debug=False)

```


PLIK: tex_service\check_lualatex_version.py

```
import subprocess

import re

import sys


out = subprocess.check_output(['lualatex', '--version'], text=True)

m = re.search(r'Version (\d+\.\d+)', out)

ver = tuple(int(x) for x in m.group(1).split('.')) if m else (0, 0)

if ver < (1, 17):

    sys.exit(f'LuaLaTeX {m.group(1) if m else "?"} < 1.17.0 – CVE-2023-32700')
```

PLIK: tex_service\compiler.py

```
"""tex_service/compiler.py

Silnik kompilacji PDF. Przyjmuje nazwę szablonu i kontekst danych,
wyrenderowuje szablon Jinja2, wywołuje lualatex,
zwraca bajty PDF lub rzuca TexCompilationError.
"""

import os

import shutil

import subprocess

import tempfile

import logging

from pathlib import Path

from jinja2 import Environment, FileSystemLoader, StrictUndefined

from sanitizer import sanitize, sanitize_date, sanitize_int

logger = logging.getLogger(__name__)

TEMPLATES_DIR = Path(__file__).parent / "templates"

COMPILE_TIMEOUT = int(os.environ.get('LATEX_TIMEOUT', '60'))

COMPILE_PASSES = 1

class TexCompilationError(RuntimeError):

    def __init__(self, message: str, log: str = ""):

        super().__init__(message)

        self.log = log

def _build_jinja_env() -> Environment:
```

```

env = Environment(

    loader=FileSystemLoader(str(TEMPLATES_DIR)),

    block_start_string="<<%",

    block_end_string="%>>",

    variable_start_string="<<",

    variable_end_string=">>",

    comment_start_string="<<#",

    comment_end_string="#>>",

    undefined=StrictUndefined,

    keep_trailing_newline=True,

)

env.filters["s"] = sanitize

env.filters["date"] = sanitize_date

env.filters["num"] = sanitize_int

return env

```

`_JINJA_ENV: Environment | None = None`

```

def get_jinja_env() -> Environment:

    global _JINJA_ENV

    if _JINJA_ENV is None:

        _JINJA_ENV = _build_jinja_env()

    return _JINJA_ENV

```

```

def _find_lualatex() -> str:

    binary = shutil.which("lualatex")

    if binary is None:

        raise EnvironmentError(

            "Nie znaleziono lualatex w PATH. "

            "Sprawdź czy texlive jest zainstalowany w kontenerze."

        )

```

```
return binary
```

```
def _run_lualatex(lualatex: str, tex_file: Path, workdir: Path) -> str:
```

```
    cmd = [  
        lualatex,  
        "-no-shell-escape",  
        "-interaction=nonstopmode",  
        "-halt-on-error",  
        "-output-directory", str(workdir),  
        str(tex_file),  
    ]
```

```
    try:
```

```
        result = subprocess.run(  
            cmd,  
            cwd=str(workdir),  
            capture_output=True,  
            timeout=COMPILE_TIMEOUT,  
            stdin=subprocess.DEVNULL,  
        )
```

```
    except subprocess.TimeoutExpired:
```

```
        raise TexCompilationError(  
            f"lualatex przekroczył limit czasu ({COMPILE_TIMEOUT}s)."  
        )
```

```
    log_file = workdir / tex_file.with_suffix(".log").name
```

```
    log_text = log_file.read_text(encoding="utf-8", errors="replace") if log_file.exists() else ""
```

```
    if result.returncode != 0:
```

```
        raise TexCompilationError(  
            f"lualatex zakończył się błędem (kod {result.returncode}).",  
            log=log_text,  
        )
```

```
return log_text
```

```
def render_tex(template_name: str, context: dict) -> str:
```

```
    """Renderuje szablon Jinja2 → surowy tekst TeX."""
```

```
    env = get_jinja_env()
```

```
    template = env.get_template(template_name)
```

```
    return template.render(**context)
```

```
def compile_pdf(template_name: str, context: dict) -> bytes:
```

```
    """Publiczne API: template + context → bajty PDF.
```

Jedyna dozwolona ścieżka kompilacji. Surowy TeX pochodzi wyłącznie z szablonu Jinja2 zaaudytowanego w systemie plików kontenera – nigdy z danych dostarczonych przez użytkownika.

Architektura allowlist: użytkownik dostarcza wyłącznie klucze JSON (dane domenowe), nie struktury TeX. Sanitizer w filtrach Jinja2 (`|s`, `|date`, `|num`) zapobiega wstrzyknięciu specjalnych znaków LaTeX nawet w polach tekstowych.

```
    """
```

```
    logger.info("Kompilacja %s", template_name)
```

```
    lualatex = _find_lualatex()
```

```
    tex_source = render_tex(template_name, context)
```

```
    with tempfile.TemporaryDirectory(prefix="tex_") as tmpdir:
```

```
        workdir = Path(tmpdir)
```

```
        tex_file = workdir / "document.tex"
```

```
        tex_file.write_text(tex_source, encoding="utf-8")
```

```
        for _ in range(COMPILE_PASSES):
```

```
            _run_lualatex(lualatex, tex_file, workdir)
```

```
pdf_file = workdir / "document.pdf"

if not pdf_file.exists():

    raise TexCompilationError("lualatex zakończył się sukcesem, ale brak pliku PDF.")


pdf_bytes = pdf_file.read_bytes()


logger.info("Wygenerowano %d bajtów", len(pdf_bytes))

return pdf_bytes
```

PLIK: tex_service\Dockfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \

    PYTHONUNBUFFERED=1 \

    DEBIAN_FRONTEND=noninteractive


WORKDIR /app


RUN apt-get update && apt-get install -y --no-install-recommends \

    texlive-luatex \

    texlive-latex-base \

    texlive-latex-recommended \

    texlive-latex-extra \

    texlive-fonts-recommended \

    texlive-lang-polish \

    lmodern \

    curl \

    && rm -rf /var/lib/apt/lists/*


COPY requirements.txt ./

RUN pip install --no-cache-dir -r requirements.txt


COPY sanitizer.py compiler.py app.py check_lualatex_version.py ./

COPY templates/ ./templates/


# Konfiguracja bezpieczeństwa TeX: openin_any=p, openout_any=p, shell_escape=f

# Plik jest kopiowany do katalogu przeszukiwanego przez kpsewhich TEXMFLOCAL.

COPY texmf.cnf /etc/texmf/texmf.d/99-security.cnf

RUN update-updmap --quiet 2>/dev/null || true && \

    texhash 2>/dev/null || true


# Weryfikacja wersji LuaLaTeX – wymagana >= 1.17.0 (naprawa CVE-2023-32700).

# Jeśli wersja jest za stara, build się nie powiedzie.
```

```
RUN lualatex --version | head -1 && python3 check_lualatex_version.py
```

```
# Bezpieczeństwo: dedykowany nieuprzywilejowany użytkownik (nie root).
```

```
# LuaLaTeX potrzebuje /tmp do plików pomocniczych – tmpfs montowany
```

```
# jest w docker-compose, więc chown /tmp nie jest wymagany.
```

```
RUN addgroup --system appgroup \
```

```
&& adduser --system --ingroup appgroup --no-create-home appuser \
```

```
&& chown -R appuser:appgroup /app
```

```
USER appuser
```

```
EXPOSE 5002
```

```
CMD ["gunicorn", "--bind", "0.0.0.0:5002", "--workers", "2", "--timeout", "120", "app:app"]
```


PLIK: tex_service\sanitizer.py

```
r"""tex_service/sanitizer.py
```

Sanityzacja danych wejściowych przed wstrzyknięciem do szablonów LaTeX.

Warstwa ochrony: dane z bazy/requestu -> sanitize() -> szablon LaTeX.

Nawet jeśli użytkownik wpisze '\\directlua{...}' w pole formularza,

po przejściu przez sanitize() zostanie to zamienione na bezpieczny

ciąg znaków LaTeX (np. \textbackslash{}directlua\{...\}).

```
"""
```

```
import re
```

```
from datetime import date, datetime
```

```
from pylatexenc.latexencode import unicode_to_latex
```

```
# Znaki specjalne LaTeX wymagające escapowania.
```

```
# unicode_to_latex z pylatexenc obsługuje je wszystkie, ale jawna lista
```

```
# służy jako dokumentacja i drugi poziom ochrony.
```

```
_LATEX_SPECIAL_CHARS = {
```

```
    '\\': r'\textbackslash{}',
```

```
    '{': r'\{',
```

```
    '}': r'\}',
```

```
    '$': r'\$',
```

```
    '&': r'\&',
```

```
    '#': r'\#',
```

```
    '^': r'\^{}',
```

```
    '_': r'\_',
```

```
    '~': r'\textasciitilde{}',
```

```
    '%': r'\%',
```

```
}
```

```
def sanitize(value, max_length: int = 500) -> str:

    """Oczyszcza wartość do bezpiecznego wstawienia w szablon LaTeX.

    Używa pylatexenc.unicode_to_latex, które:

    - Zamienia wszystkie znaki specjalne LaTeX (\\, {, }, $, &, #, ^, _, ~, %)
      na ich escapowane odpowiedniki
    - Zamienia znaki Unicode na komendy LaTeX (np. ą → \\k{a})
```

Dzięki temu nawet jeśli wartość zawiera złośliwy kod LaTeX
(np. z pola tekstowego w formularzu), zostanie on zrenderowany
jako dosłowny tekst, nie wykonany jako komenda.

```
    """

    if value is None:

        return ''

    text = str(value).strip()

    if max_length:

        text = text[:max_length]

    # unicode_to_latex escapuje wszystkie znaki specjalne LaTeX

    return unicode_to_latex(text)
```

```
def sanitize_date(value, fmt: str = '%d.%m.%Y') -> str:

    """Formatuje datę do bezpiecznego ciągu – tylko cyfry i kropki."""

    if value is None:

        return '-'

    if isinstance(value, (date, datetime)):

        return value.strftime(fmt)

    try:

        return date.fromisoformat(str(value)).strftime(fmt)

    except ValueError:

        return sanitize(str(value))
```

```
def sanitize_int(value) -> str:
```

```
"""Zwraca wyłącznie cyfry całkowite – zero ryzyka injection."""
```

```
if value is None:
```

```
    return '0'
```

```
try:
```

```
    return str(int(value))
```

```
except (ValueError, TypeError):
```

```
    return '0'
```

PLIK: tex_service__init__.py

```
"""
```

tex_service – pakiet kompilacji LaTeX do PDF.

Publiczne API:

- compile_pdf(template_name, context) -> bytes

Jedyna dozwolona ścieżka kompilacji: szablon Jinja2 + dane domenowe.

Surowy TeX generowany jest wewnętrznie – nigdy nie pochodzi od użytkownika.

- render_tex(template_name, context) -> str

Renderuje szablon → tekst TeX (używane przez testy).

- TexCompilationError

```
"""
```

```
import sys
```

```
import os
```

```
# Gwarantuj że katalog tex_service/ jest w sys.path (potrzebne dla
```

```
# lokalnych importów: 'from sanitizer import ...' w compiler.py)
```

```
_pkg_dir = os.path.dirname(__file__)
```

```
if _pkg_dir not in sys.path:
```

```
    sys.path.insert(0, _pkg_dir)
```

```
from compiler import (          # noqa: E402
```

```
    TexCompilationError,
```

```
    render_tex,
```

```
    compile_pdf,
```

```
)
```

```
__all__ = [
```

```
    "TexCompilationError",
```

```
    "render_tex",
```

```
    "compile_pdf",
```

```
]
```